

Introduction à R pour les sciences de la communication

Développé pour le cours Exercices Méthodes II (Université de Fribourg)

Julian Maitra

6 octobre 2026

Université de Fribourg (2023–2025)
Freie Universität Berlin

Table des matières

Bienvenue	5
Comment utiliser ce site	5
1 Premiers pas avec R	6
1.1 Installer les logiciels R et RStudio	6
1.2 L'interface RStudio	7
1.3 Exécuter un script R	7
1.4 Commenter votre code avec un #	9
1.5 L'opérateur d'affectation <-	9
1.6 Variables à élément unique	10
1.7 Variables à plusieurs éléments : vecteurs	12
À faire avant de continuer	13
Exercices	14
2 Vecteurs, tableaux et facteurs	19
2.1 Répétition rapide de session 1	19
2.2 Données en forme de tableau	20
2.3 Les matrices	21
2.4 Les data frames	23
2.5 Fonctions statistiques de base	26
2.6 Les facteurs	27
2.7 Le répertoire de travail	29
À faire avant de continuer	29
Exercices	30
3 Révision 1 : bases de R	36
Exercices de révision	36
Questions avancées	42
4 Paquets, jeux de données et premiers graphiques	45
4.1 Les paquets R	45
4.2 Les jeux de données intégrés	47
4.3 Introduction aux visualisations avec ggplot2	52
À faire avant de continuer	57
Exercices	57
5 Visualiser des données avec ggplot2	62
5.1 Les nuages de points (angl. <i>scatter plots</i>)	62
5.2 Les diagrammes à barres	66

5.3	Les histogrammes	69
5.4	Les boîtes à moustaches (angl. <i>boxplots</i>)	75
	À faire avant de continuer	77
	Exercices	78
6	Révision 2 : visualisation	84
	Exercices de révision	84
7	Importer et manipuler des données	93
7.1	Lire des données au format CSV dans RStudio	93
7.2	Explorez votre jeu de données importé	96
7.3	La transformation du type de variable	97
7.4	L'opérateur <code>%>%</code>	98
7.5	Manipuler les données avec <code>dplyr</code>	99
7.6	Manipulation de colonnes : <code>select()</code> et <code>mutate()</code>	100
7.7	Manipulation de lignes avec <code>filter()</code> et <code>arrange()</code>	101
7.8	Résumer, compter et regrouper avec <code>summarize()</code> , <code>count()</code> et <code>group_by()</code>	102
	À faire avant de continuer	104
	Exercices	104
8	Manipuler des données : exercices	109
8.1	Exercices avec solutions	109
	Exercices	123
9	Révision 3 : manipulation de données	126
	Exercices de révision	126
10	Statistiques descriptives	133
10.1	Analyse des réponses au questionnaire, 1re partie (stats descriptives)	134
10.2	La fonction <code>write.csv()</code>	142
	Exercices	143
11	Statistiques inférentielles I : le test t	147
11.1	Qu'est-ce que la statistique inférentielle ?	147
11.2	Le test t	148
	Exercices	154
12	Statistiques inférentielles II : ANOVA et régression	157
12.1	L'analyse de la variance (ANOVA)	157
12.2	La régression linéaire avec <code>lm()</code>	162
	Exercices	176
13	Révision 4 : statistiques	179
	Exercices de révision	180
14	Pour aller plus loin	187
14.1	Pratique régulière	187

14.2 Application académique	187
14.3 MOOC	187
14.4 Tutoriels sur YouTube	188
14.5 Intelligence artificielle pour coder	188
14.6 Communautés en ligne	188
14.7 Lecture spécialisée	189
14.8 Valorisation des compétences	189
14.9 Le mot de la fin	189
À propos	190
Auteur et contact	190
Licences	190
Données	191
Outils	192
Cookies et suivi	192
Intelligence artificielle	192
Remerciements	193
 Annexes	 194
A Corrigés	194
Chapitre 1 : Premiers pas avec R	194
Chapitre 2 : Vecteurs, tableaux et facteurs	197
Chapitre 3 : Révision 1 : bases de R	201
Chapitre 4 : Paquets, jeux de données et premiers graphiques	207
Chapitre 5 : Visualiser des données avec ggplot2	210
Chapitre 6 : Révision 2 : visualisation	212
Chapitre 7 : Importer et manipuler des données	217
Chapitre 8 : Manipuler des données : exercices	219
Chapitre 9 : Révision 3 : manipulation de données	221
Chapitre 10 : Statistiques descriptives	225
Chapitre 11 : Statistiques inférentielles I : le test t	228
Chapitre 12 : Statistiques inférentielles II : ANOVA et régression	229
Chapitre 13 : Révision 4 : statistiques	231
 Références	 236

Bienvenue

Ce site propose une introduction pas à pas au langage R pour l'analyse de données en sciences sociales, et en particulier en sciences de la communication. Chaque chapitre combine des explications, du code à exécuter dans RStudio et des exercices interactifs avec corrections.

i Origine de ce site

Ce matériel a été développé pour le cours **Exercices Méthodes II** du Bachelor en sciences de la communication de l'Université de Fribourg, que j'ai enseigné de 2023 à 2025. Il s'adresse à toute personne qui souhaite apprendre R à son rythme.

Comment utiliser ce site

- Suivez les chapitres dans l'ordre : chacun s'appuie sur les précédents.
- Exécutez vous-même le code dans RStudio et modifiez-le : on apprend R en l'utilisant.
- Testez-vous avec les exercices à la fin de chaque chapitre. Les chapitres « Révision » reprennent les tests du cours.
- Sur ce site, les exercices sont interactifs ; dans la version PDF, les réponses se trouvent dans l'annexe « Corrigés ».

Les jeux de données utilisés dans le cours se trouvent dans le dossier [data/](#) et peuvent être lus directement depuis le site, par exemple :

```
igposts <- read.csv("https://julesaberlin.github.io/intro-a-R/data/igposts.csv",  
                    encoding = "UTF-8")
```

Merci à Franck Bleu, tuteur en méthodes quantitatives, pour sa contribution aux exercices ; voir [À propos](#).

1 Premiers pas avec R

Sujet : Bienvenue, pourquoi R ? et commandes de base I

Bienvenue ! Ce premier chapitre vous guide dans vos premiers pas avec R : installer les logiciels, découvrir l'interface RStudio et écrire vos premières lignes de code.

i À noter

R est un langage de programmation et un environnement logiciel pour l'analyse statistique, la visualisation de données et le calcul scientifique. Il est largement utilisé par les statisticiens, les chercheurs en sciences des données et de plus en plus en sciences sociales y compris en sciences de la communication.

R est idéal pour la manipulation, l'analyse, la modélisation et la visualisation de données. C'est un logiciel libre et open source, ce qui signifie qu'il est gratuit à utiliser, modifier et distribuer. Il dispose d'une grande communauté d'utilisateurs et de développeurs qui contribuent à son développement et à sa maintenance.

1.1 Installer les logiciels R et RStudio

Pour utiliser R sur votre ordinateur portable, vous devez d'abord installer les deux logiciels suivants :

1. le langage de programmation [R](#) (R Core Team, 2026)
2. l'interface [RStudio](#) (Posit team, 2026)

Notez que nous n'écrirons le code R que dans RStudio, l'interface. Le langage de programmation R doit d'abord être installé, mais il fonctionnera ensuite en arrière-plan. Il n'est donc pas nécessaire d'ouvrir R directement sur votre ordinateur par la suite.

1.2 L'interface RStudio

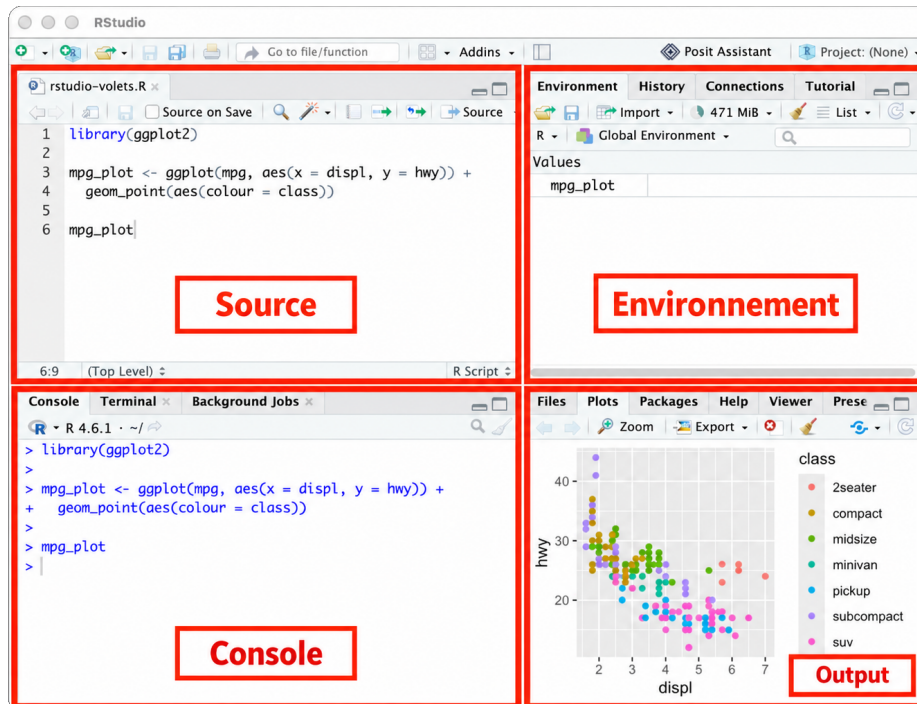


Figure 1.1 : Les quatre volets de RStudio. Capture d'écran de RStudio® Desktop (Posit Software, PBC), annotée par l'auteur.

Pour une présentation détaillée de l'interface, voir la [documentation de Posit](#).

Dans RStudio, il existe quatre volets différents :

1. Le volet **source**
2. Le volet **console**
3. Le volet **environnement**
4. Le volet **output**

1.3 Exécuter un script R

Nous travaillons principalement dans le volet **source**, c'est là que nous écrivons notre code.

Vous pouvez ouvrir le volet source en créant un nouveau **R script** (cliquez sur le signe plus vert dans le coin supérieur gauche).

Vous pouvez réécrire ou copier-coller le calcul simple suivant et l'insérer dans votre script :

```
10 + 14
```

Une fois que nous avons écrit du code R dans le script, nous pouvons **l'exécuter** en le sélectionnant et en cliquant sur le bouton d'exécution situé au-dessus du volet source (le bouton d'exécution avec la flèche verte à qui montre à droite).

C'est alors dans le volet de la **console** où le résultat va être affiché :

```
[1] 24
```

Notez que le nombre entre crochets indique la **position du premier élément** affiché sur cette ligne. Ici, il n'y a qu'un élément, donc [1]. Essayez **1:30** pour voir la différence.

Voici quelques autres calculs simples, suivi de leurs résultats :

```
1 + 2
```

```
[1] 3
```

```
99 - 98
```

```
[1] 1
```

```
2 * 10
```

```
[1] 20
```

```
100/2
```

```
[1] 50
```

```
2^10
```

```
[1] 1024
```



Astuce

Pour exécuter le code R plus rapidement, vous pouvez placer votre curseur sur une ligne de code à exécuter et utiliser les raccourcis clavier suivants :

- Sur **Windows** : CTRL + Enter
- Sur **MacOS** : Command + Enter

1.4 Commenter votre code avec un

C'est une bonne pratique de **commenter** son code R. Les commentaires expliquent ce que le code fait et facilite l'utilisation ultérieure et aussi le partage de code entre personnes (qui ne savent peut-être pas ce que vous vouliez faire avec votre code).

On commente en ajoutant un symbole **#** au début de la ligne de code, comme ceci :

```
# Ceci est un commentaire, et R l'ignorera lors de  
# l'exécution du script.
```

1.5 L'opérateur d'affectation <-

L'opérateur d'affectation **<-** est un opérateur clé de la programmation R. Il permet d'attribuer des valeurs aux variables.

```
# Par exemple, nous pouvons attribuer la valeur 5 à la  
# variable x :  
x <- 5
```

Si vous exécutez ce code, une nouvelle variable, x, sera créée dans le volet **environnement** en haut à droite de RStudio.

À noter

En R, tout ce que l'on manipule (une valeur, un vecteur, un tableau, une fonction) est un **objet** qui porte un nom. On parle aussi de **structures de données**. Les variables à un élément sont des objets très simples. Au cours de ce cours, nous découvrirons d'autres types d'objets.

Astuce

Vous pouvez créer l'opérateur d'affectation `<-` avec les raccourcis clavier suivants :

- Sur **Windows** : `Alt + -`
- Sur **MacOS** : `Option + -`

1.6 Variables à élément unique

Vous pouvez **imprimer** (afficher) la valeur d'une variable que vous avez créée, telle que `x`, en tapant le nom de la variable et en l'exécutant.

```
x
```

```
[1] 5
```

Important

- Veuillez toujours à exécuter votre code pour créer des variables avant de les utiliser pour d'autres opérations. Si vous oubliez de créer les variables en premier, vous obtiendrez une erreur comme résultat.

Par exemple, essayez d'exécuter ce qui suit : `y + z`

Pourquoi cela crée une erreur ? Réponse : les variables `y` et `z` n'ont pas encore été définies !

Pour éliminer l'erreur, nous devons donc définir `y` et `z`.

```
y <- 7
z <- 3

# Essayons encore une fois :
y + z
```

```
[1] 10
```

Maintenant ça marche! 😊

! Important

- Sachez aussi que si vous attribuez une nouvelle valeur à une variable existante, l'ancienne valeur sera écrasée!

```
# Notez que si vous exécutez toutes les lignes de code
# suivantes, la variable x se voit attribuer la dernière
# valeur exécutée : 4. Les valeurs 5 et 100 sont écrasées
# lors de l'exécution du code.
x <- 5
x <- 100
x <- 4
x
```

[1] 4

1.6.1 Les différents types de variable : numériques, textuelles et logique

Jusqu'à présent, nous avons créé des variables de **type numérique**, telles que x (valeur = 5).

Cependant, ce n'est pas le seul type de variable dans R.

i À noter

Dans R, il existe différents types de variables :

- les variables **numériques**
- les variables **textuelles**, également appelées *character strings* (chaînes de caractères).
- les variables **logiques** avec les valeurs TRUE et FALSE (VRAI et FAUX)

```
# Regardons quelques exemples avec du code ! (faut tout
# exécuter !)

# variables numériques :
a <- 100
b <- 77

age <- 21
Insta_likes <- 1539

# variables textuelles (toujours ajouter des guillemets !)
c <- "chien"
d <- "chat"

Comm_TikTok <- "wsh"
Moliere <- "il n'est rien d'égal au tabac : c'est la passion des honnêtes gens"
```

```
# variables logiques :
```

```
e <- TRUE  
f <- FALSE
```

Les variables logiques peuvent être utiles quand vous travaillez avec des **catégories binaires**.

Par exemple : il est souvent utile de classer les personnes ayant répondu à un questionnaire en catégories binaires, telles que :

- homme/femme
- mineur/majeur
- conservateur/libéral
- etc.

```
# Si Marc était mineur et Claire majeure, nous les  
# classerions comme suit :
```

```
Marc <- FALSE  
Claire <- TRUE
```

```
# Dans cet exemple, la variable logique indique si une  
# personne est majeure (TRUE) ou pas (FALSE)
```

1.7 Variables à plusieurs éléments : vecteurs

Les **vecteurs** constituent un autre type de variable (ou type d'objet) essentiel dans la programmation R. Il s'agit de variables comportant **plusieurs** éléments.

Vous pouvez créer des vecteurs avec l'opérateur d'affectation <- et la **fonction de concaténation** `c()`.

```
# Voici quelques exemples avec du code ! (faut tout  
# exécuter !)  
  
# un vecteur numérique (notez qu'il faut toujours séparer  
# les valeurs avec des virgules)  
v1 <- c(1, 2, 3)
```

Vous pouvez afficher la valeur de v1 en tapant son nom et l'exécutant

```
v1
```

```
[1] 1 2 3
```

À noter

En R, une **fonction** est un morceau de code conçu pour effectuer une tâche spécifique, souvent avec des paramètres variables. Ces paramètres d'entrée sont également appelés les **arguments** de la fonction.

Prenons l'exemple de la fonction `c()` qui crée des vecteurs :

- les arguments de cette fonction doivent être insérés entre les **parenthèses** sous la forme de valeurs séparées par des virgules (pour chaque élément du vecteur).
- Un vecteur à deux éléments : `c(2, 4)`
- Un vecteur à trois éléments : `c(3, 6, 9)`

Maintenant, regardons encore quelques exemples de vecteurs de **différents types** :

```
# vecteurs numériques
w <- c(-10, 100, 4, -88)

likes_comments_shares <- c(513, 34, 102)

# vecteurs textuels (toujours ajouter des guillemets pour
# chaque élément !)
animaux <- c("chat", "chien", "cheval", "chenille")

Comm_Insta <- c("trop cool", "wsh", "c'est quoi ?")

humains <- c("Claire", "Clarissa", "Theresa", "Marc", "Alma")

# vecteurs logiques :
femme <- c(TRUE, TRUE, TRUE, FALSE, TRUE)
```

Attention

Un vecteur ne contient qu'**un seul type** de données. Si l'on mélange des types, R les convertit sans prévenir vers le type le plus général (logique → numérique → texte) : `c(1, "a", TRUE)` donne `"1" "a" "TRUE"`.

À faire avant de continuer

- Si ce n'est pas encore fait : Installez R et RStudio sur votre ordinateur.
- Lisez ensuite encore une fois ce script et exécutez toutes les sections de code sur votre ordinateur.
- Essayez de résoudre vous-même les éventuels messages d'erreur en adaptant le code.

Exercices

Notez vos réponses, puis comparez-les avec l'annexe « Corrigés » à la fin du livre.

Question 1.1

La fonction `class()` appliquée à une variable à un élément peut donner comme résultat :

- a) "numeric"
- b) "logical"
- c) "character"
- d) 6

Pour chaque proposition : vrai ou faux ?

Question 1.2

Exécutez le code R suivant et jugez chaque proposition :

```
number <- "7"
```

- a) Le code `number^2` donne 36 comme résultat dans la console.
- b) Le code `number <- as.numeric(number)` ne change rien à ma variable.
- c) `number` peut être utilisé dans la fonction `c()` afin de créer un vecteur.
- d) `length(number)` donne une erreur.
- e) Le code `as.numeric(number)^2` donne 36 comme résultat dans la console.

Pour chaque proposition : vrai ou faux ?

Question 1.3

Exécutez le code suivant et jugez chaque proposition :

```
n <- "079 123 45 67"  
print(n)  
class(n)
```

- a) Le code crée un objet numérique avec 10 éléments.
- b) `n` est une variable de type textuel.
- c) `n` est une variable avec un élément.
- d) Si l'on a créé correctement `n` comme décrit ci-dessus et que l'on exécute `n * 2`, on obtient un message d'erreur.
- e) Dans R, il n'est jamais utile de stocker des numéros sous forme de texte.

Pour chaque proposition : vrai ou faux ?

Question 1.4

Exécutez le code suivant et jugez chaque proposition :

```
a <- 66
a <- 77
a <- 88
```

- a) Ce code génère une variable, `a`, dont la valeur finale correspond à 88.
- b) Le code écrase deux fois la variable `a`.
- c) Le code génère une erreur, car il ne peut pas créer trois variables portant exactement le même nom.
- d) Le code crée trois variables, toutes nommées `a`, et les stocke simultanément dans l'environnement.

Pour chaque proposition : vrai ou faux ?

Question 1.5

Exécutez le code R suivant et jugez chaque proposition :

```
a <- c(0:15)
print(a)
```

- a) `print(a)` est une fonction qui affiche l'objet `a` dans la console.
- b) Au lieu d'utiliser `print(a)`, on pourrait simplement afficher l'objet `a` en exécutant son nom `a`.
- c) La fonction `print()` dans R sert à imprimer le code R sur papier avec l'imprimante.
- d) Dans cet exemple de code, `a` est une fonction R.
- e) Dans cet exemple de code, un vecteur nommé `c` est créé.

Pour chaque proposition : vrai ou faux ?

Question 1.6

Exécutez le code R suivant et jugez chaque proposition :

```
a <- c(1, 22, 8, -5)
print(a)
class(a)
```

- a) **a** est un vecteur.
- b) **a** est un objet R de type numérique.
- c) Le code crée un objet dans le volet *Environment* de RStudio.
- d) Le code utilise la fonction de concaténation `c()`.

Pour chaque proposition : vrai ou faux ?

Question 1.7

Le code R suivant ouvre une page d'aide dans RStudio lorsqu'il est exécuté :

```
?typeof()
```

Vrai ou faux ?

Question 1.8

Dans R, la fonction `c()` peut être utilisée pour concaténer des éléments afin de créer des vecteurs.

Vrai ou faux ?

Question 1.9

Le `#` permet d'insérer des commentaires dans le script, sans que cela ait des conséquences sur mon codage.

Vrai ou faux ?

Question 1.10

Le code suivant crée une variable textuelle lorsqu'il est exécuté :

```
monstre <- "vampire"
```

Vrai ou faux ?

Question 1.11

Les deux lignes de code suivantes créent exactement la même variable lorsqu'elles sont exécutées :

```
a <- "Université de Fribourg"  
a = "Université de Fribourg"
```

Vrai ou faux ?

Question 1.12

Dans R, l'opérateur d'affectation (<-) peut être utilisé pour créer des objets, tels que des variables à un élément, des vecteurs, des matrices et des data frames.

Vrai ou faux ?

Question 1.13

Dans quel volet de RStudio écrivez-vous et enregistrez-vous votre script R ?

- a) Source
- b) Console
- c) Environnement
- d) Output

Une seule réponse est correcte.

Question 1.14

En général, la syntaxe des commandes R n'est pas sensible à la casse, c'est-à-dire aux lettres majuscules et minuscules.

Vrai ou faux ?

Question 1.15

Le logiciel RStudio, comme R, est un langage de programmation open source.

Vrai ou faux ?

Question 1.16

RStudio Desktop est un logiciel, dit environnement de développement, qui offre une interface utilisateur graphique permettant d'utiliser R, un langage de programmation.

Vrai ou faux ?

Question 1.17

Le code R suivant crée un vecteur avec 10 éléments numériques lorsqu'il est exécuté :

```
n <- "079 123 45 67"
```

Vrai ou faux ?

2 Vecteurs, tableaux et facteurs

2.1 Répétition rapide de session 1

```
# Script de répétition

# Avez-vous compris comment R traite les différents types
# de variables ? Quelle est la différence entre a et b ?

a <- 5
b <- "5"

# Comment afficher a et b ?

# Alternative : avec la fonction print(), qui augmente la
# compréhension ultérieure du code

# Pourquoi le code suivant produit-il une erreur ?
a * b
```

```
Error in `a * b`:
! non-numeric argument to binary operator
```

```
# Le code suivant corrige à nouveau l'erreur, pourquoi ?
b <- 5
a * b

# Vecteurs
v1 <- c(1, 2, 3, 4)
v1

# Vous pouvez créer des nombres successifs avec un colon :
v2 <- c(1:10)
v2

v3 <- c("salut", "le", "pote")
v4 <- c(TRUE, FALSE, FALSE)
```

Astuce

Vous pouvez utiliser la fonction `class()` pour vérifier le type d'un objet, p. ex. (exécutez vous-même!) :

- `class(c(1,2,3))`
- `class(c("Hello", "World"))`
- `class(c(TRUE,TRUE,FALSE))`

2.2 Données en forme de tableau

Les données sous forme de tableaux, avec des lignes et des colonnes, sont au cœur des méthodes quantitatives dans les sciences de la communication.

Dans R, il existe plusieurs structures de données en format de tableau, notamment :

- Les **matrices**
- Les **data frames**
- D'autres types, tels que les **tibbles**, qui sont en fait des data frames adaptés à des paquets R spécifiques (nous traiterons ce sujet plus tard dans le cours).

Important

- Il est de convention qu'un tableau de données présente les **unités statistiques** par **ligne** (angl. *row*).
- Les **variables** ou caractéristiques de ces unités statistiques sont indiquées en conséquence par **colonnes** (angl. *column*).

À noter

Selon le type de données, surtout en recherche empirique en sciences sociales dont font partie les sciences de la communication, les unités statistiques peuvent être :

- des **participants** à un questionnaire ou à une expérimentation
- des **documents** (p. ex. articles de presse, posts sur réseaux sociaux)
- des **observations/ cas** (p. ex. des profils sur réseaux sociaux, des entreprises médiatiques)

Dans ces jeux de données, par convention, chaque ligne représente une unité statistique (ou un participant, un document, une observation), tandis que chaque colonne représente les variables qui ont été collectées pour chaque unité statistique.

À noter

La taille d'un jeu de données est indiquée par le **nombre d'unités statistiques n**.

- Par exemple, si 49 personnes ont répondu à un questionnaire, nous parlons donc d'un échantillon de $n = 49$ participants.

Voici un modèle de la manière dont on devrait structurer les réponses à un questionnaire avec n participants et X questions :

	id	question_A	question_B	question_C	etc.	question_X
1	personne_1	réponse_a_A	réponse_a_B	réponse_a_C	...	réponse_a_X
2	personne_2	réponse_a_A	réponse_a_B	réponse_a_C	...	réponse_a_X
3	personne_3	réponse_a_A	réponse_a_B	réponse_a_C	...	réponse_a_X
4	etc.	...	...	...	...	...
5	personne_n	réponse_a_A	réponse_a_B	réponse_a_C	...	réponse_a_X

Dans ce modèle, l'unité statistique de ce jeu de données est la personne qui a répondu au questionnaire (un participant). Chaque ligne correspond donc à une personne différente. Les variables, en revanche, sont les différentes questions qui ont été enregistrées pour chaque participant.

Regardons maintenant quelques objets R sous forme de tableau :

2.3 Les matrices

Les **matrices** sont des objets plutôt mathématiques. Elles doivent notamment être composées de données de même type, c'est à dire numérique, textuel, logique, etc.

On peut créer des objets matriciels avec la commande **matrix()**, avec les arguments suivants :

- un vecteur de valeurs (p. ex. `c(1,2,3,4,5,6)`)
- le nombre de lignes (`nrow=`),
- le nombre de colonnes (`ncol=`) et
- la *direction* de remplissage des valeurs, c'est-à-dire par lignes (`byrow = TRUE`) ou par colonnes (`byrow = FALSE`).

Créons quelques matrices exemplaires avec du code R maintenant.

```
m1 <- matrix(c(1, 2, 3, 4, 5, 6), nrow = 2, ncol = 3, byrow = FALSE)
print(m1)

m2 <- matrix(c(1:6), nrow = 2, ncol = 3, byrow = TRUE)
print(m2)

# Quelle est la différence entre m1 et m2 au niveau du code
# ?
```

i À noter

Si vous ne spécifiez pas tous les arguments d'une fonction, R va tenter d'utiliser des arguments par défauts.

```
# On peut p. ex. créer une matrice sans spécifier tout les
# arguments de la fonction matrix()
m3 <- matrix(c(1:10), nrow = 5)

print(m3)
```

```
      [,1] [,2]
[1,]    1    6
[2,]    2    7
[3,]    3    8
[4,]    4    9
[5,]    5   10
```

Vous pouvez extraire des éléments individuels ou des parties d'une matrice avec des **parenthèses carrées []** en spécifiant les lignes et les colonnes concernées.

```
# Créons d'abord un matrice m4 avec 100 éléments :
m4 <- matrix(c(1:100), nrow = 10, byrow = TRUE)
print(m4)
```

```
      [,1] [,2] [,3] [,4] [,5] [,6] [,7] [,8] [,9] [,10]
[1,]    1    2    3    4    5    6    7    8    9   10
[2,]   11   12   13   14   15   16   17   18   19   20
[3,]   21   22   23   24   25   26   27   28   29   30
[4,]   31   32   33   34   35   36   37   38   39   40
[5,]   41   42   43   44   45   46   47   48   49   50
[6,]   51   52   53   54   55   56   57   58   59   60
[7,]   61   62   63   64   65   66   67   68   69   70
[8,]   71   72   73   74   75   76   77   78   79   80
[9,]   81   82   83   84   85   86   87   88   89   90
[10,]  91   92   93   94   95   96   97   98   99  100
```

```
# Extraire le deuxième élément de la troisième ligne :
m4[3, 2]
```

```
[1] 22
```

```
# Extraire les cinq premiers éléments de la dixième ligne :
m4[10, c(1:5)]
```

```
[1] 91 92 93 94 95
```

```
# Extraire tous les éléments de la première ligne :  
m4[1, ]
```

```
[1] 1 2 3 4 5 6 7 8 9 10
```

```
# Extraire tous les éléments de la troisième colonne :  
m4[, 3]
```

```
[1] 3 13 23 33 43 53 63 73 83 93
```

```
# Extraire la moitié inférieure de la matrice (lignes 6 à  
# 10) :  
m4[6:10, ]
```

	[,1]	[,2]	[,3]	[,4]	[,5]	[,6]	[,7]	[,8]	[,9]	[,10]
[1,]	51	52	53	54	55	56	57	58	59	60
[2,]	61	62	63	64	65	66	67	68	69	70
[3,]	71	72	73	74	75	76	77	78	79	80
[4,]	81	82	83	84	85	86	87	88	89	90
[5,]	91	92	93	94	95	96	97	98	99	100

```
# Et la moitié supérieure (lignes 1 à 5) :  
m4[1:5, ]
```

	[,1]	[,2]	[,3]	[,4]	[,5]	[,6]	[,7]	[,8]	[,9]	[,10]
[1,]	1	2	3	4	5	6	7	8	9	10
[2,]	11	12	13	14	15	16	17	18	19	20
[3,]	21	22	23	24	25	26	27	28	29	30
[4,]	31	32	33	34	35	36	37	38	39	40
[5,]	41	42	43	44	45	46	47	48	49	50

2.4 Les data frames

i À noter

Les **data frames** (français : *cadre de données*) sont le type de donnée en forme tableau le plus commun dans R. Cette structure de donnée peut notamment être composée de données de différents types (numérique, textuel, logique).

Créons un data frame avec la fonction `data.frame()` maintenant. Nous appelons ce type d'objets désormais un **jeu de données**.

Nos exemples reprennent les personnages du film d'animation *L'Âge de glace 3 : Le Temps des dinosaures* (2009).

```
# Créons d'abord un vecteur textuel avec les noms de
# certains animaux du film L'âge de glace.
name <- c("Manny", "Sid", "Diego", "Ellie", "Peaches", "Scrat",
          "Rudy", "Buck")

# Un second vecteur textuel indique l'espèce de chaque
# animal.
species <- c("mammoth", "sloth", "sabertooth", "mammoth", "mammoth",
             "squirrel", "dinosaur", "weasel")

# Un troisième vecteur logique indique si chaque animal est
# une femelle
female <- c(FALSE, FALSE, FALSE, TRUE, TRUE, FALSE, FALSE, FALSE)

# Un quatrième vecteur numérique indique l'âge en années de
# chaque animal.
age <- c(30, 20, 35, 25, 7, 4, 100, 28)

# Un cinquième tableau numérique indique le poids en kg de
# chaque animal.
weight <- c(5000, 80, 400, 4000, 1000, 1, 10000, 3)

# La fonction data.frame() permet d'utiliser les vecteurs
# pour former un tableau de données.
ice_age_df <- data.frame(name, species, female, age, weight)
```

Nous pouvons afficher l'objet `ice_age_df` en exécutant son nom ou en utilisant la fonction `print()` :

```
print(ice_age_df)
```

	name	species	female	age	weight
1	Manny	mammoth	FALSE	30	5000
2	Sid	sloth	FALSE	20	80
3	Diego	sabertooth	FALSE	35	400
4	Ellie	mammoth	TRUE	25	4000
5	Peaches	mammoth	TRUE	7	1000
6	Scrat	squirrel	FALSE	4	1
7	Rudy	dinosaur	FALSE	100	10000
8	Buck	weasel	FALSE	28	3

2.4.1 Quelques fonctions utiles pour analyser les data frames

Il existe plusieurs **fonctions utiles** dans R qui vous permettent d'analyser les propriétés de jeux de données de type data frame (et d'autres types d'objets R), comme p. ex. :

- `dim()` : Retourne les dimensions d'un jeu de données - le nombre de lignes et de colonnes.
- `head()` : Affiche les six premières lignes du jeu de données.

- `str()` : Fournit la structure du jeu de données, y compris
 - le type de variable de chaque colonne,
 - le nombre d'observations et
 - les premières entrées de chaque colonne.

C'est un moyen rapide de se faire une idée du contenu de votre jeu de données.

- `summary()` : Donne un résumé du jeu de données, y compris des statistiques telles que
 - la moyenne,
 - la médiane,
 - le minimum et le maximum pour les variables numériques, et
 - les fréquences pour les variables factorielles.

Cette fonction est utile pour obtenir rapidement une vue d'ensemble des statistiques.

```
# Utilisons ces fonctions pour analyser l'objet ice_age_df
```

```
dim(ice_age_df)
```

```
[1] 8 5
```

```
head(ice_age_df)
```

	name	species	female	age	weight
1	Manny	mammoth	FALSE	30	5000
2	Sid	sloth	FALSE	20	80
3	Diego	sabertooth	FALSE	35	400
4	Ellie	mammoth	TRUE	25	4000
5	Peaches	mammoth	TRUE	7	1000
6	Scrat	squirrel	FALSE	4	1

```
str(ice_age_df)
```

```
'data.frame': 8 obs. of 5 variables:
 $ name : chr "Manny" "Sid" "Diego" "Ellie" ...
 $ species: chr "mammoth" "sloth" "sabertooth" "mammoth" ...
 $ female : logi FALSE FALSE FALSE TRUE TRUE FALSE ...
 $ age : num 30 20 35 25 7 4 100 28
 $ weight : num 5000 80 400 4000 1000 1 10000 3
```

```
summary(ice_age_df)
```

	name	species	female	age
Length	:8	Length : 8	Mode :logical	Min. : 4.00
N.unique	:8	N.unique : 6	FALSE:6	1st Qu.: 16.75
N.blank	:0	N.blank : 0	TRUE :2	Median : 26.50
Min.nchar	:3	Min.nchar: 5		Mean : 31.12
Max.nchar	:7	Max.nchar:10		3rd Qu.: 31.25

```

weight
Min.   :    1.00
1st Qu.:   60.75
Median :  700.00
Mean   : 2560.50
3rd Qu.: 4250.00
Max.   :10000.00
Max.    :100.00

```

Astuce

Une autre astuce utile consiste à utiliser le **\$** pour **extraire des colonnes spécifiques** d'un jeu de données.

```
# P. ex. :
ice_age_df$name
```

```
[1] "Manny"  "Sid"    "Diego"  "Ellie"  "Peaches" "Scrat"
[7] "Rudy"   "Buck"
```

```
# Essayez vous-même et extrayez d'autres colonnes de
# ice_age_df à l'aide du $ !
```

2.5 Fonctions statistiques de base

Il existe dans R toute une palette de fonctions statistiques qui vous permettent d'évaluer principalement des données de type numérique. Quelques fonctions utiles pour les statistiques descriptives de base :

- `mean()` : Calcule la moyenne arithmétique
- `median()` : Calcule la médiane
- `sd()` : Calcule l'écart-type (angl. *standard deviation*)
- `var()` : Calcule la variance

```
a1 <- c(1, 2, 100)
```

```
mean(a1)
```

```
median(a1)
```

```
sd(a1)
```

```
var(a1)
```

```
[1] 34.33333
```

```
[1] 2
```

```
[1] 56.8712
```

```
[1] 3234.333
```

2.6 Les facteurs

Un autre type de variable clé dans R est ce que l'on appelle les **facteurs**, qui sont essentiellement des **variables catégorielles**.

i À noter

Les facteurs sont des variables qui prennent un nombre limité de valeurs différentes (appelées *levels* dans R).

Il existe deux sous-types de facteurs :

- Les **facteurs nominaux** (sans ordre) : toutes les catégories sont équivalentes les unes aux autres. Il n'y a pas de hiérarchisation.
 - Exemples : le genre et l'appartenance à un parti politique
- Les **facteurs ordinaux** (avec un ordre) : il y a un ordre ou une hiérarchie spécifiée parmi les catégories (p. ex. des niveaux tels que *faible, moyen, élevé*).
 - Exemples : niveau d'éducation, cohortes d'âge

Vous pouvez créer un facteur nominal à l'aide de la fonction `factor()` et vérifier ses catégories avec `levels()`. Voici un exemple :

```
# Créer un vecteur de données catégorielles, d'abord de
# type textuel
genre <- c("homme", "femme", "femme", "femme", "homme")

# Le convertir ensuite en facteur
facteur_genre <- factor(genre)

# Voir les niveaux du facteur genre
levels(facteur_genre)
```

```
[1] "femme" "homme"
```

```
# Imprimer le facteur (montre aussi les niveaux)
print(facteur_genre)
```

```
[1] homme femme femme femme homme
Levels: femme homme
```

Et voici comment créer un facteur ordinal :

```
# Voici un vecteur textuel avec des tailles de T-shirt
tailles <- c("Small", "Medium", "Large", "Medium", "Small")

# On le transforme en facteur ordinal en spécifiant les
# arguments suivants :
tailles_facteur <- factor(tailles, ordered = TRUE, levels = c("Small",
  "Medium", "Large"))

levels(tailles_facteur)
```

```
[1] "Small" "Medium" "Large"
```

```
print(tailles_facteur)
```

```
[1] Small Medium Large Medium Small
Levels: Small < Medium < Large
```

Transformons maintenant la colonne `species` de notre jeu de données `ice_age_df` en facteur avec la fonction `as.factor()` :

```
# Vérifions d'abord la structure de l'objet ice_age_df :
str(ice_age_df)
```

```
'data.frame': 8 obs. of 5 variables:
 $ name : chr "Manny" "Sid" "Diego" "Ellie" ...
 $ species: chr "mammoth" "sloth" "sabertooth" "mammoth" ...
 $ female : logi FALSE FALSE FALSE TRUE TRUE FALSE ...
 $ age : num 30 20 35 25 7 4 100 28
 $ weight : num 5000 80 400 4000 1000 1 10000 3
```

```
# Transformation en facteur :
ice_age_df$species <- as.factor(ice_age_df$species)

# Vérifions encore une fois la structure de l'objet :
str(ice_age_df)
```

```
'data.frame': 8 obs. of 5 variables:
 $ name : chr "Manny" "Sid" "Diego" "Ellie" ...
 $ species: Factor w/ 6 levels "dinosaur","mammoth",...: 2 4 3 2 2 5 1 6
 $ female : logi FALSE FALSE FALSE TRUE TRUE FALSE ...
 $ age : num 30 20 35 25 7 4 100 28
 $ weight : num 5000 80 400 4000 1000 1 10000 3
```

C'est maintenant à vous de transformer la colonne `name` de `ice_age_df` en facteur !

2.7 Le répertoire de travail

! Important

Dans R, le **répertoire de travail** (angl. *working directory*) fait référence au dossier de votre ordinateur dans lequel R lit et enregistre les fichiers par défaut. Il s'agit d'un concept crucial dans la programmation R, car il détermine l'endroit où R recherche les fichiers à lire et l'endroit où il place les fichiers lorsque vous les enregistrez.

Vous pouvez définir votre répertoire de travail avec la fonction `setwd()`.

Une possibilité, c'est de le faire manuellement dans le volet *output* en bas à droite :

1. Naviguez sur l'onglet **Files**
2. Cliquez sur les trois petits points ... tout à droite
3. Choisissez un dossier sur votre ordinateur qui peut servir de répertoire de travail (vous devez peut-être en créer un d'abord et lui donner un nom utile, p. ex. "intro_R")
4. Cliquez ensuite sur la petite **roue bleue** (More) et choisissez **Set as Working Directory**
5. Vous pouvez ensuite copier et coller le code R qui définit le répertoire de travail depuis l'onglet de la console et l'inclure dans votre script R

Par exemple, sous Windows : `setwd("C:/Users/votre_nom/Documents/intro_R")` ; sous macOS : `setwd("~/Documents/intro_R")`. **Attention** : Ce code ne marche naturellement pas sur votre machine. Il faut l'adapter et inclure un chemin de fichier de votre ordinateur.

Vous pouvez ensuite vérifier si le répertoire de travail a été correctement défini avec la fonction `getwd()`

💡 Bonne pratique : les projets RStudio

Créez un **projet RStudio** (*File → New Project*) pour chaque travail. Le répertoire de travail est alors défini automatiquement sur le dossier du projet, et vos chemins de fichiers relatifs (p. ex. `read.csv("donnees.csv")`) restent valables sur n'importe quel ordinateur. Avec `setwd()`, le code cesse de fonctionner dès qu'il est déplacé sur un autre ordinateur.

À faire avant de continuer

- Refaites sur votre ordinateur tout le code des chapitres 1 et 2 et vérifiez qu'il s'exécute sans message d'erreur.
- Assurez-vous de comprendre ce que fait chaque ligne, pas seulement de pouvoir l'exécuter.
- Testez-vous avec les exercices ci-dessous, puis avec le chapitre « Révision 1 ».

Exercices

Notez vos réponses, puis comparez-les avec l'annexe « Corrigés » à la fin du livre.

Question 2.1

Exécutez le code R suivant et jugez chaque proposition :

```
a <- c(1:5)
b <- c(5:10)
c <- a * b
```

- a) Les longueurs de `a` et `b` sont égales.
- b) Dans cet exemple de code, `c` ne peut être créé que si `a` et `b` sont créés avant.
- c) `length(a) <= length(b)`
- d) Dans cet exemple de code, le signe deux-points (`:`) crée des séquences de nombres entiers.

Pour chaque proposition : vrai ou faux ?

Question 2.2

Exécutez le code R suivant et jugez chaque proposition :

```
nom <- c("Kim", "Claire", "Antoine", "Mathilde", "Ali", "Charlotte", "Tiger", "Sacha")
genre <- factor(c("non binaire", "femme", "homme", "femme", "homme", "femme", "homme",
  ↪ "homme"),
               ordered = FALSE)
revenus_en_milles_chf <- c(77, 154, 15, 82, 104, 48, 540, 63)
classe_de_revenus <- factor(c("moyen", "haut", "très faible", "moyen", "haut", "faible",
  ↪ "très haut", "moyen"),
                           ordered = TRUE,
                           levels = c("très faible", "faible", "moyen", "haut", "très
  ↪ haut"))

revenus_df <- data.frame(nom, genre, revenus_en_milles_chf, classe_de_revenus)
revenus_df
```

- a) Ce data frame contient deux facteurs.
- b) `revenus_df$classe_de_revenus` est une variable catégorielle ordinale.
- c) `revenus_df$genre` est une variable catégorielle ordinale.
- d) Ce data frame contient des informations sur trois femmes et trois personnes ayant un haut revenu.

e) Ce data frame contient des informations sur cinq classes de revenus différentes.

Pour chaque proposition : vrai ou faux ?

Question 2.3

Exécutez le code R suivant et jugez chaque proposition :

```
name    <- c("Manny", "Sid", "Diego", "Ellie", "Peaches", "Scrat", "Rudy", "Buck")
species <- c("mammoth", "sloth", "sabertooth", "mammoth", "mammoth", "squirrel",
  ↪ "dinosaur", "weasel")
female  <- c(FALSE, FALSE, FALSE, TRUE, TRUE, FALSE, FALSE, FALSE)
age     <- c(30, 20, 35, 25, 7, 4, 100, 28)
weight  <- c(5000, 80, 400, 4000, 1000, 1, 10000, 3)

ice_age_df <- data.frame(name, species, female, age, weight)
ice_age_df
```

a) La colonne `female` (fr. « femelle ») du jeu de données `ice_age_df` est de type logique (*logical*).

b) Les unités statistiques du jeu de données `ice_age_df` sont les noms, les espèces, le genre (femelle), le poids et l'âge des animaux de l'ère glaciaire.

c) L'unité statistique du jeu de données `ice_age_df` est un animal de l'âge de glace, avec $n = 8$ observations.

d) Le jeu de données `ice_age_df` contient des informations sur trois mammouths différents.

e) Buck, l'écureuil, pèse 1 kg.

Pour chaque proposition : vrai ou faux ?

Question 2.4

Exécutez le code R suivant et jugez chaque proposition :

```
name    <- c("Manny", "Sid", "Diego", "Ellie", "Peaches", "Scrat", "Rudy", "Buck")
species <- c("mammoth", "sloth", "sabertooth", "mammoth", "mammoth", "squirrel",
  ↪ "dinosaur", "weasel")
female  <- c(FALSE, FALSE, FALSE, TRUE, TRUE, FALSE, FALSE, FALSE)
age     <- c(30, 20, 35, 25, 7, 4, 100, 28)
weight  <- c(5000, 80, 400, 4000, 1000, 1, 10000, 3)

ice_age_df <- data.frame(name, species, female, age, weight)
print(ice_age_df)
```

- a) Le code `ice_age_df$name` affiche les éléments de la colonne `name` du jeu de données `ice_age_df`.
- b) Le code `ice_age_df[, 2]` affiche les éléments de la deuxième colonne de `ice_age_df`.
- c) Les dimensions du jeu de données `ice_age_df` sont 8×5 (8 lignes et 5 colonnes).
- d) L'objet `ice_age_df` est de type matrice (*matrix*).
- e) Le code `mean(ice_age_df$age)` calcule la moyenne de la colonne `age` du jeu de données `ice_age_df`.

Pour chaque proposition : vrai ou faux ?

Question 2.5

Exécutez le code R suivant et jugez chaque proposition :

```
m1 <- matrix(c(1:25), nrow = 5, byrow = TRUE)
print(m1)
```

- a) `m1` est une matrice de dimension 5×5 (5 lignes et 5 colonnes).
- b) Dans le code R ci-dessus, l'argument `byrow = TRUE` spécifie que les valeurs de la matrice `m1` sont remplies par ligne.
- c) `m1` est une matrice de type textuel.
- d) `m1` est une matrice à 24 éléments.
- e) Le code R ci-dessus ne fonctionne pas et génère un message d'erreur.

Pour chaque proposition : vrai ou faux ?

Question 2.6

Jugez chaque proposition concernant les fonctions R `setwd()` et `getwd()` :

- a) Avec la fonction `setwd()`, je peux définir un dossier sur mon ordinateur à partir duquel R lit et extrait des données.
- b) Les chemins d'accès aux dossiers que l'on doit spécifier comme arguments dans `setwd()` diffèrent généralement entre différents ordinateurs.
- c) Vous ne pouvez pas utiliser R si vous ne spécifiez pas votre répertoire de travail.
- d) Toutes les variables, matrices, jeux de données, etc. que je crée dans une session R sont enregistrés sous forme de fichiers dans mon répertoire de travail.
- e) Vous pouvez afficher votre répertoire de travail actuel dans une session R avec la fonction `getwd()`.

Pour chaque proposition : vrai ou faux ?

Question 2.7

Exécutez le code R suivant et jugez chaque proposition :

```
vecteur2 <- c(1, 2, 3, 4, 1000, 2000)
```

- a) La valeur médiane de `vecteur2` est égale à 3.5.
- b) La valeur moyenne de `vecteur2` est égale à 501.00.
- c) `vecteur2` a une longueur de 3010.
- d) `vecteur2` est un objet de classe numérique.

Pour chaque proposition : vrai ou faux ?

Question 2.8

`length()` est une fonction de R qui permet de compter le nombre d'éléments d'un vecteur. Utilisez-la pour calculer la longueur du vecteur `v2` créé avec le code suivant :

```
v2 <- c(log10(100):501)
```

Réponse : _____

Question 2.9

Calculez la valeur médiane du vecteur numérique suivant :

```
vecteur1 <- c(1, 2, 3, 4, 1000, 2000)
```

Réponse : _____

Question 2.10

Calculez la valeur médiane d'un vecteur composé de nombres entiers consécutifs compris entre -66 et 702 .

Réponse : _____

Question 2.11

Exécutez le code R suivant pour créer `v1`. Ensuite, avec votre propre code, calculez la variance de `v1`.

```
v1 <- c(1:104)
```

Réponse : _____

Question 2.12

En statistique et en méthodes quantitatives en sciences sociales, c'est une convention de présenter les données dans des tableaux avec les unités statistiques (observations, cas, participants, etc.) en lignes et les variables en colonnes.

Vrai ou faux ?

Question 2.13

Les objets stockés dans l'environnement ne sont généralement stockés que temporairement, pour la session R en cours.

Vrai ou faux ?

Question 2.14

La variable *genre*, par exemple pour les participant·es à une enquête, est généralement utilisée comme facteur hiérarchique (ordonné) dans R.

Vrai ou faux ?

Question 2.15

Le code suivant crée une matrice de dimension 5×10 :

```
matrice1 <- matrix(c(5:10), nrow = 5, ncol = 10)
```

Vrai ou faux ?

Question 2.16

Si vous disposez d'un échantillon de 100 T-shirts et que chaque T-shirt est soit petit, soit moyen, soit grand, la variable *taille de T-shirt* peut être considérée comme une variable catégorielle ordinale.

Vrai ou faux ?

Question 2.17

En statistique et en méthodes quantitatives des sciences sociales, il est de convention de placer chaque observation dans une colonne différente d'un tableau de données.

Vrai ou faux ?

3 Révision 1 : bases de R

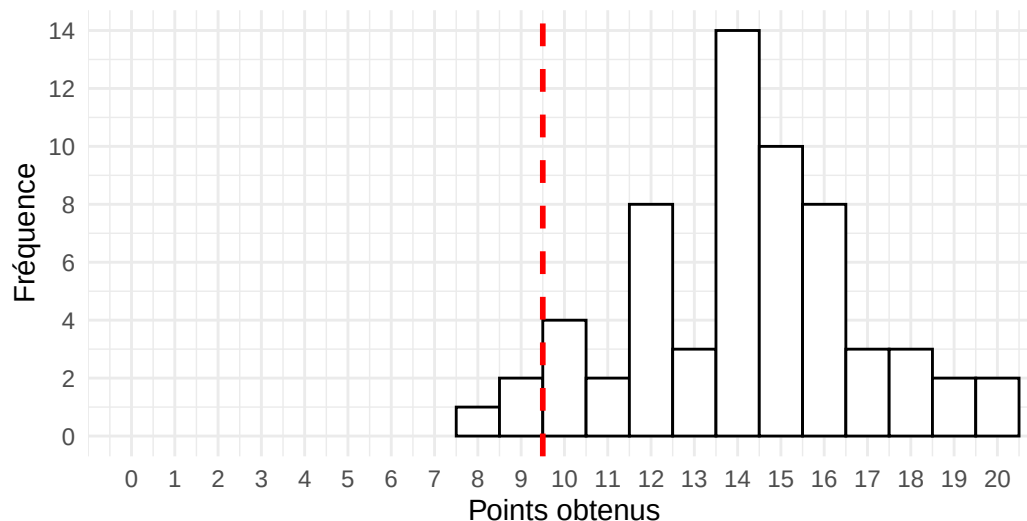
Dans le cours, le **Problem Set 1** était un test de 30 minutes sur Moodle portant sur les chapitres 1 et 2. Les questions ci-dessous en reprennent le contenu, avec la correction et une explication pour chaque réponse.

Résultats de l'édition 2025 du cours

62 étudiant·es ont passé ce test (20 points au maximum, 10 pour réussir). La moyenne était de 14,2 points et 95 % des étudiant·es l'ont réussi.

Histogramme des points obtenus dans le PS 1

n = 62 résultats de test



Source : Problem Set 1, édition 2025

Exercices de révision

Ces questions reprennent le **Problem Set 1**. Notez vos réponses, puis comparez-les avec l'annexe « Corrigés » à la fin du livre.

Question 3.1

Exécutez le code suivant et jugez chaque proposition :

```
n <- "079 123 45 67"
print(n)
class(n)
```

- a) Le code crée un objet numérique avec 10 éléments.
- b) `n` est une variable de type textuel.
- c) `n` est une variable avec un élément.
- d) Si l'on a créé correctement `n` comme décrit ci-dessus et que l'on exécute `n * 2`, on obtient un message d'erreur.
- e) Dans R, il n'est jamais utile de stocker des numéros sous forme de texte.

Pour chaque proposition : vrai ou faux ?

Question 3.2

Exécutez le code R suivant et jugez chaque proposition :

```
a <- c(0:15)
print(a)
```

- a) `print(a)` est une fonction qui affiche l'objet `a` dans la console.
- b) Au lieu d'utiliser `print(a)`, on pourrait simplement afficher l'objet `a` en exécutant son nom `a`.
- c) La fonction `print()` dans R sert à imprimer le code R sur papier avec l'imprimante.
- d) Dans cet exemple de code, `a` est une fonction R.
- e) Dans cet exemple de code, un vecteur nommé `c` est créé.

Pour chaque proposition : vrai ou faux ?

Question 3.3

Exécutez le code R suivant et jugez chaque proposition :

```
a <- c(1, 22, 8, -5)
print(a)
class(a)
```

- a) **a** est un vecteur.
- b) **a** est un objet R de type numérique.
- c) Le code crée un objet dans le volet *Environment* de RStudio.
- d) Le code utilise la fonction de concaténation `c()`.

Pour chaque proposition : vrai ou faux ?

Question 3.4

Dans R, la fonction `c()` peut être utilisée pour concaténer des éléments afin de créer des vecteurs.

Vrai ou faux ?

Question 3.5

Le `#` permet d'insérer des commentaires dans le script, sans que cela ait des conséquences sur mon codage.

Vrai ou faux ?

Question 3.6

Le code suivant crée une variable textuelle lorsqu'il est exécuté :

```
monstre <- "vampire"
```

Vrai ou faux ?

Question 3.7

Dans R, l'opérateur d'affectation (`<-`) peut être utilisé pour créer des objets, tels que des variables à un élément, des vecteurs, des matrices et des data frames.

Vrai ou faux ?

Question 3.8

Dans quel volet de RStudio écrivez-vous et enregistrez-vous votre script R ?

- a) Source
- b) Console
- c) Environnement
- d) Output

Une seule réponse est correcte.

Question 3.9

Le logiciel RStudio, comme R, est un langage de programmation open source.

Vrai ou faux ?

Question 3.10

Exécutez le code R suivant et jugez chaque proposition :

```
name    <- c("Manny", "Sid", "Diego", "Ellie", "Peaches", "Scrat", "Rudy", "Buck")
species <- c("mammoth", "sloth", "sabertooth", "mammoth", "mammoth", "squirrel",
  ↵ "dinosaur", "weasel")
female  <- c(FALSE, FALSE, FALSE, TRUE, TRUE, FALSE, FALSE, FALSE)
age     <- c(30, 20, 35, 25, 7, 4, 100, 28)
weight  <- c(5000, 80, 400, 4000, 1000, 1, 10000, 3)

ice_age_df <- data.frame(name, species, female, age, weight)
ice_age_df
```

- a) La colonne `female` (fr. « femelle ») du jeu de données `ice_age_df` est de type logique (*logical*).
- b) Les unités statistiques du jeu de données `ice_age_df` sont les noms, les espèces, le genre (femelle), le poids et l'âge des animaux de l'ère glaciaire.
- c) L'unité statistique du jeu de données `ice_age_df` est un animal de l'âge de glace, avec $n = 8$ observations.
- d) Le jeu de données `ice_age_df` contient des informations sur trois mammouths différents.
- e) Buck, l'écureuil, pèse 1 kg.

Pour chaque proposition : vrai ou faux ?

Question 3.11

Exécutez le code R suivant et jugez chaque proposition :

```
name    <- c("Manny", "Sid", "Diego", "Ellie", "Peaches", "Scrat", "Rudy", "Buck")
species <- c("mammoth", "sloth", "sabertooth", "mammoth", "mammoth", "squirrel",
  ↪ "dinosaur", "weasel")
female  <- c(FALSE, FALSE, FALSE, TRUE, TRUE, FALSE, FALSE, FALSE)
age     <- c(30, 20, 35, 25, 7, 4, 100, 28)
weight  <- c(5000, 80, 400, 4000, 1000, 1, 10000, 3)

ice_age_df <- data.frame(name, species, female, age, weight)
print(ice_age_df)
```

- a) Le code `ice_age_df$name` affiche les éléments de la colonne `name` du jeu de données `ice_age_df`.
- b) Le code `ice_age_df[, 2]` affiche les éléments de la deuxième colonne de `ice_age_df`.
- c) Les dimensions du jeu de données `ice_age_df` sont 8×5 (8 lignes et 5 colonnes).
- d) L'objet `ice_age_df` est de type matrice (*matrix*).
- e) Le code `mean(ice_age_df$age)` calcule la moyenne de la colonne `age` du jeu de données `ice_age_df`.

Pour chaque proposition : vrai ou faux ?

Question 3.12

Exécutez le code R suivant et jugez chaque proposition :

```
m1 <- matrix(c(1:25), nrow = 5, byrow = TRUE)
print(m1)
```

- a) `m1` est une matrice de dimension 5×5 (5 lignes et 5 colonnes).
- b) Dans le code R ci-dessus, l'argument `byrow = TRUE` spécifie que les valeurs de la matrice `m1` sont remplies par ligne.
- c) `m1` est une matrice de type textuel.
- d) `m1` est une matrice à 24 éléments.
- e) Le code R ci-dessus ne fonctionne pas et génère un message d'erreur.

Pour chaque proposition : vrai ou faux ?

Question 3.13

Jugez chaque proposition concernant les fonctions R `setwd()` et `getwd()` :

- a) Avec la fonction `setwd()`, je peux définir un dossier sur mon ordinateur à partir duquel R lit et extrait des données.
- b) Les chemins d'accès aux dossiers que l'on doit spécifier comme arguments dans `setwd()` diffèrent généralement entre différents ordinateurs.
- c) Vous ne pouvez pas utiliser R si vous ne spécifiez pas votre répertoire de travail.
- d) Toutes les variables, matrices, jeux de données, etc. que je crée dans une session R sont enregistrés sous forme de fichiers dans mon répertoire de travail.
- e) Vous pouvez afficher votre répertoire de travail actuel dans une session R avec la fonction `getwd()`.

Pour chaque proposition : vrai ou faux ?

Question 3.14

`length()` est une fonction de R qui permet de compter le nombre d'éléments d'un vecteur. Utilisez-la pour calculer la longueur du vecteur `v2` créé avec le code suivant :

```
v2 <- c(log10(100):501)
```

Réponse : _____

Question 3.15

Calculez la valeur médiane d'un vecteur composé de nombres entiers consécutifs compris entre -66 et 702 .

Réponse : _____

Question 3.16

Exécutez le code R suivant pour créer `v1`. Ensuite, avec votre propre code, calculez la variance de `v1`.

```
v1 <- c(1:104)
```

Réponse : _____

Question 3.17

En statistique et en méthodes quantitatives en sciences sociales, c'est une convention de présenter les données dans des tableaux avec les unités statistiques (observations, cas, participants, etc.) en lignes et les variables en colonnes.

Vrai ou faux ?

Question 3.18

La variable *genre*, par exemple pour les participant·es à une enquête, est généralement utilisée comme facteur hiérarchique (ordonné) dans R.

Vrai ou faux ?

Question 3.19

Si vous disposez d'un échantillon de 100 T-shirts et que chaque T-shirt est soit petit, soit moyen, soit grand, la variable *taille de T-shirt* peut être considérée comme une variable catégorielle ordinale.

Vrai ou faux ?

Question 3.20

En statistique et en méthodes quantitatives des sciences sociales, il est de convention de placer chaque observation dans une colonne différente d'un tableau de données.

Vrai ou faux ?

Questions avancées

Ces questions ouvertes n'ont pas de correction automatique. Essayez d'abord dans RStudio, puis comparez avec la solution proposée dans l'annexe « Corrigés ».

Question 3.A1

Écrivez une expression R vectorisée pour calculer la somme des nombres pairs de 1 à 100. Quel résultat obtenez-vous ?

Résultat : _____

Question 3.A2

On considère `v <- seq(from = 0, to = 1, length.out = 11)`.

1. Que renvoie `v[5]` ? _____
2. `v` est-il identique à `c(0, 0.1, 0.2, ..., 1)` ? Comparez avec `identical()` et `all.equal()`.

Question 3.A3

Créez un vecteur logique de longueur 20 alternant `TRUE`, `FALSE`, ... (en commençant par `TRUE`), sans boucle. Puis extrayez les éléments situés aux positions impaires.

Question 3.A4

Construisez un data frame `df` avec une colonne `A` contenant les nombres de 1 à 5 et une colonne `B` contenant leurs carrés. Quelle commande renvoie l'élément de la 4 ligne et de la 2 colonne, et quelle est sa valeur ?

Valeur : _____

Question 3.A5

```
tailles <- factor(c("S", "M", "L", "M", "S"),
                  ordered = TRUE,
                  levels = c("S", "M", "L"))
```

1. Convertissez `tailles` en numérique (`S` → 1, `M` → 2, `L` → 3).
2. Quel piège faut-il connaître avec `as.numeric()` appliqué à un facteur ?

Question 3.A6

```
m <- matrix(1:25, nrow = 5, byrow = FALSE)
```

1. Calculez la somme de la colonne 3 sans boucle. _____
2. Extrayez la sous-matrice composée des lignes 2 à 4 et des colonnes 1 et 5.

Question 3.A7

Quelle est la différence entre `class()` et `typeof()` ? Donnez un exemple.

Question 3.A8

```
x <- c(2, 4, NA, 8, 10)
```

1. Calculez la variance de `x` en ignorant la valeur manquante. _____
2. Quel est le rôle de l'argument `na.rm`?

Question 3.A9

En une ligne, utilisez `getwd()` et `setwd()` pour passer au dossier parent du répertoire de travail actuel.

Question 3.A10

Soit un data frame `df` avec une colonne numérique `X`. Ajoutez en base R une colonne `Y` qui contient les valeurs standardisées de `X`, soit $Y = (X - \text{moyenne}(X)) / \text{écart-type}(X)$.

4 Paquets, jeux de données et premiers graphiques

La session 4 a pour but d'introduire trois éléments fondamentaux en analyse de données avec R :

1. Installer et charger des **paquets R** sur son ordinateur
2. Utiliser des **jeux de données intégrés en R** pour s'entraîner en analyse de données
3. Apprendre à utiliser le paquet **ggplot2** pour créer des visualisations de données

4.1 Les paquets R

i À noter

Dans la programmation R, un **paquet** (angl. *package*) est un ensemble de fonctions, de données et de code compilé dans un format bien défini.

Les paquets étendent les fonctionnalités de R de base en fournissant des outils supplémentaires pour l'analyse des données, la visualisation, l'apprentissage automatique, la modélisation statistique et beaucoup d'autres applications spécialisées.

Les paquets R peuvent être développés par n'importe qui et partagés gratuitement avec d'autres (open source). Par conséquent, une grande multitude de paquets R sont disponibles.

Notez que la plupart des paquets R doit être installé séparément, normalement depuis le site [Comprehensive R Archive Network \(CRAN\)](https://cran.r-project.org/). Dans ce cours, nous nous concentrerons sur deux paquets célèbres :

- **ggplot2** pour créer des **visualisations de données** (Wickham, 2016)
- **dplyr** pour **manipuler les jeux de données** (Wickham et al., 2026)

Tous deux font partie du [tidyverse](https://www.tidyverse.org/), une collection de paquets conçus pour la science des données (Wickham et al., 2019). Nous utiliserons également le paquet suivant :

- **ggthemes** pour **augmenter le design des graphiques** (Arnold, 2026)

Pour utiliser un paquet R sur votre ordinateur, il faut :

1. D'abord **installer** le paquet en question avec la fonction `install.packages()`. Cela peut durer quelques minutes.

2. Ensuite **activer** le paquet avec la fonction `library()`.

Astuce

Dans RStudio, on peut aussi installer les paquets via l'onglet **Tools** et le menu déroulant : Cliquez sur *Install Packages...* et cherchez les paquets que vous voulez installer.

Utilisez le code suivant pour installer les paquets R requis pour notre cours. Enlevez les croisillons, exécutez le code avec `install.packages()` puis réajouter les croisillons pour éviter de réinstaller les paquets chaque fois que vous exécutez le code dans votre script ! Activer ensuite les paquets avec `library()`.

```
# install.packages('tidyverse')
# install.packages('ggthemes')

library(tidyverse)
library(ggthemes)
```

À noter : `ggplot2` et `dplyr` sont installés avec le `tidyverse`, il ne faut donc plus les installer séparément.

Important

Notez qu'une fois installé, vous n'avez pas besoin de le réinstaller les paquets à chaque fois avec `install.packages()`. Cependant, vous devez **activer** tous les paquets R que vous voulez utiliser dans une session avec la commande `library()`. Sans activation, vous allez recevoir des messages d'erreur.

Certains paquets ont des documentations auxquelles vous pouvez accéder grâce au point d'interrogation. Ces infos sont affichées dans le volet *Output* sous l'onglet *Help*.

- `?ggplot2`
- `?dplyr`

En ligne, vous pouvez également trouver de nombreuses ressources qui expliquent en détail les fonctionnalités des différents paquets R et vous renvoient vers d'autres livres, tutoriels, cours, etc., p. ex. :

- [Documentation officielle de ggplot2](#)
- [Documentation officielle de dplyr](#)
- [Exemples de graphiques créés avec les différents designs de ggthemes](#)

Vous pouvez vérifier quels paquets sont **actuellement installés sur votre ordinateur** avec la fonction `installed.packages()` (vous pouvez laisser vide l'espace entre parenthèses) :

```
installed.packages()
```

4.2 Les jeux de données intégrés

Une excellente façon, et probablement la plus efficace, d’apprendre à utiliser R pour l’analyse de données consiste à travailler avec des **jeux de données intégrés dans R**.

Ces jeux de données exemplaires présentent deux avantages principaux.

1. Il est facile d’y accéder et de les charger dans votre session RStudio, car ils sont intégrés à R et à certains de ses paquets.

Cela les distingue fortement des données du “monde réel”, comme les données de questionnaires ou de médias sociaux, qu’il faut souvent d’abord nettoyer et formater, puis importer dans RStudio sous forme de fichiers externes. Avec les jeux de données intégrés, vous pouvez immédiatement commencer l’analyse et la visualisation des données.

2. Ces jeux de données ont prouvé leur utilité dans d’innombrables supports d’apprentissage.

Par conséquent, vous pouvez facilement trouver des ressources supplémentaires et des exemples de code à leur sujet.

Dans ce qui suit, nous allons explorer trois jeux de données classiques de ce type :

- `iris` et `mtcars`, fournis avec R ;
- `diamonds`, fourni avec le paquet `ggplot2` (Wickham, 2016).

Nous verrons un quatrième jeu de données, `penguins` (paquet `palmerpenguins` ; Horst et al. (2020)), en session 12.

Astuce

Vous pouvez afficher les jeux de données qui sont actuellement accessibles sur votre ordinateur avec la commande `data()`.

4.2.1 Le jeu de données `iris`

Le jeu de données `iris` est un ensemble de données célèbre qui contient les mesures en centimètres des variables longueur et largeur du sépale, ainsi que longueur et largeur du pétale, pour trois espèces de fleurs d’iris (Setosa, Virginica et Versicolor). Il y a 150 observations en total (50 pour chaque espèce).

Le jeu de données a été rendu célèbre par le statisticien et biologiste britannique **Ronald Fisher** dans un article scientifique de 1936 qui avait beaucoup d’influence dans le domaine des statistiques (Fisher, 1936). Les mesures ont été publiées par le botaniste Edgar Anderson (1935) ; deux des trois espèces proviennent de la péninsule de Gaspé (Québec, Canada).

🔥 Plus sur Ronald Fisher

Ronald Fisher (1890-1962) était un statisticien, généticien et biologiste britannique qui a joué un rôle crucial dans le développement de la science statistique moderne. Il est connu pour son travail dans le développement des fondements des méthodes statistiques, incluant l'analyse de variance (ANOVA), le test exact de Fisher et l'estimation du maximum likelihood. Fisher a également apporté des contributions significatives en génétique, où son travail sur la théorie de la sélection naturelle et la génétique des populations a aidé à combler le fossé entre la génétique mendélienne et l'évolution darwinienne. Il est considéré comme l'un des architectes principaux de la synthèse néo-darwinienne. Son influence s'étend au-delà de la statistique et de la génétique aux domaines plus larges de la biologie et de la recherche agricole, faisant de lui une figure clé de la science du 20e siècle.

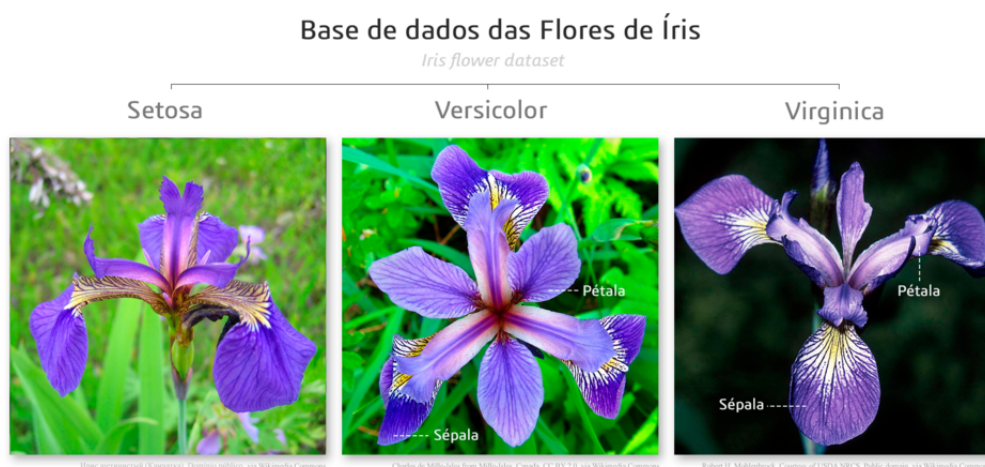


Figure 4.1 : Il y a des statistiques sur trois espèces d'iris dans le jeu de données. Photo : Diego Mariano, [CC BY-SA 4.0](#), via [Wikimedia Commons](#).

Dans l'apprentissage de R, ce jeu de données est souvent utilisé pour illustrer diverses techniques d'analyse de données, y compris le clustering, la classification et la visualisation.

Explorons un peu le jeu de données `iris` avec du code !

```
dim(iris) # Ceci affiche le nombre de lignes et de colonnes

head(iris) # Affiche les six premières lignes

str(iris) # Informations sur la structure du jeu de données

names(iris) # Affiche les noms des colonnes
```

Regardons ce que ça donne après l'exécution de chaque ligne de code :

```
dim(iris)
```

```
[1] 150  5
```

```
head(iris)
```

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
1	5.1	3.5	1.4	0.2	setosa
2	4.9	3.0	1.4	0.2	setosa
3	4.7	3.2	1.3	0.2	setosa
4	4.6	3.1	1.5	0.2	setosa
5	5.0	3.6	1.4	0.2	setosa
6	5.4	3.9	1.7	0.4	setosa

```
str(iris)
```

```
'data.frame':  150 obs. of  5 variables:
 $ Sepal.Length: num 5.1 4.9 4.7 4.6 5 5.4 4.6 5 4.4 4.9 ...
 $ Sepal.Width : num 3.5 3 3.2 3.1 3.6 3.9 3.4 3.4 2.9 3.1 ...
 $ Petal.Length: num 1.4 1.4 1.3 1.5 1.4 1.7 1.4 1.5 1.4 1.5 ...
 $ Petal.Width : num 0.2 0.2 0.2 0.2 0.2 0.4 0.3 0.2 0.2 0.1 ...
 $ Species : Factor w/ 3 levels "setosa","versicolor",...: 1 1 1 1 1 1 1
 1 1 1 ...
```

```
names(iris)
```

```
[1] "Sepal.Length" "Sepal.Width"  "Petal.Length" "Petal.Width"
[5] "Species"
```

Astuce

Vous pouvez consulter la documentation pour beaucoup de jeux de données avec un point d'interrogation : `?iris()`.

4.2.2 Le jeu de données `mtcars`

Utilisez le script suivant pour analyser `mtcars` (lisez également la documentation avec `?mtcars`)! Les données proviennent de Henderson & Velleman (1981).

```
# Chargez le jeu de données mtcars. Il devrait apparaître
# dans l'environnement
data(mtcars)

# Affichez mtcars
mtcars
```

```
# vous pouvez explorer mtcars manuellement en double
# cliquant dessus dans l'environnement

# Affichez seulement les six premières observations
head(mtcars)

# Affichez seulement les six dernières observations
tail(mtcars)

# Appliquez quelques autres commandes pour explorer ce jeu
# de données
class(mtcars)
dim(mtcars)

str(mtcars)
summary(mtcars)

glimpse(mtcars) # cette commande marche seulement si vous avez activé le tidyverse

# Extraire la colonne mpg ('miles per gallon') avec le
# signe $ et calculer la moyenne, la médiane, le minimum
# et le maximum

mtcars$mpg

mean(mtcars$mpg)

median(mtcars$mpg)

min(mtcars$mpg)

max(mtcars$mpg)
```



Figure 4.2 : Un des modèles de mtcars : la Toyota Corolla de 1974. Photo : Jeremy from Sydney, Australia, [CC BY 2.0](#), via [Wikimedia Commons](#).

4.2.3 Le jeu de données diamonds

C'est maintenant à vous d'explorer ce jeu de données avec du code !

Utilisez également `?diamonds()` !



Figure 4.3 : Qu'est-ce qui définit le prix d'un diamant ? Photo : Jerry Cone, domaine public, via [Wikimedia Commons](#).

4.3 Introduction aux visualisations avec ggplot2

i À noter

ggplot2, qui fait partie du **tidyverse**, est un paquet de visualisation de données très populaire dans R. Il a été conçu pour faciliter la création de graphiques complexes et multicouches avec une syntaxe cohérente et consistante nommée *grammar of graphics*. Le « gg » de **ggplot2** signifie *grammar of graphics* (français : *grammaire des graphiques*; Wilkinson, 2005), une théorie que Hadley Wickham a implémentée dans R ; le « 2 » distingue le paquet de sa première version, **ggplot**.

Avec **ggplot2**, les utilisateurs peuvent construire des graphiques de manière incrémentale en ajoutant des couches, ce qui le rend flexible pour une large gamme de besoins graphiques. Cela permet de créer des visualisations hautement personnalisables et esthétiquement attrayantes, allant de simples diagrammes de dispersion à des figures multi-panneaux complexes.

Le tableau suivant décrit les couches d'un graphique ggplot :

Table 4.1 : Les couches d'un graphique ggplot

Couche	Description/Variables	Obligatoire ?
<i>data</i>	le jeu de données que nous voulons visualiser	oui
<i>aesthetics</i>	le <i>mapping</i> de nos variables : axes <i>x</i> et <i>y</i> , <i>color</i> , <i>fill</i> , <i>alpha</i> , <i>line width</i> , etc.	oui
<i>geometries</i>	le type visuel du graphique : <i>point</i> , <i>line</i> , <i>histogram</i> , <i>bar</i> , <i>boxplot</i>	oui
<i>themes</i>	Le design du graphique	non
Autres couches optionnelles : <i>facets</i> , <i>statistics</i> , <i>coordinates</i>	Pour des visualisations plus complexes	non

Pour comprendre comment cela fonctionne, voici un exemple de code dans lequel les différentes couches d'un graphique *ggplot* sont ajoutées étape par étape dans la fonction `ggplot()`. (Normalement, nous ne procéderions pas ainsi, mais utiliserions directement le code en entier pour créer le graphique).

Le but dans cet exemple est de visualiser le jeu de données `diamonds` pour comprendre la relation entre les variables `price` (prix) et `carat` (poids) des diamants dans le jeu de données !

i À noter

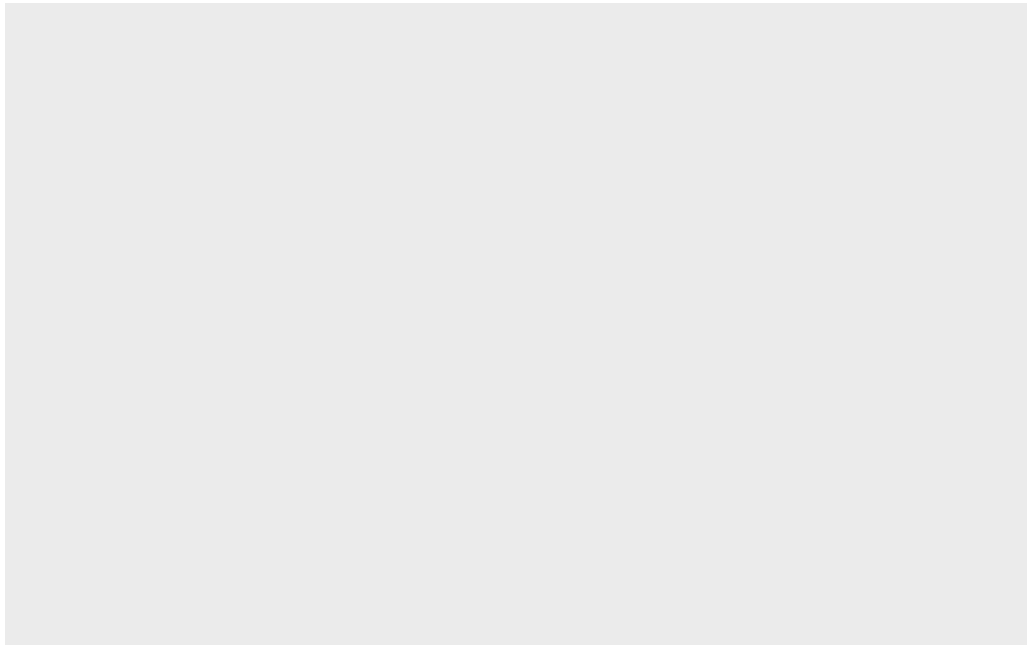
Important : avant d'analyser un ensemble de données, il faut toujours acquérir des **connaissances contextuelles** sur ce que les données représentent.

Dans notre exemple, il s'agit de diamants. Dans le monde des diamants, il est important de comprendre qu'il existe **quatre critères de qualité** qui déterminent **la valeur** (donc le prix) d'un diamant. Ce sont les suivants :

- La taille (angl. *cut*)
- La pureté (*clarity*)
- La couleur (*color*)
- Le poids en carats (*carat*)

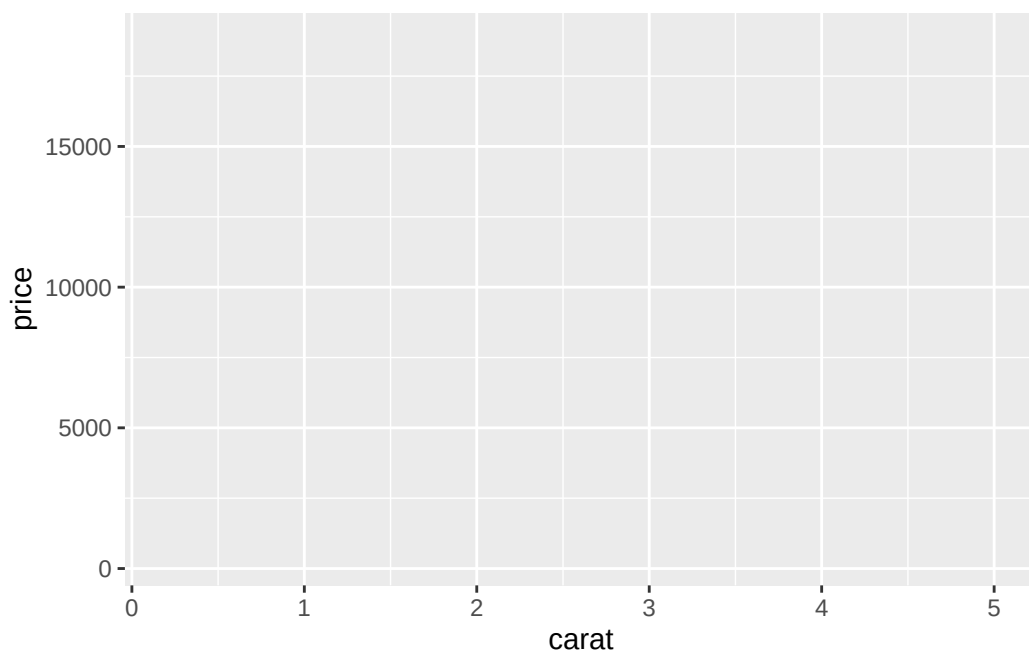
En anglais, on parle donc aussi des 4C. Le jeu de données `diamonds` contient précisément ces variables.

```
# La première couche, data, définit le jeu de données
ggplot(data = diamonds)
```



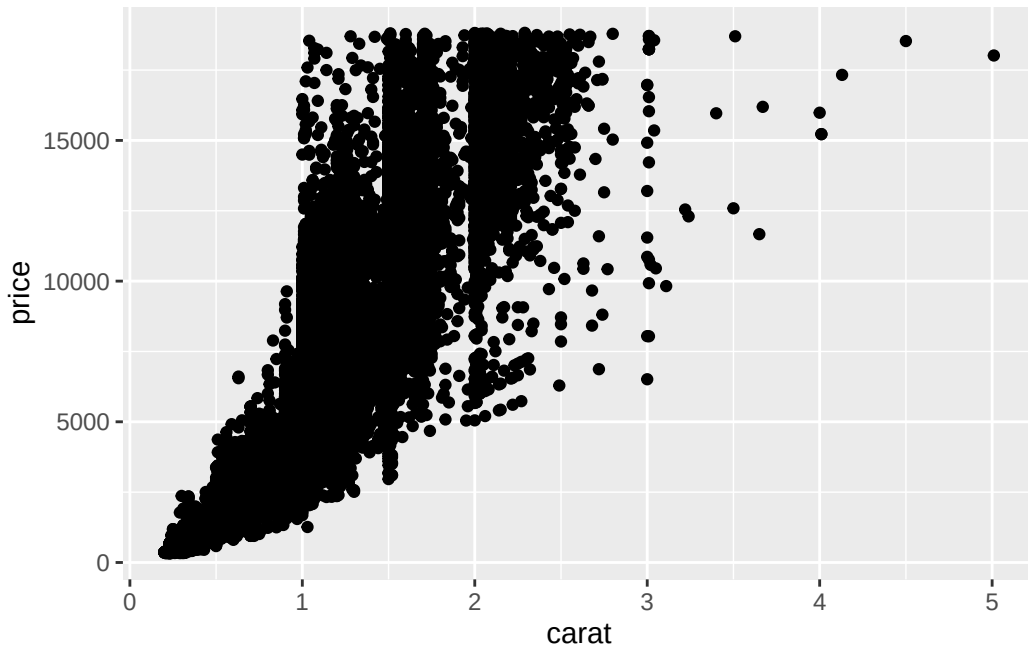
Notre graphique n'est pour l'instant qu'une zone grise! Catastrophe? Bien sûr que non 😊. Nous devons maintenant définir les autres paramètres de la fonction `ggplot()`!

```
# La deuxième couche, aesthetics, définit le mapping de nos  
# variables sur les axes y et x  
ggplot(data = diamonds, mapping = aes(x = carat, y = price))
```



La zone grise a maintenant reçu des unités pour ses axes x (`carat`) et y (`price`). Mais il n'y a toujours pas de données affichées !

```
# La troisième couche, geometries, définit le type visuel  
# de notre graphique  
ggplot(data = diamonds, mapping = aes(x = carat, y = price)) +  
  geom_point()
```

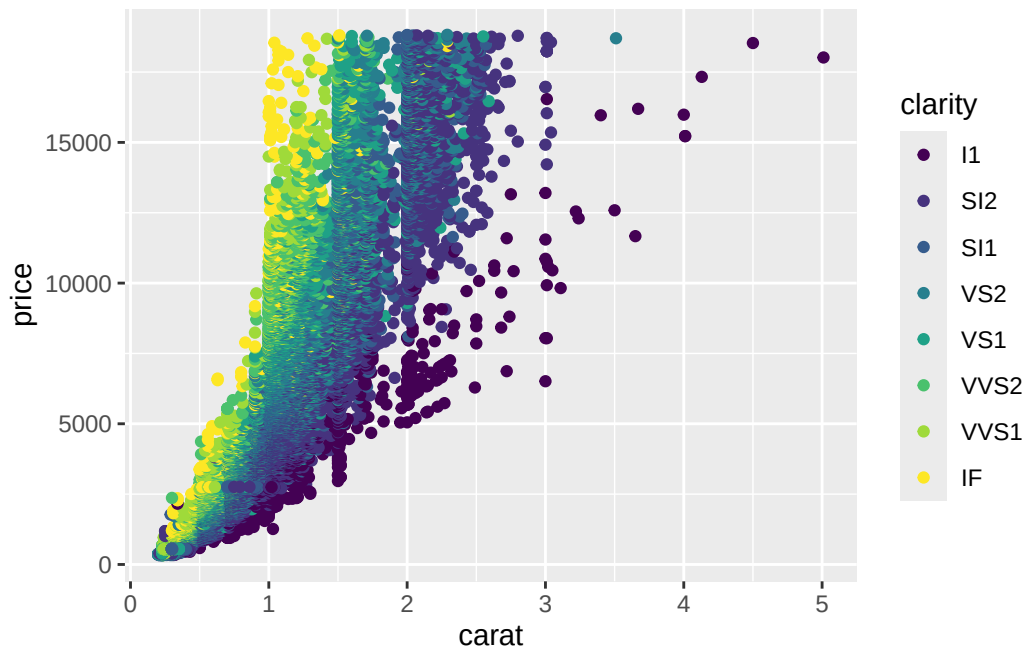


Ouf, si vous avez exécuté le dernier code, vous avez maintenant 53 940 points de données affichées dans votre graphique ! Car il y a tant d'observations dans le jeu de données `diamonds`.

Bon courage si vous avez un vieil ordinateur, cela peut prendre un certain temps avant que R ne produise le graphique...

Mais jetons un coup d'œil au graphique. Qu'est-ce qui saute aux yeux, pouvez-vous déjà identifier des tendances ?

```
# On peut maintenant encore optimiser notre graphique  
ggplot(data = diamonds, mapping = aes(x = carat, y = price, color = clarity)) +  
  geom_point()
```

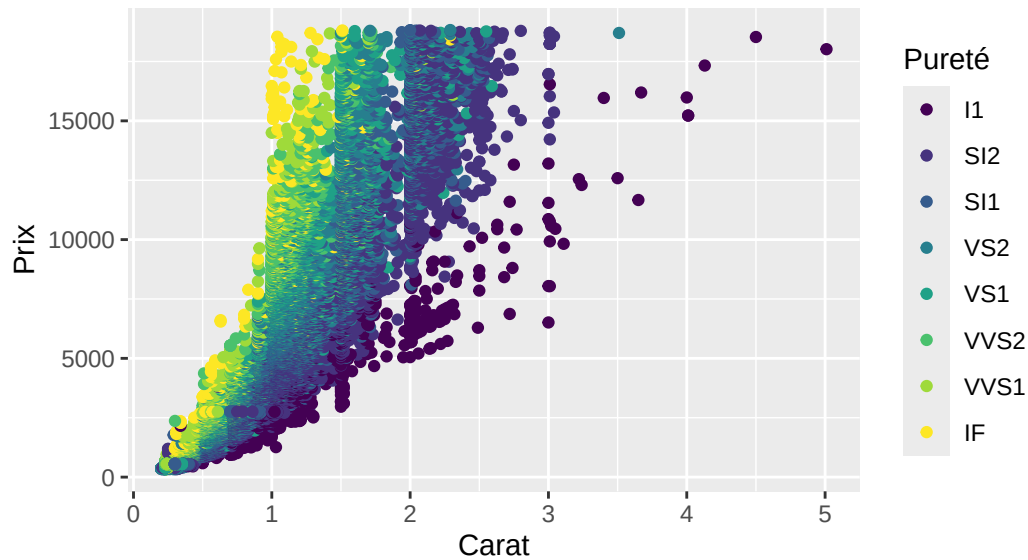


Vous avez remarqué ce qui a changé dans le code ?

```
# Ajoutons encore une couche optionnelle mais utile au
# graphique : labs() nous permet d'ajouter et de
# personnaliser des titres, sous-titres, titre d'axes,
# titres de légendes, etc. !
ggplot(data = diamonds, mapping = aes(x = carat, y = price, color = clarity)) +
  geom_point() + labs(title = "La relation entre prix et carat des diamants",
    subtitle = "n = 53 940 diamants", y = "Prix", x = "Carat",
    colour = "Pureté")
```

La relation entre prix et carat des diamants

n = 53 940 diamants



Maintenant il s'agit d'interpréter le graphique. Que nous dit-il sur la relation entre prix et carats des diamants ? Quel rôle joue la variable *pureté* (*clarity*) ?

À faire avant de continuer

- Exécutez toutes les sections de code de la session 4 sur votre ordinateur.
- Entraînez-vous à ouvrir un nouveau script R, à définir un répertoire de travail et à enregistrer le script dans un dossier sur votre ordinateur.
- Assurez-vous que le paquet R `tidyverse` est installé correctement sur votre ordinateur.
- Utilisez le code R de manière autonome pour explorer le jeu de données `diamonds`. Vous devez être en mesure de comprendre et d'expliquer comment il est structuré (nombre d'observations `n`, unité statistique, variables, origine des données, etc).

Exercices

Notez vos réponses, puis comparez-les avec l'annexe « Corrigés » à la fin du livre.

Question 4.1

Jugez chaque proposition :

- a) Le paquet `ggplot2` est inclus dans `tidyverse`.

- b) Le paquet `ggplot` est à utiliser pour faire des graphiques avec une variable, tandis que le paquet `ggplot2` est utile pour voir graphiquement la corrélation entre deux variables.
- c) Le responsable de `ggplot2` (qui maintient le paquet) est Thomas Lin Pedersen.
- d) `ggplot2` est très utile pour modifier des données.
- e) `ggplot2` construit des graphiques de manière incrémentale, en travaillant avec des couches.

Pour chaque proposition : vrai ou faux ?

Question 4.2

Jugez chaque proposition relative au jeu de données `iris`, intégré à R :

- a) Le jeu de données contient 150 observations de 5 variables.
- b) Le jeu de données contient des observations de trois espèces d'iris différentes.
- c) L'unité statistique du jeu de données est `Petal.Length` (longueur de pétale) des fleurs d'iris.
- d) Les mesures ont été publiées par Edgar Anderson (1935).

Pour chaque proposition : vrai ou faux ?

Question 4.3

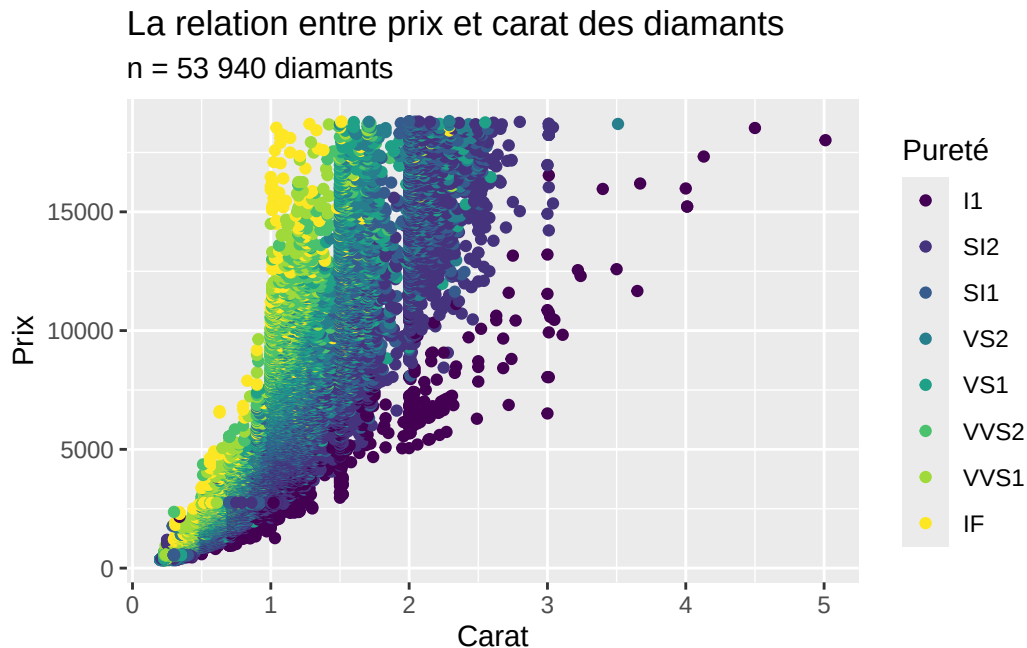
Jugez chaque proposition relative au jeu de données `mtcars`, intégré à R :

- a) `mtcars` est un `"data.frame"`.
- b) Le jeu de données `mtcars` contient 32 observations de 11 variables.
- c) Avec 33,9 milles par gallon (`mpg`), la Toyota Corolla est le modèle de voiture le plus efficace en termes de consommation de carburant dans le jeu de données.
- d) La Ford Pantera L a une boîte de vitesses manuelle.

Pour chaque proposition : vrai ou faux ?

Question 4.4

Jugez chaque proposition concernant le graphique suivant :



- a) Le graphique montre différents niveaux de qualité de la couleur des diamants.
- b) En moyenne, la relation entre le carat et le prix semble linéaire (le prix et le carat augmentent proportionnellement).
- c) En moyenne, la relation entre le carat et le prix semble exponentielle (le prix augmente plus vite que le carat).
- d) À carat égal, les diamants plus purs semblent plus chers dans ce graphique.
- e) Le diamant le plus lourd est aussi le plus cher.

Pour chaque proposition : vrai ou faux ?

Question 4.5

Jugez chaque proposition concernant les couches d'un graphique `ggplot2` (*Data*, *Aesthetics*, *Geometries*, *Facets*, *Statistics*, *Coordinates*, *Theme*) :

- a) Pour créer un graphique `ggplot2`, il faut toujours définir les paramètres de toutes les couches les unes après les autres.
- b) La couche *Geometries* définit le type de graphique (par ex. nuage de points, diagramme à barres, boîtes à moustaches, etc.).

c) En général, on construit un graphique ggplot2 par couches en commençant par la couche *Data*.

d) La couche *Aesthetics* définit le design du graphique (par exemple, elle permet de générer le graphique dans le design du Wall Street Journal ou du magazine The Economist).

e) Les couches *Facets*, *Statistics*, *Coordinates* et *Theme* ne doivent pas obligatoirement être spécifiées dans le code, car R utilise pour celles-ci des paramètres standard prédéfinis.

Pour chaque proposition : vrai ou faux ?

Question 4.6

Quel est le prix en USD du diamant le moins cher du jeu de données `diamonds` ? (Donnez seulement la valeur numérique.)

Réponse : _____

Question 4.7

Quelle est la longueur du pétale le plus court du jeu de données `iris` ? (Donnez seulement la valeur numérique.)

Réponse : _____

Question 4.8

Combien de chevaux-vapeur (*gross horsepower*, variable `hp`) a la Chrysler Imperial dans le jeu de données `mtcars` ?

Réponse : _____

Question 4.9

Le code R suivant charge le paquet `tidyverse` pour la session R en cours, à condition que ce paquet ait été préalablement installé sur votre ordinateur :

```
library(tidyverse)
```

Vrai ou faux ?

Question 4.10

`ggplot2` est un paquet R qui fait partie du `tidyverse`.

Vrai ou faux ?

Question 4.11

`ggthemes` est un paquet R qui fait partie du `tidyverse`.

Vrai ou faux ?

Question 4.12

Le **tidyverse**, contraction de *tidy* et de *universe*, est une collection de paquets R (*R packages*) open source.

Vrai ou faux ?

5 Visualiser des données avec ggplot2

La session 5 a pour but d'approfondir les techniques de visualisation de données avec le paquet `ggplot2`, notamment la création des types de graphique suivants :

1. Les nuages de points (angl. *scatter plots*)
2. Les diagrammes à barres (angl. *bar plots*)
3. Les histogrammes (angl. *histograms*)
4. Les boîtes à moustaches (angl. *box plots*)

```
# N'oubliez pas de charger les paquets tidyverse et  
# ggthemes ! Exécutez le code suivant pour le faire :  
library(tidyverse)  
library(ggthemes)
```

5.1 Les nuages de points (angl. *scatter plots*)

Commençons par le **nuage de points**, que nous avons déjà vu lors de la dernière session, lorsque nous avons visualisé la relation entre les carats et les prix dans le jeu de données `diamonds`.

i À noter

Un **nuage de points** (auss appelé diagramme de dispersion) est utilisé dans `ggplot2` pour représenter les valeurs de **deux variables numériques différentes** par des points, avec une variable sur l'axe des X et l'autre sur l'axe des Y.

Ce type de graphique est idéal pour l'analyse exploratoire de données, permettant d'observer les relations entre les variables, de détecter des tendances, des regroupements, des valeurs extrêmes (angl. *outliers*) ou de vérifier des hypothèses de corrélation.

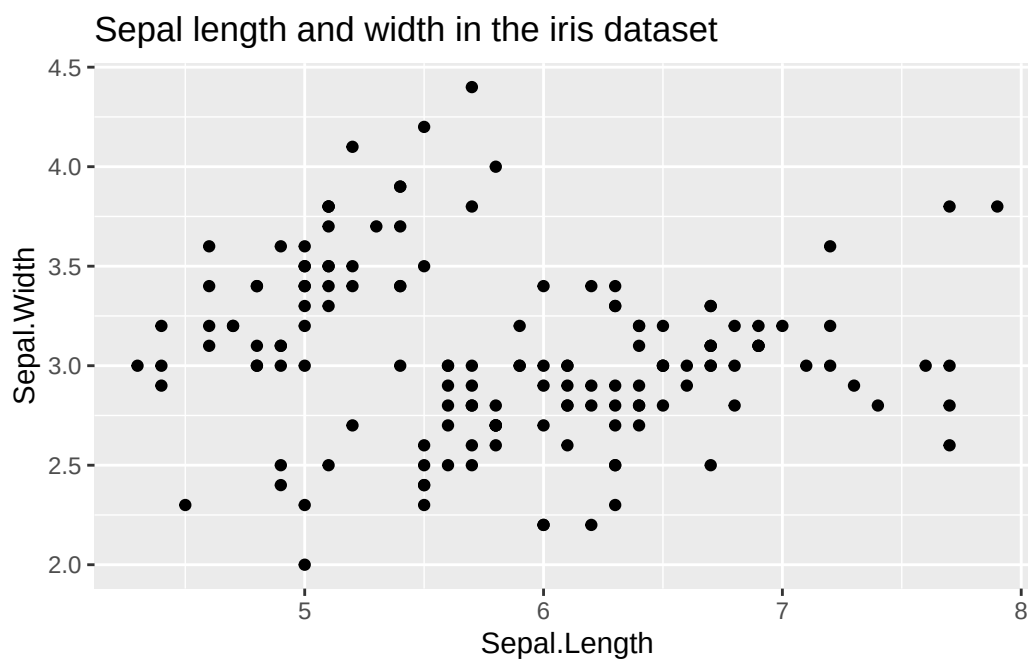
Dans la fonction `ggplot()`, un scatter plot est créé avec la couche géométrique `geom_point()`.

Créons un nuage de points avec le jeu de données `iris`, en comparant les longueurs et les largeurs des sépales des iris (les variables `Sepal.Length` et `Sepal.Width`).

Notez que, dans le code suivant, nous ne créons pas directement un graphique `ggplot` mais nous stockons le graphique comme un objet dans l'environnement, nommé `p1`, que nous pouvons ensuite afficher avec la fonction `print()`. Nous verrons dans un instant pourquoi cette pratique est utile.

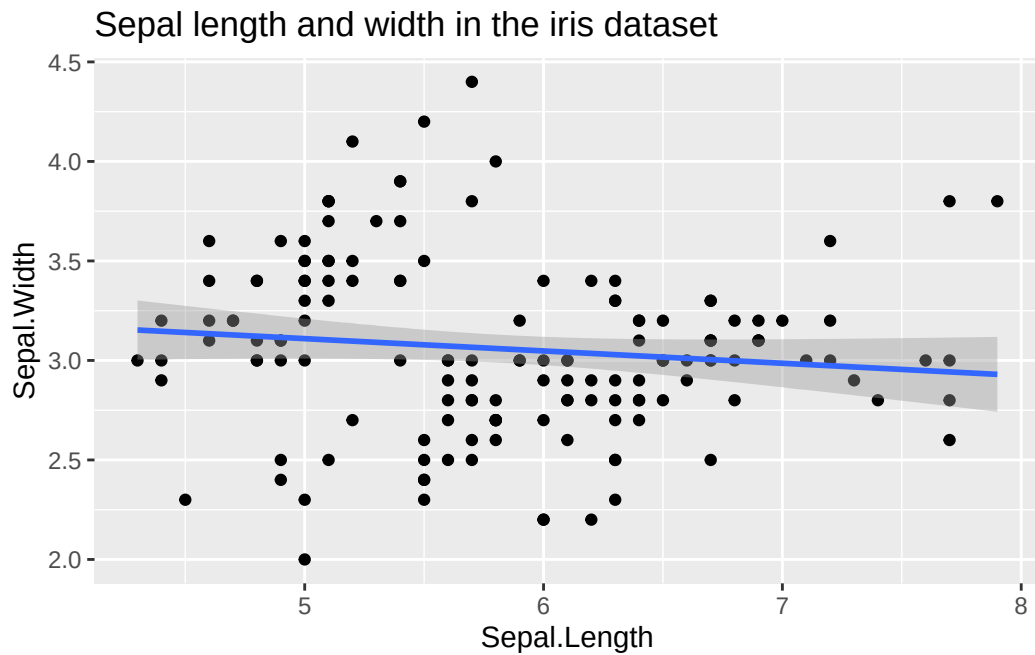
```
# nuage de points basique pour comparer la longueur et la
# largeur des sépales des iris
np1 <- ggplot(data = iris, mapping = aes(x = Sepal.Length, y = Sepal.Width)) +
  geom_point() + labs(title = "Sepal length and width in the iris dataset")

print(np1)
```



Nous pouvons ajouter des couches à l'objet `np1` sans l'écraser, p. ex. une ligne de régression linéaire avec `geom_smooth(method = lm)` :

```
np1 + geom_smooth(method = lm)
```

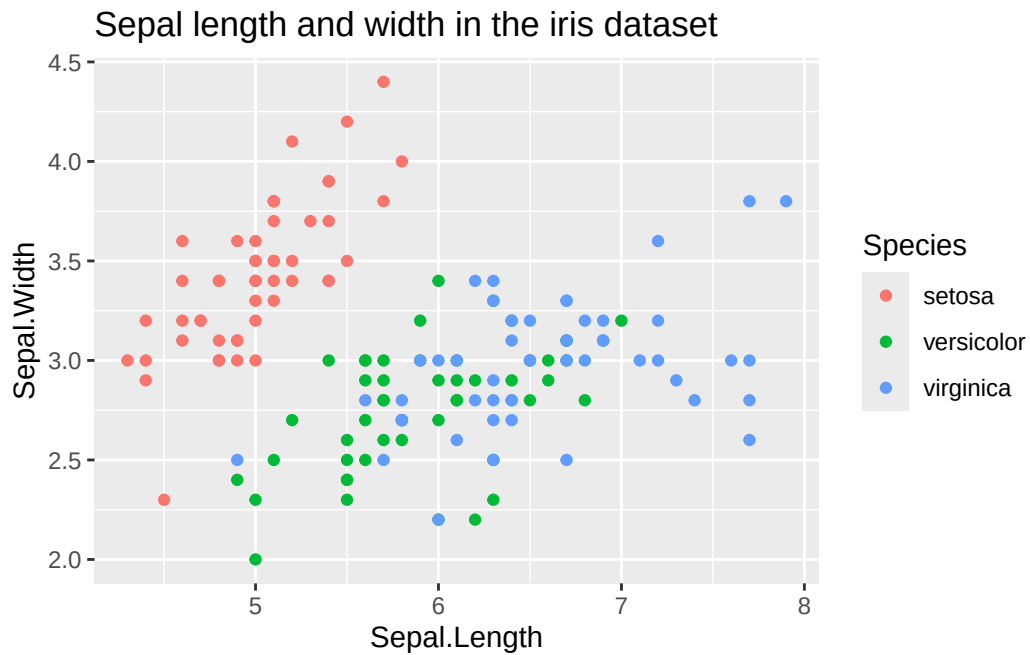


Que nous indique la ligne de régression ? En principe, elle indique une corrélation légèrement négative entre les variables `Sepal.Length` et `Sepal.Width` ! (la zone grise autour de la ligne bleue est l'intervalle de confiance à 95 % de la droite de régression).

Cela signifie que les iris dont les sépales sont plus longs ont tendance à avoir des sépales un peu moins larges ! Gardez cette constatation à l'esprit pour les étapes de codage suivantes.

```
# Ecrasons np1 en changeant un peu le code. Découvrez
# vous-même ce qui a changé dans le code !
np1 <- ggplot(iris, aes(x = Sepal.Length, y = Sepal.Width, color = Species)) +
  geom_point() + labs(title = "Sepal length and width in the iris dataset")

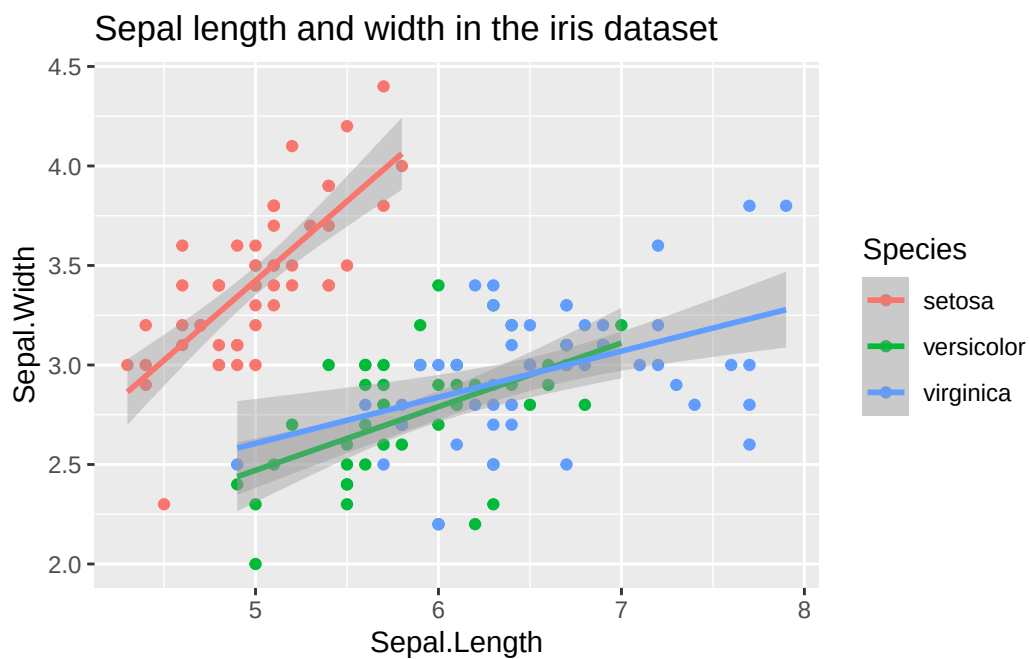
print(np1)
```



Vous pouvez voir dans le graphique que nous pouvons maintenant différencier les trois espèces d'iris. La dispersion des points semble moins aléatoire, ce qui permet de voir des groupes de points appartenant à l'une ou l'autre espèce (angl. *cluster*).

Maintenant, ajoutons à nouveau une couche `geom_smooth()` à l'objet `np1` !

```
np1 + geom_smooth(method = lm)
```



Les trois lignes de régression (une pour chaque type d'iris) montrent maintenant un tout autre résultat : en effet, la corrélation entre `Sepal.Length` et `Sepal.Width` est clairement positive.

Ce renversement s'appelle le **paradoxe de Simpson** : une tendance globale peut s'inverser lorsqu'on tient compte d'une variable de groupe (ici l'espèce).

Cette constatation est un exemple d'analyse exploratoire des données, rendue possible par les possibilités de visualisation de `ggplot2`.

5.2 Les diagrammes à barres

i À noter

Les **diagrammes à barres** dans R avec `ggplot2` sont utilisés pour représenter visuellement des **données catégorielles**, permettant de comparer des quantités entre différentes catégories.

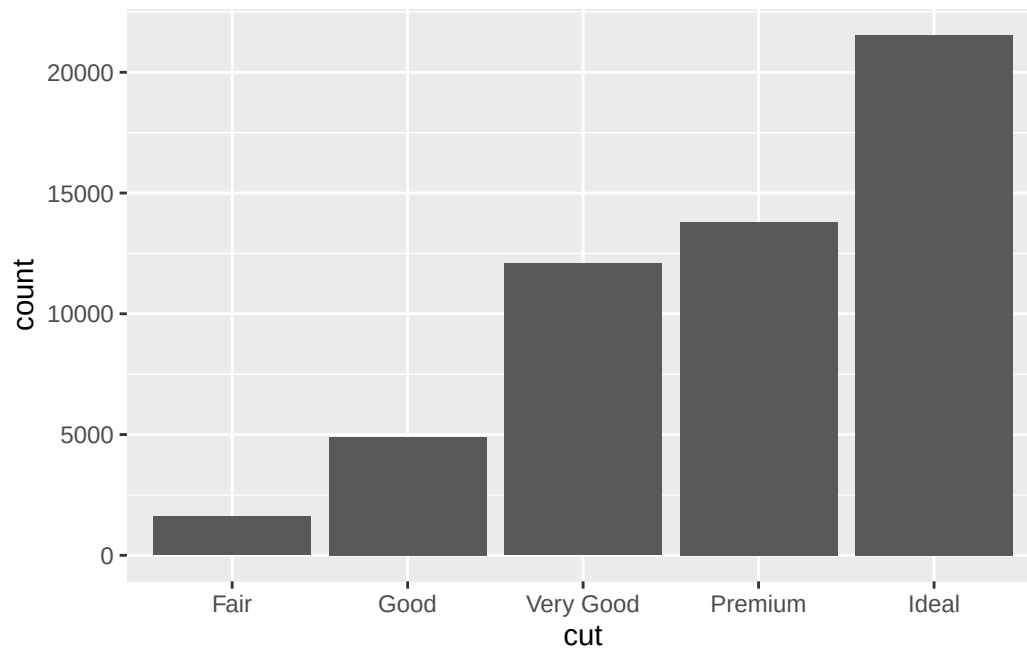
On les utilise donc souvent pour montrer la fréquence ou la proportion d'observations dans chaque catégorie, ce qui les rend utiles pour analyser des tendances ou des différences au sein de données catégorielles.

Dans la fonction `ggplot()`, un diagramme à barres est créé avec la couche géométrique `geom_bar()`.

Par exemple, nous pouvons utiliser le code simple suivant pour comparer la fréquence des différentes tailles (angl. `cut`) dans le jeu de données `diamonds`. Cette-fois ci, nous sauvegardons et nommons le graphique `db1`.

```
db1 <- ggplot(data = diamonds, mapping = aes(x = cut)) + geom_bar()

print(db1)
```

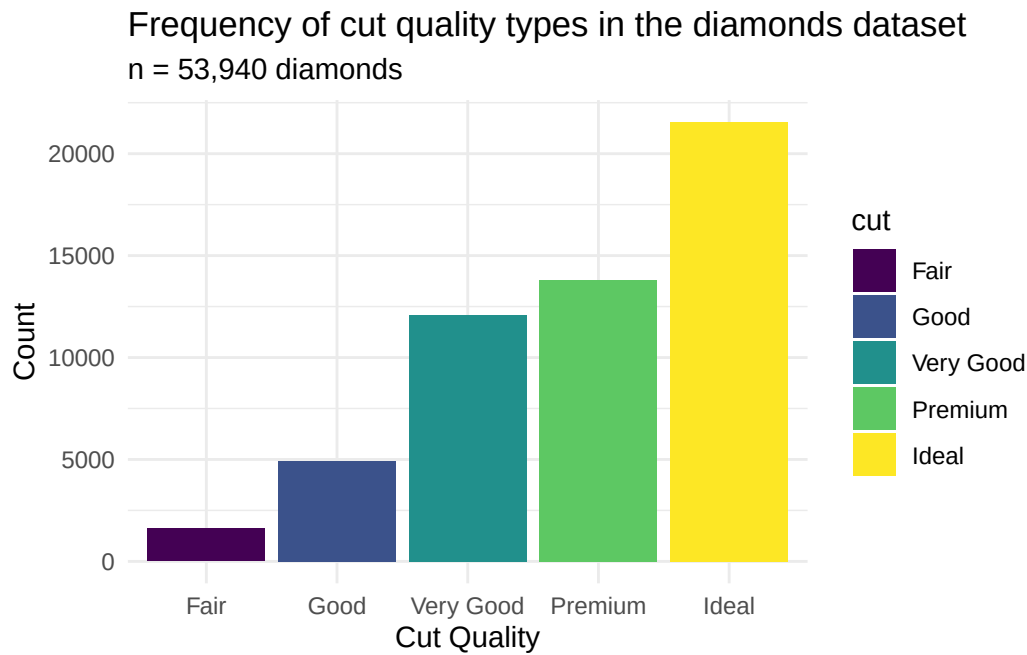


Améliorons le graphique avec des paramètres et des couches supplémentaires de la fonction `ggplot()` (Notez que nous allons simplement écraser l'objet `db1` créé précédemment avec une nouvelle version de `db1`!) :

```
library(ggthemes)

db1 <- ggplot(data = diamonds, mapping = aes(x = cut, fill = cut)) +
  geom_bar() + labs(title = "Frequency of cut quality types in the diamonds dataset",
    subtitle = "n = 53,940 diamonds", x = "Cut Quality", y = "Count") +
  theme_minimal()

print(db1)
```



Que nous dit le graphique ? Il est intéressant de constater que dans le jeu de données **diamonds**, le niveau de qualité le plus élevé de la taille, à savoir **ideal**, est également le plus fréquent.

🔥 Attention

Dans la fonction `ggplot()`, il y a une différence si vous voulez ajouter de la couleur aux points (qui ont mathématiquement zéro dimension) ou aux surfaces (qui ont deux dimensions), comme dans les diagrammes à barres.

Si vous voulez ajouter de la couleur à une surface, vous devez utiliser l'argument `fill = "xyz"`. L'argument `color = "xyz"` que nous avons utilisé pour ajouter de la couleur aux nuages de points ne fera qu'ajouter de la couleur à la ligne autour de la surface !

5.2.1 C'est maintenant à vous de jouer !

Reprenez le code que nous avons utilisé pour créer **db1** et modifiez-le pour créer un nouvel objet, **db2**, qui visualise la fréquence des types de qualité de couleur (**color**) dans le jeu de données **diamonds** sous la forme d'un diagramme à barres !

5.3 Les histogrammes

À noter

Un **histogramme** est un type de graphique utilisé pour représenter **la distribution d'une variable numérique** à travers des barres.

Un “histogramme sépare les valeurs possibles des données en classes ou groupes. Pour chaque groupe, on construit un rectangle dont la base correspond aux valeurs de ce groupe et la hauteur correspond au nombre d'observations dans le groupe.

L'histogramme a une apparence semblable au graphique à barres verticales, mais il n'y a pas d'écart entre les barres.

En règle générale, l'histogramme possède des barres d'une largeur égale” (Source : [Statistique Canada, 2021](#)).

L'histogramme est particulièrement utile pour :

- **Analyser la distribution** : Comprendre si les données sont normalement distribuées, asymétriques (angl. *skewness*), ou si elles ont des queues plus ou moins épaisses que la loi normale (*kurtosis*).
- **Détecter les valeurs extrêmes** : Identifier les données qui s'écartent du reste de l'ensemble des données de manière significative.
- **Comparer des distributions** : Observer comment différentes sous-populations se comparent les unes aux autres en termes de distribution de données.

Avec `ggplot2` en R, créer un histogramme se fait avec la couche géométrique `geom_histogram()`. Par défaut, `geom_histogram()` découpe les données en 30 intervalles (angl. *bins*) et affiche un message invitant à choisir une meilleure valeur. On peut fixer la largeur des intervalles avec l'argument `binwidth` (p. ex. `geom_histogram(binwidth = 30)`) ou leur nombre avec `bins`.

Avant de créer un histogramme avec le code R, introduisons d'abord une nouvelle fonction : `rnorm()`. Elle peut être utilisée pour créer des valeurs aléatoires qui suivent une **loi normale** (angl. *normal distribution*).

Les arguments que nous pouvons entrer dans la fonction sont :

- le nombre `n` de valeurs aléatoires que nous voulons créer
- la moyenne `mean` de la distribution des valeurs aléatoires
- l'écart-type `sd` de la distribution des valeurs aléatoires

Notez que par défaut, la fonction utilise `mean = 0` et `sd = 1`, ce qui correspond à une **loi normale centrée réduite** (angl. *standard normal distribution*).

Utilisez aussi `?rnorm()` pour accéder à plus d'infos.

Astuce

La fonction `set.seed()` dans R est utilisée pour définir la graine (seed en anglais) du générateur de nombres aléatoires. La graine est un nombre entier qui sert de point de

départ pour le générateur de nombres aléatoires.

En définissant une graine spécifique, par exemple avec `set.seed(123)`, vous garantisiez que la séquence de nombres aléatoires générée par la suite avec des fonctions telles que `rnorm()`, sera la même à chaque fois que vous exécuterez le code, ce qui est utile pour la reproductibilité des résultats.

Dans le code R suivant, nous utilisons `rnorm()` pour simuler les distributions des tailles des hommes et des femmes en Suisse.

D'après l'Enquête suisse sur la santé 2017 (Müller et al., 2022), les femmes mesurent en moyenne 164,6 cm (écart-type 6,5 cm) et les hommes 177,0 cm (écart-type 7,1 cm). Dans la suite, nous utilisons des valeurs proches : une moyenne de 164.7 cm pour les femmes et de 177.4 cm pour les hommes, avec des écarts-types un peu plus petits de 5.6 cm et de 6.1 cm.

```
# Simulation de tailles de femmes en Suisse Créer 500  
# valeurs aléatoires de taille (moyenne = 164.7 et  
# écart-type = 5.6) avec la fonction rnorm().  
set.seed(123)
```

```
taille_f <- rnorm(500, mean = 164.7, sd = 5.6)
```

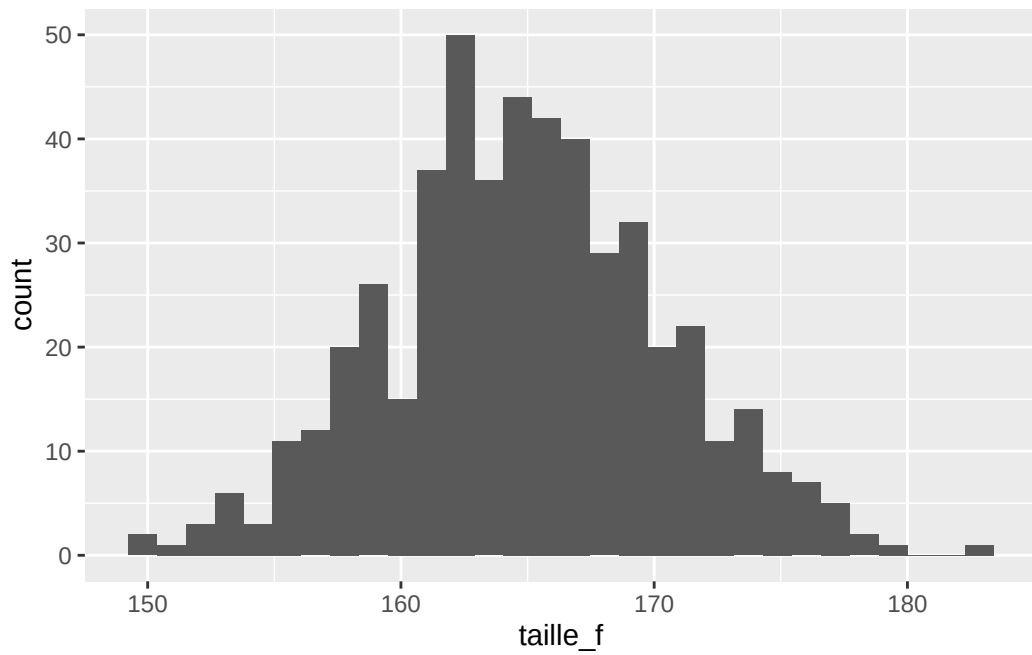
```
head(taille_f)
```

```
[1] 161.5613 163.4110 173.4288 165.0948 165.4240 174.3044
```

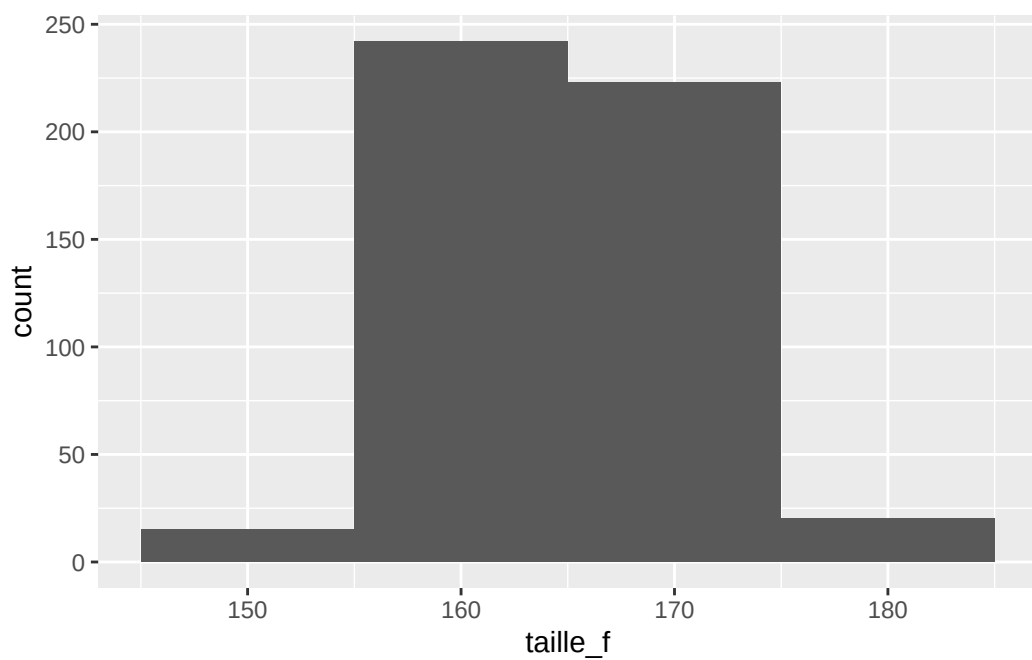
```
# stocker taille_f comme objet de type 'data.frame'  
taille_f <- as.data.frame(taille_f)
```

```
# visualiser taille_f avec un histogramme basique
```

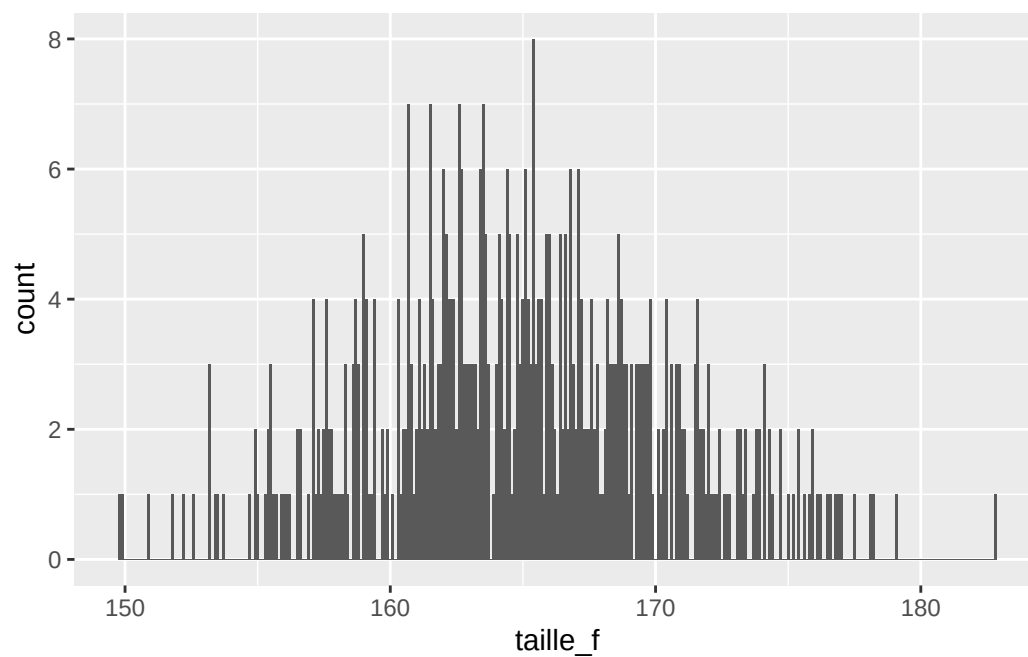
```
ggplot(data = taille_f, mapping = aes(x = taille_f)) + geom_histogram()
```



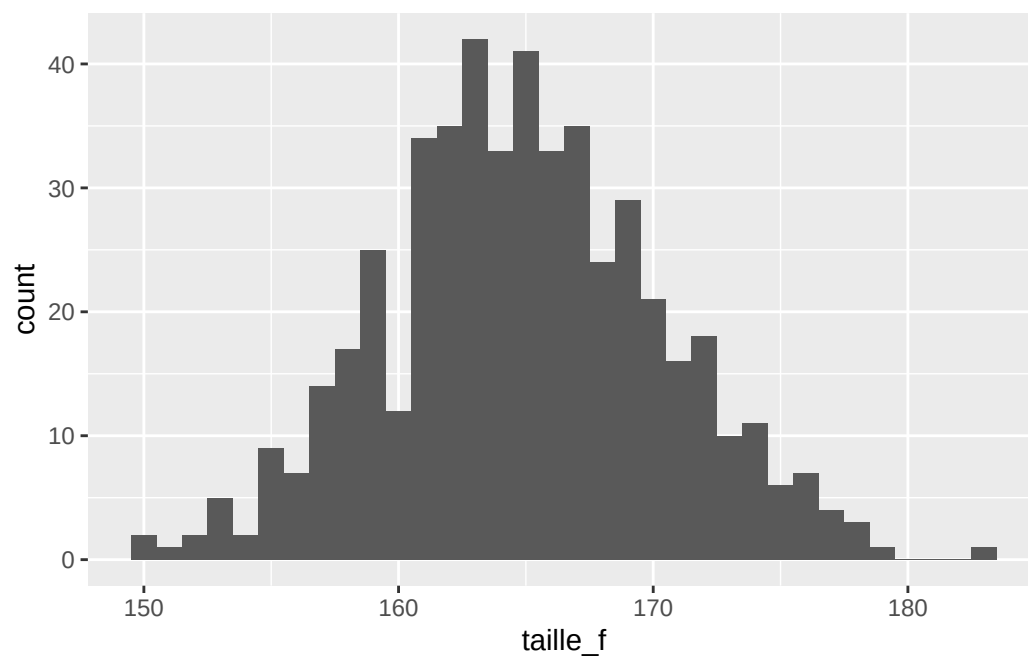
```
# jouer avec le paramètre 'binwidth' de l'histogramme qui
# définit l'intervalle de la base de ses barres
ggplot(data = taille_f, mapping = aes(x = taille_f)) + geom_histogram(binwidth = 10)
```



```
ggplot(data = taille_f, mapping = aes(x = taille_f)) + geom_histogram(binwidth = 0.1)
```

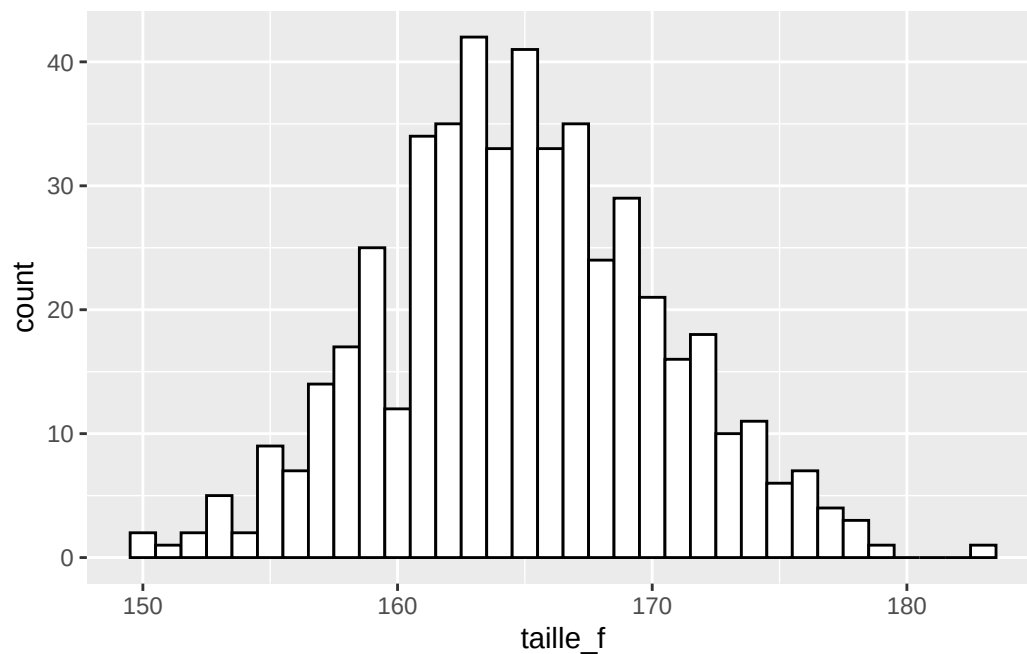


```
ggplot(data = taille_f, mapping = aes(x = taille_f)) + geom_histogram(binwidth = 1)
```



```
# stocker l'histogramme comme objet, nommé hist1, et
# modifier sa couleur
hist1 <- ggplot(data = taille_f, mapping = aes(x = taille_f)) +
  geom_histogram(binwidth = 1, color = "black", fill = "white")

print(hist1)
```

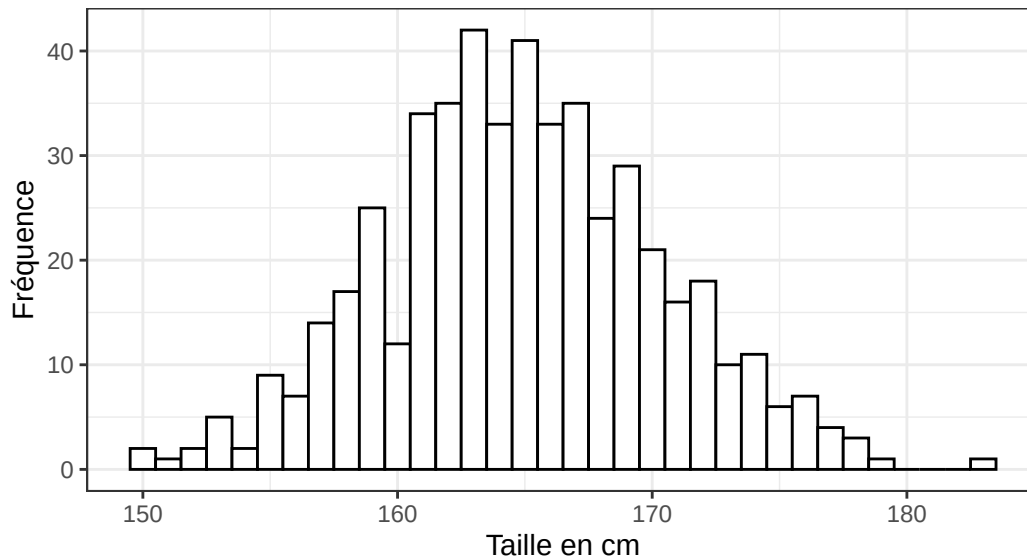


```
# Ajouter des couches supplémentaires pour améliorer la
# visualisation p. ex. des titres
hist1 <- hist1 + labs(title = "Histogramme des tailles de femmes en Suisse",
  subtitle = "Simulation basée sur 500 observations", x = "Taille en cm",
  y = "Fréquence") + theme_bw()

print(hist1)
```

Histogramme des tailles de femmes en Suisse

Simulation basée sur 500 observations



💡 Astuce

Un élément clé des visualisations de données est la **couleur**. R connaît 657 noms de couleurs : la fonction `colors()` les liste, et vous pouvez utiliser chacun de ces noms dans `ggplot2` (p. ex. `fill = "steelblue"`). Pour un aperçu visuel, voir *Colors in R* (Brown, 2021).

```
head(colors(), 20)
```

```
[1] "white"          "aliceblue"      "antiquewhite"   "antiquewhite1"
[5] "antiquewhite2" "antiquewhite3" "antiquewhite4"  "aquamarine"
[9] "aquamarine1"   "aquamarine2"   "aquamarine3"    "aquamarine4"
[13] "azure"         "azure1"        "azure2"         "azure3"
[17] "azure4"        "beige"         "bisque"         "bisque1"
```

5.3.1 C'est maintenant à vous de jouer !

Reprenez le code que nous avons utilisé pour créer `taille_f` et `hist1` et modifiez-le pour créer une nouvelle simulation de taille d'hommes, `taille_h`, et un nouvel histogramme qui montre sa distribution, `hist2`. Jouez avec les paramètres `binwidth` et la couleur de votre visualisation. Pour cette tâche, vous pouvez supposer que les hommes en Suisse ont une taille moyenne (`mean`) de 177,4 cm avec un écart-type (`sd`) de 6,1 cm.

5.4 Les boîtes à moustaches (angl. *boxplots*)

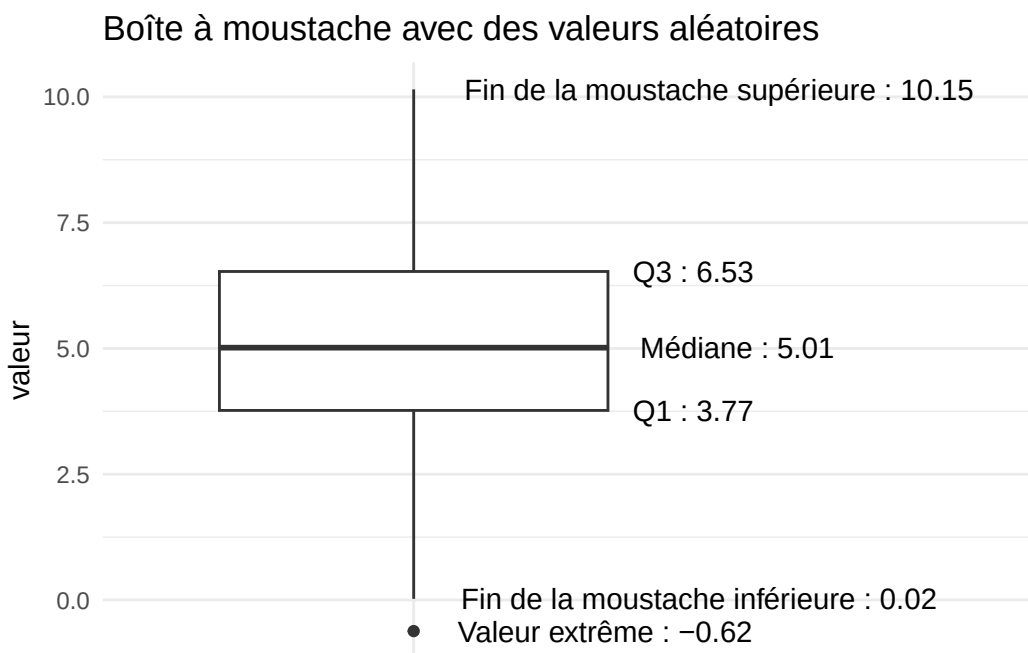
i À noter

Les **boîtes à moustaches**, ou **boxplots**, sont un type de graphique pour représenter la distribution d'un ensemble de données à travers cinq mesures clés :

1. l'extrémité de la moustache inférieure
2. le premier quartile (Q1 ; début de la boîte)
3. la médiane (Q2 ; ligne qui "coupe la boîte en deux")
4. le troisième quartile (Q3 ; fin de la boîte) et
5. l'extrémité de la moustache supérieure.

Les moustaches s'étendent jusqu'aux valeurs les plus extrêmes situées à moins de $1,5 \times$ l'écart interquartile de la boîte ; les valeurs au-delà sont dessinées comme des points (valeurs extrêmes potentielles). Les moustaches correspondent donc au minimum et au maximum seulement s'il n'y a pas de valeurs extrêmes.

Ces diagrammes permettent d'identifier rapidement la médiane, l'étendue interquartile (angl. *interquartile range* = *IQR*) et les valeurs extrêmes potentielles. Ils sont particulièrement utiles pour comparer les distributions entre plusieurs groupes ou variables.



Le dataset `mtcars` en R, qui contient des données sur les caractéristiques de différents modèles de voitures, peut être utilisé pour illustrer l'utilisation de boxplots.

Par exemple, pour visualiser la distribution de l'efficacité énergétique (`mpg`, miles par gallon) des voitures en fonction du nombre de cylindres (`cyl`), on peut utiliser le code suivant :

```
# Avant de créer le boxplot, vérifions d'abord la structure
# de mtcars :
str(mtcars)
```

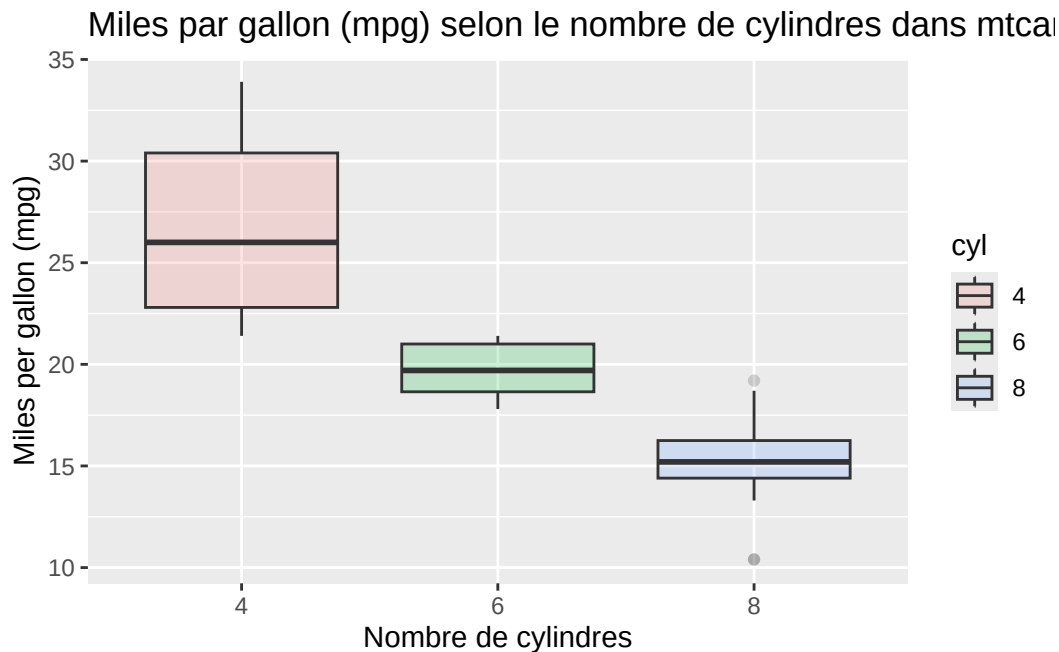
```
'data.frame':  32 obs. of  11 variables:
 $ mpg : num 21 21 22.8 21.4 18.7 18.1 14.3 24.4 22.8 19.2 ...
 $ cyl : num 6 6 4 6 8 6 8 4 4 6 ...
 $ disp: num 160 160 108 258 360 ...
 $ hp  : num 110 110 93 110 175 105 245 62 95 123 ...
 $ drat: num 3.9 3.9 3.85 3.08 3.15 2.76 3.21 3.69 3.92 3.92 ...
 $ wt  : num 2.62 2.88 2.32 3.21 3.44 ...
 $ qsec: num 16.5 17 18.6 19.4 17 ...
 $ vs  : num 0 0 1 1 0 1 0 1 1 1 ...
 $ am  : num 1 1 1 0 0 0 0 0 0 0 ...
 $ gear: num 4 4 4 3 3 3 3 4 4 4 ...
 $ carb: num 4 4 1 1 2 1 4 2 2 4 ...
```

Nous pouvons constater que la colonne `cyl` (cylindre) est stockée au format numérique.

C'est un problème, car nous voulons comparer l'efficacité énergétique (`mpg`) des voitures à 4, 6 ou 8 cylindres. Nous devons traiter chacune d'entre elles comme un sous-groupe distinct. Comment faire ?

```
# Transformons donc cyl en facteur
mtcars$cyl <- as.factor(mtcars$cyl)

# Et maintenant nous pouvons construire notre boxplot
box1 <- ggplot(data = mtcars, aes(x = cyl, y = mpg, fill = cyl)) +
  geom_boxplot(alpha = 0.2) + labs(title = "Miles par gallon (mpg) selon le nombre de
    ↪ cylindres dans mtcars",
    x = "Nombre de cylindres", y = "Miles per gallon (mpg)")
print(box1)
```



Ce code génère un boxplot, `box1`, pour chaque groupe de cylindres dans le dataset `mtcars`, permettant de comparer facilement l'efficacité énergétique (`mpg`) entre les voitures avec des nombres de cylindres différents.

i À noter

Le paramètre `alpha = 0.2` à l'intérieur de `geom_boxplot()` rend les boxplots transparents. En général, `alpha` peut être compris entre 0 (totalement transparent) et 1 (non transparent). C'est un paramètre supplémentaire qui permet de personnaliser votre graphique `ggplot2`.

On voit dans le graphique des boxplots que dans `mtcars`, les voitures avec des moteurs ayant plus de cylindres ont aussi tendance à être moins efficaces en termes de consommation de carburant. Il existe toutefois quelques intersections entre les groupes. C'est-à-dire qu'il y a des voitures à 8 cylindres qui sont relativement efficaces en termes de consommation de carburant, surpassant quelques voitures à 6 cylindres, par exemple.

À faire avant de continuer

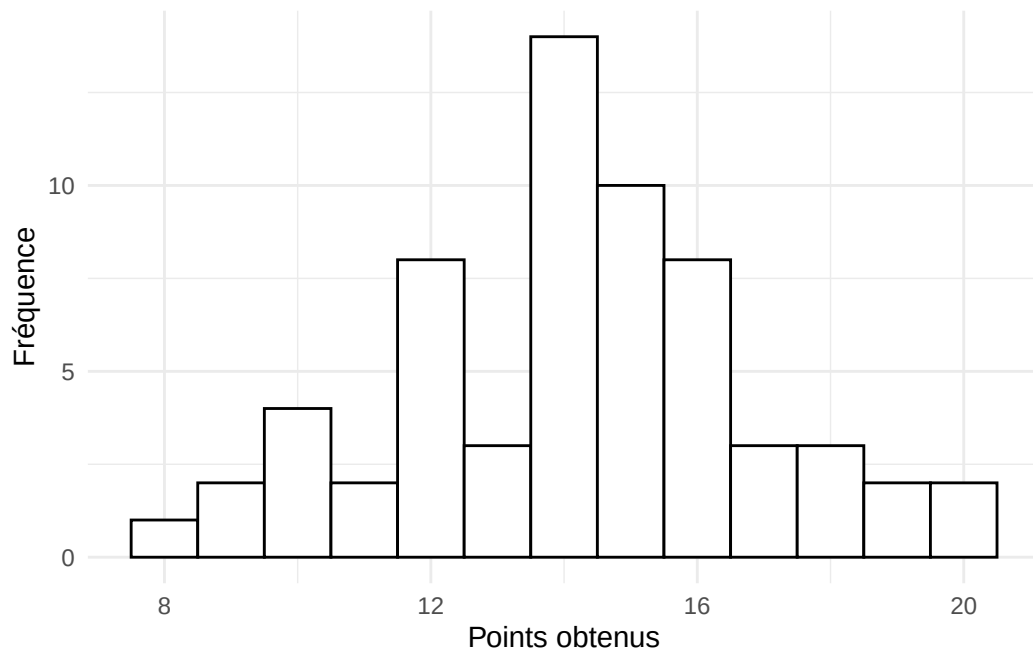
- Refaites les graphiques de ce chapitre et modifiez leurs paramètres (couleurs, `binwidth`, titres) pour voir ce qui change.
- Révissez les chapitres 4 et 5, puis testez-vous avec le chapitre « Révision 2 ».

Exercices

Notez vos réponses, puis comparez-les avec l'annexe « Corrigés » à la fin du livre.

Question 5.1

Jugez chaque proposition concernant l'histogramme suivant des points obtenus au Problem Set 1 :



- a) Les étudiants ont obtenu le plus fréquemment 13 points dans le Problem Set 1.
- b) Le graphique montre qu'en médiane, 16 points ont été obtenus dans le Problem Set 1.
- c) Le graphique montre qu'en moyenne, 16 points ont été obtenus dans le Problem Set 1.
- d) L'histogramme montre que la distribution des points dans le Problem Set 1 suit une loi normale centrée réduite.

Pour chaque proposition : vrai ou faux ?

Question 5.2

Comment peut-on ajuster la **largeur des intervalles** dans un histogramme créé avec `ggplot2` ?

- a) En utilisant l'argument `bins`.
- b) En ajustant l'argument `width`.

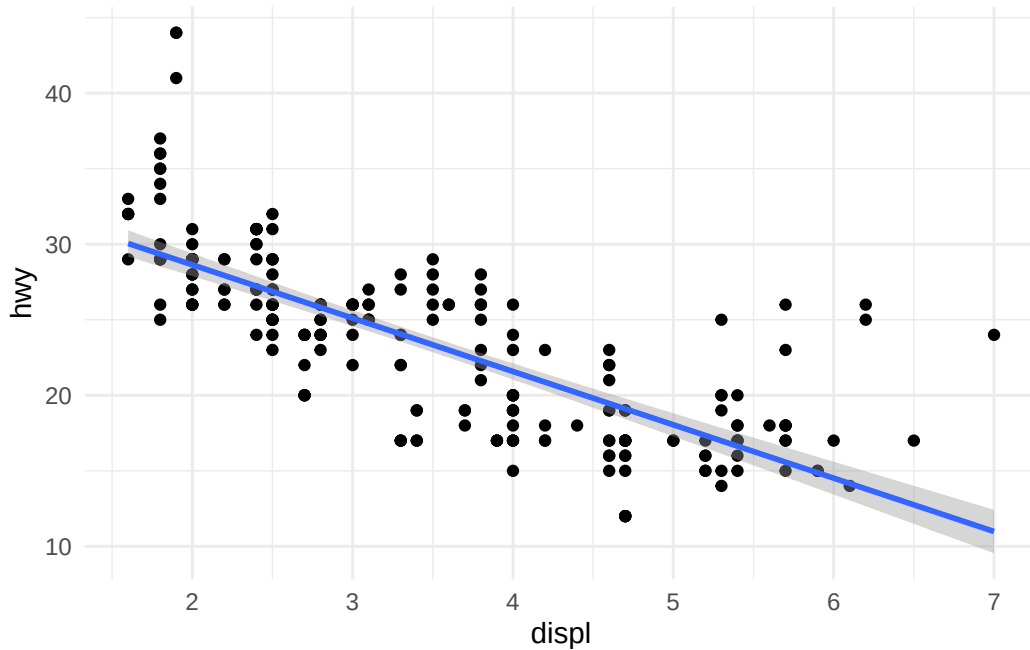
c) En modifiant l'argument `binwidth`.

d) En changeant l'argument `interval`.

Pour chaque proposition : vrai ou faux ?

Question 5.3

Jugez chaque proposition concernant le graphique suivant, créé avec le jeu de données `mpg` (inclus dans le paquet `ggplot2`) :



a) La ligne de régression indique que la corrélation entre la taille du moteur et l'efficacité du carburant est négative.

b) Les données présentées proviennent du jeu de données `mtcars`.

c) Le jeu de données `mpg` contient des données relatives à l'économie de carburant de 1999 à 2008 pour 38 modèles de voitures populaires.

d) L'axe des x de ce graphique indique les miles par gallon (`mpg`).

e) Le graphique montre que les moteurs de plus grande taille ont tendance à être associés à un plus grand nombre de miles par gallon.

Pour chaque proposition : vrai ou faux ?

Question 5.4

Exécutez le code R suivant et jugez chaque proposition :

```
library(ggplot2)
data(mtcars)
g <- ggplot(mtcars, aes(mpg, wt)) + geom_point()
print(g)
```

- a) `g` est un objet stocké dans le volet *Environment* de RStudio.
- b) Le graphique `g` affiche une relation entre le poids des voitures (`wt`) et leur efficacité énergétique (`mpg`, miles par gallon).
- c) Le code produit un histogramme.
- d) Chaque point dans le graphique représente une voiture du jeu de données `mtcars`.
- e) Le graphique indique que les deux variables affichées sont négativement corrélées.

Pour chaque proposition : vrai ou faux ?

Question 5.5

Exécutez le code R suivant et jugez chaque proposition :

```
library(ggplot2)
data(mtcars)
# Transformations :
mtcars$cyl <- as.factor(mtcars$cyl)
mtcars$gear <- as.factor(mtcars$gear)
# Diagramme à barres :
ggplot(mtcars, aes(x = cyl, fill = gear)) +
  geom_bar() +
  labs(title = "Distribution of cylinders and gears in mtcars")
```

- a) Ce code produit un diagramme à barres représentant le nombre de voitures par nombre de cylindres.
- b) La couleur des barres est déterminée par le nombre de vitesses (`gear`).
- c) Les cylindres et les vitesses sont transformés en variables de type facteur.
- d) Le graphique montre qu'il n'y a pas de voitures avec 8 cylindres et 4 vitesses dans `mtcars`.
- e) Les voitures à 6 cylindres sont moins fréquentes que celles à 4 ou 8 cylindres.

Pour chaque proposition : vrai ou faux ?

Question 5.6

Exécutez le code R suivant et jugez chaque proposition :

```
rnorm(1000)
```

- a) Ce code génère 1000 valeurs « aléatoires » qui se rapprochent d'une loi normale centrée réduite (*standard normal distribution*).
- b) Ce code génère un message d'erreur, car il y a des arguments manquants dans la fonction `rnorm()`.
- c) Ce code génère 1000 valeurs « aléatoires » qui se rapprochent d'une loi normale avec une moyenne = 1 et un écart-type = 0.
- d) Ce code génère des valeurs numériques positives et négatives.
- e) Toutes les lois normales ont le même écart-type.

Pour chaque proposition : vrai ou faux ?

Question 5.7

Quels types de visualisation sont bien adaptés pour afficher la relation entre deux variables numériques ?

- a) Histogramme
- b) Diagramme à barres
- c) Boîte à moustaches
- d) Nuage de points

Pour chaque proposition : vrai ou faux ?

Question 5.8

Une visualisation des données peut servir à l'analyse exploratoire des données.

Vrai ou faux ?

Question 5.9

Un boxplot peut être utilisé pour visualiser la distribution, la moyenne, les déciles et les valeurs extrêmes d'un ensemble de données.

Vrai ou faux ?

Question 5.10

Le code suivant crée un histogramme qui indique une loi normale :

```
library(ggplot2)
ggplot(data = diamonds, aes(x = price)) +
  geom_histogram(binwidth = 600, fill = "steelblue", color = "black") +
  labs(title = "Histogramme des prix dans le jeu de données diamonds",
       subtitle = "n = 53 940 observations",
       x = "Prix en USD", y = "Fréquence") +
  theme_minimal()
```

Vrai ou faux ?

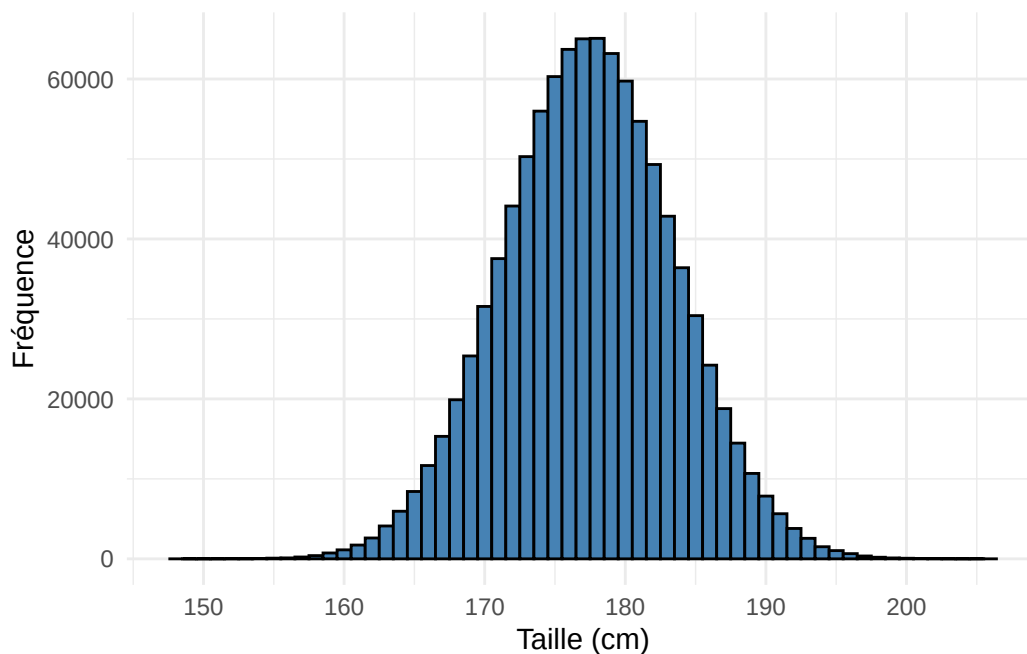
Question 5.11

La couche `geom_smooth(method = lm)` peut être utilisée pour ajouter une ligne de régression à un nuage de points dans `ggplot2`.

Vrai ou faux ?

Question 5.12

Le graphique suivant montre une simulation d'un million de tailles aléatoires d'hommes en Suisse, réalisée avec la fonction `rnorm()`. Il illustre le fait que plus le nombre de simulations est élevé, plus une loi normale est approchée avec précision.



Vrai ou faux ?

6 Révision 2 : visualisation

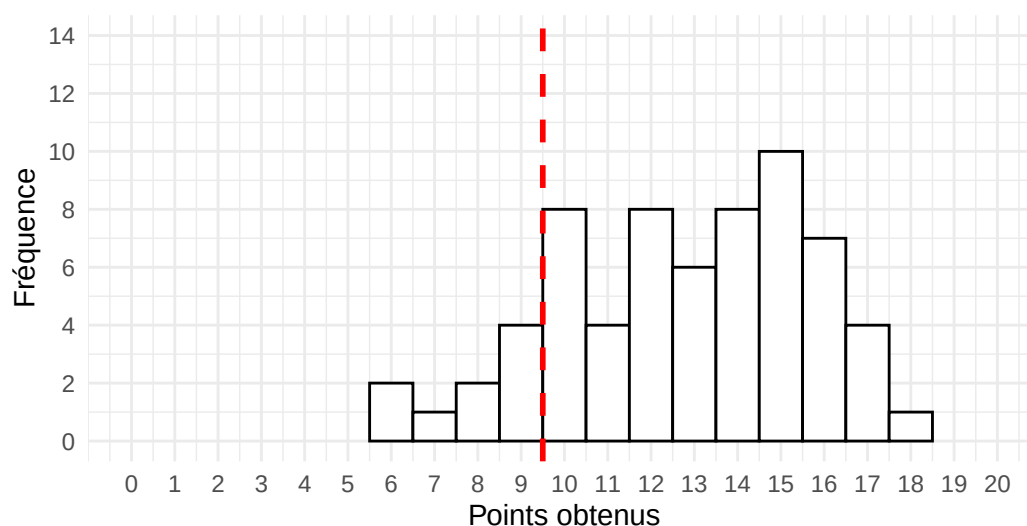
Dans le cours, le **Problem Set 2** était un test de 30 minutes sur Moodle portant sur les chapitres 4 et 5. Les questions ci-dessous en reprennent le contenu, avec la correction et une explication pour chaque réponse.

i Résultats de l'édition 2025 du cours

65 étudiant·es ont passé ce test (20 points au maximum, 10 pour réussir). La moyenne était de 12,8 points et 86 % des étudiant·es l'ont réussi.

Histogramme des points obtenus dans le PS 2

n = 65 résultats de test



Source : Problem Set 2, édition 2025

Exercices de révision

Ces questions reprennent le **Problem Set 2**. Notez vos réponses, puis comparez-les avec l'annexe « Corrigés » à la fin du livre.

Question 6.1

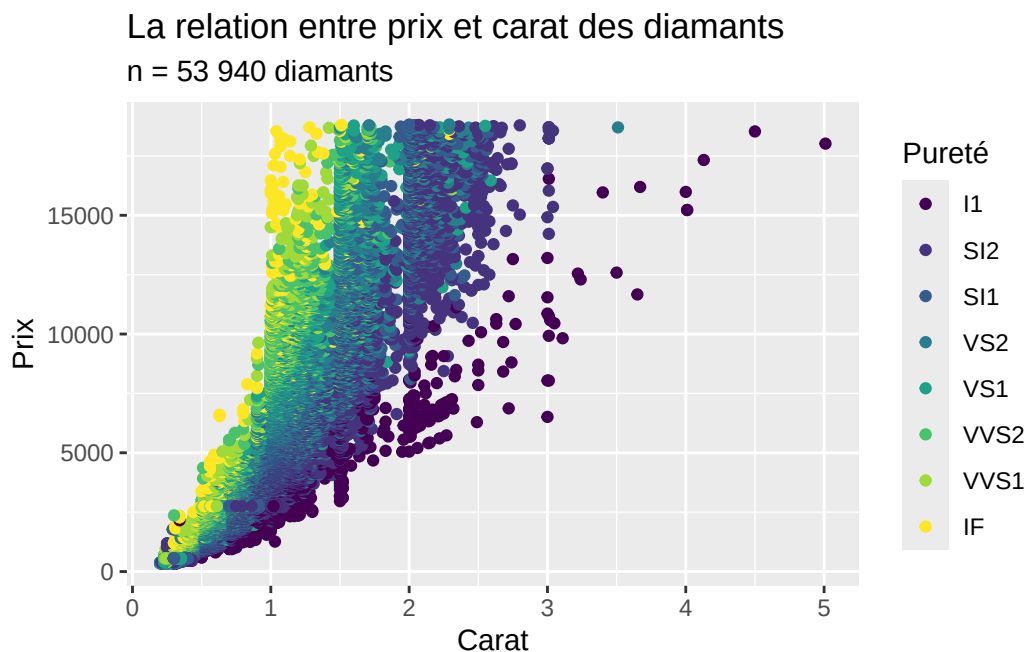
Jugez chaque proposition :

- a) Le paquet `ggplot2` est inclus dans `tidyverse`.
- b) Le paquet `ggplot` est à utiliser pour faire des graphiques avec une variable, tandis que le paquet `ggplot2` est utile pour voir graphiquement la corrélation entre deux variables.
- c) Le responsable de `ggplot2` (qui maintient le paquet) est Thomas Lin Pedersen.
- d) `ggplot2` est très utile pour modifier des données.
- e) `ggplot2` construit des graphiques de manière incrémentale, en travaillant avec des couches.

Pour chaque proposition : vrai ou faux ?

Question 6.2

Jugez chaque proposition concernant le graphique suivant :



- a) Le graphique montre différents niveaux de qualité de la couleur des diamants.
- b) En moyenne, la relation entre le carat et le prix semble linéaire (le prix et le carat augmentent proportionnellement).
- c) En moyenne, la relation entre le carat et le prix semble exponentielle (le prix augmente plus vite que le carat).
- d) À carat égal, les diamants plus purs semblent plus chers dans ce graphique.
- e) Le diamant le plus lourd est aussi le plus cher.

Pour chaque proposition : vrai ou faux ?

Question 6.3

Jugez chaque proposition concernant les couches d'un graphique `ggplot2` (*Data*, *Aesthetics*, *Geometries*, *Facets*, *Statistics*, *Coordinates*, *Theme*) :

- a) Pour créer un graphique `ggplot2`, il faut toujours définir les paramètres de toutes les couches les unes après les autres.
- b) La couche *Geometries* définit le type de graphique (par ex. nuage de points, diagramme à barres, boîtes à moustaches, etc.).
- c) En général, on construit un graphique `ggplot2` par couches en commençant par la couche *Data*.
- d) La couche *Aesthetics* définit le design du graphique (par exemple, elle permet de générer le graphique dans le design du Wall Street Journal ou du magazine The Economist).
- e) Les couches *Facets*, *Statistics*, *Coordinates* et *Theme* ne doivent pas obligatoirement être spécifiées dans le code, car R utilise pour celles-ci des paramètres standard prédéfinis.

Pour chaque proposition : vrai ou faux ?

Question 6.4

Quel est le prix en USD du diamant le moins cher du jeu de données `diamonds` ? (Donnez seulement la valeur numérique.)

Réponse : _____

Question 6.5

Quelle est la longueur du pétale le plus court du jeu de données `iris` ? (Donnez seulement la valeur numérique.)

Réponse : _____

Question 6.6

Combien de chevaux-vapeur (*gross horsepower*, variable `hp`) a la Chrysler Imperial dans le jeu de données `mtcars` ?

Réponse : _____

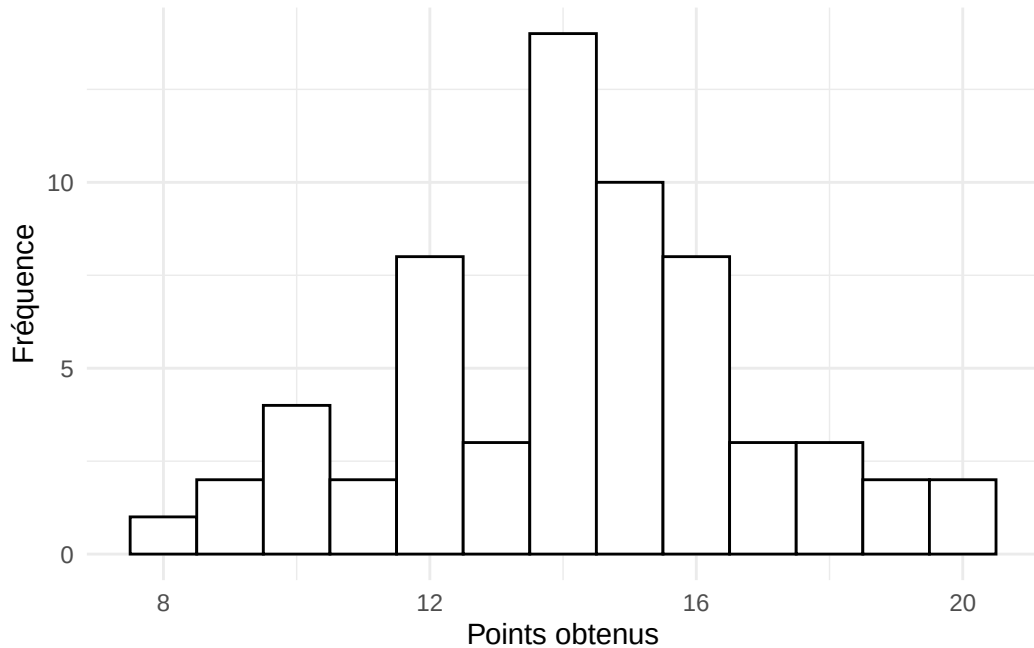
Question 6.7

`ggplot2` est un paquet R qui fait partie du `tidyverse`.

Vrai ou faux ?

Question 6.8

Jugez chaque proposition concernant l'histogramme suivant des points obtenus au Problem Set 1 :



- a) Les étudiant·es ont obtenu le plus fréquemment 13 points dans le Problem Set 1.
- b) Le graphique montre qu'en médiane, 16 points ont été obtenus dans le Problem Set 1.
- c) Le graphique montre qu'en moyenne, 16 points ont été obtenus dans le Problem Set 1.
- d) L'histogramme montre que la distribution des points dans le Problem Set 1 suit une loi normale centrée réduite.

Pour chaque proposition : vrai ou faux ?

Question 6.9

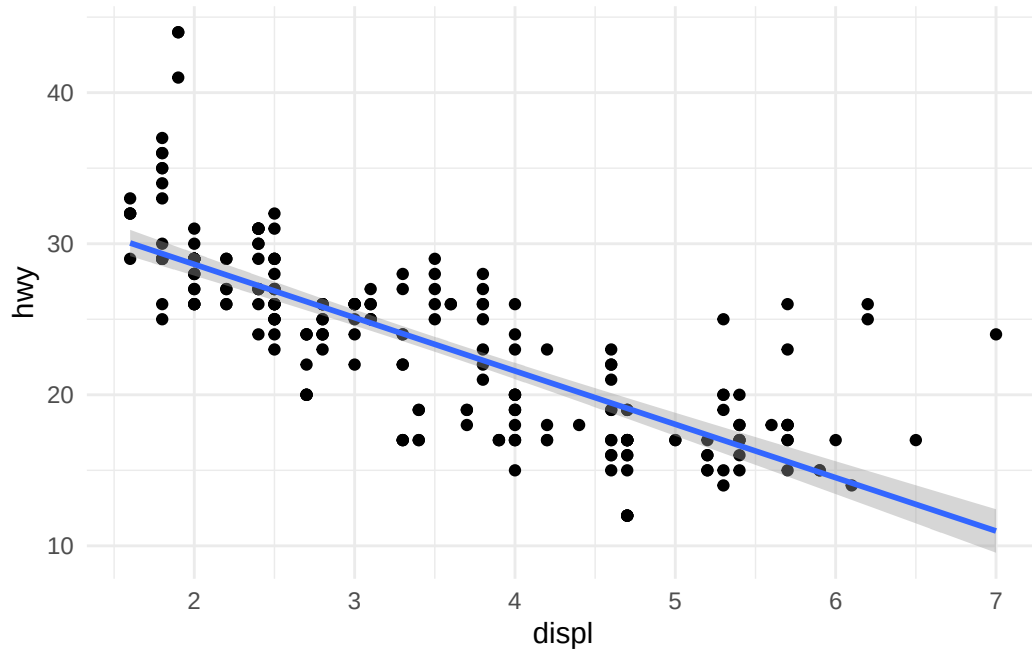
Comment peut-on ajuster la **largeur des intervalles** dans un histogramme créé avec `ggplot2` ?

- a) En utilisant l'argument `bins`.
- b) En ajustant l'argument `width`.
- c) En modifiant l'argument `binwidth`.
- d) En changeant l'argument `interval`.

Pour chaque proposition : vrai ou faux ?

Question 6.10

Jugez chaque proposition concernant le graphique suivant, créé avec le jeu de données `mpg` (inclus dans le paquet `ggplot2`) :



- a) La ligne de régression indique que la corrélation entre la taille du moteur et l'efficacité du carburant est négative.
- b) Les données présentées proviennent du jeu de données `mtcars`.
- c) Le jeu de données `mpg` contient des données relatives à l'économie de carburant de 1999 à 2008 pour 38 modèles de voitures populaires.
- d) L'axe des x de ce graphique indique les miles par gallon (`mpg`).
- e) Le graphique montre que les moteurs de plus grande taille ont tendance à être associés à un plus grand nombre de miles par gallon.

Pour chaque proposition : vrai ou faux ?

Question 6.11

Exécutez le code R suivant et jugez chaque proposition :

```
library(ggplot2)
data(mtcars)
g <- ggplot(mtcars, aes(mpg, wt)) + geom_point()
print(g)
```

- a) `g` est un objet stocké dans le volet *Environment* de RStudio.
 - b) Le graphique `g` affiche une relation entre le poids des voitures (`wt`) et leur efficacité énergétique (`mpg`, miles par gallon).
 - c) Le code produit un histogramme.
 - d) Chaque point dans le graphique représente une voiture du jeu de données `mtcars`.
 - e) Le graphique indique que les deux variables affichées sont négativement corrélées.
- Pour chaque proposition : vrai ou faux ?*

Question 6.12

Exécutez le code R suivant et jugez chaque proposition :

```
library(ggplot2)
data(mtcars)
# Transformations :
mtcars$cyl <- as.factor(mtcars$cyl)
mtcars$gear <- as.factor(mtcars$gear)
# Diagramme à barres :
ggplot(mtcars, aes(x = cyl, fill = gear)) +
  geom_bar() +
  labs(title = "Distribution of cylinders and gears in mtcars")
```

- a) Ce code produit un diagramme à barres représentant le nombre de voitures par nombre de cylindres.
 - b) La couleur des barres est déterminée par le nombre de vitesses (`gear`).
 - c) Les cylindres et les vitesses sont transformés en variables de type facteur.
 - d) Le graphique montre qu'il n'y a pas de voitures avec 8 cylindres et 4 vitesses dans `mtcars`.
 - e) Les voitures à 6 cylindres sont moins fréquentes que celles à 4 ou 8 cylindres.
- Pour chaque proposition : vrai ou faux ?*

Question 6.13

Exécutez le code R suivant et jugez chaque proposition :

```
rnorm(1000)
```

- a) Ce code génère 1000 valeurs « aléatoires » qui se rapprochent d'une loi normale centrée réduite (*standard normal distribution*).
- b) Ce code génère un message d'erreur, car il y a des arguments manquants dans la fonction `rnorm()`.
- c) Ce code génère 1000 valeurs « aléatoires » qui se rapprochent d'une loi normale avec une moyenne = 1 et un écart-type = 0.
- d) Ce code génère des valeurs numériques positives et négatives.
- e) Toutes les lois normales ont le même écart-type.

Pour chaque proposition : vrai ou faux ?

Question 6.14

Une visualisation des données peut servir à l'analyse exploratoire des données.

Vrai ou faux ?

Question 6.15

Un boxplot peut être utilisé pour visualiser la distribution, la moyenne, les déciles et les valeurs extrêmes d'un ensemble de données.

Vrai ou faux ?

Question 6.16

Le code suivant crée un histogramme qui indique une loi normale :

```
library(ggplot2)
ggplot(data = diamonds, aes(x = price)) +
  geom_histogram(binwidth = 600, fill = "steelblue", color = "black") +
  labs(title = "Histogramme des prix dans le jeu de données diamonds",
        subtitle = "n = 53 940 observations",
        x = "Prix en USD", y = "Fréquence") +
  theme_minimal()
```

Vrai ou faux ?

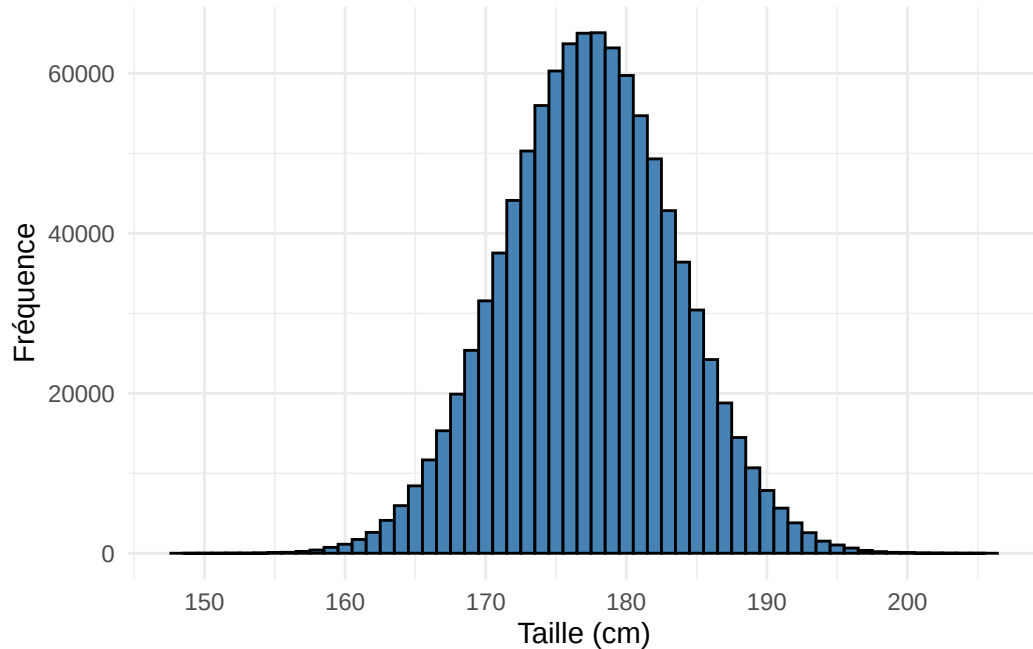
Question 6.17

La couche `geom_smooth(method = lm)` peut être utilisée pour ajouter une ligne de régression à un nuage de points dans `ggplot2`.

Vrai ou faux ?

Question 6.18

Le graphique suivant montre une simulation d'un million de tailles aléatoires d'hommes en Suisse, réalisée avec la fonction `rnorm()`. Il illustre le fait que plus le nombre de simulations est élevé, plus une loi normale est approchée avec précision.



Vrai ou faux ?

Question 6.19

Exécutez le code R suivant et jugez chaque proposition :

```
library(tidyverse)
data(diamonds)

diamonds %>%
  group_by(clarity) %>%
  summarize(n = n(), prix_moyen = mean(price))
```

- a) La sortie du code indique les prix moyens en USD des diamants pour chaque type de pureté dans le jeu de données `diamonds`.
- b) En moyenne, les diamants du type de pureté le plus élevé coûtent 1790 USD.
- c) Il y a le même nombre de diamants pour chaque type de pureté dans le jeu de données `diamonds`.

d) L'argument `n = n()` dans la fonction `summarize()` nous permet de compter le nombre d'observations dans chaque catégorie.

e) La fonction `group_by(clarity)` regroupe les diamants par type de pureté dans le résultat.

Pour chaque proposition : vrai ou faux ?

Question 6.20

La fonction `as.factor()` peut être utilisée pour transformer des variables textuelles et numériques en facteurs.

Vrai ou faux ?

7 Importer et manipuler des données

Sujet : Manipuler les données (partie 1)

Le premier but de cette session est d'apprendre à importer des données dans RStudio à partir de **fichiers CSV** et de les stocker en forme de data frame (format tabulaire).

Le deuxième but est de se familiariser avec **dplyr** (prononcé « dee-ply-er » en anglais), un paquet du **tidyverse** conçu pour manipuler les données en forme de data frame.

Contrairement aux jeux de données intégrés à R (tels que **diamonds**, etc.), la plupart des jeux de données du monde réel nécessitent des **transformations** avant de pouvoir être analysés et visualisés. À cette fin, nous examinerons également certaines transformations de types de variables et un nouvel opérateur, le `%>%` (angl. « pipe » ; trad. franç. « tuyau »), qui est très utile pour les manipulations de données.

7.1 Lire des données au format CSV dans RStudio

Par la suite, nous allons appliquer **dplyr** pour manipuler les données d'un jeu de données concret : un fichier CSV nommé **igposts.csv**. Ce fichier contient des posts Instagram de célébrités qui ont reçu beaucoup d'interactions d'utilisateurs. Pour cette application, il faut d'abord charger ce jeu de données dans notre session RStudio en cours.

i D'où viennent ces données ?

igposts.csv reprend les posts d'un échantillon de février–mars 2023 qui étaient encore publics le 5 octobre 2026. Les métriques (**Likes**, **Comments**, **Reposts**) et le nombre de followers ont été relevés à cette date : ils décrivent l'état des posts en 2026, pas au moment de leur publication. Voir [À propos](#) pour les détails.

i À noter

Un fichier de type **CSV** (angl. comma-separated values) est un format textuel simple pour sauvegarder des données en forme de tableau, c'est à dire avec des valeurs organisé par lignes et colonnes et séparées par virgules.

Le format CSV présente l'avantage d'être assez léger et compatible avec tous les systèmes d'exploitation. En outre, il est utile pour stocker des données quantitatives en sciences sociales, qui se présentent souvent sous forme de tableaux.

Voici un exemple de format CSV simple :

nom, prénom, âge
Leclerc, Antoine, 53
Gomez, Hugo, 25
Nguyen, Laetitia, 19

Que nous pouvons afficher comme tableau (à noter que la première ligne désigne les noms des colonnes) :

nom	prénom	âge
Leclerc	Antoine	53
Gomez	Hugo	25
Nguyen	Laetitia	19

Avant l'importation de fichier CSV dans RStudio (et comme déjà vu pleins de fois dans le cours 😊), il faut d'abord charger les paquets nécessaires avec `library()` et définir son répertoire de travail (vous devez adapter les paramètres de `setwd()` pour qu'il marche sur votre ordinateur) :

```
library(tidyverse)
```

```
-- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
v dplyr      1.2.1      v readr      2.2.0
v forcats    1.0.1      v stringr    1.6.0
v ggplot2    4.0.3      v tibble     3.3.1
v lubridate  1.9.5      v tidyr      1.3.2
v purrr      1.2.2
-- Conflicts ----- tidyverse_conflicts() --
x dplyr::filter() masks stats::filter()
x dplyr::lag()     masks stats::lag()
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to
  ↳ become errors
```

```
# Adaptez le chemin, puis enlevez le # :
# setwd('~/Documents/exercices_methodes_2')
```

Pour charger le jeu de données `igposts.csv` dans RStudio, vous pouvez suivre le processus suivant :

1. Téléchargez le fichier [igposts.csv](#) (clic droit → « Enregistrer le lien sous... »).
2. Stockez le fichier `igposts.csv` dans le dossier que vous avez défini comme répertoire de travail.
3. Ensuite, vous utilisez le menu interactif dans RStudio : File -> Import Dataset -> From Text (base)... et choisissez `igposts.csv`.

4. Une fenêtre avec des paramètres apparaît, que vous pouvez remplir de la manière suivante :

Import Dataset

Name:

Encoding:

Heading: ☒ Yes ☐ No

Row names:

Separator:

Decimal:

Quote:

Comment:

na.strings:

☐ Strings as factors

Input File:

Data Frame

Account	User.Name	Followers.at.Observat
Leo Messi	leomessi	516000000
Selena Gomez	selenagomez	402000000
KIARA	kiaraaliaadvani	359000000
Cristiano Ronaldo	cristiano	679000000
Cristiano Ronaldo	cristiano	679000000
Cristiano Ronaldo	cristiano	679000000
Jin of BTS	jin	566000000
V	thv	768000000
Selena Gomez	selenagomez	402000000
Cristiano Ronaldo	cristiano	679000000
Leo Messi	leomessi	516000000
badgalriri	badgalriri	145000000

Figure 7.1 : Importer un fichier CSV dans RStudio. Capture d'écran de RStudio Desktop (Posit Software, PBC).

- Explications des paramètres :
 - **Encoding = UTF-8** : Indique à RStudio d'utiliser l'encodage de caractères UTF-8 (qui est un standard très répandu), ce qui est important pour lire correctement toutes les lettres (notamment les caractères avec des accents : é, ä, ê, etc.).
 - **Heading = YES** : Indique à RStudio que la première ligne du fichier CSV contient les noms des colonnes.
 - **Separator = Comma** : Indique à RStudio que les valeurs du tableau sont séparées par des virgules « , » (attention, certains documents francophones utilisent le point-virgule « ; » à la place de la virgule, ce qu'il faudrait tenir en compte, si c'était le cas).
 - **Decimal = Period** : Indique à RStudio que le point « . » est utilisé pour les

décimales (et non la virgule « , » comme c'est souvent le cas dans les documents en français).

- **na.strings = NA** : valeur par défaut. Ce fichier n'a pas de valeurs manquantes ; dans d'autres fichiers (p. ex. exports Excel), elles peuvent être codées autrement, p. ex. #N/A.

5. Finalement, vous pouvez copier et coller le code qui apparaît dans la console dans votre script R pour facilement recharger le fichier CSV ultérieurement.

```
# Par exemple, en lisant le fichier directement depuis le
# site du cours :

igposts <- read.csv("https://julesaberlin.github.io/intro-a-R/data/igposts.csv",
  encoding = "UTF-8")

# Si tout a bien marché, l'objet igposts devrait apparaître
# dans votre environnement Nous pouvons afficher la
# structure de igposts pour une première impression :

str(igposts)
```

```
'data.frame':  84 obs. of  11 variables:
 $ Account : chr "Leo Messi" "Selena Gomez" "KIARA" "Cristiano Ronaldo"
   ...
 $ User.Name : chr "leomessi" "selenagomez" "kiaraaliaadvani"
   "cristiano" ...
 $ Followers.at.Observation: int 516000000 402000000 359000000 679000000
   679000000 679000000 566000000 768000000 402000000 679000000 ...
 $ Post.Created.Date : chr "27/02/2023" "23/02/2023" "07/02/2023"
   "09/02/2023" ...
 $ Post.Created.Time : chr "23:17:47" "02:32:34" "16:55:59" "21:19:37"
   ...
 $ Type : chr "Album" "Photo" "Album" "Album" ...
 $ Total.Interactions : int 22626965 20881240 15796982 15674570 14122758
   14160795 15541444 14992404 13333475 12339166 ...
 $ Likes : int 22386730 20636437 15598197 15511140 14051375 14047774
   15541443 14991958 13257263 12267359 ...
 $ Comments : int 240235 244803 198785 163430 71383 113021 1 446 76212
   71807 ...
 $ Reposts : int 812 511 184 97 134 80 3141 18265 21 159 ...
 $ URL : chr "https://www.instagram.com/p/CpLxnFlnJZn/"
   "https://www.instagram.com/p/Co_P7YgrF0/"
   "https://www.instagram.com/p/CoXmBSLvj7A/"
   "https://www.instagram.com/p/CodNx8BL1-X/" ...
```

7.2 Explorez votre jeu de données importé

Cela nous montre qu'il s'agit d'un data frame avec 84 observations et 11 variables. Chaque ligne (unité statistique) représente une observation associant un post Instagram à un compte

qui l’a publié ou co-publié : les 84 lignes correspondent à 82 posts distincts provenant de 33 comptes. Chaque colonne est une variable différente associée à cette observation.

Notez que les variables sont soit de type textuel (chr = « character » ; p. ex. `Account=` le nom du compte Insta qui a publié le post), soit numérique (int = « integer » ; p. ex. `Total.Interactions` = la somme des Likes et Comments qu’un post a reçu).

Vous pouvez aussi cliquer sur l’objet et explorer le tableau manuellement pour le comprendre davantage (comme nous l’avons fait avec `mtcars`, etc.).

Attention

- N’enregistrez pas un fichier CSV après l’avoir ouvert dans Excel (ou un autre tableur) : Excel peut modifier les dates, les zéros initiaux, le séparateur ou l’encodage, ce qui entraîne ensuite des erreurs dans R.
- Vous devriez enregistrer le fichier CSV directement dans votre répertoire de travail sans l’ouvrir, puis l’importer dans RStudio à l’aide du menu interactif.
- Vous devriez également éviter d’ouvrir le fichier CSV avec des applications ou autres (p. ex. une application mobile), ce qui peut également entraîner des erreurs.
- Il est préférable de télécharger les fichiers CSV avec des navigateurs standard tels que Chrome ou Safari.

Avant de continuer avec `dplyr`, regardons encore deux éléments cruciaux pour la manipulation de données avec R :

- la transformation du type de variable (p. ex. de variables textuelles en facteurs)
- L’opérateur `%>%` (angl. « pipe » ; trad. française « tuyau »)

7.3 La transformation du type de variable

Lorsque l’on analyse des jeux de données, on constate parfois que toutes les variables **ne sont pas enregistrées dans un format utile pour l’analyse**. Parfois, par exemple, les valeurs numériques sont enregistrées sous forme de texte. Il est alors utile de les transformer dans un format numérique (par exemple avec la fonction `as.numeric()`).

Mais il y a aussi le cas suivant : les variables sous forme de texte ne représentent pas simplement des textes, mais certaines **catégories**. Dans ce cas, il est judicieux de les transformer en une variable catégorielle, c’est-à-dire un facteur, à l’aide de la fonction `as.factor()`.

Dans notre exemple de jeu de données `igposts`, qui contient des métriques sur les posts Instagram, nous pouvons justement constater cela. Certes, certaines variables textuelles, telles que `URL` (le lien vers le post), sont tout à fait pertinentes. D’autres, en revanche, sont des catégories. La plus évidente est la variable `Type`, qui indique si un post Instagram est une photo ou un album (les deux catégories présentes dans `igposts`). Factorisons ci-dessous quelques-unes de ces variables catégorielles à l’aide de code :

```
# Transformons les colonnes 'Account', 'User.Name' et  
# 'Type' en facteurs :
```

```
igposts$Account <- as.factor(igposts$Account)
```

```
igposts$User.Name <- as.factor(igposts$User.Name)
```

```
igposts$Type <- as.factor(igposts$Type)
```

```
# Vérifier, si ça a marché :
```

```
str(igposts)
```

```
'data.frame': 84 obs. of 11 variables:  
 $ Account : Factor w/ 33 levels "Anastasia Karanikolaou",...: 20 26 17 6  
   6 6 12 30 26 6 ...  
 $ User.Name : Factor w/ 33 levels "agustd","arianagrande",...: 20 25 17  
   6 6 6 12 30 25 6 ...  
 $ Followers.at.Observation: int 516000000 402000000 359000000 679000000  
   679000000 679000000 566000000 768000000 402000000 679000000 ...  
 $ Post.Created.Date : chr "27/02/2023" "23/02/2023" "07/02/2023"  
   "09/02/2023" ...  
 $ Post.Created.Time : chr "23:17:47" "02:32:34" "16:55:59" "21:19:37"  
   ...  
 $ Type : Factor w/ 2 levels "Album","Photo": 1 2 1 1 2 1 2 1 1 2 ...  
 $ Total.Interactions : int 22626965 20881240 15796982 15674570 14122758  
   14160795 15541444 14992404 13333475 12339166 ...  
 $ Likes : int 22386730 20636437 15598197 15511140 14051375 14047774  
   15541443 14991958 13257263 12267359 ...  
 $ Comments : int 240235 244803 198785 163430 71383 113021 1 446 76212  
   71807 ...  
 $ Reposts : int 812 511 184 97 134 80 3141 18265 21 159 ...  
 $ URL : chr "https://www.instagram.com/p/CpLxnFlnJZn/"  
   "https://www.instagram.com/p/Co_P7YgrrF0/"  
   "https://www.instagram.com/p/CoXmBSLvj7A/"  
   "https://www.instagram.com/p/CodNx8BL1-X/" ...
```

Nous allons voir par la suite, pourquoi ces factorisations sont utiles.

7.4 L'opérateur %>%

La fonction principale du pipe %>% : construire **une séquence de fonctions** (« un tuyau ») qui est plus facile à comprendre et à lire dans le code.

```
# Par exemple, regardez la fonction suivante : quel est le  
# problème ?
```

```
log(sqrt(mean(c(1:100))))
```

```
[1] 1.960987
```

```
# Le problème est que cette fonction est imbriquée et, à  
# cause de cela, difficile à lire  
  
# Avec le pipe, nous pouvons réécrire cette fonction comme  
# ça et obtenir le même résultat après exécution du code :  
  
c(1:100) %>%  
  mean() %>%  
  sqrt() %>%  
  log()
```

```
[1] 1.960987
```

Astuce

Vous pouvez créer le pipe `%>%` avec les raccourcis clavier suivants :

- Sur **Windows** : Control + Shift + M
- Sur **MacOS** : Command + Shift + M

De plus, vous pouvez aussi utiliser le pipe natif de R, `|>` (depuis R 4.1). Il fonctionne sans paquet et s'utilise de la même façon que `%>%` dans les cas simples.

Vous pouvez déjà constater que travailler avec des jeux de données du monde réel, tels que le jeu de données Instagram `igposts`, nécessite un travail préparatoire substantiel. Mais rassurez-vous et ne vous découragez pas, plus vous pratiquez, plus cela devient facile et, bientôt, vous serez en mesure de charger facilement des ensembles de données dans RStudio ! Par ailleurs : il est normal de rencontrer des messages d'erreur. C'est une partie essentielle du processus d'apprentissage que de les surmonter en bricolant et en jouant avec vos données et votre code R jusqu'à ce que vous puissiez le faire fonctionner.

Après avoir chargé notre fichier CSV dans RStudio, appliqué des transformations et notre connaissance de l'opérateur pipe, nous pouvons maintenant manipuler les données avec le package `dplyr`.

7.5 Manipuler les données avec `dplyr`

Le paquet `dplyr` est basé sur une logique de *verbes* pour les différents types de manipulation de données, notamment :

- Les verbes pour manipuler les données au niveau de **colonnes** :
 - `select()` : pour extraire une ou plusieurs colonnes d'un data frame.
 - `mutate()` : pour créer une ou plusieurs nouvelles colonnes dans un data frame.
- Les verbes pour manipuler les données au niveau de **lignes** :
 - `filter()` : pour filtrer les lignes d'un data frame selon un ou plusieurs critères.
 - `arrange()` : pour trier les lignes d'un data frame selon un ou plusieurs critères.

- Les verbes pour **résumer, compter et regrouper** les données :
 - `summarize()` : pour réduire une ou plusieurs colonnes d'un data frame en une seule ligne, notamment pour résumer des données.
 - `count()` : pour compter le nombre d'observations d'une ou plusieurs catégories dans un data frame.
 - `group_by()` : pour regrouper des données d'un data frame selon une ou plusieurs catégories.

Essayons maintenant d'appliquer tout cela avec du code et les données `igposts` pour comprendre ce que ça veut dire en pratique !

7.6 Manipulation de colonnes : `select()` et `mutate()`

```
# select() : fonction pour extraire une ou plusieurs
# colonnes d'un data frame

# Par exemple, nous pouvons extraire les quatres colonnes
# suivantes d'igposts (voir comment le pipe est utilisé) :

igposts %>%
  select(Account, Type, Followers.at.Observation, Total.Interactions)

# Avec les deux points dans la fonction select(), vous
# pouvez sélectionner plusieurs colonnes successives. Notez
# que cette fois-ci, nous affichons que les six premières
# lignes avec la fonction head() :

igposts %>%
  select(Account:Reposts) %>%
  head()

# Vous pouvez également créer un nouvel objet, df1, qui est
# essentiellement un data frame plus petit que igposts avec
# un sous-ensemble de variables (un tel data frame plus
# petit peut être plus utile à analyser et à afficher) :

df1 <- igposts %>%
  select(Account, Type, Followers.at.Observation, Total.Interactions,
         URL)

#####

# mutate() : fonction pour créer une ou plusieurs nouvelles
# colonnes dans un data frame

# Par exemple, nous pouvons créer une nouvelle colonne qui
```

```
# compte le nombre d'interactions par follower :

df1 %>%
  mutate(TI_par_Follower = Total.Interactions/Followers.at.Observation)

# Attention, sans opérateur <-, nous ne changeons pas
# l'objet df1 ! Pour ce faire et sauvegarder notre
# nouvelle colonne, nous pouvons écraser df1 avec une
# nouvelle version de df1 :

df1 <- df1 %>%
  mutate(TI_par_Follower = Total.Interactions/Followers.at.Observation)

str(df1)
```

7.7 Manipulation de lignes avec filter() et arrange()

```
# filter() : fonction pour filtrer les lignes selon un ou
# plusieurs critères

# Pour filtrer pour une valeur spécifique d'une variable,
# p.ex. les posts de Cristiano Ronaldo :

df1 %>%
  filter(Account == "Cristiano Ronaldo")

# Pour plusieurs valeurs spécifique de la même variable, p.
# ex. les posts de Ronaldo et Messi :

df1 %>%
  filter(Account %in% c("Cristiano Ronaldo", "Leo Messi"))

# Pour plusieurs valeurs spécifiques de variables
# différentes, p. ex. les posts de type Photo de Ronaldo :

df1 %>%
  filter(Account == "Cristiano Ronaldo", Type == "Photo")

# Filtrer à partir de valeurs numériques, p. ex. les posts
# avec au moins 10 millions d'interactions :

options(scipen = 999) # Cette commande désactive la notation scientifique des nombres

df1 %>%
  filter(Total.Interactions >= 1e+07)

# Filtrer pour les posts de Selena Gomez avec au moins 10
# millions d'interactions :
```

```

df1 %>%
  filter(Account == "Selena Gomez", Total.Interactions >= 1e+07)

#####

# arrange() : fonction pour trier les lignes selon un ou
# plusieurs critères

# Trier des valeurs/facteurs textuelles par ordre
# alphabétique :

df1 %>%
  arrange(Account) %>%
  head(10)

# Trier par le nombre d'interactions (ordre croissant)

df1 %>%
  arrange(Total.Interactions) %>%
  head(10)

# Trier par le nombre d'interactions par follower (ordre
# décroissant)

df1 %>%
  arrange(desc(TI_par_Follower)) %>%
  head(10)

```

7.8 Résumer, compter et regrouper avec summarize(), count() et group_by()

```

# summarize() : fonction qui réduit une ou plusieurs
# colonnes en une seule ligne, notamment pour résumer des
# données

# Nous pouvons, par exemple, résumer la somme totale de
# Total.Interactions dans notre ensemble de données. Ce
# nombre est assez impressionnant et constitue une
# caractéristique importante de notre ensemble de données.

df1 %>%
  summarize(sum_TI = sum(Total.Interactions))

# Nous pouvons aussi calculer les moyennes des interactions
# et de followers de tout les comptes qui ont publié les
# posts :

```

```

df1 %>%
  summarize(mean_TI = mean(Total.Interactions), mean_Followers =
    ↪ mean(Followers.at.Observation))

#####

# count() : fonction qui compte le nombre d'observations
# d'une ou plusieurs catégories

# Compter l'occurrence des types de posts Instagram (Photo
# et Album, dans ce cas) dans notre jeu de données :

df1 %>%
  count(Type)

# Compter l'occurrence des différents comptes dans notre jeu
# de données :

df1 %>%
  count(Account)

# Compter l'occurrence des différents comptes dans notre jeu
# de données en les triant par cette occurrence de manière
# décroissante (avec le paramètre arrange(-n)) :

df1 %>%
  count(Account) %>%
  arrange(-n) %>%
  head(10)

#####

# group_by() : fonction qui regroupe des données selon une
# ou plusieurs catégories

# Regroupement par type de post et comparaison du nombre
# moyen d'interactions par type de post :

df1 %>%
  group_by(Type) %>%
  summarize(mean_TI = mean(Total.Interactions))

# En utilisant le paramètre n=n() dans la fonction
# summarize(), nous pouvons également compter le nombre
# d'observations dans chaque catégorie :

df1 %>%

```

```

group_by(Type) %>%
  summarize(n = n(), mean_TI = mean(Total.Interactions))

# Utilisons plusieurs verbes dplyr pour trouver les 10
# comptes qui apparaissent le plus fréquemment dans nos
# ensembles de données de posts Instagram. Nous résumons
# également leur nombre moyen de followers et
# d'interactions.

df1 %>%
  group_by(Account) %>%
  summarize(n = n(), mean_Followers = mean(Followers.at.Observation),
            mean_TI = mean(Total.Interactions)) %>%
  arrange(desc(n)) %>%
  head(10)

# Utilisons plusieurs verbes dplyr pour comparer les Likes
# et Comments des posts de Ronaldo et Messi :

igposts %>%
  group_by(Account) %>%
  filter(Account %in% c("Cristiano Ronaldo", "Leo Messi")) %>%
  summarize(n = n(), mean_Followers = mean(Followers.at.Observation),
            mean_Likes = mean(Likes), mean_Comments = mean(Comments))

```

À faire avant de continuer

- Vérifiez que vous savez importer `igposts.csv`, depuis votre ordinateur ou directement depuis le site : les chapitres suivants utilisent d'autres fichiers CSV.
- Si les exercices vous posent problème, refaites le code des chapitres précédents et assurez-vous de comprendre chaque étape.

Exercices

Notez vos réponses, puis comparez-les avec l'annexe « Corrigés » à la fin du livre.

Question 7.1

Quel compte (`User.Name`) a publié le post de `igposts.csv` ayant reçu le plus de `Total.Interactions`?

Réponse : _____

Question 7.2

Quel est le rôle principal de l'opérateur `%>%` dans `dplyr` ?

- a) Importer des données.
- b) Assigner des variables.
- c) Créer des séquences de fonctions pour simplifier le code.
- d) Convertir des types de données.
- e) Aucune des autres réponses n'est correcte.

Une seule réponse est correcte.

Question 7.3

Exécutez le code R suivant et jugez chaque proposition :

```
library(tidyverse)
data(diamonds)

diamonds %>%
  group_by(clarity) %>%
  summarize(n = n(), prix_moyen = mean(price))
```

- a) La sortie du code indique les prix moyens en USD des diamants pour chaque type de pureté dans le jeu de données `diamonds`.
- b) En moyenne, les diamants du type de pureté le plus élevé coûtent 1790 USD.
- c) Il y a le même nombre de diamants pour chaque type de pureté dans le jeu de données `diamonds`.
- d) L'argument `n = n()` dans la fonction `summarize()` nous permet de compter le nombre d'observations dans chaque catégorie.
- e) La fonction `group_by(clarity)` regroupe les diamants par type de pureté dans le résultat.

Pour chaque proposition : vrai ou faux ?

Question 7.4

Exécutez le code R suivant et jugez chaque proposition :

```
library(tidyverse)
data(mtcars)
mtcars %>% arrange(mpg)
```

- a) La fonction `arrange(mpg)` trie les voitures de `mtcars` par valeurs `mpg` croissantes.
- b) La fonction `arrange(mpg)` trie les voitures de `mtcars` par valeurs `mpg` décroissantes.
- c) Plus la valeur `mpg` d'une voiture est élevée dans `mtcars`, plus elle peut aller loin avec une unité (gallon) de carburant.
- d) Le code R modifie l'objet `mtcars` dans l'environnement de RStudio lorsqu'il est exécuté.
- e) La Cadillac Fleetwood utilise le carburant le plus efficacement de toutes les voitures du jeu de données.

Pour chaque proposition : vrai ou faux ?

Question 7.5

Exécutez le code R suivant et jugez chaque proposition. La fonction `rnorm()` simule des valeurs aléatoires qui se rapprochent d'une loi normale ; on suppose ici qu'il s'agit de tailles de femmes en Suisse (en cm).

```
library(tidyverse)
taille_f <- rnorm(500, mean = 164.7, sd = 5.6) %>% as.data.frame()
taille_f %>% count(taille_f <= 160)
```

- a) La fonction `count(taille_f <= 160)` compte le nombre de valeurs inférieures ou égales à 160 (femmes de 160 cm ou moins) ainsi que celles supérieures à 160 (femmes de plus de 160 cm).
- b) En général, la fonction `count()` de `dplyr` est utile pour compter le nombre d'observations d'un jeu de données qui correspondent à certains critères.
- c) Si vous exécutez le code plusieurs fois de suite, vous obtiendrez toujours un résultat légèrement différent, car la fonction `rnorm()` génère des valeurs « aléatoires ».
- d) `taille_f` est un objet de type *data frame* dans RStudio.
- e) `taille_f` comporte 100 lignes (observations).

Pour chaque proposition : vrai ou faux ?

Question 7.6

Ouvrez le post Instagram suivant, qui fait partie de `igposts.csv`, et jugez chaque proposition : <https://www.instagram.com/p/CpHHbGWunhI/>

- a) Le post a été publié par le compte Instagram dont le `User.Name` est **zendaya**.
- b) Il s'agit d'un post Instagram de type **Album**.
- c) Il s'agit d'un post Instagram de type **Video**.
- d) Le post a reçu plus de **250 Reposts**.
- e) Zendaya est une actrice et chanteuse **britannique**.

Pour chaque proposition : vrai ou faux ?

Question 7.7

Le terme CSV désigne un format de texte ouvert permettant de représenter des données tabulaires sous forme de valeurs séparées par des virgules.

Vrai ou faux ?

Question 7.8

La fonction `as.factor()` peut être utilisée pour transformer des variables textuelles et numériques en facteurs.

Vrai ou faux ?

Question 7.9

Un fichier CSV stocke des données textuelles dans lesquelles les valeurs sont généralement séparées par des espaces.

Vrai ou faux ?

Question 7.10

Lors de l'importation de fichiers CSV dans RStudio, il est important de savoir si les séparateurs décimaux des valeurs numériques sont des points (p. ex. 4.5) ou des virgules (p. ex. 4,5).

Vrai ou faux ?

Question 7.11

Dans `igposts.csv`, les valeurs de `Likes` correspondent au nombre de likes au moment de la publication des posts en 2023.

Vrai ou faux ?

Question 7.12

L'unité statistique (ou unité d'analyse) du jeu de données `igposts.csv` est le compte Instagram de célébrités telles que Cristiano Ronaldo et Kendall Jenner.

Vrai ou faux ?

Question 7.13

Dans le code R suivant, l'argument `na.strings = "#N/A"` indique à R comment les valeurs manquantes sont désignées dans le fichier CSV importé.

```
donnees <- read.csv("donnees.csv", encoding = "UTF-8", na.strings = "#N/A")
```

Vrai ou faux ?

8 Manipuler des données : exercices

Sujet : Manipuler les données (partie 2)

L'objectif de la session 8 est de vous entraîner à résoudre de manière autonome des problèmes de manipulation de données à l'aide du code R et de vous préparer ainsi avant le chapitre « Révision 3 ».

Les exercices suivants se composent d'une question et d'une solution modèle sous forme de code R. Notez qu'en programmation R, il y a souvent plusieurs solutions pour arriver à la réponse correcte.

Conseil

Pour un meilleur effet d'apprentissage : essayez d'abord de résoudre la question avec votre propre script R sans consulter immédiatement la solution modèle. Comparez ensuite votre propre code avec celui de la solution modèle.

8.1 Exercices avec solutions

8.1.1 Préparez-vous pour l'exercice

- Commencez en ouvrant un nouveau script R.
- Nommez-le `script_session_8` et sauvegardez-le sur votre ordinateur. (Définissez votre répertoire de travail avec `setwd()` et sauvegardez le code de la console dans le script).
- Chargez le paquet `tidyverse` avec `library(tidyverse)`.

8.1.2 Exercice 1 : `mtcars` I

- Affichez la structure de `mtcars` avec `str(mtcars)`.
- Sélectionnez ensuite les colonnes `mpg` et `cyl` de `mtcars` avec `dplyr`.

Solution

```
str(mtcars)

mtcars %>%
  select(mpg, cyl)
```

8.1.3 Exercice 2 : mtcars II

- Filtrez mtcars pour mpg >= 25 !
- Combien de modèles de voitures dans mtcars répondent à ce critère ?

Solution

```
mtcars %>%  
  filter(mpg >= 25)
```

	mpg	cyl	disp	hp	drat	wt	qsec	vs	am	gear	carb
Fiat 128	32.4	4	78.7	66	4.08	2.200	19.47	1	1	4	1
Honda Civic	30.4	4	75.7	52	4.93	1.615	18.52	1	1	4	2
Toyota Corolla	33.9	4	71.1	65	4.22	1.835	19.90	1	1	4	1
Fiat X1-9	27.3	4	79.0	66	4.08	1.935	18.90	1	1	4	1
Porsche 914-2	26.0	4	120.3	91	4.43	2.140	16.70	0	1	5	2
Lotus Europa	30.4	4	95.1	113	3.77	1.513	16.90	1	1	5	2

```
# Six modèles de voitures ont une valeur mpg d'au moins 25  
# dans mtcars.
```

8.1.4 Exercice 3 : mtcars III

- Triez mtcars par mpg dans l'ordre décroissant !

Solution

```
mtcars %>%  
  arrange(desc(mpg))
```

	mpg	cyl	disp	hp	drat	wt	qsec	vs	am	gear	carb
Toyota Corolla	33.9	4	71.1	65	4.22	1.835	19.90	1	1	4	1
Fiat 128	32.4	4	78.7	66	4.08	2.200	19.47	1	1	4	1
Honda Civic	30.4	4	75.7	52	4.93	1.615	18.52	1	1	4	2
Lotus Europa	30.4	4	95.1	113	3.77	1.513	16.90	1	1	5	2
Fiat X1-9	27.3	4	79.0	66	4.08	1.935	18.90	1	1	4	1
Porsche 914-2	26.0	4	120.3	91	4.43	2.140	16.70	0	1	5	2
Merc 240D	24.4	4	146.7	62	3.69	3.190	20.00	1	0	4	2
Datsun 710	22.8	4	108.0	93	3.85	2.320	18.61	1	1	4	1
Merc 230	22.8	4	140.8	95	3.92	3.150	22.90	1	0	4	2
Toyota Corona	21.5	4	120.1	97	3.70	2.465	20.01	1	0	3	1
Hornet 4 Drive	21.4	6	258.0	110	3.08	3.215	19.44	1	0	3	1
Volvo 142E	21.4	4	121.0	109	4.11	2.780	18.60	1	1	4	2
Mazda RX4	21.0	6	160.0	110	3.90	2.620	16.46	0	1	4	4
Mazda RX4 Wag	21.0	6	160.0	110	3.90	2.875	17.02	0	1	4	4
Ferrari Dino	19.7	6	145.0	175	3.62	2.770	15.50	0	1	5	6

Merc 280	19.2	6	167.6	123	3.92	3.440	18.30	1	0	4	4
Pontiac Firebird	19.2	8	400.0	175	3.08	3.845	17.05	0	0	3	2
Hornet Sportabout	18.7	8	360.0	175	3.15	3.440	17.02	0	0	3	2
Valiant	18.1	6	225.0	105	2.76	3.460	20.22	1	0	3	1
Merc 280C	17.8	6	167.6	123	3.92	3.440	18.90	1	0	4	4
Merc 450SL	17.3	8	275.8	180	3.07	3.730	17.60	0	0	3	3
Merc 450SE	16.4	8	275.8	180	3.07	4.070	17.40	0	0	3	3
Ford Pantera L	15.8	8	351.0	264	4.22	3.170	14.50	0	1	5	4
Dodge Challenger	15.5	8	318.0	150	2.76	3.520	16.87	0	0	3	2
Merc 450SLC	15.2	8	275.8	180	3.07	3.780	18.00	0	0	3	3
AMC Javelin	15.2	8	304.0	150	3.15	3.435	17.30	0	0	3	2
Maserati Bora	15.0	8	301.0	335	3.54	3.570	14.60	0	1	5	8
Chrysler Imperial	14.7	8	440.0	230	3.23	5.345	17.42	0	0	3	4
Duster 360	14.3	8	360.0	245	3.21	3.570	15.84	0	0	3	4
Camaro Z28	13.3	8	350.0	245	3.73	3.840	15.41	0	0	3	4
Cadillac Fleetwood	10.4	8	472.0	205	2.93	5.250	17.98	0	0	3	4
Lincoln Continental	10.4	8	460.0	215	3.00	5.424	17.82	0	0	3	4

8.1.5 Exercice 4 : mtcars IV

- Créez une nouvelle variable (colonne) dans `mtcars`, nommé `mpg_wt`, définie comme le nombre de `mpg` par unité de poids (`wt`) !
- Sauvegardez la version modifiée de `mtcars` avec l'opérateur d'affectation sous l'objet `mtcars_v2`.
- Affichez la nouvelle colonne `mpg_wt` de `mtcars_v2` en la triant par ordre décroissant !
- Quel modèle de voiture de `mtcars` est le plus économe en carburant par rapport à son poids ?

Solution

```
mtcars_v2 <- mtcars %>%
  mutate(mpg_wt = mpg/wt)

mtcars_v2 %>%
  select(mpg_wt) %>%
  arrange(desc(mpg_wt))
```

	mpg_wt
Lotus Europa	20.092531
Honda Civic	18.823529
Toyota Corolla	18.474114
Fiat 128	14.727273
Fiat X1-9	14.108527
Porsche 914-2	12.149533
Datsun 710	9.827586
Toyota Corona	8.722110
Mazda RX4	8.015267

Volvo 142E	7.697842
Merc 240D	7.648903
Mazda RX4 Wag	7.304348
Merc 230	7.238095
Ferrari Dino	7.111913
Hornet 4 Drive	6.656299
Merc 280	5.581395
Hornet Sportabout	5.436047
Valiant	5.231214
Merc 280C	5.174419
Pontiac Firebird	4.993498
Ford Pantera L	4.984227
Merc 450SL	4.638070
AMC Javelin	4.425036
Dodge Challenger	4.403409
Maserati Bora	4.201681
Merc 450SE	4.029484
Merc 450SLC	4.021164
Duster 360	4.005602
Camaro Z28	3.463542
Chrysler Imperial	2.750234
Cadillac Fleetwood	1.980952
Lincoln Continental	1.917404

```
# Le Lotus Europa est le plus économe en carburant par
# rapport à son poids.
```

8.1.6 Exercice 5 : diamonds I

- Affichez encore une fois `str(diamonds)` !
- Utilisez des verbes `dplyr` pour compter le nombre de diamants dans `diamonds` qui pèsent plus de 3 carats !

Solution

```
diamonds %>%
  count(carat > 3)
```

```
# A tibble: 2 x 2
  `carat > 3`     n
  <lgl>         <int>
1 FALSE      53908
2 TRUE         32
```

```
# Solution alternative :

diamonds %>%
  filter(carat > 3) %>%
  count()

# A tibble: 1 x 1
      n
  <int>
1    32

# Le critère carat > 3 s'applique à 32 diamants dans
# diamonds.
```

8.1.7 Exercice 6 : diamonds II

— Calculez le prix moyen en USD des diamants dans **diamonds** !

Solution

```
diamonds %>%
  summarize(prix_moyen = mean(price))

# A tibble: 1 x 1
  prix_moyen
    <dbl>
1    3933.

# Solution alternative (sans dplyr) :

mean(diamonds$price)

[1] 3932.8
```

8.1.8 Exercice 7 : diamonds III

- Calculez le prix moyen en USD des diamants des différentes catégories de pureté (clarity) dans **diamonds** !
- Triez le résultat par ordre décroissant des prix moyens
- Quelle catégorie de pureté est la plus chère en moyenne dans **diamonds** ?

Solution

```
diamonds %>%  
  group_by(clarity) %>%  
  summarize(prix_moyen = mean(price)) %>%  
  arrange(desc(prix_moyen))
```

```
# A tibble: 8 x 2  
  clarity prix_moyen  
  <ord>      <dbl>  
1 SI2        5063.  
2 SI1        3996.  
3 VS2        3925.  
4 I1         3924.  
5 VS1        3839.  
6 VVS2       3284.  
7 IF         2865.  
8 VVS1       2523.
```

```
# Avec un prix par diamant de 5063 USD en moyenne, la  
# catégorie de pureté SI2 est la plus chère dans diamonds.
```

Attention : cela ne veut pas dire qu'une pureté plus faible rend un diamant plus cher. Les diamants SI2 sont en moyenne plus **lourds** (1,08 carat, contre 0,51 pour la pureté maximale IF), et le poids influence fortement le prix. Comparez avec `diamonds %>% group_by(clarity) %>% summarize(mean(carat))`. C'est un exemple de **variable confondante**.

8.1.9 Exercice 8 : mtcars V

- Regroupez `mtcars` par `gear`, puis, à l'aide de `summarize()`, calculez `n`, la médiane et la variance de `mpg`.

Solution

```
mtcars %>%  
  group_by(gear) %>%  
  summarize(n = n(), median_mpg = median(mpg), var_mpg = var(mpg))
```

```
# A tibble: 3 x 4  
  gear      n median_mpg var_mpg  
  <dbl> <int>      <dbl>   <dbl>  
1    3     15      15.5    11.4  
2    4     12      22.8    27.8  
3    5      5      19.7    44.3
```

8.1.10 Exercice 9 : iris I

- Sélectionner `Petal.Length`, `Petal.Width`, et `Species` dans `iris`.
- Ensuite, utilisez `mutate()` pour créer une nouvelle colonne, appelée `Petal.Surface`, qui est le produit de `Petal.Length` et `Petal.Width`.
- Regroupez ensuite les données par espèce et utilisez `summarize()` pour calculer une nouvelle variable : `mean_Petal.Surface`.

Solution

```
str(iris)
```

```
'data.frame':  150 obs. of  5 variables:
 $ Sepal.Length: num 5.1 4.9 4.7 4.6 5 5.4 4.6 5 4.4 4.9 ...
 $ Sepal.Width : num 3.5 3 3.2 3.1 3.6 3.9 3.4 3.4 2.9 3.1 ...
 $ Petal.Length: num 1.4 1.4 1.3 1.5 1.4 1.7 1.4 1.5 1.4 1.5 ...
 $ Petal.Width : num 0.2 0.2 0.2 0.2 0.2 0.4 0.3 0.2 0.2 0.1 ...
 $ Species : Factor w/ 3 levels "setosa","versicolor",...: 1 1 1 1 1 1 1 1 1 1 ...
 1 1 1 ...
```

```
iris %>%
  select(Petal.Length, Petal.Width, Species) %>%
  mutate(Petal.Surface = Petal.Length * Petal.Width) %>%
  group_by(Species) %>%
  summarise(mean_Petal.Surface = mean(Petal.Surface))
```

```
# A tibble: 3 x 2
  Species    mean_Petal.Surface
  <fct>          <dbl>
1 setosa          0.366
2 versicolor      5.72
3 virginica       11.3
```

```
# Les iris de l'espèce virginica ont les plus grandes
# surfaces de pétales (en moyenne).
```

8.1.11 Exercice 10 : diamonds IV

- Utilisez les verbes de `dplyr` pour calculer la moyenne des prix en USD par carat dans `diamonds`.
- Montrez les résultats, triés par ordre décroissant de dollars par carat, pour les différentes catégories de taille (`cut`).
- Montrez aussi `n`, le nombre de diamants par catégorie de taille.

Solution

```
diamonds %>%
  select(carat, cut, price) %>%
  mutate(USD_per_carat = price/carat) %>%
  group_by(cut) %>%
  summarize(n = n(), mean_USD_per_carat = mean(USD_per_carat)) %>%
  arrange(desc(mean_USD_per_carat))
```

```
# A tibble: 5 x 3
  cut          n mean_USD_per_carat
<ord>      <int>          <dbl>
1 Premium  13791          4223.
2 Very Good 12082          4014.
3 Ideal    21551          3920.
4 Good      4906          3860.
5 Fair     1610          3767.
```

```
# En moyenne, les diamants de la meilleure qualité de
# taille (Ideal) ne sont pas les plus chers dans diamonds.
```

Ici aussi, le poids joue un rôle : même **par carat**, les gros diamants coûtent plus cher, et les diamants de taille *Ideal* sont en moyenne plus petits (0,70 carat) que les *Premium* (0,89 carat).

8.1.12 Exercice 11 : igposts I

- Chargez le jeu de données `igposts` que nous avons vu pendant session 7.
- Si nécessaire, téléchargez-le à nouveau depuis le [site](#) et sauvegardez-le dans le dossier de votre ordinateur que vous avez défini comme répertoire de travail.
- Si nécessaire, transformez encore une fois les colonnes `igposts$Account`, `igposts$User.Name` et `igposts$Type` en variables catégorielles avec `as.factor()`.

Solution

```
# Vous pouvez copier-coller le code que nous avons utilisé
# la dernière fois (modifiez-le pour votre ordi) :
igposts <- read.csv("https://julesaberlin.github.io/intro-a-R/data/igposts.csv",
  encoding = "UTF-8")

# Appliquez encore une fois les transformations de
# variables, si nécessaire :

igposts$Account <- as.factor(igposts$Account)

igposts$User.Name <- as.factor(igposts$User.Name)

igposts$Type <- as.factor(igposts$Type)

# Vérifier, si ça a marché :

str(igposts)

'data.frame': 84 obs. of 11 variables:
 $ Account : Factor w/ 33 levels "Anastasia Karanikolaou",...: 20 26 17 6
   6 6 12 30 26 6 ...
 $ User.Name : Factor w/ 33 levels "agustd","arianagrande",...: 20 25 17
   6 6 6 12 30 25 6 ...
 $ Followers.at.Observation: int 516000000 402000000 359000000 679000000
   679000000 679000000 566000000 768000000 402000000 679000000 ...
 $ Post.Created.Date : chr "27/02/2023" "23/02/2023" "07/02/2023"
   "09/02/2023" ...
 $ Post.Created.Time : chr "23:17:47" "02:32:34" "16:55:59" "21:19:37"
   ...
 $ Type : Factor w/ 2 levels "Album","Photo": 1 2 1 1 2 1 2 1 1 2 ...
 $ Total.Interactions : int 22626965 20881240 15796982 15674570 14122758
   14160795 15541444 14992404 13333475 12339166 ...
 $ Likes : int 22386730 20636437 15598197 15511140 14051375 14047774
   15541443 14991958 13257263 12267359 ...
 $ Comments : int 240235 244803 198785 163430 71383 113021 1 446 76212
   71807 ...
 $ Reposts : int 812 511 184 97 134 80 3141 18265 21 159 ...
 $ URL : chr "https://www.instagram.com/p/CpLxnFtNJZn/"
   "https://www.instagram.com/p/Co_P7YgrrF0/"
   "https://www.instagram.com/p/CoXmBSLvj7A/"
   "https://www.instagram.com/p/CodNx8BL1-X/" ...
```

8.1.13 Exercice 12 : igposts II

- Utilisez `dplyr` pour trouver le nombre `n` de posts Instagram regroupés par compte des influenceurs dans `igposts`.
- Affichez les 10 premiers par ordre décroissant de `n`.

Solution

```
igposts %>%  
  group_by(Account) %>%  
  summarize(n = n()) %>%  
  arrange(desc(n)) %>%  
  head(10)
```

```
# A tibble: 10 x 2  
  Account      n  
  <fct>      <int>  
1 Cristiano Ronaldo 11  
2 LISA           6  
3 Selena Gomez     6  
4 badgalriri       5  
5 Kylie           5  
6 Leo Messi        5  
7 J               4  
8 jhope           3  
9 KAROL G         3  
10 KIARA          3
```

8.1.14 Exercice 13 : igposts III

- Utilisez dplyr pour calculer le nombre moyen de Likes, Commentaires et Reposts par compte (Account) des influenceurs dans igposts.
- Triez-les par ordre décroissant de la moyenne des Likes.
- Affichez le top 10.

Solution

```
igposts %>%  
  group_by(Account) %>%  
  summarize(mean_Likes = mean(Likes), mean_Comments = mean(Comments),  
            mean_Reposts = mean(Reposts)) %>%  
  arrange(desc(mean_Likes)) %>%  
  head(10)
```

```
# A tibble: 10 x 4  
  Account      mean_Likes mean_Comments mean_Reposts  
  <fct>      <dbl>      <dbl>      <dbl>  
1 Jin of BTS 15541443      1      3141  
2 V         14991958     446     18265  
3 Leo Messi 12960542.    92625.    1033  
4 KIARA     11115012     82133     120.  
5 Selena Gomez 10978163.    80699.    107.  
6 Cristiano Ronaldo 10003869.    67541.     82.8
```

7	badgalriri	9970143.	52486.	80.6
8	Noah Schnapp	9253770	17981	7270
9	Kendall	9210722	40433	133
10	Georgina Rodríguez	8849248.	29210.	64.5

8.1.15 Exercice 14 : igposts IV

- Utilisez `dplyr` pour afficher les variables `Account`, `Likes` et `URL` des 5 posts dans `igposts` qui ont reçu le plus de `Likes`.
- Allez voir l'URL du post avec le plus de `Likes` : De quoi s'agit-il ?

Solution

```
igposts %>%
  select(Account, Likes, URL) %>%
  arrange(desc(Likes)) %>%
  head(5)
```

	Account	Likes	URL
1	Leo Messi	22386730	https://www.instagram.com/p/CpLxnFlnJZn/
2	Selena Gomez	20636437	https://www.instagram.com/p/Co_P7YgrrF0/
3	KIARA	15598197	https://www.instagram.com/p/CoXmBSLvJ7A/
4	Jin of BTS	15541443	https://www.instagram.com/p/CpW0CpzhZyS/
5	Cristiano Ronaldo	15511140	https://www.instagram.com/p/CodNx8BL1-X/

```
# Le post Instagram le plus liké dans igposts est un album
# publié par Leo Messi.
```

[URL Instagram du post de Leo Messi](https://www.instagram.com/p/CpLxnFlnJZn/)

8.1.16 Exercice 15 : igposts V

- Faites la même chose que dans la question 14, mais maintenant pour les `Comments` au lieu des `Likes` !
- Allez voir l'URL du post avec le plus de `Comments` : De quoi s'agit-il ?

Solution

```
igposts %>%
  select(Account, Comments, URL) %>%
  arrange(desc(Comments)) %>%
  head(5)
```

Account Comments

URL

```

1 Selena Gomez 244803 https://www.instagram.com/p/Co_P7YgrrF0/
2 Leo Messi 240235 https://www.instagram.com/p/CpLxnFlnJZn/
3 KIARA 198785 https://www.instagram.com/p/CoXmBSLvj7A/
4 Shakira 179418 https://www.instagram.com/p/Co8GDELuSpG/
5 KAROL G 179418 https://www.instagram.com/p/Co8GDELuSpG/

```

```

# Le post Instagram ayant reçu le plus de commentaires dans
# igposts est de type photo et a été publié par Selena
# Gomez.

```

[URL du post Instagram de Selena Gomez](https://www.instagram.com/p/Co_P7YgrrF0/)

8.1.17 Exercice 16 : igposts VI

- Faites la même chose que dans les exercices 14 et 15, mais maintenant pour les **Reposts** (nombre de reposts relevé le 5 octobre 2026 ; fonction introduite par Instagram en 2025) !
- Allez voir l'URL du post avec le plus de **Reposts** : De quoi s'agit-il ?

Solution

```

igposts %>%
  select(Account, Reposts, URL) %>%
  arrange(desc(Reposts)) %>%
  head(5)

```

	Account	Reposts	URL
1	SUGA of BTS 민윤기	23257	https://www.instagram.com/p/Copd1XvJT78/
2	V	18265	https://www.instagram.com/p/CpJn09fPvud/
3	Noah Schnapp	7270	https://www.instagram.com/p/Co2oCgz0lEr/
4	Leo Messi	3869	https://www.instagram.com/p/CoZDmfZN0Ys/
5	Jin of BTS	3141	https://www.instagram.com/p/CpW0CpzhZyS/

```

# Le post Instagram ayant reçu le plus de Reposts dans
# igposts est une photo publiée par SUGA (agustd).

```

[URL du post Instagram de SUGA](https://www.instagram.com/p/Copd1XvJT78/)

8.1.18 Exercice 17 : igposts VII

- Utilisez dplyr pour créer une nouvelle colonne : **Likes_par_Follower** (définie comme Likes divisé par Followers.at.Observation). Les followers ont été relevés le 5 octobre 2026 : ce ratio compare les posts à la taille actuelle des comptes, ce n'est pas un taux d'engagement au moment de la publication.

- Trouvez maintenant le post avec le plus grand nombre de Likes_par_Follower et naviguez jusqu'à son URL : que montre-t-il ?

Solution

```
igposts %>%  
  mutate(Likes_par_Follower = Likes/Followers.at.Observation) %>%  
  select(Account, Likes_par_Follower, URL) %>%  
  arrange(desc(Likes_par_Follower)) %>%  
  head(5)
```

	Account	Likes_par_Follower
1	Anastasia Karanikolaou	0.8123424
2	KIARA	0.4344902
3	Noah Schnapp	0.3643217
4	Jin of BTS	0.2745838
5	KIARA	0.2502900

	URL
1	https://www.instagram.com/p/CoptMRoJtUD/
2	https://www.instagram.com/p/CoXmBSLvj7A/
3	https://www.instagram.com/p/Co2oCgz0lEr/
4	https://www.instagram.com/p/CpW0CpzhZyS/
5	https://www.instagram.com/p/Co7takyvRw-/

```
# Le post Instagram qui a reçu le plus de Likes par  
# follower dans igposts est un album publié par Anastasia  
# Karanikolaou.
```

[URL du post Instagram de Anastasia Karanikolaou](https://www.instagram.com/p/CoptMRoJtUD/)

8.1.19 Exercice 18 : igposts VIII

- Faites la même chose qu'à la question 17, mais cette fois créez la variable Comments_par_Follower.

Solution

```
igposts %>%  
  mutate(Comments_par_Follower = Comments/Followers.at.Observation) %>%  
  select(Account, Comments_par_Follower, URL) %>%  
  arrange(desc(Comments_par_Follower)) %>%  
  head(5)
```

	Account	Comments_par_Follower
1	KIARA	0.005537187
2	KAROL G	0.002548551
3	Shakira	0.001741922

```

4           harrystyles           0.001735974
5 Anastasia Karanikolaou         0.001685600
                                URL
1 https://www.instagram.com/p/CoXmBSLvj7A/
2 https://www.instagram.com/p/Co8GDELuSpG/
3 https://www.instagram.com/p/Co8GDELuSpG/
4 https://www.instagram.com/p/CoWJZRTobJH/
5 https://www.instagram.com/p/CoptMRoJtUD/

```

```

# Le post Instagram qui a reçu le plus de commentaires par
# follower dans igposts est un album publié par l'actrice
# indienne Kiara Advani.

```

[URL du post Instagram de KIARA](#)

8.1.20 Exercice 19 : igposts IX

- Faites la même chose que dans les exercices 17 et 18, mais cette fois pour `Reposts_par_Follower`.

Solution

```

igposts %>%
  mutate(Reposts_par_Follower = Reposts/Followers.at.Observation) %>%
  select(Account, Reposts_par_Follower, URL) %>%
  arrange(desc(Reposts_par_Follower)) %>%
  head(5)

```

```

              Account Reposts_par_Follower
1 SUGA of BTS 민윤기           4.220871e-04
2      Noah Schnapp           2.862205e-04
3              V             2.378255e-04
4      Jin of BTS           5.549470e-05
5          jhope            3.003527e-05
                                URL
1 https://www.instagram.com/p/Copd1XvJT78/
2 https://www.instagram.com/p/Co2oCgz0lEr/
3 https://www.instagram.com/p/CpJn09fPvud/
4 https://www.instagram.com/p/CpW0CpzhZyS/
5 https://www.instagram.com/p/CpQDcEXJA0G/

```

```

# Le post Instagram qui a reçu le plus de Reposts par
# follower est la même photo de SUGA (agustd) qu'à
# l'exercice 16.

```

[URL du post Instagram de SUGA](#)

Exercices

Notez vos réponses, puis comparez-les avec l'annexe « Corrigés » à la fin du livre.

Question 8.1

Exécutez le code R suivant et jugez chaque proposition. Les unités américaines sont converties : 1 pouce (*inch*) = 2,54 cm et 1 livre (*pound*) = 0,4536 kg.

```
library(tidyverse)
data(women)
print(women)
?women
women %>% summarize(n = n(),
                    taille_moyenne_cm = mean(height * 2.54),
                    poids_moyen_kg = mean(weight * 0.4536))
```

- a) `women` est un jeu de données contenant les tailles et les poids de 15 femmes américaines âgées de 20 à 39 ans.
- b) La femme moyenne du jeu de données `women` mesure environ 165 cm et pèse environ 62 kg.
- c) Les données indiquent que les femmes américaines pèsent en moyenne beaucoup plus que les femmes européennes.
- d) Le jeu de données `women` est composé de 15 lignes et de 2 colonnes.
- e) Le code R transforme les unités américaines, pouces (*inches*) et livres (*pounds*), en centimètres et kilogrammes.

Pour chaque proposition : vrai ou faux ?

Question 8.2

Combien de posts Instagram de Rihanna (`User.Name = badgalriri`) le jeu de données `igposts.csv` contient-il ?

Réponse : _____

Question 8.3

Combien de posts Instagram de Selena Gomez (`User.Name = selenagomez`) le jeu de données `igposts.csv` contient-il ?

Réponse : _____

Question 8.4

Combien d'observations (lignes) de type **Album** le jeu de données `igposts.csv` contient-il ?

Réponse : _____

Question 8.5

Combien de voitures du jeu de données `mtcars` ont une valeur `mpg` d'au moins 20 ?

Réponse : _____

Question 8.6

Quel est le `User.Name` du compte Instagram ayant le plus de `Followers.at.Observation` parmi les comptes du jeu de données `igposts.csv` ? (Tapez le `User.Name` exact.)

Réponse : _____

Question 8.7

Quel est le `User.Name` du compte Instagram qui a publié le post ayant reçu le plus grand nombre de `Reposts` dans le jeu de données `igposts.csv` ? (Tapez le `User.Name` exact.)

Réponse : _____

Question 8.8

Dans le jeu de données `igposts.csv`, l'actrice de Bollywood Kiara Advani (`Account = KIARA`) présente le ratio de commentaires par follower le plus élevé de tous les comptes Instagram du jeu de données.

Vrai ou faux ?

Question 8.9

Exécutez le code R suivant pour savoir si l'affirmation est vraie ou fausse : la Toyota Corolla est le modèle de voiture le plus économe en carburant par rapport à son poids dans le jeu de données `mtcars`.

```
library(tidyverse)
data(mtcars)
mtcars %>%
  select(mpg, wt) %>%
  mutate(mpg_par_livre = mpg / wt) %>%
  arrange(desc(mpg_par_livre))
```

Vrai ou faux ?

9 Révision 3 : manipulation de données

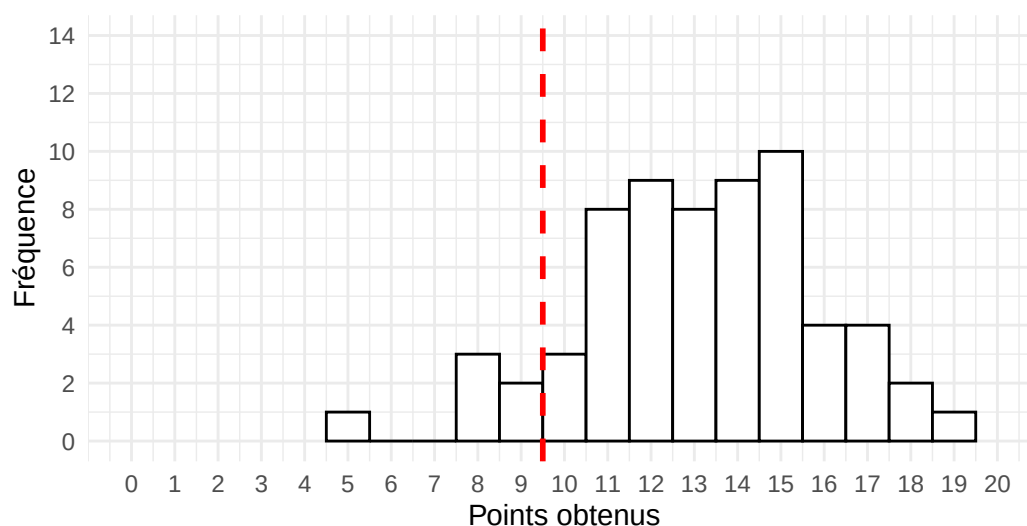
Dans le cours, le **Problem Set 3** était un test sur Moodle portant sur les chapitres 7 et 8. Les questions ci-dessous en reprennent le contenu, avec la correction et une explication pour chaque réponse.

i Résultats de l'édition 2025 du cours

64 étudiant·es ont passé ce test (20 points au maximum, 10 pour réussir). La moyenne était de 13,1 points et 91 % des étudiant·es l'ont réussi.

Histogramme des points obtenus dans le PS 3

n = 64 résultats de test



Source : Problem Set 3, édition 2025

Exercices de révision

Ces questions reprennent le **Problem Set 3**. Notez vos réponses, puis comparez-les avec l'annexe « Corrigés » à la fin du livre.

Question 9.1

Quel compte (User.Name) a publié le post de `igposts.csv` ayant reçu le plus de `Total.Interactions` ?

Réponse : _____

Question 9.2

Quel est le rôle principal de l'opérateur `%>%` dans `dplyr` ?

- a) Importer des données.
- b) Assigner des variables.
- c) Créer des séquences de fonctions pour simplifier le code.
- d) Convertir des types de données.
- e) Aucune des autres réponses n'est correcte.

Une seule réponse est correcte.

Question 9.3

Exécutez le code R suivant et jugez chaque proposition :

```
library(tidyverse)
data(diamonds)

diamonds %>%
  group_by(clarity) %>%
  summarize(n = n(), prix_moyen = mean(price))
```

- a) La sortie du code indique les prix moyens en USD des diamants pour chaque type de pureté dans le jeu de données `diamonds`.
- b) En moyenne, les diamants du type de pureté le plus élevé coûtent 1790 USD.
- c) Il y a le même nombre de diamants pour chaque type de pureté dans le jeu de données `diamonds`.
- d) L'argument `n = n()` dans la fonction `summarize()` nous permet de compter le nombre d'observations dans chaque catégorie.
- e) La fonction `group_by(clarity)` regroupe les diamants par type de pureté dans le résultat.

Pour chaque proposition : vrai ou faux ?

Question 9.4

Exécutez le code R suivant et jugez chaque proposition :

```
library(tidyverse)
data(mtcars)
mtcars %>% arrange(mpg)
```

- a) La fonction `arrange(mpg)` trie les voitures de `mtcars` par valeurs `mpg` croissantes.
- b) La fonction `arrange(mpg)` trie les voitures de `mtcars` par valeurs `mpg` décroissantes.
- c) Plus la valeur `mpg` d'une voiture est élevée dans `mtcars`, plus elle peut aller loin avec une unité (gallon) de carburant.
- d) Le code R modifie l'objet `mtcars` dans l'environnement de RStudio lorsqu'il est exécuté.
- e) La Cadillac Fleetwood utilise le carburant le plus efficacement de toutes les voitures du jeu de données.

Pour chaque proposition : vrai ou faux ?

Question 9.5

Exécutez le code R suivant et jugez chaque proposition. La fonction `rnorm()` simule des valeurs aléatoires qui se rapprochent d'une loi normale ; on suppose ici qu'il s'agit de tailles de femmes en Suisse (en cm).

```
library(tidyverse)
taille_f <- rnorm(500, mean = 164.7, sd = 5.6) %>% as.data.frame()
taille_f %>% count(taille_f <= 160)
```

- a) La fonction `count(taille_f <= 160)` compte le nombre de valeurs inférieures ou égales à 160 (femmes de 160 cm ou moins) ainsi que celles supérieures à 160 (femmes de plus de 160 cm).
- b) En général, la fonction `count()` de `dplyr` est utile pour compter le nombre d'observations d'un jeu de données qui correspondent à certains critères.
- c) Si vous exécutez le code plusieurs fois de suite, vous obtiendrez toujours un résultat légèrement différent, car la fonction `rnorm()` génère des valeurs « aléatoires ».
- d) `taille_f` est un objet de type *data frame* dans RStudio.
- e) `taille_f` comporte 100 lignes (observations).

Pour chaque proposition : vrai ou faux ?

Question 9.6

Ouvrez le post Instagram suivant, qui fait partie de `igposts.csv`, et jugez chaque proposition : <https://www.instagram.com/p/CpHHbGWunhI/>

- a) Le post a été publié par le compte Instagram dont le `User.Name` est **zendaya**.
- b) Il s'agit d'un post Instagram de type **Album**.
- c) Il s'agit d'un post Instagram de type **Video**.
- d) Le post a reçu plus de **250 Reposts**.
- e) Zendaya est une actrice et chanteuse **britannique**.

Pour chaque proposition : vrai ou faux ?

Question 9.7

Un fichier CSV stocke des données textuelles dans lesquelles les valeurs sont généralement séparées par des espaces.

Vrai ou faux ?

Question 9.8

Lors de l'importation de fichiers CSV dans RStudio, il est important de savoir si les séparateurs décimaux des valeurs numériques sont des points (p. ex. 4.5) ou des virgules (p. ex. 4,5).

Vrai ou faux ?

Question 9.9

Dans `igposts.csv`, les valeurs de `Likes` correspondent au nombre de likes au moment de la publication des posts en 2023.

Vrai ou faux ?

Question 9.10

L'unité statistique (ou unité d'analyse) du jeu de données `igposts.csv` est le compte Instagram de célébrités telles que Cristiano Ronaldo et Kendall Jenner.

Vrai ou faux ?

Question 9.11

Dans le code R suivant, l'argument `na.strings = "#N/A"` indique à R comment les valeurs manquantes sont désignées dans le fichier CSV importé.

```
donnees <- read.csv("donnees.csv", encoding = "UTF-8", na.strings = "#N/A")
```

Vrai ou faux ?

Question 9.12

Exécutez le code R suivant et jugez chaque proposition. Les unités américaines sont converties : 1 pouce (*inch*) = 2,54 cm et 1 livre (*pound*) = 0,4536 kg.

```
library(tidyverse)
data(women)
print(women)
?women
women %>% summarize(n = n(),
                    taille_moyenne_cm = mean(height * 2.54),
                    poids_moyen_kg = mean(weight * 0.4536))
```

- a) `women` est un jeu de données contenant les tailles et les poids de 15 femmes américaines âgées de 20 à 39 ans.
- b) La femme moyenne du jeu de données `women` mesure environ 165 cm et pèse environ 62 kg.
- c) Les données indiquent que les femmes américaines pèsent en moyenne beaucoup plus que les femmes européennes.
- d) Le jeu de données `women` est composé de 15 lignes et de 2 colonnes.
- e) Le code R transforme les unités américaines, pouces (*inches*) et livres (*pounds*), en centimètres et kilogrammes.

Pour chaque proposition : vrai ou faux ?

Question 9.13

Combien de posts Instagram de Selena Gomez (`User.Name = selenagomez`) le jeu de données `igposts.csv` contient-il ?

Réponse : _____

Question 9.14

Combien d'observations (lignes) de type **Album** le jeu de données `igposts.csv` contient-il ?

Réponse : _____

Question 9.15

Combien de voitures du jeu de données `mtcars` ont une valeur `mpg` d'au moins 20 ?

Réponse : _____

Question 9.16

Quel est le `User.Name` du compte Instagram ayant le plus de `Followers.at.Observation` parmi les comptes du jeu de données `igposts.csv` ? (Tapez le `User.Name` exact.)

Réponse : _____

Question 9.17

Dans le jeu de données `igposts.csv`, l'actrice de Bollywood Kiara Advani (`Account = KIARA`) présente le ratio de commentaires par follower le plus élevé de tous les comptes Instagram du jeu de données.

Vrai ou faux ?

Question 9.18

Exécutez le code R suivant pour savoir si l'affirmation est vraie ou fausse : la Toyota Corolla est le modèle de voiture le plus économe en carburant par rapport à son poids dans le jeu de données `mtcars`.

```
library(tidyverse)
data(mtcars)
mtcars %>%
  select(mpg, wt) %>%
  mutate(mpg_par_livre = mpg / wt) %>%
  arrange(desc(mpg_par_livre))
```

Vrai ou faux ?

Question 9.19

Importez le jeu de données `reponses_socio.csv` dans RStudio. Combien de **variables** contient-il ?

Réponse : _____

Question 9.20

Importez le jeu de données `reponses_socio.csv` dans RStudio. Pour la variable `genre` : combien d'**hommes** y a-t-il dans ce jeu de données ?

Réponse : _____

10 Statistiques descriptives

Sujet : La statistique descriptive

i À noter

La **statistique descriptive** est généralement la première étape d'une analyse statistique. Elle aide à comprendre le jeu de données et ses caractéristiques à un niveau descriptif.

La statistique descriptive **résume** les données de l'échantillon. La statistique **inférentielle** permet ensuite de généraliser de l'échantillon à la population (avec une marge d'incertitude). Établir une **relation de cause à effet** demande en plus un dispositif adapté, par exemple une expérience randomisée.

Les mesures suivantes font souvent partie des statistiques descriptives (souvent appelés *summary statistics* en anglais) et sont utilisées pour une première analyse des variables de type numérique d'un jeu de données.

- La taille de l'échantillon (nombre d'observations n)
- Les mesures de tendance centrale (moyenne, médiane, mode)
- Les mesures de dispersion (étendue, écart interquartile, variance, écart-type)
- Les mesures de forme (asymétrie, *kurtosis*)
- Les mesure de position (percentiles, quartiles)

Ces mesures permettent d'obtenir une première idée de la structure d'un jeu de données. Elles permettent également d'identifier des valeurs extrêmes (angl. *outliers*) qui peuvent jouer un rôle important dans un jeu de données.

Des mesures concernant les **variables catégorielles** peuvent également faire partie d'une analyse descriptive, notamment :

- La fréquence de chaque catégorie (p. ex. combien de participant·es à un questionnaire sont des femmes ?)
- La répartition des différentes catégories (p. ex. quel est le pourcentage de femmes dans l'échantillon ?)

Dans la mise en œuvre d'une analyse statistique, par exemple dans un travail scientifique, les statistiques descriptives sont souvent présentées sous deux formats :

- des **tableaux** pour les mesures telles que la taille de l'échantillon, les moyennes, etc. Souvent aussi pour différentes catégories, p. ex. hommes et femmes.
- des **graphiques** pour la visualisation de distributions, par exemple des histogrammes, et des répartitions de variables catégorielles, par exemple par le biais de diagrammes à barres.

Lorsque l'on veut représenter des statistiques descriptives dans R, il est souvent utile de combiner les deux paquets R `dplyr` et `ggplot2`. C'est une bonne nouvelle, car nous avons déjà examiné les principaux codes R nécessaires pour utiliser ces deux paquets dans ce cours.

Pour nous entraîner à nouveau, cette fois avec nos propres données, nous allons procéder ci-dessous à une analyse descriptive des réponses au questionnaire rempli par les étudiant·es du cours !

i Données synthétiques

Le fichier `reponses_socio.csv` utilisé sur ce site est **synthétique** : ses données ont été simulées à partir du questionnaire d'une classe, et il ne contient **aucune personne réelle**. Il a les mêmes variables que le questionnaire, avec des distributions et des relations semblables. Pour protéger les participant·es, les valeurs rares ont été regroupées (p. ex. « Autre canton »). Les résultats ci-dessous peuvent donc différer de ceux présentés en cours.

Vous verrez donc par la suite comment ce que nous avons appris jusqu'à présent dans le cours peut être combiné de manière utile.

10.1 Analyse des réponses au questionnaire, 1re partie (stats descriptives)

Veuillez ouvrir un nouveau script R et charger les paquets `tidyverse` et `ggthemes`. Téléchargez ensuite le fichier `reponses_socio.csv` dans votre répertoire de travail, ou lisez-le directement depuis le site comme dans le code ci-dessous.

Si vous importez le fichier avec *Import Dataset* → *From Text (base)...*, utilisez les paramètres suivants :

- *Name* : `reponses_socio`
- *Encoding* : UTF-8
- *Heading* : Yes
- *Row names* : Automatic
- *Separator* : Comma; *Decimal* : Period; *Quote* : Double (")
- *na.strings* : NA
- *Strings as factors* : coché

```
# Script session 10
```

```
library(tidyverse)
```

```
-- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
v dplyr      1.2.1      v readr      2.2.0
v forcats    1.0.1      v stringr    1.6.0
v ggplot2    4.0.3      v tibble     3.3.1
v lubridate  1.9.5      v tidyr      1.3.2
```

```
v purrr      1.2.2
-- Conflicts ----- tidyverse_conflicts() --
x dplyr::filter() masks stats::filter()
x dplyr::lag()   masks stats::lag()
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to
  ↳ become errors
```

```
library(ggthemes)

# vous devez modifier les fonctions setwd() et read.csv()
# pour correspondre à votre ordinateur (ou lire le fichier
# directement depuis le site du cours, comme ici) :
# setwd('~\Documents/exercices_methodes_2')

reponses_socio <-
  ↳ read.csv("https://julesaberlin.github.io/intro-a-R/data/reponses_socio.csv",
            encoding = "UTF-8", stringsAsFactors = TRUE)

# Contrôle, si ça a marché
str(reponses_socio)
```

```
'data.frame': 74 obs. of 10 variables:
 $ naissance : int 2003 2002 2002 2001 2001 2002 2002 2003 2003 2001 ...
 $ genre : Factor w/ 3 levels "Autre","Femme",...: 3 2 2 2 3 2 3 2 2 2
 ...
 $ permis : Factor w/ 2 levels "Non","Oui": 2 2 2 2 2 2 2 1 2 ...
 $ suisse : Factor w/ 2 levels "Non","Oui": 2 2 2 1 2 2 1 2 2 2 ...
 $ origine : Factor w/ 2 levels "Autre continent",...: 2 1 2 2 2 2 2 2 2
 2 ...
 $ fraterie : int 1 2 2 1 1 4 1 1 2 1 ...
 $ domicile : Factor w/ 7 levels "Autre canton",...: 3 6 1 3 5 3 3 7 3 3
 ...
 $ trajet : int 35 90 10 5 15 30 15 10 20 15 ...
 $ note_ecole: num 4.8 4.4 NA 5 4.3 6 4.2 5.3 4.5 5 ...
 $ note_uni : num 5 4 4 5.5 4.7 4.9 5.3 4.8 4.7 5.2 ...
```

```
# Une formidable fonction pour analyser les stats
# descriptives : summary()
summary(reponses_socio)
```

naissance	genre	permis	suisse	origine
Min. :1996	Autre: 6	Non:21	Non:10	Autre continent: 8
1st Qu.:2001	Femme:51	Oui:52	Oui:64	Europe :66
Median :2002	Homme:17	NAs: 1		
Mean :2002				
3rd Qu.:2003				
Max. :2005				

fraterie	domicile	trajet	note_ecole
Min. :0.000	Autre canton: 5	Min. : 5.00	Min. :4.200
1st Qu.:1.000	Berne : 3	1st Qu.:10.00	1st Qu.:4.500

Median :1.000	Fribourg :39	Median :20.00	Median :4.800
Mean :1.811	Genève : 3	Mean :31.89	Mean :4.811
3rd Qu.:2.000	Tessin : 4	3rd Qu.:56.25	3rd Qu.:5.000
Max. :5.000	Valais : 3	Max. :90.00	Max. :6.000
	Vaud :17		NAs :4


```

note_uni
Min. :4.000
1st Qu.:4.700
Median :4.900
Mean :4.862
3rd Qu.:5.200
Max. :5.500

```

```

# Abordons maintenant quelques questions plus spécifiques
# auxquelles nous pouvons répondre à l'aide de statistiques
# descriptives

```

```

# 1. Trajet pour arriver à l'université

```

```

# *****

```

```

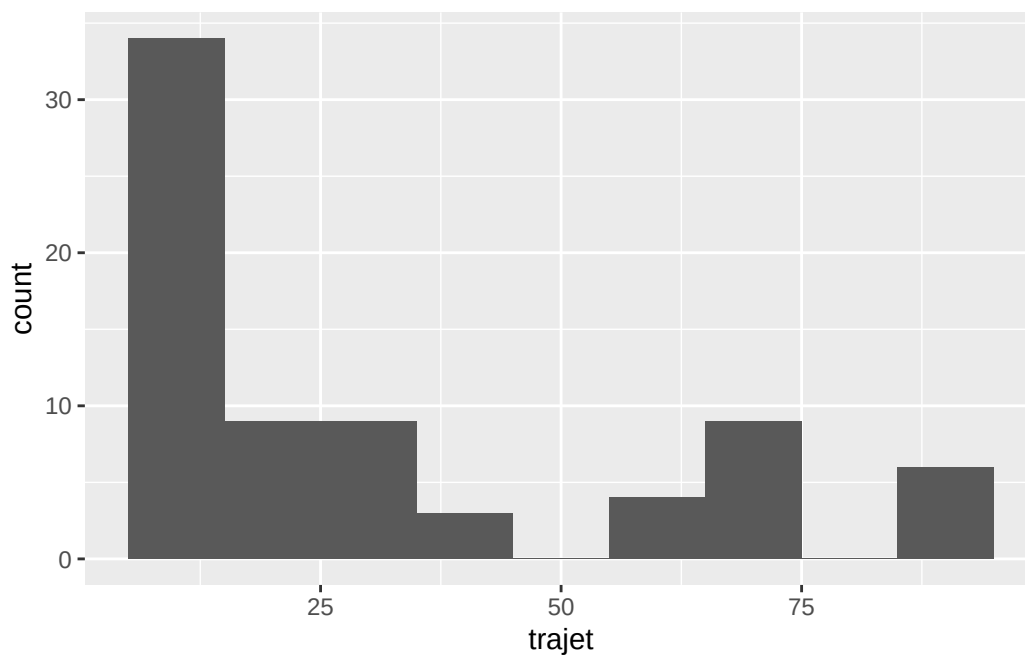
# histogramme très basique

```

```

ggplot(data = reponses_socio, mapping = aes(x = trajet)) + geom_histogram(binwidth = 10)

```



```

# plus sophistiqué

```

```

ggplot(data = reponses_socio, mapping = aes(x = trajet)) + geom_histogram(binwidth = 10,
  fill = "steelblue", color = "black") + labs(title = "Trajets en minutes des étudiant·es
  ↪ pour arriver à l'UniFR",

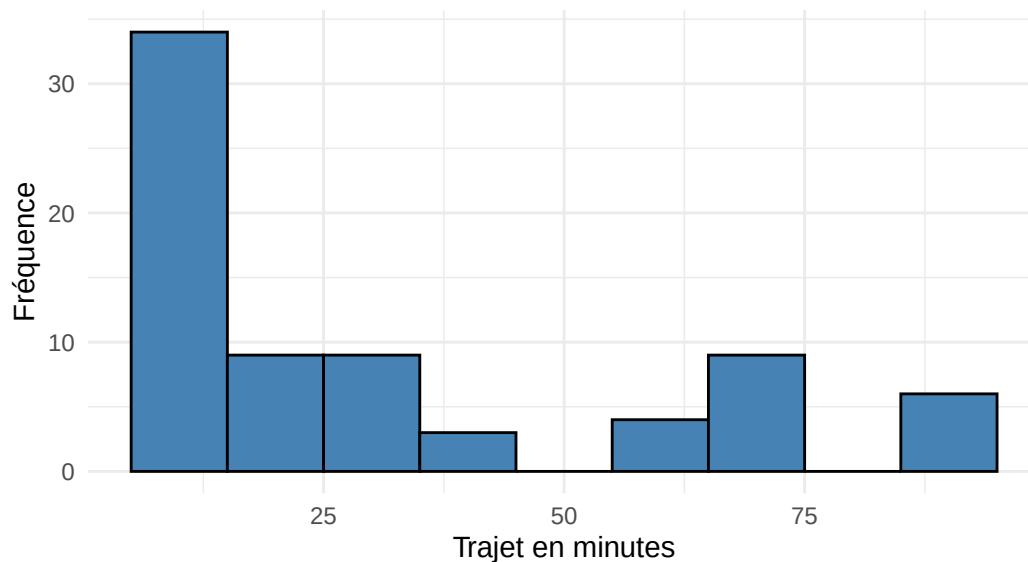
```

```

subtitle = "Questionnaire avec n = 74 participant·es", x = "Trajet en minutes",
y = "Fréquence") + theme_minimal()

```

Trajets en minutes des étudiant·es pour arriver à l'UniFR
Questionnaire avec n = 74 participant·es



```

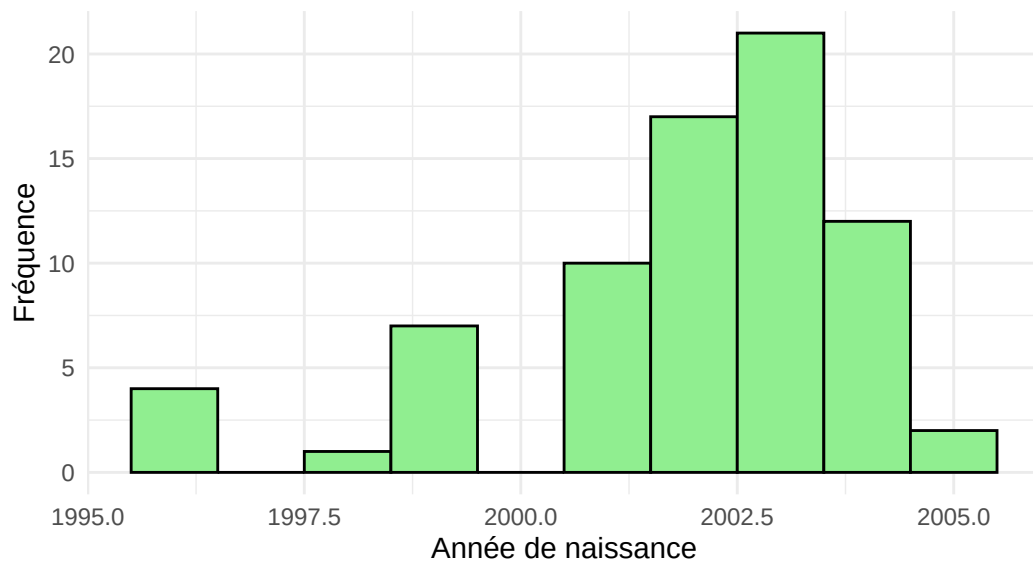
# 2. histogramme des années de naissance
# *****

ggplot(data = reponses_socio, mapping = aes(x = naissance)) +
  geom_histogram(binwidth = 1, fill = "lightgreen", color = "black") +
  labs(title = "Années de naissance", subtitle = "n = 74 répondants à un questionnaire",
        x = "Année de naissance", y = "Fréquence") + theme_minimal()

```

Années de naissance

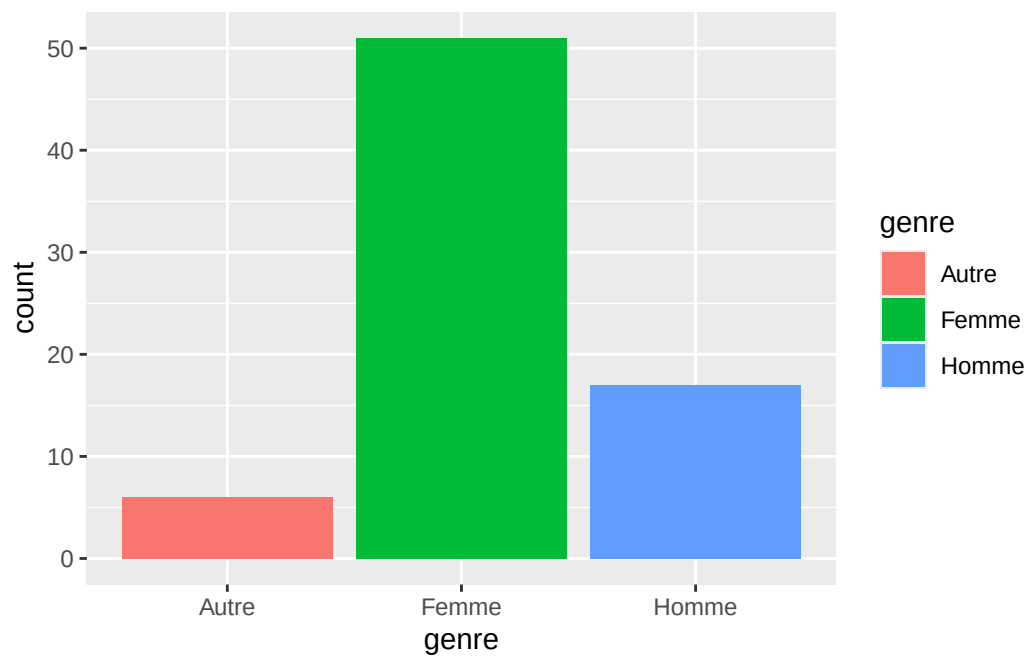
n = 74 répondants à un questionnaire



```
# 3. diagramme à barres des genres
```

```
# *****
```

```
ggplot(data = reponses_socio, mapping = aes(x = genre, fill = genre)) +  
  geom_bar()
```



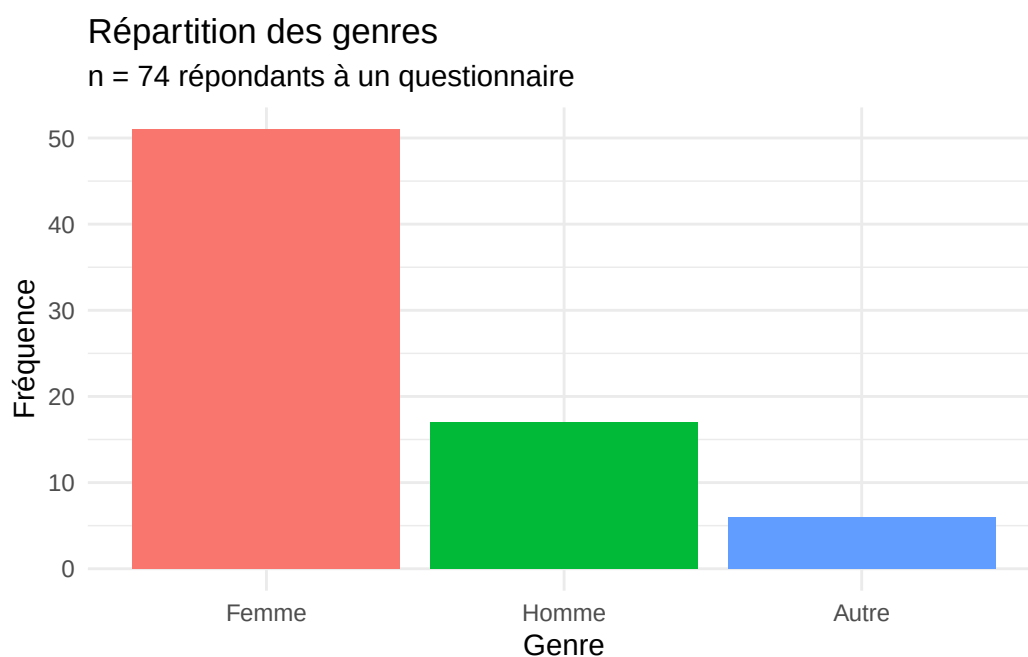
```
# graphique plus sophistiquée

# changer d'abord l'ordre des catégories (commençant avec
# la plus fréquente)

ordre = c("Femme", "Homme", "Autre")

# Puis combiner dplyr avec ggplot2 !
reponses_socio %>%
  select(genre) %>%
  mutate(genre = factor(genre, levels = ordre)) %>%
  arrange(genre) %>%
  ggplot(aes(x = genre, fill = genre)) + geom_bar(show.legend = FALSE) +
  labs(title = "Répartition des genres", subtitle = "n = 74 répondants à un
  ↪ questionnaire",
       x = "Genre", y = "Fréquence", legend = "") + theme_minimal()
```

Ignoring unknown labels:
 * legend : ""



```
# 4. Moyennes des notes
# *****

mean(reponses_socio$note_ecole, na.rm = TRUE)
```

```
[1] 4.811429
```

```
mean(reponses_socio$note_uni, na.rm = TRUE)
```

```
[1] 4.862162
```

```
min(reponses_socio$note_uni, na.rm = TRUE)
```

```
[1] 4
```

```
min(reponses_socio$note_ecole, na.rm = TRUE)
```

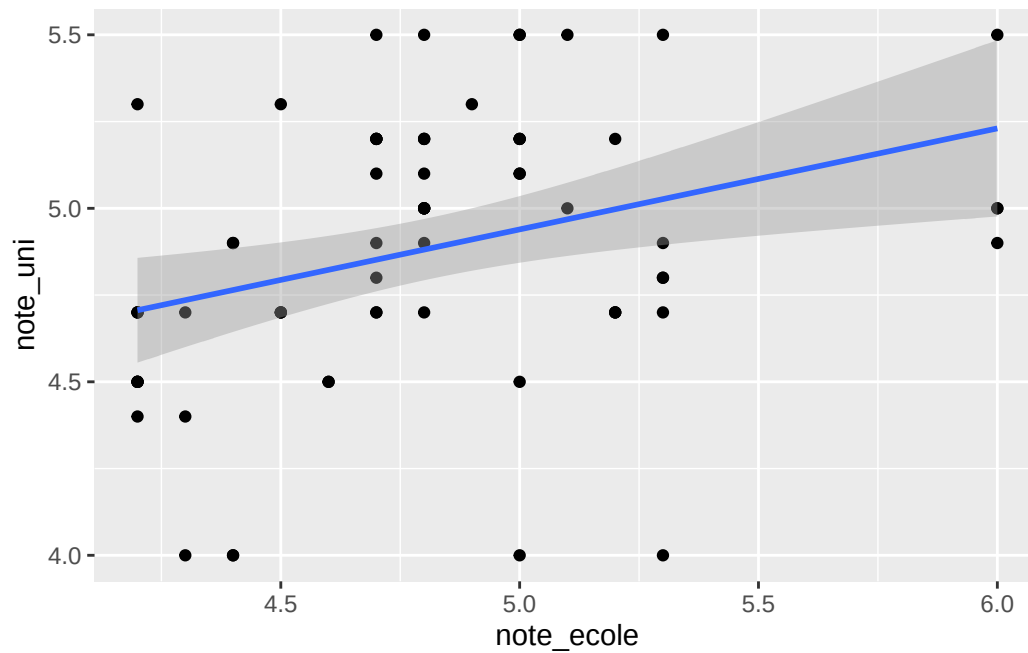
```
[1] 4.2
```

```
# On exclut les réponses sans note (valeurs manquantes, NA)
# ; sans ce filtre, mean() renverrait NA pour les groupes
# concernés
reponses_socio %>%
  select(note_ecole, note_uni, genre) %>%
  group_by(genre) %>%
  filter(!is.na(note_ecole), !is.na(note_uni)) %>%
  summarize(n = n(), note_ecole_moyenne = mean(note_ecole),
            note_uni_moyenne = mean(note_uni))
```

```
# A tibble: 3 x 4
  genre      n note_ecole_moyenne note_uni_moyenne
<fct> <int>          <dbl>          <dbl>
1 Autre      4           4.35           4.68
2 Femme     50           4.94           4.94
3 Homme     16           4.52           4.75
```

```
# Visualisation de la relation entre note_ecole et note_uni
# (la note d'école vient avant la note à l'université :
# elle va sur l'axe x)
reponses_socio %>%
  select(note_ecole, note_uni) %>%
  filter(!is.na(note_ecole), !is.na(note_uni)) %>%
  ggplot(aes(x = note_ecole, y = note_uni)) + geom_point() +
  geom_smooth(method = "lm")
```

```
`geom_smooth()` using formula = 'y ~ x'
```



```
# 5. Permis de conduire
# *****

table(reponses_socio$permis)
```

```
Non Oui
21  52
```

```
table(reponses_socio$suisse)
```

```
Non Oui
10  64
```

```
reponses_socio %>%
  select(permis, suisse) %>%
  filter(!is.na(permis)) %>%
  group_by(suisse, permis) %>%
  summarize(n = n())
```

```
`summarise()` has regrouped the output.
i Summaries were computed grouped by suisse and permis.
i Output is grouped by suisse.
i Use `summarise(.groups = "drop_last")` to silence this message.
i Use `summarise(.by = c(suisse, permis))` for per-operation grouping
  (`?dplyr::dplyr_by`) instead.
```

```
# A tibble: 4 x 3
# Groups:   suisse [2]
  suisse permis    n
  <fct> <fct> <int>
1 Non   Non     2
2 Non   Oui     8
3 Oui   Non    19
4 Oui   Oui    44
```

10.2 La fonction write.csv()

Avec R, vous ne pouvez pas seulement lire des données au format CSV (entrée de données ; avec des fonctions telles que `read.csv()`), mais vous pouvez également créer des fichiers CSV à partir de votre session en cours (sortie de données), notamment avec la fonction `write.csv()` !

La syntaxe de la fonction nécessite comme arguments le nom du data frame que vous voulez sauvegarder comme CSV ainsi que le chemin du fichier sur votre ordinateur qui inclut le nom que vous voulez donner au fichier CSV : `write.csv(nom_df, "chemin_du_fichier/nom_du_fichier.csv")`

```
# Voici un exemple avec du code (recréons le tableau avec
# les animaux de Ice Age !)

name <- c("Manny", "Sid", "Diego", "Ellie", "Peaches", "Scrat",
          "Rudy", "Buck")

species <- c("mammoth", "sloth", "sabertooth", "mammoth", "mammoth",
            "squirrel", "dinosaur", "weasel")

female <- c(FALSE, FALSE, FALSE, TRUE, TRUE, FALSE, FALSE, FALSE)

age <- c(30, 20, 35, 25, 7, 4, 100, 28)

weight <- c(5000, 80, 400, 4000, 1000, 1, 10000, 3)

ice_age_df <- data.frame(name, species, female, age, weight)

print(ice_age_df)
```

	name	species	female	age	weight
1	Manny	mammoth	FALSE	30	5000
2	Sid	sloth	FALSE	20	80
3	Diego	sabertooth	FALSE	35	400
4	Ellie	mammoth	TRUE	25	4000
5	Peaches	mammoth	TRUE	7	1000
6	Scrat	squirrel	FALSE	4	1
7	Rudy	dinosaur	FALSE	100	10000
8	Buck	weasel	FALSE	28	3

```
# Maintenant, notre but est de créer un fichier CSV qui  
# contient les données du data frame ice_age_df
```

```
# Il faut adapter le chemin du fichier suivant pour qu'il  
# marche sur votre ordinateur :
```

```
write.csv(ice_age_df, file = "~/Documents/exercices_methodes_2/ice_age_df.csv")
```

Si tout a bien marché, vous devrez maintenant avoir un fichier CSV, nommé `ice_age_df.csv` dans votre répertoire de travail!

Exercices

Notez vos réponses, puis comparez-les avec l'annexe « Corrigés » à la fin du livre.

Question 10.1

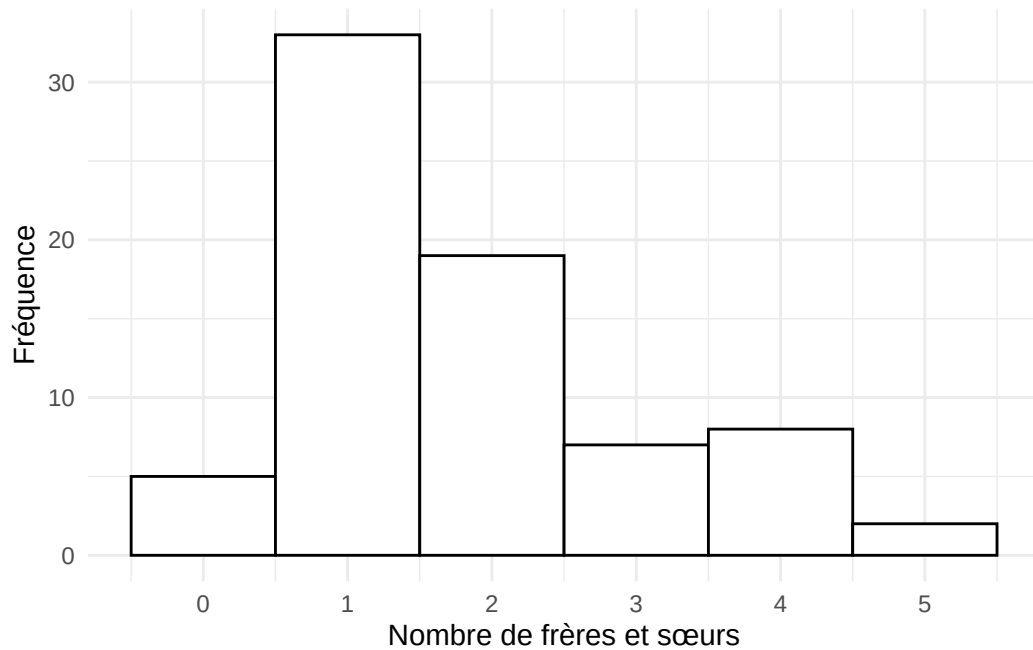
Jugez chaque affirmation concernant les **statistiques descriptives** :

- a) L'objectif des statistiques descriptives est principalement d'analyser les relations causales entre les variables.
- b) La partie descriptive d'une analyse statistique fournit généralement des indications telles que la taille de l'échantillon (n), les fréquences des variables catégorielles et les moyennes des variables numériques.
- c) Les tests d'hypothèse, par exemple avec un test t , sont une étape courante des statistiques descriptives.
- d) Les statistiques descriptives précèdent souvent les statistiques inférentielles dans une analyse de données quantitatives en sciences sociales.
- e) Certaines visualisations de données, comme les histogrammes, se prêtent bien à la présentation de statistiques descriptives.

Pour chaque proposition : vrai ou faux ?

Question 10.2

Jugez chaque proposition concernant l'histogramme suivant (source des données : questionnaire du cours) :



- a) L'histogramme montre que le nombre de frères et sœurs (variable `fraterie`) suit une loi normale.
- b) L'histogramme montre que la plupart des étudiant·es de la classe ont grandi dans des constellations familiales traditionnelles.
- c) L'histogramme montre qu'il y a plus d'étudiant·es qui ont 3 frères et sœurs que d'étudiant·es qui sont enfants uniques.
- d) La constellation la plus fréquente est celle des étudiant·es ayant un seul frère ou une seule sœur.
- e) Une personne ayant participé au questionnaire a indiqué qu'elle avait 7 frères et sœurs.

Pour chaque proposition : vrai ou faux ?

Question 10.3

Importez le jeu de données `reponses_socio.csv` dans RStudio. Combien d'**observations** contient-il ?

Réponse : _____

Question 10.4

Importez le jeu de données `reponses_socio.csv` dans RStudio. Combien de **variables** contient-il ?

Réponse : _____

Question 10.5

Importez le jeu de données `reponses_socio.csv` dans RStudio. Pour la variable **genre** : combien de **femmes** y a-t-il dans ce jeu de données ?

Réponse : _____

Question 10.6

Importez le jeu de données `reponses_socio.csv` dans RStudio. Pour la variable **genre** : combien d'**hommes** y a-t-il dans ce jeu de données ?

Réponse : _____

Question 10.7

Dans le jeu de données `reponses_socio.csv`, combien de minutes les étudiant·es **possédant un passeport suisse** mettent-ils en moyenne pour se rendre à l'université ? Arrondissez à la minute.

Réponse : _____

Question 10.8

Dans le jeu de données `reponses_socio.csv`, quel **genre** a le **trajet moyen** le plus court pour se rendre à l'université ? (Tapez le genre exactement comme il figure dans les données.)

Réponse : _____

Question 10.9

En général, je peux enregistrer les données issues de ma session R sur mon ordinateur (par exemple, un data frame que j'ai créé) sous la forme de fichiers CSV.

Vrai ou faux ?

Question 10.10

Le code suivant fournit des statistiques descriptives du jeu de données `mtcars` :

```
data(mtcars)
summary(mtcars)
```

Vrai ou faux ?

Question 10.11

Dans le jeu de données `reponses_socio.csv`, en moyenne, les étudiant·es **sans** passeport suisse ont besoin de moins de temps pour se rendre à l'université que leurs camarades suisses.

Vrai ou faux ?

Question 10.12

Les statistiques descriptives révèlent souvent des informations et des tendances intéressantes dans les données, même avant l'application de statistiques inférentielles.

Vrai ou faux ?

11 Statistiques inférentielles I : le test t

Sujet : La statistique inférentielle, Partie 1 (tests t)

Dans cette et la prochaine session, nous allons nous concentrer sur trois méthodes de statistiques inférentielles couramment utilisées :

- Les **tests t** (angl. Student's t-test)
- l'**analyse de variance** (angl. **ANOVA** = ANalysis Of VAriance)
- La **régression linéaire** (angl. linear regression).

Mais d'abord, il faut répondre à la question suivante :

11.1 Qu'est-ce que la statistique inférentielle ?

i À noter

Les statistiques inférentielles sont une branche des statistiques qui permet de tirer des conclusions générales sur une population entière en se basant sur l'analyse statistique d'un échantillon représentatif de cette population.

En sciences sociales, les statistiques inférentielles sont souvent utilisées pour évaluer si une différence ou une association observée dans l'échantillon est susceptible d'exister aussi dans la population. Nous voulons savoir, par exemple, si les notes de maturité (variable y) diffèrent selon le genre (variable x). Une relation **causale** ne peut être conclue que si le dispositif le permet (p. ex. une expérience).

En général, pour analyser de telles relations, nous formulons souvent des hypothèses basées sur des théories ou des suppositions tirées de la littérature d'un domaine de recherche, que nous pouvons ensuite tester sur notre échantillon. Les résultats obtenus nous permettent de rejeter ou non l'hypothèse nulle, et de généraliser prudemment les conclusions à la population entière. Ceci nous permet de contribuer à la création de nouvelles connaissances dans notre domaine de recherche

11.2 Le test t

À noter

Le test t est un outil statistique qui permet d'évaluer les moyennes d'un ou deux échantillons à l'aide d'un test d'hypothèse. Il est aussi appelé test t de Student, car le statisticien qui l'a inventé au début du 20ème siècle, William Sealy Gosset, a publié sous le pseudonyme de "Student" ([Wikipédia, 2024](#)).

Attention

Pour effectuer un test t, il y a plusieurs hypothèses qui doivent être satisfaites :

1. **Les données sont continues**
2. **L'échantillonnage est aléatoire**
 - Les données de chaque échantillon ont été prélevées au hasard dans une population.
3. **Indépendance**
 - Les observations doivent être indépendantes les unes des autres. Cela signifie que la valeur d'une observation ne doit pas dépendre de la valeur d'une autre observation.
 - Pour les tests t à deux échantillons, les échantillons doivent être indépendants. Si les échantillons ne sont pas indépendants, un test t pour échantillons appariés peut être approprié.
4. **Normalité**
 - Ce qui compte, c'est que la **moyenne** de l'échantillon suive approximativement une loi normale. C'est le cas si la variable étudiée est à peu près normale dans chaque groupe, ou si les échantillons sont assez grands (théorème central limite).
 - Cette hypothèse est donc surtout importante pour les **petits** échantillons. Vérifiez-la plutôt visuellement (histogramme, [graphique Q-Q](#) avec `qqnorm()`) que par un test : un test de normalité comme Shapiro-Wilk (`shapiro.test()`) manque de puissance avec peu de données et signale des écarts sans importance avec beaucoup de données.
5. **Homogénéité des variances (homoscédasticité)**, uniquement pour le test t « classique » de Student (`var.equal = TRUE`)
 - Les variances des données dans chaque groupe sont similaires.
 - La variante de Welch, utilisée par défaut dans R, n'exige pas cette hypothèse.
 - Cette hypothèse peut être vérifiée à l'aide [du test de Levene](#) ou [du test de Bartlett](#).

En pratique, les tests t sont relativement robustes aux déviations de ces hypothèses, surtout si le nombre d'observations (n) de chaque échantillon est important.

Néanmoins, si une ou plusieurs de ces hypothèses n'est pas satisfaite, alors des alternatives non paramétriques au test t peuvent être utilisées : [le test de Wilcoxon-Mann-Whitney](#) (`wilcox.test()`) pour deux groupes indépendants, ou [le test des rangs signés](#)

de Wilcoxon (`wilcox.test(..., paired = TRUE)`) pour un échantillon ou des données appariées. (Pour plus de deux groupes, l'équivalent non paramétrique de l'ANOVA est le test de **Kruskal-Wallis**, `kruskal.test()`.)

Dans R, le test t est effectué avec la fonction `t.test()`.

Il existe **trois types** de tests t :

1. **Le test t à un échantillon** (angl. *one sample t-test*) : pour évaluer si la moyenne d'un seul échantillon diffère de manière significative d'une valeur connue.
 - Par exemple, si la moyenne des notes d'un groupe de 30 lycéens, qui est de 4.9, est significativement différente de 5.0 ou non.
2. **Le test t à deux échantillons indépendants** (angl. *two sample t-test*) : pour évaluer si les moyennes de deux échantillons indépendants diffèrent de manière significative. Par défaut, R effectue la variante de **Welch** (*Welch two sample t-test*), qui n'exige pas que les variances soient égales (`var.equal = FALSE`). Le test t « classique » de Student (`var.equal = TRUE`) suppose des variances égales.
 - Par exemple, si la moyenne des notes de 30 lycéennes, qui est de 5.0, est significativement différente ou non de la moyenne des notes de 30 lycéens, qui est de 4.8.
3. **Le test t pour échantillons appariés** (angl. *paired t-test*) : pour comparer les moyennes de deux groupes qui sont liés ou appariés d'une certaine manière. Dans ce type de test, chaque participant est mesuré deux fois, une fois dans chaque groupe, ce qui permet de réduire les effets des variations individuelles.
 - Par exemple, nous pourrions enregistrer la moyenne des notes d'une classe de 30 élèves avant et après un cours de soutien scolaire en ligne. Il y a donc deux mesures pour chaque élève. Le cours de soutien scolaire est dans ce cas un traitement.
 - (à noter : nous n'allons pas traiter ce type de test t plus en détail dans l'exercice).

Voyons maintenant des exemples avec du code !

11.2.1 Exemple 1 : test t à un échantillon

Problème : Tester si la moyenne de QI d'un échantillon d'étudiant·es, qui est de 105, est différente de manière significative de la moyenne de la population, qui est de 100.

- L'**hypothèse nulle H0** (angl. *null hypothesis*) : la moyenne de QI des étudiant·es dans la population est égale à 100 ($\mu = 100$).
- L'**hypothèse alternative H1** (angl. *alternative hypothesis*) : elle est différente de 100 ($\mu \neq 100$).

Les hypothèses portent sur la **population**, pas sur le résultat du test. Le test ci-dessous est bilatéral (il cherche un écart dans les deux sens). Pour tester seulement si le QI est *plus élevé*, on ajouterait `alternative = "greater"` dans `t.test()`.

À noter : dans le code qui suit, nous allons encore une fois utiliser la fonction `rnorm()` que nous avons vue au chapitre 5 pour créer des valeurs “aléatoires” qui suivent une loi normale. Nous pouvons utiliser ces valeurs pour simuler notre problème.

```
# Générer des données d'échantillon avec n = 25
# observations hypothétiques de QI d'étudiant·es. Nous
# simulons, avec rnorm() 25 observations qui suivent une
# loi normale avec une moyenne de 105 et un écart-type de
# 15
set.seed(123)
scores_qi <- rnorm(25, mean = 105, sd = 15)

# Test t à un échantillon où il faut 2 arguments dans la
# fonction t.test() : (1) les données à tester (scores_qi)
# et (2) la valeur à laquelle on veut comparer les données
# (mu = 100)
resultat_test_t <- t.test(scores_qi, mu = 100)
print(resultat_test_t)
```

One Sample t-test

```
data: scores_qi
t = 1.5844, df = 24, p-value = 0.1262
alternative hypothesis: true mean is not equal to 100
95 percent confidence interval:
 98.63817 110.36192
sample estimates:
mean of x
 104.5
```

Il s'agit maintenant d'interpréter le résultat du test t dans R.

- **La statistique t** quantifie la différence entre les moyennes des groupes par rapport à la variabilité de l'échantillon. Le signe de la statistique t indique si la moyenne de l'échantillon est supérieure (t positif) ou inférieure (t négatif) à la moyenne de la comparaison. L'ampleur reflète combien d'erreurs standard la différence est par rapport à zéro. En général, des valeurs absolues plus élevées suggèrent des preuves plus solides contre l'hypothèse nulle, ce qui se traduit par des valeurs p plus faibles.
- **df** signifie “degrees of freedom”. Dans un test t, les degrés de liberté font référence au nombre de valeurs dans le calcul final d'une statistique qui sont libres de varier. Il s'agit d'une mesure de la quantité d'informations disponibles pour estimer le paramètre statistique.
- La **valeur p** (angl. p-value) est la probabilité d'observer un résultat aussi extrême que celui obtenu dans l'échantillon sous l'hypothèse nulle. Elle est utilisée pour déterminer si le résultat du test est statistiquement significatif en comparant sa valeur à un seuil de signification prédéfini (habituellement 5% = 0.05). La p-value est calculée à partir de la statistique de test t et des degrés de liberté associés à l'échantillon.

La valeur p indique si l'écart observé est compatible avec H_0 . Mais regardez aussi la **moyenne estimée** et l'**intervalle de confiance à 95 %** affichés par `t.test()` : ils indiquent l'ampleur de la différence et sa précision.

Dans notre cas, la valeur p (0,13) est supérieure au seuil $\alpha = 0,05$: **nous ne rejetons pas H_0** . Les données sont compatibles avec une moyenne de 100 ; cela ne prouve pas que la moyenne *est* 100. L'intervalle de confiance à 95 % (de 98,6 à 110,4), qui contient 100, le montre aussi. Avec seulement 25 observations, le test manque peut-être simplement de puissance.

```
# Regénérons encore une fois des données d'échantillon,  
# mais cette fois-ci avec n = 50 observations  
set.seed(123)  
scores_qi <- rnorm(50, mean = 105, sd = 15)  
  
# Test t à un échantillon  
resultat_test_t <- t.test(scores_qi, mu = 100)  
print(resultat_test_t)
```

One Sample t-test

```
data: scores_qi  
t = 2.8085, df = 49, p-value = 0.007129  
alternative hypothesis: true mean is not equal to 100  
95 percent confidence interval:  
 101.5691 109.4630  
sample estimates:  
mean of x  
 105.5161
```

La valeur p est maintenant beaucoup plus petite. Si la vraie moyenne était 100, obtenir un écart aussi grand ou plus grand serait très improbable ($p = 0,007$). **Nous rejetons H_0** au seuil de 5 %. La moyenne estimée (105,5) est supérieure à 100.

À retenir : pour une même différence réelle, un **échantillon plus grand** donne au test plus de puissance.

11.2.2 Exercice 1

Selon l'étude [World Internet Project de l'Université de Zurich](#) (Latzner et al., 2023), qui a évalué les données pour la Suisse, les personnes en Suisse ont passé en moyenne 5.6 heures en ligne en 2023. Chez les 20-29 ans, ce chiffre s'élève même à 7.9 heures en moyenne.

Réalisez un test T en utilisant ces statistiques comme suit :

- Simulez avec `rnorm()` 30 valeurs pour l'utilisation d'Internet en heures par jour pour le groupe d'âge 20-29 ans (`mean = 7.9, sd = 2`).

- Testez si la durée moyenne d'utilisation d'Internet en heures de votre échantillon simulé de 30 suisses de 20 à 29 ans se distingue significativement de la moyenne nationale ($\mu = 5.6$).

Solution

```
# Générons des données d'échantillon hypothétique avec n =  
# 30 observations  
set.seed(123)  
  
usage_20_29 <- rnorm(30, mean = 7.9, sd = 2)  
  
# Test t à un échantillon  
resultat_test_t <- t.test(usage_20_29, mu = 5.6)  
print(resultat_test_t)
```

One Sample t-test

```
data: usage_20_29  
t = 6.1576, df = 29, p-value = 1.035e-06  
alternative hypothesis: true mean is not equal to 5.6  
95 percent confidence interval:  
 7.073147 8.538438  
sample estimates:  
mean of x  
 7.805792
```

La valeur p est très faible : des données comme celles-ci seraient très improbables si la moyenne des 20-29 ans était de 5,6 heures. Nous rejetons H_0 . La moyenne estimée de l'échantillon (7,8 heures) est supérieure à la moyenne nationale.
(Les données étant simulées avec une moyenne de 7,9 heures, ce résultat était attendu.)

11.2.3 Exemple 2 : test t à deux échantillons

Utilisons encore une fois la fonction `rnorm()`, mais maintenant pour simuler deux échantillons hypothétiques :

- Un échantillon de 5 tailles de femmes suisses
- Un échantillon de 5 tailles d'hommes suisses

D'après l'Enquête suisse sur la santé 2017 ([Staub et al., 2022](#)), que nous avons déjà vue en session 5, les femmes mesurent en moyenne 164,6 cm (écart-type 6,5 cm) et les hommes 177,0 cm (écart-type 7,1 cm). Dans la suite, nous utilisons des valeurs proches : une moyenne de 164.7 cm pour les femmes et de 177.4 cm pour les hommes, avec des écarts-types un peu plus petits de 5.6 cm et de 6.1 cm.

```
# Données exemplaires :
set.seed(123)
tailles_f <- rnorm(5, mean = 164.7, sd = 5.6)
tailles_h <- rnorm(5, mean = 177.4, sd = 6.1)

# Two-sample t-test
t_test_two_sample <- t.test(tailles_f, tailles_h)
print(t_test_two_sample)
```

Welch Two Sample t-test

```
data: tailles_f and tailles_h
t = -3.0109, df = 6.8056, p-value = 0.02031
alternative hypothesis: true difference in means is not equal to 0
95 percent confidence interval:
 -20.308039 -2.383283
sample estimates:
mean of x mean of y
 165.7840  177.1297
```

Avec `set.seed(123)`, la différence est significative ($p \approx 0,02$) même avec $n = 5$, car l'écart entre hommes et femmes (environ 11 cm) est grand par rapport à la variabilité. Essayez d'autres graines : avec `set.seed(8)`, par exemple, la différence n'est plus significative ($p \approx 0,15$). Avec des échantillons aussi petits, le résultat varie d'une simulation à l'autre. Avec $n = 20$ ou $n = 100$, le test détecte la différence de manière beaucoup plus fiable.

11.2.4 Exercice 2

Supposons maintenant qu'un avion en provenance d'Amérique atterrisse à Genève-Cointrin. Il transporte une sélection de 24 joueuses de la Women's NBA, la ligue professionnelle féminine de basket-ball aux États-Unis. Les joueuses de la WNBA mesurent en moyenne environ 1,83 à 1,85 m ; nous utilisons ici une valeur approximative de 184.5 cm.

Testez si la taille moyenne de ces 24 joueuses diffère significativement de celle de 24 hommes suisses tirés au hasard. (Supposons que l'écart-type des joueuses est de 6.5 cm.)

Solution

```
# Générons des données pour deux échantillons hypothétique
# avec n = 24 observations chacun
set.seed(123)
tailles_f_WNBA <- rnorm(24, mean = 184.5, sd = 6.5)
tailles_h <- rnorm(24, mean = 177.4, sd = 6.1)

# Two-sample t-test
t_test_two_sample <- t.test(tailles_f_WNBA, tailles_h)
print(t_test_two_sample)
```

Welch Two Sample t-test

```
data: tailles_f_WNBA and tailles_h
t = 3.8971, df = 45.668, p-value = 0.0003162
alternative hypothesis: true difference in means is not equal to 0
95 percent confidence interval:
 3.253468 10.207582
sample estimates:
mean of x mean of y
184.4436 177.7131
```

La valeur p est très faible ($p < 0,001$) : nous rejetons H_0 . Dans ces données, les joueuses sont en moyenne plus grandes que les hommes suisses (environ 184,4 cm contre 177,7 cm).

À noter : la fonction `t.test()`, comme on l'a utilisé dans ces exemples de code jusqu'à présent, utilise plusieurs paramètres par défaut, si nous ne les spécifions pas comme arguments dans la fonction. Voici quelques-uns de ces paramètres, qu'on peut aussi visualiser avec `?t.test()` :

```
t.test(x, y = NULL,
       alternative = c("two.sided", "less", "greater"),
       mu = 0, paired = FALSE, var.equal = FALSE,
       conf.level = 0.95, ...)
```

En fonction de votre problématique et de vos données, par exemple dans un travail de bachelor, il peut être nécessaire de spécifier explicitement certains de ces paramètres. Pour ce faire, il peut être utile de lire les pages d'aide de R et de rechercher d'autres documents en ligne pertinents pour votre problématique spécifique.

— Un tutoriel qui approfondit le test t dans R est par exemple [celui de Datacamp \(2024\)](#).

Exercices

Notez vos réponses, puis comparez-les avec l'annexe « Corrigés » à la fin du livre.

Question 11.1

Vous avez effectué un test t à un échantillon et obtenu une statistique t de 4.0869 avec 49 degrés de liberté et une valeur p de 0.0001618. Quelle conclusion pouvez-vous en tirer au niveau de signification de 5 % ?

- a) Ne pas rejeter l'hypothèse nulle (H_0).
- b) Rejeter l'hypothèse nulle (H_0).
- c) Il n'y a pas assez d'informations pour conclure.

- d) Augmenter la taille de l'échantillon.
- e) Aucune des autres réponses n'est correcte.

Une seule réponse est correcte.

Question 11.2

Lesquelles des hypothèses suivantes pourraient être testées dans un **test t à deux échantillons** ?

- a) Les deux échantillons proviennent de populations avec la même moyenne.
- b) Les deux échantillons proviennent de populations avec des moyennes différentes.
- c) L'une des populations est beaucoup plus nombreuse que l'autre.
- d) Les populations ont la même moyenne mais des variances très différentes.
- e) Trois groupes d'humains sont de tailles moyennes très différentes.

Pour chaque proposition : vrai ou faux ?

Question 11.3

Jugez chaque proposition relative aux **statistiques inférentielles** :

- a) Les statistiques inférentielles permettent de tirer des conclusions sur une population entière en se basant sur l'analyse d'un échantillon représentatif de cette population.
- b) Si deux variables quantitatives, par exemple y et x, sont significativement corrélées, il existe toujours une relation de cause à effet entre ces deux variables.
- c) L'échantillonnage aléatoire est une méthode de sélection au hasard d'un sous-groupe de participants dans une population, où chaque membre a une chance égale d'être choisi, offrant ainsi une représentation fiable de la population entière.
- d) Un échantillon de 100 femmes suisses d'une taille moyenne de 180 cm est représentatif de la distribution de taille des femmes en Suisse.
- e) Les tests t, l'ANOVA et la régression linéaire peuvent être des techniques utiles dans une analyse de statistiques inférentielles.

Pour chaque proposition : vrai ou faux ?

Question 11.4

Supposons que nous fassions un test t pour savoir si la taille moyenne de 100 joueuses de la WNBA sélectionnées au hasard diffère de celle de 100 hommes néerlandais sélectionnés au hasard. Le code suivant simule les deux échantillons et effectue le test. Exécutez-le et jugez chaque affirmation.

```
set.seed(2026)
wnba <- rnorm(100, mean = 184.5, sd = 6.5)
pays_bas <- rnorm(100, mean = 183.8, sd = 7.1)
t.test(wnba, pays_bas)
```

- a) Dans l'échantillon simulé, la joueuse de basket-ball moyenne mesure 183,86 cm.
- b) La valeur p du test indique que la différence entre les tailles moyennes est statistiquement significative au niveau de 5 %.
- c) L'hypothèse nulle du test t est que les moyennes des deux populations (joueuses de la WNBA et hommes néerlandais) sont égales.
- d) Le test t indique que les hommes néerlandais sont significativement plus grands que les joueuses de basket-ball de la WNBA.
- e) Les résultats du test t indiquent que, dans les échantillons simulés, il n'y a pas de différence significative entre les tailles moyennes des joueuses de basket et des hommes néerlandais.

Pour chaque proposition : vrai ou faux ?

Question 11.5

Que signifie une valeur p inférieure à 0,05 dans le contexte d'un test statistique ?

- a) Il y a une probabilité de 5 % que l'hypothèse nulle soit vraie.
- b) Il y a moins de 5 % de chances que les résultats observés soient dus au hasard.
- c) La probabilité que l'hypothèse nulle soit vraie est supérieure à 95 %.
- d) Les données ne suivent pas une loi normale.
- e) Toutes les autres réponses sont incorrectes.

Une seule réponse est correcte.

Question 11.6

Avec des échantillons de grande taille, même des différences (apparemment) faibles entre les moyennes de deux groupes peuvent devenir statistiquement significatives (par exemple, la taille moyenne des hommes dans deux pays différents).

Vrai ou faux ?

12 Statistiques inférentielles II : ANOVA et régression

Sujet : La statistique inférentielle, Partie 2 (ANOVA et régression linéaire)

12.1 L'analyse de la variance (ANOVA)

À noter

L'analyse de la variance (abbrégée **ANOVA**) est un ensemble de modèles statistiques et de procédures d'estimation associées utilisées pour analyser les différences entre les moyennes. L'ANOVA a été développée par le statisticien britannique [Ronald Fisher \(1890-1962\)](#).

L'ANOVA permet d'étudier le comportement d'une variable quantitative à expliquer en fonction d'une ou de plusieurs variables catégorielles.

Dans sa forme la plus simple, l'ANOVA fournit un test statistique permettant de déterminer si deux moyennes de population ou plus sont égales, et généralise donc le test t au-delà de deux moyennes. En d'autres termes, l'ANOVA est utilisée pour tester si les différences entre deux ou plusieurs moyennes d'une variable quantitative à expliquer sont significatives ou si elles peuvent être attribuées au hasard.

Le principe de l'ANOVA est basé sur la loi de la variance totale, selon laquelle la variance observée d'une variable particulière est divisée en composantes attribuables à différentes sources de variation, telles que la variation entre les groupes et la variation à l'intérieur des groupes.

En fonction du nombre de facteurs et du nombre de niveaux dans chaque facteur, il existe différents types d'ANOVA, tels que l'ANOVA à un facteur, l'ANOVA à deux facteurs, etc.

(Source : [Wikipédia, 2024](#)).

Attention

Comme pour le test t, l'ANOVA nécessite certaines hypothèses concernant les données : l'indépendance des observations ; dans chaque groupe, une distribution à peu près normale (vérifiable sur les résidus du modèle) ; des variances semblables entre les groupes (homoscédasticité).

Dans R, la **fonction** `aov()` est utilisée pour effectuer des ANOVAs à un ou plusieurs facteurs.

- Elle prend en entrée une formule qui spécifie la variable réponse et les facteurs explicatifs, ainsi qu'un data frame contenant les données. Syntaxe : `aov(formula, data)`
- La fonction `aov()` renvoie un objet de classe *aov* qui contient les résultats de l'analyse de variance, y compris
 - les sommes de carrés,
 - les degrés de liberté,
 - les valeurs de F et
 - les valeurs de p pour chaque effet du modèle.

En résumé, `aov()` est utilisée pour ajuster des modèles linéaires à effets fixes et pour effectuer des analyses de variance.

12.1.1 Exemple 1 : ANOVA avec le jeu de données `palmerpenguins`



Figure 12.1 : Logo du paquet `palmerpenguins`. Illustration : Allison Horst, [CC0](#).

Pour cet exemple, il faut télécharger le paquet R `palmerpenguins` (Horst et al., 2020). Celui-ci contient un jeu de données avec des informations sur trois espèces de manchots (Adélie, à jugulaire et papou ; en anglais Adelie, Chinstrap et Gentoo) qui vivent dans l'archipel de Palmer en Antarctique.

Les données ont été collectées entre 2007 et 2009 (Gorman et al., 2014) et comprennent des mesures telles que la longueur du bec (`bill_length_mm`), la longueur des nageoires (`flipper_length_mm`), le poids corporel (`body_mass_g`) et le sexe des manchots.

- Utilisez le code suivant pour la première installation du paquet :

```
install.packages("palmerpenguins")
```
- Après l'installation, il suffit d'activer le paquet avec `library(palmerpenguins)`

- Il est toujours recommandé de comprendre un nouveau jeu de données au niveau descriptif, utilisez donc `?palmerpenguins` et naviguez un peu sur [le site des créatrices de ce paquet R!](#)
- Le **but de notre ANOVA** est de déterminer si les différences de longueur moyenne des nageoires `flipper_length_mm` entre les trois espèces de manchots sont statistiquement significatives.
 - `flipper_length_mm` est donc notre variable quantitative à expliquer et
 - `species` est notre variable catégorielle qui indique à quelle espèce de manchots (Adelie, Chinstrap et Gentoo) chaque observation de `flipper_length_mm` appartient.

```
library(tidyverse)
```

```
-- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
v dplyr      1.2.1      v readr      2.2.0
v forcats    1.0.1      v stringr    1.6.0
v ggplot2    4.0.3      v tibble     3.3.1
v lubridate  1.9.5      v tidyr      1.3.2
v purrr      1.2.2
-- Conflicts ----- tidyverse_conflicts() --
x dplyr::filter() masks stats::filter()
x dplyr::lag()     masks stats::lag()
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to
  ↳ become errors
```

```
library(palmerpenguins)
```

```
Attaching package: 'palmerpenguins'
```

```
The following objects are masked from 'package:datasets':
```

```
  penguins, penguins_raw
```

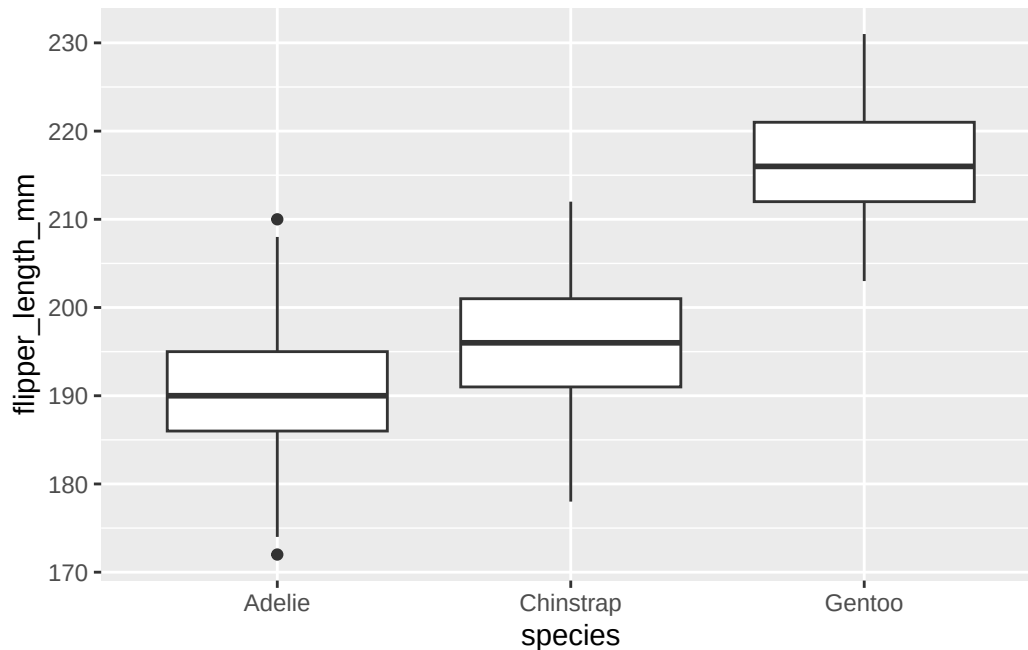
```
# Explorez d'abord un peu la structure du jeu de données
str(penguins)
```

```
tibble [344 x 8] (S3: tbl_df/tbl/data.frame)
 $ species : Factor w/ 3 levels "Adelie","Chinstrap",...: 1 1 1 1 1 1 1 1
   1 1 ...
 $ island  : Factor w/ 3 levels "Biscoe","Dream",...: 3 3 3 3 3 3 3 3
   ...
 $ bill_length_mm : num [1:344] 39.1 39.5 40.3 NA 36.7 39.3 38.9 39.2
   34.1 42 ...
 $ bill_depth_mm : num [1:344] 18.7 17.4 18 NA 19.3 20.6 17.8 19.6 18.1
   20.2 ...
 $ flipper_length_mm: int [1:344] 181 186 195 NA 193 190 181 195 193 190
   ...
 $ body_mass_g : int [1:344] 3750 3800 3250 NA 3450 3650 3625 4675 3475
   4250 ...
```

```
$ sex : Factor w/ 2 levels "female","male": 2 1 1 NA 1 2 1 2 NA NA ...
$ year : int [1:344] 2007 2007 2007 2007 2007 2007 2007 2007 2007 2007 2007
...
```

```
# Visualisez les distributions de flipper length avec des
# boxplots
ggplot(penguins, aes(x = species, y = flipper_length_mm)) + geom_boxplot()
```

Warning: Removed 2 rows containing non-finite outside the scale range (`stat\_boxplot()`).

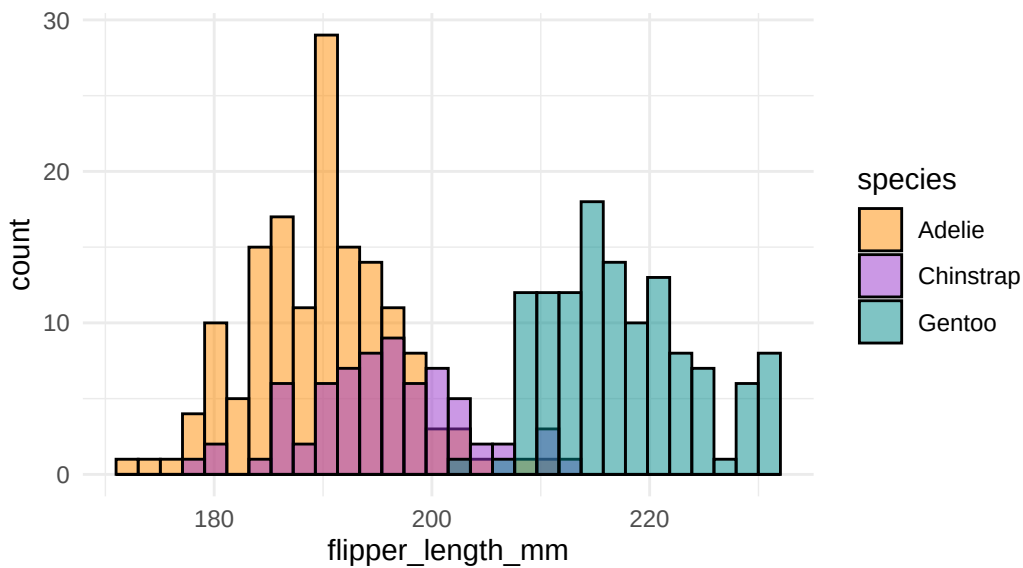


```
# Comme alternative, vous pouvez aussi créer un histogramme
# plus avancé pour comparer flipper length par espèce
ggplot(data = penguins, aes(x = flipper_length_mm)) + geom_histogram(aes(fill = species),
  color = "black", alpha = 0.5, position = "identity", bins = 30) +
  scale_fill_manual(values = c("darkorange", "darkorchid",
    "cyan4")) + theme_minimal() + labs(title = "Histograms of flipper length by penguin
    ↪ species",
  subtitle = "n = 344 ; data source : palmerpenguin R package")
```

Warning: Removed 2 rows containing non-finite outside the scale range (`stat\_bin()`).

Histograms of flipper length by penguin species

n = 344 ; data source : palmerpenguin R package



```
# Calculs des variances de flipper length avec dplyr par
# espèce de manchot, pour vérifier si l'hypothèse des
# variances similaire entre groupes est satisfaite :
penguins %>%
  group_by(species) %>%
  summarize(var_fl = var(flipper_length_mm, na.rm = TRUE))
```

```
# A tibble: 3 x 2
  species var_fl
  <fct>    <dbl>
1 Adelie  42.8
2 Chinstrap 50.9
3 Gentoo  42.1
```

```
# ANOVA avec aov()
anova_1 <- aov(flipper_length_mm ~ species, data = penguins)
summary(anova_1)
```

```
              Df Sum Sq Mean Sq F value Pr(>F)
species        2  52473    26237   594.8 <2e-16 ***
Residuals     339  14953         44
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
2 observations deleted due to missingness
```

```
# à noter, avec TukeyHSD() nous pouvons vérifier d'avantage
# les différences entre chaque paire d'espèces :
TukeyHSD(anova_1)
```

```
Tukey multiple comparisons of means
95% family-wise confidence level
```

```
Fit: aov(formula = flipper_length_mm ~ species, data = penguins)
```

```
$species
```

	diff	lwr	upr	p adj
Chinstrap-Adelie	5.869887	3.586583	8.153191	0
Gentoo-Adelie	27.233349	25.334376	29.132323	0
Gentoo-Chinstrap	21.363462	19.000841	23.726084	0

```
# Nous pouvons rejeter H0 avec une grande certitude !
```

12.2 La régression linéaire avec `lm()`

12.2.1 Explications théoriques de la régression linéaire

Dans le monde réel, nous supposons souvent qu'il existe des **relations de cause à effet**.

- Par exemple, nous pouvons supposer que les personnes ayant un niveau d'éducation élevé ont des salaires plus élevés. Il s'agit bien sûr d'une généralisation et il se peut qu'elle ne soit vraie qu'en moyenne (il se peut que certaines personnes ayant un très faible niveau d'éducation aient des salaires très élevés et vice versa). Néanmoins, malgré quelques exceptions à la règle, nous pouvons raisonnablement supposer qu'en moyenne, une telle relation de cause à effet entre le niveau d'éducation et le salaire existe.

La question suivante est donc de savoir quelle est l'ampleur d'un tel effet et s'il est possible de le quantifier.

- Est-il possible, par exemple, de prédire une augmentation moyenne du salaire en % pour chaque année d'études supplémentaire? Répondre à une telle question peut s'avérer compliqué pour différentes raisons.
- Si l'éducation est importante, un certain nombre d'autres facteurs peuvent également influencer le salaire d'une personne, tels que l'âge, le sexe, l'origine, la classe sociale, l'intelligence, les capacités de communication, la résistance au stress et peut-être même la taille, l'attrait physique et l'attitude politique. Si ces autres facteurs ont également une influence sur le salaire, comment déterminer l'impact fractionnel du niveau d'éducation?

Un autre problème, en particulier dans la pratique, est que nombre de ces autres facteurs peuvent ne pas être disponibles ou être difficiles à mesurer.

- Et même si tous ces facteurs pouvaient être observés et contrôlés, la relation entre le niveau d'éducation et le salaire sera toujours soumise à un certain degré de hasard. Peut-être y a-t-il d'autres facteurs inconnus, ou peut-être y a-t-il simplement une fluctuation inexplicable du niveau de salaire entre tous les individus, même ceux qui ont des qualifications et des profils socio-économiques très similaires.

La **régression linéaire** permet de **quantifier l'association** entre x et y , éventuellement en tenant compte d'autres variables mesurées (« toutes choses égales par ailleurs »). Elle ne prouve pas à elle seule une relation causale : si des facteurs importants ne sont pas mesurés (p. ex. les capacités ou l'origine sociale), le coefficient de l'éducation peut être biaisé.

i À noter

La régression linéaire est une méthode statistique utilisée pour modéliser la relation linéaire entre une **variable à expliquer** (y) et une ou plusieurs **variables explicatives** (x_1, x_2 , etc.).

- La variable à expliquer, y , est parfois aussi appelée **variable dépendante** ou **variable de réponse** et, en anglais, **output variable**, **response variable**, **predicted variable**
- Les variables explicatives, x , quant à eux, sont aussi appelées **variables indépendantes** ou **prédicteurs** et, en anglais, **input variables**, **predictor variables**.

Nous parlons de régression linéaire **simple**, s'il y a seulement une variable explicative x et de régression linéaire **multiple** s'il y a plus d'une variable explicative, p. ex. x_1, x_2, x_3 , etc.

En analyse de données, un modèle de régression a deux principaux types d'applications possibles :

- **Explication** : nous pouvons décrire comment les variations observées de y sont associées aux variations observées de x . Une interprétation causale demande en plus des arguments théoriques ou un dispositif adapté (p. ex. une expérience).
- Par exemple, considérons un échantillon aléatoire de données sur la taille et le poids des personnes. En moyenne, les personnes plus grandes pèsent plus que les personnes plus petites (association). Nous pourrions donc développer un modèle de régression qui décrit les variations de poids (y) en fonction des variations de taille (x).
- **Prédiction** : Si nous disposons d'un échantillon représentatif de données contenant des observations des valeurs y et x , nous pouvons adapter un modèle de régression aux données. Par la suite, nous pouvons utiliser le modèle de régression pour un nouvel ensemble de données qui ne contient que des valeurs x et prédire les valeurs y correspondantes.
- Supposons par exemple que vous soyez un agent immobilier. Vous disposez d'un ensemble de données de 200 ventes de maisons dans une ville au cours des cinq derniers mois, qui contient le prix de vente (y) et la taille de la maison en mètres carrés (x). Vous pouvez maintenant adapter un modèle de régression à vos données et l'utiliser pour prédire les prix de vente attendus des maisons actuellement sur le marché en fonction de leur superficie (x), que vous connaissez.

Notez qu'un modèle de régression linéaire, comme tout modèle, est toujours **une simplification de la réalité**. Par conséquent, la précision des modèles de régression linéaire appliqués

à des données réelles ne sera jamais de 100 %, c'est-à-dire qu'elle est sujette à une erreur d'estimation.

Comprendre (et aussi quantifier) cette déviation du modèle de régression par rapport aux valeurs observées est également une étape importante de toute analyse de régression, comme nous le verrons par la suite.

Au niveau technique, l'objectif de la régression linéaire est de spécifier une équation mathématique, appelée **modèle de régression**, qui quantifie la relation entre les variables y et x sur la base d'un ensemble d'observations.

Par exemple, dans le cas d'une régression linéaire simple avec une variable x , nous utiliserions l'équation suivante :

$$y = \beta_0 + \beta_1 x + \epsilon$$

où :

- y est la variable à expliquer
- x est la variable explicative.
- β_0 est l'ordonnée à l'origine de la droite de régression.
- β_1 est la pente de la droite de régression.
- ϵ est le terme d'erreur aléatoire

i À noter

Si cela est trop abstrait pour certaines personnes : Nous pourrions également dire que, d'un point de vue pratique, l'objectif d'une régression linéaire simple est d'ajuster une ligne droite à un nuage de points. Le modèle de régression serait la droite de régression qui modélise la relation entre les variables y et x (les observations sous la forme d'un nuage de points).

12.2.2 La fonction `lm()`

La fonction `lm()` dans R est utilisée pour ajuster un modèle de régression linéaire. Elle est couramment utilisée dans le contexte de la régression linéaire pour étudier la relation entre une variable dépendante (y) et une ou plusieurs variables indépendantes (x_1 , x_2 , etc.).

La syntaxe de base de la fonction `lm()` est la suivante : `lm(formula, data)`.

- L'argument `formula` désigne une formule qui décrit le modèle à ajuster. Par exemple, `y ~ x1 + x2` signifie que la variable dépendante y est modélisée en fonction des variables indépendantes x_1 et x_2 .
- L'argument `data` requiert l'indication du jeu de données que la régression doit utiliser en tant qu'entrée contenant les variables mentionnées dans la formule.

12.2.3 Exemple 2 : La régression linéaire à une variable avec mtcars

Voici un exemple d'utilisation de la fonction `lm()` avec le jeu de données `mtcars` : `lm(mpg ~ wt, data = mtcars)`

- Notre but est de modéliser la relation entre la variable dépendante `mpg` (miles per gallon) et la variable indépendante `wt` (weight).
- Notre **hypothèse** est qu'il existe une relation négative entre `wt` et `mpg` des véhicules dans `mtcars`. En d'autres termes, nous nous attendons à ce que les voitures plus lourdes aient tendance à parcourir moins de distance avec une unité de carburant que les voitures plus légères.
- Nous supposons qu'il s'agit d'une **relation de cause à effet**, car les lois de la physique nous apprennent que l'accélération et le déplacement d'objets plus lourds nécessitent une force et une énergie plus importantes. Par conséquent, nous supposons que les voitures plus lourdes consomment généralement plus d'énergie en forme de carburant que les voitures plus légères.

```
# Charger le jeu de données mtcars
data(mtcars)

# Ajuster un modèle de régression linéaire avec mpg comme
# variable dépendante et wt comme variable indépendante
modele_1 <- lm(mpg ~ wt, data = mtcars)

# Afficher le résumé du modèle que nous pouvons interpréter
summary(modele_1)
```

Call:

```
lm(formula = mpg ~ wt, data = mtcars)
```

Residuals:

Min	1Q	Median	3Q	Max
-4.5432	-2.3647	-0.1252	1.4096	6.8727

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	37.2851	1.8776	19.858	< 2e-16 ***
wt	-5.3445	0.5591	-9.559	1.29e-10 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 3.046 on 30 degrees of freedom

Multiple R-squared: 0.7528, Adjusted R-squared: 0.7446

F-statistic: 91.38 on 1 and 30 DF, p-value: 1.294e-10

Le résultat de la régression linéaire indique que le modèle linéaire `modele_1` prédit la variable dépendante `mpg` en fonction de la variable indépendante `wt`. Regardons le résultat plus en détail :

- Les coefficients du modèle sont présentés dans la section *Coefficients*.
- L'interception est estimée à 37,285 et le coefficient pour *wt* est estimé à -5,345. Cela signifie que pour chaque augmentation d'une unité de *wt*, la valeur prédite de *mpg* diminue de 5,345 unités en moyenne. En d'autres termes, pour chaque tranche de 1 000 livres (environ 454 kg) de plus, une voiture parcourt en moyenne **5,3 miles de moins par gallon** (elle consomme donc davantage).
- La valeur de p ($\Pr(>|t|)$) pour le coefficient de *wt* est très faible ($< 0,000000000129$), ce qui indique que le coefficient est significativement différent de zéro et que la variable *wt* a un effet significatif sur la variable dépendante *mpg*.
- La section *Residuals* indique les résidus minimaux, du premier quartile, de la médiane, du troisième quartile et maximaux. Ces valeurs peuvent être utilisées pour vérifier si les hypothèses du modèle sont satisfaites.
- Le *Residual standard error* est l'écart type résiduel, qui mesure l'écart entre les valeurs prédites et les valeurs réelles de la variable dépendante. Plus cette valeur est faible, mieux le modèle s'adapte aux données. Dans ce cas, la valeur est de 3,046.
- Le *Multiple R-squared* est le coefficient de détermination, qui mesure la proportion de la variance de la variable dépendante expliquée par la variable indépendante. Dans ce cas, le coefficient de détermination est de 0,7528 : le poids rend compte d'environ 75 % de la variance de *mpg* dans ces données (« expliquer » au sens statistique, pas causal). C'est une valeur élevée pour un modèle à une seule variable, mais ce qui est « élevé » dépend du domaine : les données physiques comme celles-ci donnent souvent des R^2 élevés, les données d'enquêtes en sciences sociales beaucoup plus rarement.
- Le *Adjusted R-squared* est une mesure de la proportion de la variance de la variable dépendante qui est expliquée par les variables indépendantes dans un modèle de régression linéaire, tout en tenant compte du nombre de variables indépendantes dans le modèle. Il est calculé en ajustant le *R-squared* en fonction du nombre de variables indépendantes dans le modèle, et est toujours inférieur ou égal au *R-squared*. Comme il pénalise les variables supplémentaires, le R^2 ajusté est un critère utile pour comparer des modèles ayant un nombre différent de variables, parmi d'autres (AIC, théorie, validation sur d'autres données).
- Enfin, la *F-statistic* est utilisée pour tester l'hypothèse nulle que le coefficient de régression est égal à zéro. Dans ce cas, la valeur de p associée à la statistique F est très faible ($< 0,0000000001294$), ce qui indique que l'hypothèse nulle peut être rejetée et que le modèle est significatif.

La vidéo YouTube suivante de StatQuest, un créateur de contenus qui explique très bien les concepts statistiques (en anglais), décrit comment interpréter les résultats d'une régression linéaire dans R (StatQuest with Josh Starmer, 2017b) :

Vidéo : *Linear Regression in R, Step-by-Step*, https://www.youtube.com/watch?v=ulcc1r_Y7M0.

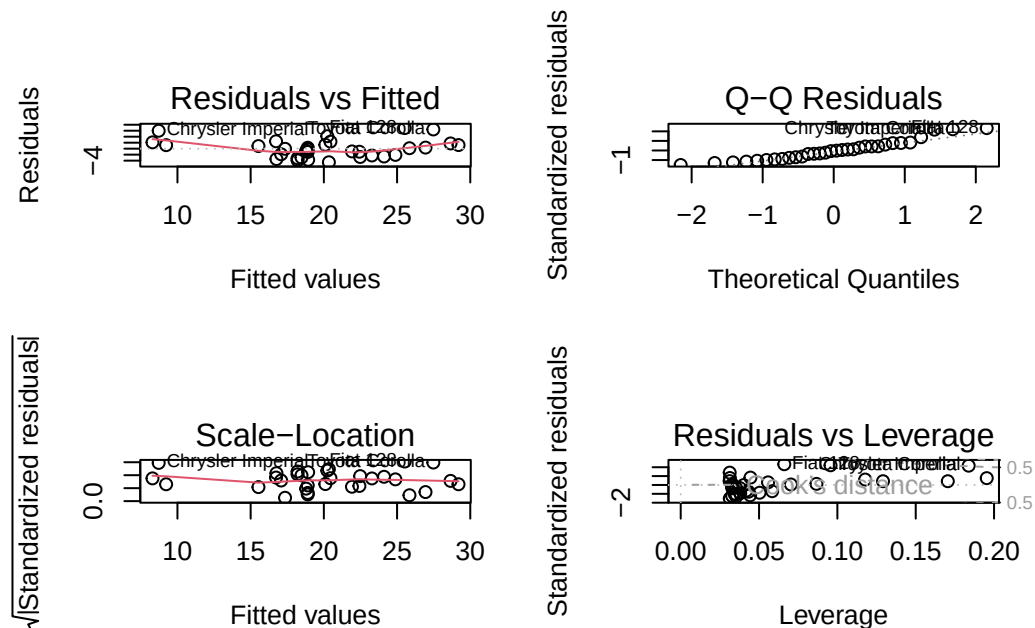
Les graphiques de diagnostic

Les graphiques de diagnostic (angl. diagnostic plots) sont des graphiques utilisés pour évaluer la qualité d'un modèle de régression linéaire et vérifier si les hypothèses du modèle sont satisfaites (p. ex. si les **résidus** suivent approximativement une loi normale).

En utilisant la fonction `plot()` après avoir calculé un modèle de régression linéaire dans R, nous pouvons visualiser quatre types de plots diagnostiques :

```
# Cette commande fait en sorte que les graphes de
# diagnostic soient tous affichés en même temps, sur un
# panneau 2 X 2 :
par(mfrow = c(2, 2))

# Afficher ensuite les graphiques de diagnostic :
plot(modele_1)
```



Explications des quatre graphiques diagnostiques :

1. **Residuals vs Fitted** : Ce graphique montre si les résidus (la distance entre les valeurs modélisées et observées) présentent des tendances non linéaires. Il peut y avoir une relation non linéaire entre les variables explicatives x et une variable expliquée y , et le schéma peut apparaître dans ce graphique si le modèle ne capture pas la relation non linéaire. Si vous trouvez des résidus plus ou moins uniformément répartis (approximés par la ligne rouge) autour de la ligne horizontale en pointillés sans tendances distinctes, c'est une bonne indication que vous n'avez pas de relations non linéaires.
2. **Q-Q Residuals** : Ce graphique montre si les résidus (les points) suivent une loi normale. Les résidus se situent-ils plus ou moins sur la ligne droite diagonale en pointillés ou s'en écartent-ils fortement ? C'est une bonne chose si les résidus sont bien alignés sur la ligne droite en pointillés.
3. **Scale-Location** : Ce graphique affiche les valeurs ajustées d'un modèle de régression sur l'axe des abscisses et la racine carrée des résidus standardisés sur l'axe des ordonnées. C'est ainsi que vous pouvez vérifier l'hypothèse de variance égale.

(homoscédasticité). C'est une bonne chose si vous voyez une ligne rouge plus ou moins horizontale avec des points répartis de manière égale (au hasard) autour d'elle.

4. **Residuals vs Leverage** : Ce graphique permet d'identifier les valeurs aberrantes influentes (valeurs extrêmes) dans les données qui peuvent affecter le modèle de régression linéaire. Toutes les valeurs aberrantes ne sont pas nécessairement influentes ; certaines peuvent ne pas avoir d'impact significatif sur le modèle de régression, ce qui signifie que les résultats de l'analyse de régression restent similaires, que ces cas soient inclus ou exclus. Cependant, certaines valeurs aberrantes peuvent avoir une grande influence et modifier les résultats si elles sont exclues, car elles ne s'alignent pas sur la tendance générale. Dans ce graphique, nous nous concentrons sur les valeurs aberrantes situées dans les coins supérieurs ou inférieurs droits, en dehors des lignes pointillées, qui indiquent des scores élevés de « distance de Cook ». Ces cas ont un impact significatif sur les résultats de la régression et doivent être notés.

En examinant ces graphiques de diagnostic, nous pouvons évaluer si les hypothèses du modèle de régression linéaire sont satisfaites, telles que l'indépendance des résidus, l'homoscédasticité, la linéarité et la normalité des résidus.

Si les graphiques de diagnostic montrent que les hypothèses ne sont pas satisfaites, cela peut indiquer que le modèle n'est pas adéquat et qu'une transformation des données ou une sélection de variables différente peut être nécessaire.

Dans le cas du modèle `modele_1` ($\text{mpg} \sim \text{wt}$), le graphique *Residuals vs Fitted* montre une légère courbure en forme de U : la relation n'est pas tout à fait linéaire (on pourrait essayer $\text{mpg} \sim \text{wt} + \text{I}(\text{wt}^2)$ ou $\log(\text{mpg})$). Quelques voitures (Fiat 128, Toyota Corolla, Chrysler Imperial) s'écartent nettement du modèle. Pour un modèle simple d'illustration, cela reste acceptable.

12.2.4 Exemple 3 : La régression linéaire multiple avec `mtcars`

Pouvons-nous encore améliorer `modele_1` qui prédit `mpg` dans le jeu de données `mtcars` ? Une possibilité évidente est d'inclure d'autres variables x , en plus de `wt`, dans notre modèle de régression pour prédire `mpg`.

En d'autres termes, nous devrions effectuer une régression linéaire multiple !

Si nous vérifions à nouveau les variables de `mtcars` (par exemple, avec `?mtcars`), une autre variable nous saute aux yeux qui pourrait avoir un impact sur `mpg` : la variable `hp`, qui indique la puissance brute en chevaux des voitures.

On peut supposer que les voitures avec plus de chevaux, c'est-à-dire avec des moteurs plus puissants, parcourent **moins de miles par gallon** que les voitures avec moins de `hp`.

```
# Ajuster un modèle de régression linéaire multiple avec
# mpg comme variable dépendante et wt et hp comme variables
# indépendantes
modele_2 <- lm(mpg ~ wt + hp, data = mtcars)
```

```
# Afficher le résumé du modèle que nous pouvons interpréter
summary(modele_2)
```

Call:

```
lm(formula = mpg ~ wt + hp, data = mtcars)
```

Residuals:

	Min	1Q	Median	3Q	Max
Residuals	-3.941	-1.600	-0.182	1.050	5.854

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	37.22727	1.59879	23.285	< 2e-16 ***
wt	-3.87783	0.63273	-6.129	1.12e-06 ***
hp	-0.03177	0.00903	-3.519	0.00145 **

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 2.593 on 29 degrees of freedom

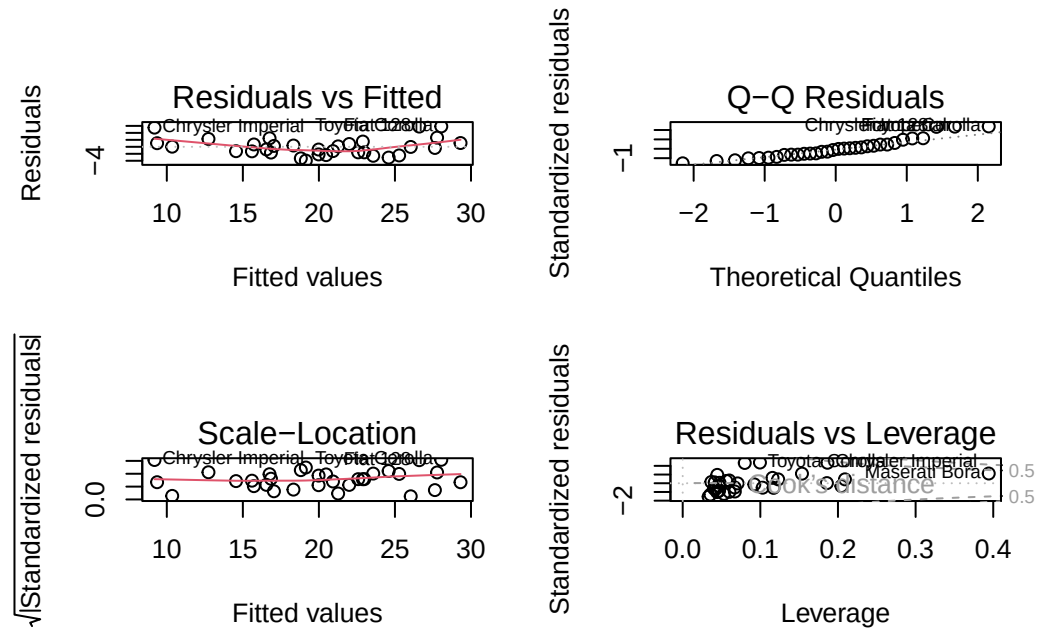
Multiple R-squared: 0.8268, Adjusted R-squared: 0.8148

F-statistic: 69.21 on 2 and 29 DF, p-value: 9.109e-12

```
# Afficher les graphiques de diagnostic
```

```
par(mfrow = c(2, 2))
```

```
plot(modele_2)
```



Les résultats montrent qu'en effet, la puissance en chevaux `hp` a également un effet négatif sur `mpg`. La valeur `p` de `hp` indique un effet significatif au niveau de 1% (mais pas au niveau de 0,1% comme pour `wt`).

Le R-carré de notre modèle a également augmenté pour atteindre 82,68% (R-carré ajusté à 81,48%), ce qui est une valeur très élevée. Cela signifie que `modele_2` peut effectivement expliquer une plus grande partie de la variabilité de `mpg` que `modele_1`.

Pour comparer deux modèles emboîtés, on utilise `anova(modele_1, modele_2)` :

```
anova(modele_1, modele_2)
```

Analysis of Variance Table

Model 1: `mpg ~ wt`

Model 2: `mpg ~ wt + hp`

	Res.Df	RSS	Df	Sum of Sq	F	Pr(>F)
1	30	278.32				
2	29	195.05	1	83.274	12.381	0.001451 **

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Ici, l'ajout de `hp` améliore significativement l'ajustement ($F = 12,4$; $p = 0,0015$). Le R^2 ajusté et l'AIC sont d'autres critères. La statistique `F` de `summary()` teste seulement si le modèle explique quelque chose; elle ne sert pas à comparer des modèles.

Une question est alors : **comment choisir le meilleur modèle de régression ?**

12.2.5 Exemple 4 : La régression par étape avec `mtcars`

Une possible solution est d'utiliser ce qu'on appelle **la régression par étape**. C'est une méthode d'ajustement des modèles de régression dans laquelle le choix des variables prédictives est effectué par une procédure automatique.

Dans R, la régression par étapes peut être effectuée à l'aide de différentes approches, notamment la fonction `step()`, qui base ses sélections sur les valeurs AIC (critère d'information d'Akaike).

- Pour notre exemple de régression par étapes, nous allons prédire `mpg` en utilisant toutes les autres variables de l'ensemble de données `mtcars`.
- À partir de ce « modèle complet », la procédure retire (et éventuellement réintroduit) des variables une à une, tant que cela fait **baisser l'AIC**.
- L'AIC équilibre l'ajustement du modèle et sa complexité : il pénalise chaque variable supplémentaire. La procédure s'arrête lorsqu'aucun changement ne fait plus baisser l'AIC.
- R propose ainsi le modèle ayant l'AIC le plus faible **parmi ceux explorés**. Ce n'est pas forcément « le » meilleur modèle.

⚠ Attention

La sélection automatique pas à pas est pratique pour explorer, mais elle est **critiquée** : elle tend à surestimer la qualité du modèle (valeurs p et R^2 trop optimistes), et le modèle choisi peut changer avec d'autres données. En recherche, on choisit plutôt les variables à partir de la théorie et de la question de recherche.

Voyons comment ça marche avec du code R !

```
# Créons le modèle complet avec toutes les variables de
# mtcars (avec ~ .)
full_model <- lm(mpg ~ ., data = mtcars)
summary(full_model)
```

Call:

```
lm(formula = mpg ~ ., data = mtcars)
```

Residuals:

	Min	1Q	Median	3Q	Max
	-3.4506	-1.6044	-0.1196	1.2193	4.6271

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	12.30337	18.71788	0.657	0.5181
cyl	-0.11144	1.04502	-0.107	0.9161
disp	0.01334	0.01786	0.747	0.4635
hp	-0.02148	0.02177	-0.987	0.3350
drat	0.78711	1.63537	0.481	0.6353
wt	-3.71530	1.89441	-1.961	0.0633 .
qsec	0.82104	0.73084	1.123	0.2739
vs	0.31776	2.10451	0.151	0.8814
am	2.52023	2.05665	1.225	0.2340
gear	0.65541	1.49326	0.439	0.6652
carb	-0.19942	0.82875	-0.241	0.8122

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 2.65 on 21 degrees of freedom

Multiple R-squared: 0.869, Adjusted R-squared: 0.8066

F-statistic: 13.93 on 10 and 21 DF, p-value: 3.793e-07

```
# Nous utilisons la fonction step(), en commençant par le
# modèle complet et en permettant à la fois la sélection en
# avant et l'élimination en arrière sur la base de l'AIC (avec
# direction = 'both')
stepwise_model <- step(full_model, direction = "both")
```

Start: AIC=70.9

mpg ~ cyl + disp + hp + drat + wt + qsec + vs + am + gear + carb

	Df	Sum of Sq	RSS	AIC
- cyl	1	0.0799	147.57	68.915
- vs	1	0.1601	147.66	68.932
- carb	1	0.4067	147.90	68.986
- gear	1	1.3531	148.85	69.190
- drat	1	1.6270	149.12	69.249
- disp	1	3.9167	151.41	69.736
- hp	1	6.8399	154.33	70.348
- qsec	1	8.8641	156.36	70.765
<none>			147.49	70.898
- am	1	10.5467	158.04	71.108
- wt	1	27.0144	174.51	74.280

Step: AIC=68.92

mpg ~ disp + hp + drat + wt + qsec + vs + am + gear + carb

	Df	Sum of Sq	RSS	AIC
- vs	1	0.2685	147.84	66.973
- carb	1	0.5201	148.09	67.028
- gear	1	1.8211	149.40	67.308
- drat	1	1.9826	149.56	67.342
- disp	1	3.9009	151.47	67.750
- hp	1	7.3632	154.94	68.473
<none>			147.57	68.915
- qsec	1	10.0933	157.67	69.032
- am	1	11.8359	159.41	69.384
+ cyl	1	0.0799	147.49	70.898
- wt	1	27.0280	174.60	72.297

Step: AIC=66.97

mpg ~ disp + hp + drat + wt + qsec + am + gear + carb

	Df	Sum of Sq	RSS	AIC
- carb	1	0.6855	148.53	65.121
- gear	1	2.1437	149.99	65.434
- drat	1	2.2139	150.06	65.449
- disp	1	3.6467	151.49	65.753
- hp	1	7.1060	154.95	66.475
<none>			147.84	66.973
- am	1	11.5694	159.41	67.384
- qsec	1	15.6830	163.53	68.200
+ vs	1	0.2685	147.57	68.915
+ cyl	1	0.1883	147.66	68.932
- wt	1	27.3799	175.22	70.410

Step: AIC=65.12

mpg ~ disp + hp + drat + wt + qsec + am + gear

	Df	Sum of Sq	RSS	AIC
- gear	1	1.565	150.09	63.457
- drat	1	1.932	150.46	63.535
<none>			148.53	65.121

- disp	1	10.110	158.64	65.229
- am	1	12.323	160.85	65.672
- hp	1	14.826	163.35	66.166
+ carb	1	0.685	147.84	66.973
+ vs	1	0.434	148.09	67.028
+ cyl	1	0.414	148.11	67.032
- qsec	1	26.408	174.94	68.358
- wt	1	69.127	217.66	75.350

Step: AIC=63.46

mpg ~ disp + hp + drat + wt + qsec + am

	Df	Sum of Sq	RSS	AIC
- drat	1	3.345	153.44	62.162
- disp	1	8.545	158.64	63.229
<none>			150.09	63.457
- hp	1	13.285	163.38	64.171
+ gear	1	1.565	148.53	65.121
+ cyl	1	1.003	149.09	65.242
+ vs	1	0.645	149.45	65.319
+ carb	1	0.107	149.99	65.434
- am	1	20.036	170.13	65.466
- qsec	1	25.574	175.67	66.491
- wt	1	67.572	217.66	73.351

Step: AIC=62.16

mpg ~ disp + hp + wt + qsec + am

	Df	Sum of Sq	RSS	AIC
- disp	1	6.629	160.07	61.515
<none>			153.44	62.162
- hp	1	12.572	166.01	62.682
+ drat	1	3.345	150.09	63.457
+ gear	1	2.977	150.46	63.535
+ cyl	1	2.447	150.99	63.648
+ vs	1	1.121	152.32	63.927
+ carb	1	0.011	153.43	64.160
- qsec	1	26.470	179.91	65.255
- am	1	32.198	185.63	66.258
- wt	1	69.043	222.48	72.051

Step: AIC=61.52

mpg ~ hp + wt + qsec + am

	Df	Sum of Sq	RSS	AIC
- hp	1	9.219	169.29	61.307
<none>			160.07	61.515
+ disp	1	6.629	153.44	62.162
+ carb	1	3.227	156.84	62.864
+ drat	1	1.428	158.64	63.229
- qsec	1	20.225	180.29	63.323
+ cyl	1	0.249	159.82	63.465
+ vs	1	0.249	159.82	63.466

```
+ gear 1      0.171 159.90 63.481
- am 1      25.993 186.06 64.331
- wt 1      78.494 238.56 72.284
```

Step: AIC=61.31
mpg ~ wt + qsec + am

	Df	Sum of Sq	RSS	AIC
<none>			169.29	61.307
+ hp	1	9.219	160.07	61.515
+ carb	1	8.036	161.25	61.751
+ disp	1	3.276	166.01	62.682
+ cyl	1	1.501	167.78	63.022
+ drat	1	1.400	167.89	63.042
+ gear	1	0.123	169.16	63.284
+ vs	1	0.000	169.29	63.307
- am	1	26.178	195.46	63.908
- qsec	1	109.034	278.32	75.217
- wt	1	183.347	352.63	82.790

Voici comment la régression par étape choisit le modèle retenu :

- **AIC le plus faible** : le modèle ayant l'AIC le plus faible est généralement préféré car il suggère un meilleur ajustement du modèle compte tenu du nombre de variables explicatives utilisées. L'AIC permet d'équilibrer l'adéquation et la complexité du modèle. Dans notre exemple, le modèle retenu est donc `mpg ~ wt + qsec + am`.

Après avoir exécuté la régression par étape, le résumé de `stepwise_model` affiche les coefficients pour chaque variable explicative ainsi que des statistiques telles que le R-carré, la statistique F et les valeurs p pour les variables explicatives.

```
summary(stepwise_model)
```

Call:

```
lm(formula = mpg ~ wt + qsec + am, data = mtcars)
```

Residuals:

	Min	1Q	Median	3Q	Max
	-3.4811	-1.5555	-0.7257	1.4110	4.6610

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	9.6178	6.9596	1.382	0.177915
wt	-3.9165	0.7112	-5.507	6.95e-06 ***
qsec	1.2259	0.2887	4.247	0.000216 ***
am	2.9358	1.4109	2.081	0.046716 *

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

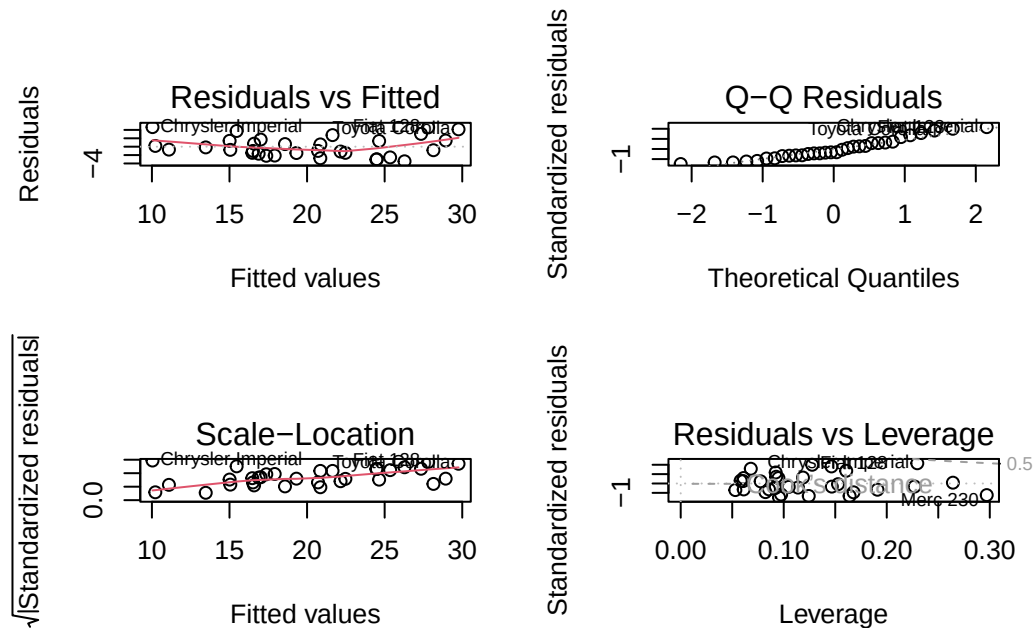
Residual standard error: 2.459 on 28 degrees of freedom

Multiple R-squared: 0.8497, Adjusted R-squared: 0.8336

F-statistic: 52.75 on 3 and 28 DF, p-value: 1.21e-11

Graphiques de diagnostic : Après avoir choisi un modèle, il est utile de vérifier les graphiques des résidus pour valider les hypothèses d'homoscédasticité, de normalité et d'indépendance des résidus :

```
par(mfrow = c(2, 2))
plot(stepwise_model)
```



Les graphiques de diagnostic sont acceptables pour notre modèle !

12.2.6 Ressources pour aller plus loin

Je vous recommande de jeter un coup d'œil à d'autres vidéos YouTube de StatQuest qui expliquent clairement les concepts statistiques, dont beaucoup sont très pertinents pour les sciences sociales et pour vos études en général, p. ex. :

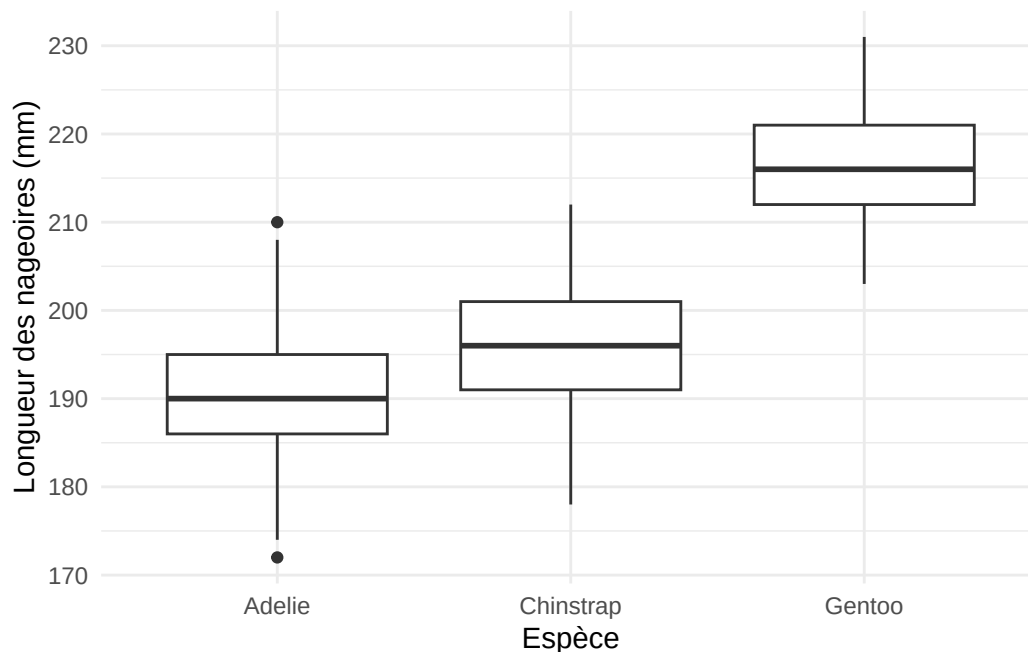
- [Linear Regression, Clearly Explained !!!](#) (StatQuest with Josh Starmer, 2017a)
- [Multiple Regression in R, Step by Step !!!](#) (StatQuest with Josh Starmer, 2022)
- [The Normal Distribution, Clearly Explained !!!](#) (StatQuest with Josh Starmer, 2017c)
- [p-values : What they are and how to interpret them](#) (StatQuest with Josh Starmer, 2020a)
- [R-squared, Clearly Explained !!!](#) (StatQuest with Josh Starmer, 2015)
- [Hypothesis Testing and The Null Hypothesis, Clearly Explained !!!](#) (StatQuest with Josh Starmer, 2020b)
- [Logistic Regression in R, Clearly Explained !!!!](#) (StatQuest with Josh Starmer, 2018)

Exercices

Notez vos réponses, puis comparez-les avec l'annexe « Corrigés » à la fin du livre.

Question 12.1

Jugez chaque proposition concernant le graphique suivant :



- a) La ligne noire épaisse au milieu des boxplots indique la longueur moyenne des nageoires pour chaque espèce de manchots.
- b) Au moins un manchot Adélie de l'échantillon a une nageoire plus longue qu'au moins un manchot papou (Gentoo).
- c) Les boxplots nous indiquent qu'il existe des différences statistiquement significatives dans la longueur des nageoires entre les trois espèces de manchots.
- d) Sur la seule base des boxplots, nous ne pouvons pas savoir si les différences de longueur des nageoires des différentes espèces de manchots sont statistiquement significatives (nous pourrions faire une ANOVA pour le savoir).
- e) Les boxplots indiquent qu'il y a deux valeurs extrêmes (*outliers*) dans l'ensemble de données.

Pour chaque proposition : vrai ou faux ?

Question 12.2

Exécutez le code suivant et jugez chaque proposition :

```
data(mtcars)
modele_1 <- lm(formula = mpg ~ wt + qsec + am, data = mtcars)
summary(modele_1)
```

- a) Ce modèle de régression pourrait être représenté visuellement par une ligne droite passant par un nuage de points.
- b) mpg est la variable dépendante.
- c) wt, qsec et am sont les variables indépendantes.
- d) Toutes les variables x de `modele_1` sont significatives au niveau de 5 %.

Pour chaque proposition : vrai ou faux ?

Question 12.3

Quel est le poids moyen en grammes des manchots du jeu de données `penguins` (paquet `palmerpenguins`) ? Arrondissez au gramme.

Réponse : _____

Question 12.4

Quel est le poids moyen en grammes des manchots **femelles** du jeu de données `penguins` ? Arrondissez au gramme.

Réponse : _____

Question 12.5

En raison des lois de la physique, on peut raisonnablement supposer que les voitures plus lourdes consomment généralement plus de carburant par kilomètre que les voitures plus légères. En d'autres termes, nous pouvons supposer qu'il existe **une relation de cause à effet** entre le poids du véhicule et l'efficacité énergétique.

Vrai ou faux ?

Question 12.6

Le **R carré** d'un modèle de régression linéaire indique le pourcentage de la variabilité de la variable dépendante (y) qui peut être expliquée par la variabilité des variables indépendantes (x_1 , x_2 , etc.).

Vrai ou faux ?

13 Révision 4 : statistiques

Dans le cours, le **Problem Set 4** était un test sur Moodle portant sur les chapitres 10 à 12. Les questions ci-dessous en reprennent le contenu, avec la correction et une explication pour chaque réponse.

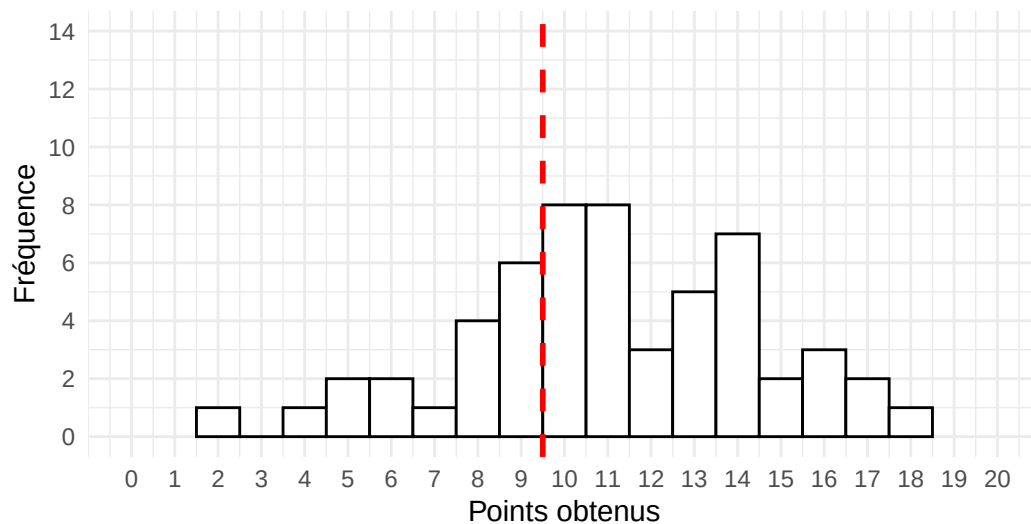
i Résultats de l'édition 2025 du cours

56 étudiant·es ont passé ce test (20 points au maximum, 10 pour réussir). La moyenne était de 11,0 points et 70 % des étudiant·es l'ont réussi.

```
-- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
v dplyr      1.2.1      v readr      2.2.0
v forcats    1.0.1      v stringr    1.6.0
v ggplot2    4.0.3      v tibble     3.3.1
v lubridate  1.9.5      v tidyr      1.3.2
v purrr      1.2.2
-- Conflicts ----- tidyverse_conflicts() --
x dplyr::filter() masks stats::filter()
x dplyr::lag()     masks stats::lag()
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts
  ↳ to become errors
```

Histogramme des points obtenus dans le PS 4

n = 56 résultats de test



Source : Problem Set 4, édition 2025

Exercices de révision

Ces questions reprennent le **Problem Set 4**. Notez vos réponses, puis comparez-les avec l'annexe « Corrigés » à la fin du livre.

Question 13.1

Quels types de visualisation sont bien adaptés pour afficher la relation entre deux variables numériques ?

- a) Histogramme
- b) Diagramme à barres
- c) Boîte à moustaches
- d) Nuage de points

Pour chaque proposition : vrai ou faux ?

Question 13.2

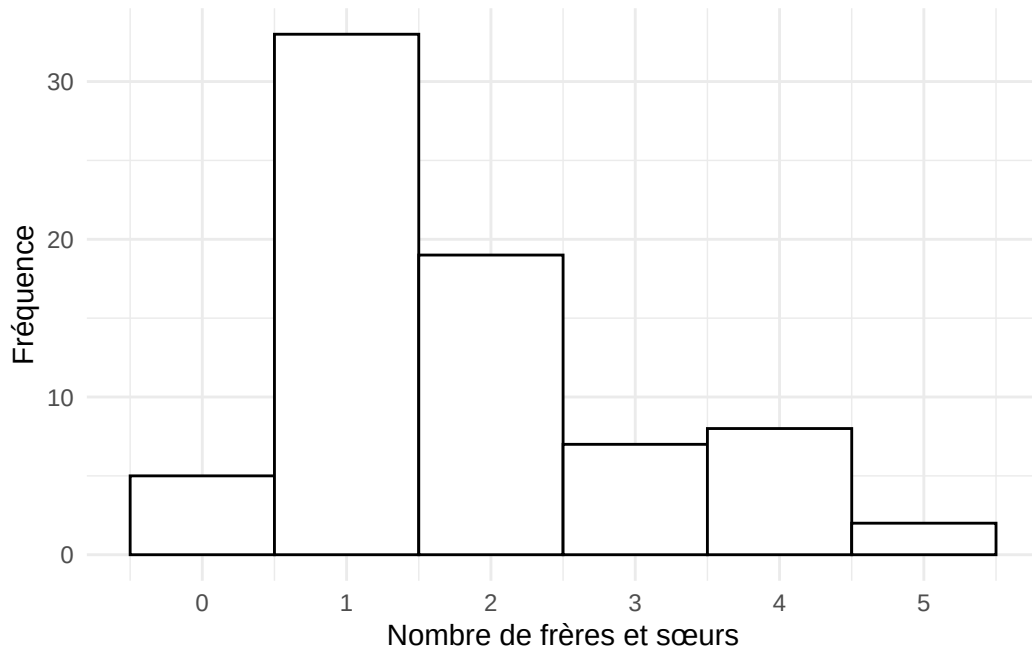
Jugez chaque affirmation concernant les **statistiques descriptives** :

- a) L'objectif des statistiques descriptives est principalement d'analyser les relations causales entre les variables.
- b) La partie descriptive d'une analyse statistique fournit généralement des indications telles que la taille de l'échantillon (n), les fréquences des variables catégorielles et les moyennes des variables numériques.
- c) Les tests d'hypothèse, par exemple avec un test t , sont une étape courante des statistiques descriptives.
- d) Les statistiques descriptives précèdent souvent les statistiques inférentielles dans une analyse de données quantitatives en sciences sociales.
- e) Certaines visualisations de données, comme les histogrammes, se prêtent bien à la présentation de statistiques descriptives.

Pour chaque proposition : vrai ou faux ?

Question 13.3

Jugez chaque proposition concernant l'histogramme suivant (source des données : questionnaire du cours) :



- a) L'histogramme montre que le nombre de frères et sœurs (variable `fraterie`) suit une loi normale.
- b) L'histogramme montre que la plupart des étudiant·es de la classe ont grandi dans des constellations familiales traditionnelles.
- c) L'histogramme montre qu'il y a plus d'étudiant·es qui ont 3 frères et sœurs que d'étudiant·es qui sont enfants uniques.
- d) La constellation la plus fréquente est celle des étudiant·es ayant un seul frère ou une seule sœur.
- e) Une personne ayant participé au questionnaire a indiqué qu'elle avait 7 frères et sœurs.

Pour chaque proposition : vrai ou faux ?

Question 13.4

Dans le jeu de données `reponses_socio.csv`, combien de minutes les étudiant·es **possédant un passeport suisse** mettent-ils en moyenne pour se rendre à l'université ? Arrondissez à la minute.

Réponse : _____

Question 13.5

Dans le jeu de données `reponses_socio.csv`, quel **genre** a le **trajet moyen** le plus court pour se rendre à l'université ? (Tapez le genre exactement comme il figure dans les données.)

Réponse : _____

Question 13.6

Le code suivant fournit des statistiques descriptives du jeu de données `mtcars` :

```
data(mtcars)
summary(mtcars)
```

Vrai ou faux ?

Question 13.7

Dans le jeu de données `reponses_socio.csv`, en moyenne, les étudiant·es **sans** passeport suisse ont besoin de moins de temps pour se rendre à l'université que leurs camarades suisses.

Vrai ou faux ?

Question 13.8

Les statistiques descriptives révèlent souvent des informations et des tendances intéressantes dans les données, même avant l'application de statistiques inférentielles.

Vrai ou faux ?

Question 13.9

Vous avez effectué un test t à un échantillon et obtenu une statistique t de 4.0869 avec 49 degrés de liberté et une valeur p de 0.0001618. Quelle conclusion pouvez-vous en tirer au niveau de signification de 5 % ?

- a) Ne pas rejeter l'hypothèse nulle (H_0).
- b) Rejeter l'hypothèse nulle (H_0).
- c) Il n'y a pas assez d'informations pour conclure.
- d) Augmenter la taille de l'échantillon.
- e) Aucune des autres réponses n'est correcte.

Une seule réponse est correcte.

Question 13.10

Lesquelles des hypothèses suivantes pourraient être testées dans un **test t à deux échantillons** ?

- a) Les deux échantillons proviennent de populations avec la même moyenne.
- b) Les deux échantillons proviennent de populations avec des moyennes différentes.
- c) L'une des populations est beaucoup plus nombreuse que l'autre.
- d) Les populations ont la même moyenne mais des variances très différentes.
- e) Trois groupes d'humains sont de tailles moyennes très différentes.

Pour chaque proposition : vrai ou faux ?

Question 13.11

Jugez chaque proposition relative aux **statistiques inférentielles** :

- a) Les statistiques inférentielles permettent de tirer des conclusions sur une population entière en se basant sur l'analyse d'un échantillon représentatif de cette population.
- b) Si deux variables quantitatives, par exemple y et x, sont significativement corrélées, il existe toujours une relation de cause à effet entre ces deux variables.
- c) L'échantillonnage aléatoire est une méthode de sélection au hasard d'un sous-groupe de participants dans une population, où chaque membre a une chance égale d'être choisi, offrant ainsi une représentation fiable de la population entière.
- d) Un échantillon de 100 femmes suisses d'une taille moyenne de 180 cm est représentatif de la distribution de taille des femmes en Suisse.
- e) Les tests t, l'ANOVA et la régression linéaire peuvent être des techniques utiles dans une analyse de statistiques inférentielles.

Pour chaque proposition : vrai ou faux ?

Question 13.12

Supposons que nous fassions un test t pour savoir si la taille moyenne de 100 joueuses de la WNBA sélectionnées au hasard diffère de celle de 100 hommes néerlandais sélectionnés au hasard. Le code suivant simule les deux échantillons et effectue le test. Exécutez-le et jugez chaque affirmation.

```
set.seed(2026)
wnba <- rnorm(100, mean = 184.5, sd = 6.5)
pays_bas <- rnorm(100, mean = 183.8, sd = 7.1)
t.test(wnba, pays_bas)
```

- a) Dans l'échantillon simulé, la joueuse de basket-ball moyenne mesure 183,86 cm.
- b) La valeur p du test indique que la différence entre les tailles moyennes est statistiquement significative au niveau de 5 %.
- c) L'hypothèse nulle du test t est que les moyennes des deux populations (joueuses de la WNBA et hommes néerlandais) sont égales.
- d) Le test t indique que les hommes néerlandais sont significativement plus grands que les joueuses de basket-ball de la WNBA.
- e) Les résultats du test t indiquent que, dans les échantillons simulés, il n'y a pas de différence significative entre les tailles moyennes des joueuses de basket et des hommes néerlandais.

Pour chaque proposition : vrai ou faux ?

Question 13.13

Que signifie une valeur p inférieure à 0,05 dans le contexte d'un test statistique ?

- a) Il y a une probabilité de 5 % que l'hypothèse nulle soit vraie.
- b) Il y a moins de 5 % de chances que les résultats observés soient dus au hasard.
- c) La probabilité que l'hypothèse nulle soit vraie est supérieure à 95 %.
- d) Les données ne suivent pas une loi normale.
- e) Toutes les autres réponses sont incorrectes.

Une seule réponse est correcte.

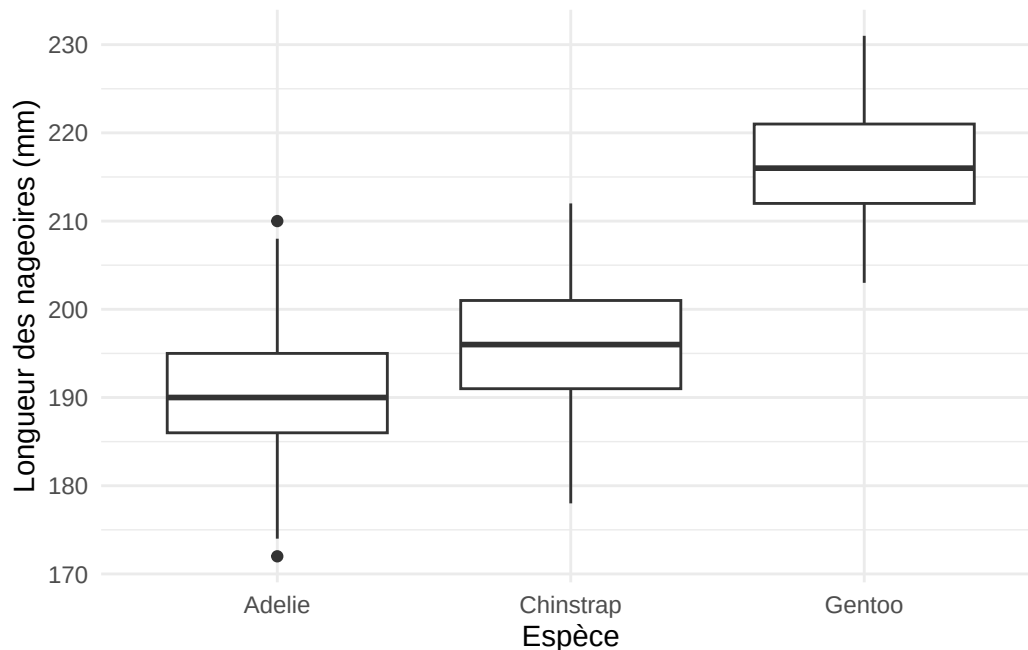
Question 13.14

Avec des échantillons de grande taille, même des différences (apparemment) faibles entre les moyennes de deux groupes peuvent devenir statistiquement significatives (par exemple, la taille moyenne des hommes dans deux pays différents).

Vrai ou faux ?

Question 13.15

Jugez chaque proposition concernant le graphique suivant :



- a) La ligne noire épaisse au milieu des boxplots indique la longueur moyenne des nageoires pour chaque espèce de manchots.
- b) Au moins un manchot Adélie de l'échantillon a une nageoire plus longue qu'au moins un manchot papou (Gentoo).
- c) Les boxplots nous indiquent qu'il existe des différences statistiquement significatives dans la longueur des nageoires entre les trois espèces de manchots.
- d) Sur la seule base des boxplots, nous ne pouvons pas savoir si les différences de longueur des nageoires des différentes espèces de manchots sont statistiquement significatives (nous pourrions faire une ANOVA pour le savoir).
- e) Les boxplots indiquent qu'il y a deux valeurs extrêmes (*outliers*) dans l'ensemble de données.

Pour chaque proposition : vrai ou faux ?

Question 13.16

Exécutez le code suivant et jugez chaque proposition :

```
data(mtcars)
modele_1 <- lm(formula = mpg ~ wt + qsec + am, data = mtcars)
summary(modele_1)
```

- a) Ce modèle de régression pourrait être représenté visuellement par une ligne droite passant par un nuage de points.
- b) `mpg` est la variable dépendante.
- c) `wt`, `qsec` et `am` sont les variables indépendantes.
- d) Toutes les variables `x` de `modele_1` sont significatives au niveau de 5 %.

Pour chaque proposition : vrai ou faux ?

Question 13.17

Quel est le poids moyen en grammes des manchots du jeu de données `penguins` (paquet `palmerpenguins`) ? Arrondissez au gramme.

Réponse : _____

Question 13.18

Quel est le poids moyen en grammes des manchots **femelles** du jeu de données `penguins` ? Arrondissez au gramme.

Réponse : _____

Question 13.19

En raison des lois de la physique, on peut raisonnablement supposer que les voitures plus lourdes consomment généralement plus de carburant par kilomètre que les voitures plus légères. En d'autres termes, nous pouvons supposer qu'il existe **une relation de cause à effet** entre le poids du véhicule et l'efficacité énergétique.

Vrai ou faux ?

Question 13.20

Le **R carré** d'un modèle de régression linéaire indique le pourcentage de la variabilité de la variable dépendante (`y`) qui peut être expliquée par la variabilité des variables indépendantes (`x1`, `x2`, etc.).

Vrai ou faux ?

14 Pour aller plus loin

Quelques pistes pour continuer à progresser avec R après cette introduction.

14.1 Pratique régulière

- Il est essentiel de pratiquer régulièrement pour devenir plus à l'aise avec R.
- Intégrez l'utilisation de R dans vos projets académiques et personnels autant que possible.

14.2 Application académique

- Demandez aux enseignant·es d'autres cours si vous pouvez utiliser R dans leurs cours.
- Proposez d'utiliser R pour l'analyse de données dans vos travaux de séminaire et de Bachelor.
- Collaborez avec d'autres étudiant·es utilisateurs·trices de R sur des projets de recherche en groupe à l'université.

14.3 MOOC

Pour répéter et approfondir vos connaissances en R, des **MOOC** (massive open online courses) sont disponibles gratuitement en tant qu'auditeur/auditrice (après inscription) :

1. **Datacamp** : Introduction to R Programming : <https://www.datacamp.com/courses/free-introduction-to-r> (DataCamp, s.d.)
2. **Edx (Harvard)** : R Basics : <https://www.edx.org/learn/r-programming/harvard-university-data-science-r-basics> (HarvardX, s.d.)
3. **Edx (Stanford)** : R Programming Fundamentals : <https://www.edx.org/learn/r-programming/stanford-university-r-programming-fundamentals> (StanfordOnline, s.d.)
4. **Udacity** : Data Analysis with R : <https://www.udacity.com/course/data-analysis-with-r--ud651> (Udacity, s.d.)

Si vous payez entre 50 et 200 USD, il est également possible de valider de tels MOOC avec un certificat. Ces certificats peuvent notamment être reliés à LinkedIn et y être affichés.

14.4 Tutoriels sur YouTube

Il existe également de nombreux tutoriels R sur Youtube. Il peut s'agir de cours R complets de différents niveaux ou de courtes vidéos montrant comment utiliser une technique R particulière (par ex. un certain type de visualisation).

1. **Introduction à R et RStudio (ft. mon chat radioactif)** : Tutoriel facile à comprendre en français d'environ 1h par le chercheur belge Nathan Uyttendaele (Il aborde certains aspects et logiciels que nous ne traitons pas dans notre cours. Mais dans l'ensemble, c'est une bonne introduction à R et RStudio pour les débutants) : https://www.youtube.com/watch?v=sav3Mbe0_DM (Chat Sceptique, 2020)
2. **Introduction to R & RStudio and other videos on Data Analysis with R** by Wouter van Atteveldt (computational communication scientist, VU Amsterdam) : https://www.youtube.com/watch?v=PVhZD5MINYM&list=PLjXODJ_IGN_V2ntvV2CN_GvzZ6Qm5km9L (van Atteveldt, 2020)
3. **Boot Camp of the Summer Institute in Computational Social Science**, SICSS, taught by Chris Bail from Duke University (multiple videos, social science focus) : https://sicss.io/boot_camp/ (Summer Institutes in Computational Social Science, s.d.)
4. **R Programming Tutorial : Learn the basics of statistical computing** (by Barton Poulson, 2h comprehensive general R introduction) : https://www.youtube.com/watch?v=_V8eKsto3Ug (freeCodeCamp.org, 2019)

14.5 Intelligence artificielle pour coder

Les assistants d'intelligence artificielle générative peuvent proposer du code R, expliquer un message d'erreur ou commenter votre script. Utilisez-les comme un·e collègue à qui l'on pose des questions, pas comme un pilote automatique : exécutez le code proposé, vérifiez le résultat et assurez-vous de comprendre chaque ligne. Ces outils se trompent souvent avec assurance. Respectez aussi les règles de votre université sur l'usage de l'IA dans les travaux évalués.

14.6 Communautés en ligne

Participez à des communautés en ligne pour rester connecté·e avec les dernières tendances et partager vos connaissances :

- [GitHub](#) : Le réseau social des développeurs
- [Stackoverflow](#) : Un forum d'entraide pour programmeurs
- [Kaggle](#) : Une plateforme pour le machine learning et l'analyse de données

14.7 Lecture spécialisée

Approfondissez vos connaissances en R avec des livres numérique open-source :

- **R for Data Science** [anglais] Une référence bien établie en sciences des données avec R (Wickham et al., 2023) : <https://r4ds.hadley.nz/>
- **R Cookbook** [anglais] Référence établie pour l'introduction, les statistiques et les graphiques avec R (Long & Teetor, 2019) : <https://rc2e.com/>
- **R Graphics Cookbook** [anglais] Ouvrage de référence pour concevoir des graphiques attrayants avec R (Chang, 2018) : <https://r-graphics.org/>
- **Programmer avec R** [français] de Vincent Goulet (Université Laval) (2026) : <https://vigou3.gitlab.io/programmer-avec-r/>
- **guide-R** [français] Guide pour l'analyse de données d'enquêtes avec R (Larmarange, 2026) : <https://larmarange.github.io/guide-R/>
- **Introduction à R et au tidyverse** [français] (Barnier, 2024) : <https://juba.github.io/tidyverse/>
- **utilitR** [français] Documentation collaborative de l'Insee (Insee, s.d.) : <https://book.utilitr.org/>
- **Computational Analysis of Communication** [anglais], pour aller vers les méthodes computationnelles en communication (van Atteveldt et al., 2022) : <https://cssbook.net/>

14.8 Valorisation des compétences

- Mettez en avant vos compétences en programmation R dans votre CV et sur LinkedIn, notamment les certificats obtenus via des MOOCs.

Ces conseils vous aideront à utiliser efficacement R dans le domaine des sciences de la communication et à développer une expertise approfondie.

14.9 Le mot de la fin

Ce chapitre clôt cette introduction à R. J'espère que vous y avez appris des techniques utiles pour vos études et vos projets! 🍌

N'oubliez pas qu'au final, l'essentiel est de faire preuve de bon sens pour mettre des outils comme R au service de questions intéressantes en sciences de la communication. 😊

Julian Maitra

À propos

Auteur et contact

Ce site accompagne le cours **Exercices Méthodes II** du Bachelor en sciences de la communication de l'[Université de Fribourg](#), où Julian Maitra a enseigné ces exercices de 2023 à 2025.

Julian Maitra est aujourd'hui rattaché à l'[Institut für Publizistik- und Kommunikationswissenschaft \(IfPuK\)](#) de la Freie Universität Berlin.

ORCID : [0000-0003-3989-5158](#)

Vous avez repéré une erreur ou une question mal formulée ? Signalez-la en ouvrant une *issue* sur [GitHub](#).

Licences

- **Textes, exercices, explications et figures** produites pour ce site : [CC BY-NC-SA 4.0](#). Vous pouvez les partager et les adapter à des fins non commerciales, en citant l'auteur et en partageant vos adaptations sous la même licence.
- **Code R** (extraits de code, `_common.R`, scripts du dossier `data-raw/`) : [licence MIT](#).
- **Éléments de tiers** : ils restent sous leur propre licence.

Élément	Où	Auteur·e	Licence
Photo de trois espèces d'iris	Chapitre 4	Diego Mariano, via Wikimedia Commons	CC BY-SA 4.0
Photo d'une Toyota Corolla de 1974	Chapitre 4	Jeremy from Sydney, Australia, via Wikimedia Commons	CC BY 2.0
Photo d'un diamant	Chapitre 4	Jerry Cone, via Wikimedia Commons	Domaine public
Logo du paquet palmerpenguins	Chapitre 12	Allison Horst (palmerpenguins)	CC0

Élément	Où	Auteur·e	Licence
Exercices interactifs (<code>webex.css</code> , <code>webex.js</code>)	Toutes les pages	Dale Barr, Lisa DeBruine et Caspar J. Van Lissa (webexercices)	CC BY-SA 4.0
Jeux de données intégrés à R (<code>iris</code> , <code>mtcars</code> , <code>women</code> , <code>diamonds</code> , <code>mpg</code> , <code>penguins</code>)	Chapitres 4 à 13	<code>iris</code> : Fisher (1936) et Anderson (1935); <code>mtcars</code> : Henderson & Velleman (1981); <code>women</code> : McNeil (1977); <code>diamonds</code> : Wickham (2016); <code>mpg</code> : U.S. Environmental Protection Agency (s.d.); <code>penguins</code> : Gorman et al. (2014) et Horst et al. (2020). Voir aussi la page d’aide de chaque jeu de données (p. ex. <code>?iris</code>)	Licences de R et des paquets concernés
Captures d’écran de RStudio (volets ; importation d’un fichier CSV)	Chapitres 1 et 7	Julian Maitra ; captures d’écran de RStudio Desktop © Posit Software, PBC	Logiciel sous licence AGPL v3 ; exclues de la licence CC du site

Posit, RStudio, and Shiny are trademarks of Posit Software, PBC, all rights reserved, and may be registered in the United States Patent and Trademark Office and in other countries.

Données

- **reponses_socio.csv** est un fichier **synthétique** : ses données ont été simulées à partir du questionnaire d’une classe, et il ne contient **aucune personne réelle**. Les valeurs rares ont été regroupées pour protéger les participant·es (voir la [description des données](#)).
- **igposts.csv** contient 84 observations compte-post (82 posts Instagram distincts de 33 comptes, publiés en février–mars 2023), avec leurs métriques et leurs URL uniquement.
- **Origine** : l’échantillon initial a été constitué en 2023 à partir de CrowdTangle (CrowdTangle Team, 2023). Seuls les posts encore publiquement accessibles le 5 octobre 2026 ont été gardés ; seize URL originales n’étaient plus disponibles et ont été exclues sans être remplacées par des posts plus récents.

- **Réobservation** : les métriques Likes, Comments et Reposts ont été relevées le 5 octobre 2026 sur les pages Instagram publiques avec Zeeschuimer (Peeters, 2026).
- **Followers et heures** : les nombres de followers sont un instantané du 5 octobre 2026, arrondi selon l’affichage d’Instagram. Les ratios par follower ne sont donc pas des taux d’engagement historiques. Les dates et heures sont en UTC (l’ancien fichier les donnait avec 5 heures de retard) ; Account et User.Name gardent les noms de 2023 (p. ex. justinbieber, aujourd’hui lilbieber).
- **Ce qui n’est pas publié** : ni légendes, ni texte de commentaires, ni données brutes Zeeschuimer. Les photos et textes des posts restent la propriété de leurs auteur·es ; le site renvoie aux posts par des liens et n’intègre aucun contenu d’Instagram.

La méthode complète est décrite dans la [description des données](#).

- Les fichiers **distribution_ps_1.csv** à **distribution_ps_4.csv** contiennent les distributions agrégées des résultats réels (nombre d’étudiant·es par nombre de points) aux Problem Sets. Les résultats individuels ne sont pas publiés.

Outils

Ce livre est écrit avec Quarto (Allaire et al., 2026) et R (R Core Team, 2026) ; le code est exécuté avec knitr (Xie, 2026) et les exercices interactifs utilisent le paquet webexercices (Barr et al., 2026). La liste complète des sources se trouve dans les [Références](#).

Cookies et suivi

Ce site ne dépose **aucun cookie** et n’utilise **aucun outil de mesure d’audience** ni de suivi. Il est hébergé par GitHub Pages ; GitHub peut enregistrer l’adresse IP des visiteurs (voir la [déclaration de confidentialité de GitHub](#)).

Seule la session 12 charge des ressources depuis d’autres serveurs : MathJax (jsDelivr) et une bibliothèque de compatibilité (cdnjs) pour afficher les formules, ainsi qu’une vidéo YouTube intégrée en mode « confidentialité renforcée » (youtube-nocookie.com).

Intelligence artificielle

Les explications des exercices ont été en partie rédigées avec l’aide d’un outil d’intelligence artificielle (Claude, d’Anthropic), puis relues par l’auteur.

Remerciements

Je remercie chaleureusement **Franck Bleu**, tuteur en méthodes quantitatives à l'Université de Fribourg (semestre de printemps 2025), qui a contribué à l'élaboration et à la relecture des questions à choix multiples des Problem Sets. Ces questions sont devenues les exercices interactifs de ce site. Il a aussi accompagné les étudiant·es dans leur apprentissage de R.

Je remercie également les étudiant·es des Exercices Méthodes II de l'Université de Fribourg (2023–2025), dont les questions, les erreurs fécondes et les retours ont façonné ce matériel au fil des semestres.

A Corrigés

Cette annexe donne les réponses et les explications des exercices, chapitre par chapitre. Les numéros reprennent ceux des questions : la « Question 3.1 » est la première question du chapitre 3.

Chapitre 1 : Premiers pas avec R

Question 1.1

Réponse : a) Vrai ; b) Vrai ; c) Vrai ; d) Faux

`class()` renvoie toujours un **texte** décrivant la classe de l'objet, p. ex. `class(5) → "numeric"`, `class(TRUE) → "logical"`, `class("a") → "character"`. Elle ne renvoie jamais un nombre comme 6.

Question 1.2

Réponse : a) Faux ; b) Faux ; c) Vrai ; d) Faux ; e) Faux

`number` contient le **texte** "7".

- a) Un calcul sur du texte produit une erreur.
- b) `as.numeric()` transforme le texte en nombre : la classe de `number` change.
- c) `c(number, "8")` crée un vecteur textuel.
- d) `length(number)` renvoie 1.
- e) `as.numeric(number)^2` vaut 49, pas 36.

Question 1.3

Réponse : a) Faux ; b) Vrai ; c) Vrai ; d) Vrai ; e) Faux

Les guillemets font de `n` une chaîne de caractères à **un** élément. Une opération arithmétique sur du texte (`n * 2`) produit une erreur.

- e) Stocker des numéros de téléphone, codes postaux ou identifiants en texte est souvent utile : on ne calcule pas avec eux, et les zéros initiaux sont conservés.

Question 1.4

Réponse : a) Vrai ; b) Vrai ; c) Faux ; d) Faux

Chaque nouvelle affectation remplace l'ancienne valeur : il n'existe qu'un seul objet **a**, qui vaut 88 à la fin. R ne produit pas d'erreur lorsqu'on réutilise un nom.

Question 1.5

Réponse : a) Vrai ; b) Vrai ; c) Faux ; d) Faux ; e) Faux

`print(a)` affiche le contenu de l'objet **a** dans la console. En session interactive, taper simplement **a** a le même effet. **a** est un vecteur (pas une fonction), et `c()` est la fonction de concaténation qui le crée : aucun objet nommé **c** n'est créé.

Question 1.6

Réponse : a) Vrai ; b) Vrai ; c) Vrai ; d) Vrai

Toutes les propositions sont correctes : `c()` crée un vecteur numérique (`class(a)` renvoie "numeric"), et l'affectation `<-` le place dans l'environnement.

Question 1.7

Réponse : Vrai

Le point d'interrogation ouvre l'aide de la fonction qui suit, avec ou sans parenthèses : `?typeof` et `?typeof()` affichent la même page. `help(typeof)` fait de même.

Question 1.8

Réponse : Vrai

`c()` (pour *combine* / concaténer) assemble ses arguments en un vecteur.

Question 1.9

Réponse : Vrai

Tout ce qui suit `#` sur une ligne est ignoré par R lors de l'exécution.

Question 1.10

Réponse : Vrai

Les guillemets font de "vampire" une chaîne de caractères : `class(monstre)` renvoie "character".

Question 1.11

Réponse : Vrai

Pour une affectation, = et <- sont équivalents. Par convention, on utilise <- pour les affectations et on réserve = aux arguments des fonctions.

Question 1.12

Réponse : Vrai

<- assigne n'importe quel objet R à un nom : scalaire, vecteur, matrice, data frame, liste, etc.

Question 1.13

Réponse : a) Source

Le script s'écrit et s'enregistre dans le volet Source (en haut à gauche) ; la console affiche le code exécuté et les résultats.

Question 1.14

Réponse : Faux

R est sensible à la casse : `Mean(x)` produit une erreur alors que `mean(x)` fonctionne, et `a` et `A` sont deux objets différents.

Question 1.15

Réponse : Faux

R est le langage de programmation. RStudio est un *environnement de développement intégré* (IDE) qui facilite le travail avec R, mais n'est pas un langage en soi.

Question 1.16

Réponse : Vrai

RStudio est un IDE : il réunit éditeur de script, console, environnement et fenêtre de graphiques. R tourne en arrière-plan.

Question 1.17

Réponse : Faux

Les guillemets créent **un seul** élément de type texte : `length(n)` vaut 1 et `class(n)` renvoie "character".

Chapitre 2 : Vecteurs, tableaux et facteurs

Question 2.1

Réponse : a) Faux ; b) Vrai ; c) Vrai ; d) Vrai

a) `a` a 5 éléments, `b` en a 6 : les longueurs diffèrent (et `length(a) <= length(b)` renvoie TRUE).

b) R doit connaître `a` et `b` pour calculer `a * b`.

d) `1:5` crée la suite 1, 2, 3, 4, 5. Remarque : comme les longueurs diffèrent, R « recycle » le vecteur le plus court et affiche un avertissement, mais `c` est quand même créé.

Question 2.2

Réponse : a) Vrai ; b) Vrai ; c) Faux ; d) Faux ; e) Vrai

a) `genre` et `classe_de_revenus` sont des facteurs (`factor()`). `nom` est une variable textuelle, qui sert d'identifiant.

b) `classe_de_revenus` est un facteur ordonné (`ordered = TRUE`).

c) `genre` est créé avec `ordered = FALSE` : il est nominal.

d) Il y a bien trois femmes, mais seulement deux personnes dans la classe « haut ».

e) Les cinq niveaux de `classe_de_revenus` apparaissent tous dans les données.

Question 2.3

Réponse : a) Vrai ; b) Faux ; c) Vrai ; d) Vrai ; e) Faux

- a) `female` est créé à partir d'un vecteur de TRUE/FALSE, donc de type logique.
- b) Ce sont les **variables** (colonnes), pas les unités statistiques.
- c) Chaque ligne correspond à un animal, soit 8 observations.
- d) « mammoth » apparaît trois fois dans `species` (Manny, Ellie, Peaches).
- e) Buck est une belette (*weasel*) de 3 kg. C'est Scrat, l'écureuil, qui pèse 1 kg.

Question 2.4

Réponse : a) Vrai ; b) Vrai ; c) Vrai ; d) Faux ; e) Vrai

- a) et e) Le `$` extrait une colonne par son nom ; `mean()` en calcule la moyenne.
- b) Dans `[lignes, colonnes]`, laisser les lignes vides sélectionne toutes les lignes de la 2 colonne.
- c) `dim(ice_age_df)` renvoie 8 5.
- d) `class(ice_age_df)` renvoie "data.frame" : contrairement à une matrice, un data frame peut contenir des colonnes de types différents.

Question 2.5

Réponse : a) Vrai ; b) Vrai ; c) Faux ; d) Faux ; e) Faux

25 valeurs réparties sur 5 lignes donnent une matrice 5×5 de type numérique (*integer*), remplie ligne par ligne grâce à `byrow = TRUE`.

Question 2.6

Réponse : a) Vrai ; b) Vrai ; c) Faux ; d) Faux ; e) Vrai

- c) R a toujours un répertoire de travail par défaut : on peut l'utiliser sans appeler `setwd()`.
- d) Les objets vivent en mémoire (volet *Environment*). Ils ne sont écrits sur le disque que si on les exporte explicitement (p. ex. avec `write.csv()` ou `save()`).

Question 2.7

Réponse : a) Vrai ; b) Faux ; c) Faux ; d) Vrai

a) La médiane d'un nombre pair de valeurs est la moyenne des deux valeurs centrales : $(3 + 4) / 2 = 3.5$.

b) `mean(vecteur2)` vaut 501.67.

c) `length(vecteur2)` vaut 6 ; 3010 est la **somme** des éléments.

Question 2.8

Réponse : 500

`log10(100)` vaut 2, donc `v2` contient les entiers de 2 à 501, soit $501 - 2 + 1 = 500$ éléments.

Question 2.9

Réponse : 3.5 (à 0.001 près)

```
median(vecteur1)
```

Avec 6 valeurs, la médiane est la moyenne des 3 et 4 valeurs triées : $(3 + 4) / 2 = 3.5$. Les deux valeurs extrêmes (1000 et 2000) n'ont aucun effet sur la médiane, contrairement à la moyenne (501.67).

Question 2.10

Réponse : 318

```
median(-66:702)
```

La suite de -66 à 702 comporte 769 valeurs. La médiane est la 385^e valeur, soit $-66 + 384 = 318$.

Question 2.11

Réponse : 910

```
var(v1)
```

Pour les entiers de 1 à n , la variance (d'échantillon) vaut $n(n + 1) / 12 = 104 \times 105 / 12 = 910$.

Question 2.12

Réponse : Vrai

C'est la convention des données « tidy » : une ligne par observation, une colonne par variable.

Question 2.13

Réponse : Vrai

Les objets vivent en mémoire et disparaissent quand on ferme R, sauf si on les exporte (p. ex. avec `write.csv()`). C'est pourquoi on garde tout son code dans un script : il permet de recréer les objets.

Question 2.14

Réponse : Faux

Le genre est une variable **nominale** : ses modalités n'ont pas d'ordre. On l'encode donc comme facteur simple, pas comme facteur ordonné.

Question 2.15

Réponse : Vrai

`nrow = 5` et `ncol = 10` fixent les dimensions ; `dim(matrice1)` renvoie `5 10`. Les 6 valeurs de `5:10` sont recyclées pour remplir les 50 cases. Comme 6 ne divise pas 5, R affiche un **avertissement**, mais la matrice est bien créée.

Question 2.16

Réponse : Vrai

Les modalités « petit », « moyen », « grand » sont des catégories qui ont un ordre naturel : il s'agit donc d'une variable ordinale (dans R, un facteur ordonné).

Question 2.17

Réponse : Faux

C'est l'inverse : les observations sont en **lignes**, les variables en colonnes.

Chapitre 3 : Révision 1 : bases de R

Question 3.1

Réponse : a) Faux ; b) Vrai ; c) Vrai ; d) Vrai ; e) Faux

Les guillemets font de `n` une chaîne de caractères à **un** élément. Une opération arithmétique sur du texte (`n * 2`) produit une erreur.

e) Stocker des numéros de téléphone, codes postaux ou identifiants en texte est souvent utile : on ne calcule pas avec eux, et les zéros initiaux sont conservés.

Question 3.2

Réponse : a) Vrai ; b) Vrai ; c) Faux ; d) Faux ; e) Faux

`print(a)` affiche le contenu de l'objet `a` dans la console. En session interactive, taper simplement `a` a le même effet. `a` est un vecteur (pas une fonction), et `c()` est la fonction de concaténation qui le crée : aucun objet nommé `c` n'est créé.

Question 3.3

Réponse : a) Vrai ; b) Vrai ; c) Vrai ; d) Vrai

Toutes les propositions sont correctes : `c()` crée un vecteur numérique (`class(a)` renvoie `"numeric"`), et l'affectation `<-` le place dans l'environnement.

Question 3.4

Réponse : Vrai

`c()` (pour *combine* / concaténer) assemble ses arguments en un vecteur.

Question 3.5

Réponse : Vrai

Tout ce qui suit `#` sur une ligne est ignoré par R lors de l'exécution.

Question 3.6

Réponse : Vrai

Les guillemets font de `"vampire"` une chaîne de caractères : `class(monstre)` renvoie `"character"`.

Question 3.7

Réponse : Vrai

`<-` assigne n'importe quel objet R à un nom : scalaire, vecteur, matrice, data frame, liste, etc.

Question 3.8

Réponse : a) Source

Le script s'écrit et s'enregistre dans le volet Source (en haut à gauche) ; la console affiche le code exécuté et les résultats.

Question 3.9

Réponse : Faux

R est le langage de programmation. RStudio est un *environnement de développement intégré* (IDE) qui facilite le travail avec R, mais n'est pas un langage en soi.

Question 3.10

Réponse : a) Vrai ; b) Faux ; c) Vrai ; d) Vrai ; e) Faux

- a) `female` est créé à partir d'un vecteur de TRUE/FALSE, donc de type logique.
- b) Ce sont les **variables** (colonnes), pas les unités statistiques.
- c) Chaque ligne correspond à un animal, soit 8 observations.
- d) « mammoth » apparaît trois fois dans `species` (Manny, Ellie, Peaches).
- e) Buck est une belette (*weasel*) de 3 kg. C'est Scrat, l'écureuil, qui pèse 1 kg.

Question 3.11

Réponse : a) Vrai ; b) Vrai ; c) Vrai ; d) Faux ; e) Vrai

- a) et e) Le `$` extrait une colonne par son nom ; `mean()` en calcule la moyenne.
- b) Dans `[lignes, colonnes]`, laisser les lignes vides sélectionne toutes les lignes de la 2 colonne.
- c) `dim(ice_age_df)` renvoie 8 5.
- d) `class(ice_age_df)` renvoie "data.frame" : contrairement à une matrice, un data frame peut contenir des colonnes de types différents.

Question 3.12

Réponse : a) Vrai ; b) Vrai ; c) Faux ; d) Faux ; e) Faux

25 valeurs réparties sur 5 lignes donnent une matrice 5×5 de type numérique (*integer*), remplie ligne par ligne grâce à `byrow = TRUE`.

Question 3.13

Réponse : a) Vrai ; b) Vrai ; c) Faux ; d) Faux ; e) Vrai

c) R a toujours un répertoire de travail par défaut : on peut l'utiliser sans appeler `setwd()`.

d) Les objets vivent en mémoire (volet *Environment*). Ils ne sont écrits sur le disque que si on les exporte explicitement (p. ex. avec `write.csv()` ou `save()`).

Question 3.14

Réponse : 500

`log10(100)` vaut 2, donc `v2` contient les entiers de 2 à 501, soit $501 - 2 + 1 = 500$ éléments.

Question 3.15

Réponse : 318

```
median(-66:702)
```

La suite de -66 à 702 comporte 769 valeurs. La médiane est la 385^e valeur, soit $-66 + 384 = 318$.

Question 3.16

Réponse : 910

```
var(v1)
```

Pour les entiers de 1 à n , la variance (d'échantillon) vaut $n(n + 1) / 12 = 104 \times 105 / 12 = 910$.

Question 3.17

Réponse : Vrai

C'est la convention des données « tidy » : une ligne par observation, une colonne par variable.

Question 3.18

Réponse : Faux

Le genre est une variable **nominale** : ses modalités n'ont pas d'ordre. On l'encode donc comme facteur simple, pas comme facteur ordonné.

Question 3.19

Réponse : Vrai

Les modalités « petit », « moyen », « grand » sont des catégories qui ont un ordre naturel : il s'agit donc d'une variable ordinale (dans R, un facteur ordonné).

Question 3.20

Réponse : Faux

C'est l'inverse : les observations sont en **lignes**, les variables en colonnes.

Question 3.A1

Réponse : 2550

```
sum(seq(from = 2, to = 100, by = 2))  
# ou :  
x <- 1:100  
sum(x[x %% 2 == 0])
```

Les 50 nombres pairs de 2 à 100 ont une moyenne de 51, d'où $50 \times 51 = 2550$. Aucune boucle n'est nécessaire, car `seq()`, `%%` et `sum()` opèrent sur tout le vecteur.

Question 3.A2

Réponse : 0.4

```
v <- seq(from = 0, to = 1, length.out = 11)
v[5] # 0.4 (5e élément ; l'indexation commence à 1)
w <- c(0, 0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 1)
all.equal(v, w) # TRUE
identical(v, w) # FALSE
v == w # quelques FALSE
```

Les valeurs sont « les mêmes » à l'œil, mais certains nombres décimaux comme 0,3 ne peuvent pas être représentés exactement en binaire. `seq()` les calcule ($0 + 3 \times 0,1$), alors que `c()` reprend les littéraux : de minuscules écarts d'arrondi apparaissent. Pour comparer des nombres décimaux, utilisez `all.equal()` plutôt que `==` ou `identical()`.

Question 3.A3

```
x <- rep(c(TRUE, FALSE), times = 10)
length(x) # 20

x[seq(1, 20, by = 2)] # positions impaires
x[c(TRUE, FALSE)] # astuce : le vecteur logique est recyclé
```

`rep()` répète un motif. Pour l'extraction, un vecteur d'indices (`seq()`) ou un vecteur logique recyclé fonctionnent tous les deux.

Question 3.A4

Réponse : 16

```
df <- data.frame(A = 1:5, B = (1:5)^2)
df[4, 2] # 16
df$B[4] # équivalent
```

Question 3.A5

```
as.numeric(tailles) # 1 2 3 2 1
```

`as.numeric()` renvoie les **codes internes** du facteur, c'est-à-dire la position de chaque modalité dans `levels`. Ici, c'est exactement ce que l'on veut, car les niveaux sont dans l'ordre S, M, L.

Le piège : si les niveaux ne sont pas dans l'ordre voulu, ou si le facteur contient des **nombre**s stockés comme texte, le résultat est trompeur :

```
f <- factor(c("10", "5", "20"))
as.numeric(f)      # 1 3 2 (codes, pas les valeurs !)
as.numeric(as.character(f)) # 10 5 20
```

Question 3.A6

Réponse : 65

```
sum(m[, 3])      # 65 (11 + 12 + 13 + 14 + 15)
m[2:4, c(1, 5)]
#      [,1] [,2]
# [1,]    2   22
# [2,]    3   23
# [3,]    4   24
```

Question 3.A7

`class()` renvoie la classe de l'objet, c'est-à-dire la façon dont R et les fonctions le *traitent*.
`typeof()` renvoie le type de stockage interne.

```
df <- data.frame(x = 1:3)
class(df)      # "data.frame"
typeof(df)     # "list" (un data frame est une liste de colonnes)

f <- factor(c("a", "b"))
class(f)       # "factor"
typeof(f)      # "integer" (les codes internes)
```

Question 3.A8

Réponse : 13.33

```
var(x)          # NA
var(x, na.rm = TRUE) # 13.33333
```

Par défaut, toute valeur **NA** « contamine » le résultat. `na.rm = TRUE` (*NA remove*) demande à la fonction d'écarter les valeurs manquantes avant le calcul. Le même argument existe pour `mean()`, `sum()`, `sd()`, `median()`, etc.

Question 3.A9

```
setwd(dirname(getwd()))  
# ou plus simplement :  
setwd("../")
```

`dirname()` retire le dernier élément d'un chemin. `".."` désigne le dossier parent dans tous les systèmes d'exploitation.

Question 3.A10

```
df <- data.frame(X = c(4, 8, 15, 16, 23, 42)) # exemple  
df$Y <- (df$X - mean(df$X)) / sd(df$X)  
df  
# Vérification : mean(df$Y) vaut ~0 et sd(df$Y) vaut 1
```

`as.vector(scale(df$X))` donne le même résultat.

Chapitre 4 : Paquets, jeux de données et premiers graphiques

Question 4.1

Réponse : a) Vrai ; b) Faux ; c) Vrai ; d) Faux ; e) Vrai

- a) `ggplot2` fait partie des paquets chargés par `library(tidyverse)`.
- b) Il n'existe pas de paquet `ggplot` distinct : `ggplot()` est la fonction principale de `ggplot2`.
- c) `packageDescription("ggplot2")$Maintainer` l'indique.
- d) Pour modifier des données, on utilise `dplyr` ; `ggplot2` sert à les visualiser.
- e) On ajoute les couches les unes aux autres avec `+`.

Question 4.2

Réponse : a) Vrai ; b) Vrai ; c) Faux ; d) Vrai

- a) `dim(iris)` renvoie `150 5`.
- b) `levels(iris$Species)` : `setosa`, `versicolor` et `virginica`.
- c) L'unité statistique est une **fleur** ; `Petal.Length` est une variable.
- d) La page d'aide `?iris` cite Anderson (1935), *The irises of the Gaspé Peninsula*. 1935 est l'année de publication ; selon Anderson, deux des trois espèces proviennent de la péninsule de Gaspé (Québec).

Question 4.3

Réponse : a) Vrai ; b) Vrai ; c) Vrai ; d) Vrai

a) et b) `class(mtcars)` renvoie `"data.frame"` et `dim(mtcars)` renvoie `32 11`.

c) `mtcars[which.max(mtcars$mpg), 1]` donne la Toyota Corolla (33,9 mpg).

d) `mtcars["Ford Pantera L", "am"]` vaut 1, ce qui signifie « manuelle » (voir `?mtcars`).

Question 4.4

Réponse : a) Faux ; b) Faux ; c) Vrai ; d) Vrai ; e) Faux

a) Les couleurs du graphique représentent la **pureté** (`clarity`), pas la couleur des diamants (`color`).

b) et c) Le nuage de points s'incurve vers le haut : le prix augmente plus vite que le carat.

d) Pour un même carat, les points des puretés élevées se situent plus haut. Parmi les diamants d'environ 1 carat, le prix médian passe de 2 808 USD (I1) à 9 764 USD (IF).

e) Le diamant le plus lourd (5,01 carats) coûte 18 018 USD, alors que le plus cher coûte 18 823 USD.

Question 4.5

Réponse : a) Faux ; b) Vrai ; c) Vrai ; d) Faux ; e) Vrai

a) et e) Seules *Data*, *Aesthetics* et *Geometries* sont nécessaires ; les autres couches ont des valeurs par défaut.

c) On commence par `ggplot(data = ...)`, puis on ajoute les couches.

d) *Aesthetics* (`aes()`) relie les variables aux axes, couleurs, tailles, etc. Le style « WSJ » ou « Economist » relève de la couche *Theme* (p. ex. `theme_wsj()` du paquet `ggthemes`).

Question 4.6

Réponse : 326

```
min(diamonds$price)
```

Question 4.7

Réponse : 1 (à 0.001 près)

```
min(iris$Petal.Length)
```

Question 4.8

Réponse : 230

Les noms des voitures sont les noms de lignes de `mtcars` :

```
mtcars["Chrysler Imperial", "hp"]
```

Question 4.9

Réponse : Vrai

Un paquet s'installe une fois avec `install.packages("tidyverse")`, puis se charge à chaque session avec `library(tidyverse)`. Si le paquet n'est pas installé, `library()` renvoie une erreur.

Question 4.10

Réponse : Vrai

`tidyverse::tidyverse_packages()` liste les paquets du tidyverse; `ggplot2` en fait partie.

Question 4.11

Réponse : Faux

`ggthemes` complète `ggplot2` avec des thèmes supplémentaires, mais il ne fait pas partie du tidyverse : il faut l'installer et le charger séparément.

Question 4.12

Réponse : Vrai

`library(tidyverse)` charge d'un coup plusieurs paquets conçus pour fonctionner ensemble, dont `ggplot2`, `dplyr`, `tidyr` et `readr`.

Chapitre 5 : Visualiser des données avec ggplot2

Question 5.1

Réponse : a) Faux ; b) Faux ; c) Faux ; d) Faux

a) La barre la plus haute est à 14 points (le mode).

b) et c) Un histogramme ne montre ni la médiane ni la moyenne exactes. Ici, la médiane vaut 14 et la moyenne 14,2.

d) Une loi normale *centrée réduite* a une moyenne de 0 et un écart-type de 1 : ce n'est pas le cas de points compris entre 8 et 20.

Question 5.2

Réponse : a) Vrai ; b) Faux ; c) Vrai ; d) Faux

`binwidth` fixe directement la largeur de chaque barre ; `bins` fixe le **nombre** de barres, ce qui change aussi leur largeur. `geom_histogram()` n'a pas d'argument `interval`.

Question 5.3

Réponse : a) Vrai ; b) Faux ; c) Vrai ; d) Faux ; e) Faux

a) et e) La droite descend : plus la cylindrée (`displ`) est grande, moins on parcourt de miles par gallon (`hwy`).

b) Les données viennent de `mpg`, pas de `mtcars`.

c) C'est la description de `?mpg`.

d) L'axe des x montre la cylindrée (`displ`, en litres) ; les miles par gallon sont sur l'axe des y.

Question 5.4

Réponse : a) Vrai ; b) Vrai ; c) Faux ; d) Vrai ; e) Vrai

a) L'affectation `<-` stocke le graphique dans l'objet `g`.

c) `geom_point()` produit un **nuage de points**, pas un histogramme.

d) Chaque ligne de `mtcars` (une voiture) donne un point.

e) Les voitures lourdes parcourent moins de miles par gallon : `cor(mtcars$mpg, mtcars$wt)` vaut $-0,87$.

Question 5.5

Réponse : a) Vrai ; b) Vrai ; c) Vrai ; d) Vrai ; e) Vrai

- a) `geom_bar()` compte les voitures par valeur de `cyl`.
- b) `fill = gear` colore les barres selon le nombre de vitesses.
- c) `as.factor()` transforme `cyl` et `gear` en facteurs.
- d) La barre des 8 cylindres ne contient que des voitures à 3 et 5 vitesses.
- e) Il y a 7 voitures à 6 cylindres, contre 11 à 4 cylindres et 14 à 8 cylindres.

Question 5.6

Réponse : a) Vrai ; b) Faux ; c) Faux ; d) Vrai ; e) Faux

Sans autres arguments, `rnorm()` utilise `mean = 0` et `sd = 1` : c'est la loi normale centrée réduite, qui produit des valeurs positives et négatives autour de 0.

- e) L'écart-type est un paramètre de la loi : il varie d'une loi normale à l'autre.

Question 5.7

Réponse : a) Faux ; b) Faux ; c) Faux ; d) Vrai

Le nuage de points place une variable numérique sur chaque axe. L'histogramme montre la distribution d'une variable numérique ; le diagramme à barres compte des catégories ; la boîte à moustaches compare une variable numérique entre catégories.

Question 5.8

Réponse : Vrai

Les graphiques sont un outil central de l'analyse exploratoire : ils révèlent la forme des distributions, les tendances et les valeurs aberrantes.

Question 5.9

Réponse : Faux

Un boxplot montre la **médiane** et les **quartiles** (plus les moustaches et les valeurs extrêmes), pas la moyenne ni les déciles.

Question 5.10

Réponse : Faux

La distribution des prix est fortement asymétrique à droite : beaucoup de diamants bon marché, peu de diamants très chers. La moyenne (3933 USD) dépasse d'ailleurs nettement la médiane (2401 USD).

Question 5.11

Réponse : Vrai

`geom_smooth(method = "lm")` ajoute la droite de régression linéaire (et son intervalle de confiance) au nuage de points.

Question 5.12

Réponse : Vrai

Avec un million de valeurs, l'histogramme prend presque parfaitement la forme en cloche de la loi normale. Avec seulement quelques dizaines de valeurs, il serait beaucoup plus irrégulier.

Chapitre 6 : Révision 2 : visualisation

Question 6.1

Réponse : a) Vrai ; b) Faux ; c) Vrai ; d) Faux ; e) Vrai

- a) `ggplot2` fait partie des paquets chargés par `library(tidyverse)`.
- b) Il n'existe pas de paquet `ggplot` distinct : `ggplot()` est la fonction principale de `ggplot2`.
- c) `packageDescription("ggplot2")$Maintainer` l'indique.
- d) Pour modifier des données, on utilise `dplyr` ; `ggplot2` sert à les visualiser.
- e) On ajoute les couches les unes aux autres avec `+`.

Question 6.2

Réponse : a) Faux ; b) Faux ; c) Vrai ; d) Vrai ; e) Faux

a) Les couleurs du graphique représentent la **pureté** (*clarity*), pas la couleur des diamants (*color*).

b) et c) Le nuage de points s'incurve vers le haut : le prix augmente plus vite que le carat.

d) Pour un même carat, les points des puretés élevées se situent plus haut. Parmi les diamants d'environ 1 carat, le prix médian passe de 2 808 USD (I1) à 9 764 USD (IF).

e) Le diamant le plus lourd (5,01 carats) coûte 18 018 USD, alors que le plus cher coûte 18 823 USD.

Question 6.3

Réponse : a) Faux ; b) Vrai ; c) Vrai ; d) Faux ; e) Vrai

a) et e) Seules *Data*, *Aesthetics* et *Geometries* sont nécessaires ; les autres couches ont des valeurs par défaut.

c) On commence par `ggplot(data = ...)`, puis on ajoute les couches.

d) *Aesthetics* (`aes()`) relie les variables aux axes, couleurs, tailles, etc. Le style « WSJ » ou « Economist » relève de la couche *Theme* (p. ex. `theme_wsj()` du paquet `ggthemes`).

Question 6.4

Réponse : 326

```
min(diamonds$price)
```

Question 6.5

Réponse : 1 (à 0.001 près)

```
min(iris$Petal.Length)
```

Question 6.6

Réponse : 230

Les noms des voitures sont les noms de lignes de `mtcars` :

```
mtcars["Chrysler Imperial", "hp"]
```

Question 6.7

Réponse : Vrai

`tidyverse::tidyverse_packages()` liste les paquets du tidyverse ; `ggplot2` en fait partie.

Question 6.8

Réponse : a) Faux ; b) Faux ; c) Faux ; d) Faux

a) La barre la plus haute est à 14 points (le mode).

b) et c) Un histogramme ne montre ni la médiane ni la moyenne exactes. Ici, la médiane vaut 14 et la moyenne 14,2.

d) Une loi normale *centrée réduite* a une moyenne de 0 et un écart-type de 1 : ce n'est pas le cas de points compris entre 8 et 20.

Question 6.9

Réponse : a) Vrai ; b) Faux ; c) Vrai ; d) Faux

`binwidth` fixe directement la largeur de chaque barre ; `bins` fixe le **nombre** de barres, ce qui change aussi leur largeur. `geom_histogram()` n'a pas d'argument `interval`.

Question 6.10

Réponse : a) Vrai ; b) Faux ; c) Vrai ; d) Faux ; e) Faux

a) et e) La droite descend : plus la cylindrée (`displ`) est grande, moins on parcourt de miles par gallon (`hwy`).

b) Les données viennent de `mpg`, pas de `mtcars`.

c) C'est la description de `?mpg`.

d) L'axe des x montre la cylindrée (`displ`, en litres) ; les miles par gallon sont sur l'axe des y.

Question 6.11

Réponse : a) Vrai ; b) Vrai ; c) Faux ; d) Vrai ; e) Vrai

- a) L'affectation `<-` stocke le graphique dans l'objet `g`.
- c) `geom_point()` produit un **nuage de points**, pas un histogramme.
- d) Chaque ligne de `mtcars` (une voiture) donne un point.
- e) Les voitures lourdes parcourent moins de miles par gallon : `cor(mtcars$mpg, mtcars$wt)` vaut $-0,87$.

Question 6.12

Réponse : a) Vrai ; b) Vrai ; c) Vrai ; d) Vrai ; e) Vrai

- a) `geom_bar()` compte les voitures par valeur de `cyl`.
- b) `fill = gear` colore les barres selon le nombre de vitesses.
- c) `as.factor()` transforme `cyl` et `gear` en facteurs.
- d) La barre des 8 cylindres ne contient que des voitures à 3 et 5 vitesses.
- e) Il y a 7 voitures à 6 cylindres, contre 11 à 4 cylindres et 14 à 8 cylindres.

Question 6.13

Réponse : a) Vrai ; b) Faux ; c) Faux ; d) Vrai ; e) Faux

Sans autres arguments, `rnorm()` utilise `mean = 0` et `sd = 1` : c'est la loi normale centrée réduite, qui produit des valeurs positives et négatives autour de 0.

- e) L'écart-type est un paramètre de la loi : il varie d'une loi normale à l'autre.

Question 6.14

Réponse : Vrai

Les graphiques sont un outil central de l'analyse exploratoire : ils révèlent la forme des distributions, les tendances et les valeurs aberrantes.

Question 6.15

Réponse : Faux

Un boxplot montre la **médiane** et les **quartiles** (plus les moustaches et les valeurs extrêmes), pas la moyenne ni les déciles.

Question 6.16

Réponse : Faux

La distribution des prix est fortement asymétrique à droite : beaucoup de diamants bon marché, peu de diamants très chers. La moyenne (3933 USD) dépasse d'ailleurs nettement la médiane (2401 USD).

Question 6.17

Réponse : Vrai

`geom_smooth(method = "lm")` ajoute la droite de régression linéaire (et son intervalle de confiance) au nuage de points.

Question 6.18

Réponse : Vrai

Avec un million de valeurs, l'histogramme prend presque parfaitement la forme en cloche de la loi normale. Avec seulement quelques dizaines de valeurs, il serait beaucoup plus irrégulier.

Question 6.19

Réponse : a) Vrai ; b) Faux ; c) Faux ; d) Vrai ; e) Vrai

a), d) et e) `group_by()` crée un groupe par pureté ; `summarize()` calcule ensuite, pour chaque groupe, le nombre de diamants (`n()`) et le prix moyen.

b) 1790 est le **nombre** de diamants de pureté maximale (IF) ; leur prix moyen est d'environ 2865 USD.

c) Les effectifs varient de 741 (I1) à 13 065 (SI1).

Question 6.20

Réponse : Vrai

`as.factor()` convertit un vecteur textuel ou numérique en facteur, p. ex. `mtcars$cyl <- as.factor(mtcars$cyl)`.

Chapitre 7 : Importer et manipuler des données

Question 7.1

Réponse : leomessi

Triez `igposts` par `Total.Interactions` décroissant : le premier post est celui de leomessi (22 626 965 interactions). Ouvrez son URL pour voir de quoi il s'agit : <https://www.instagram.com/p/CpLxnFLNjZn/>

Question 7.2

Réponse : c) Créer des séquences de fonctions pour simplifier le code.

Le *pipe* `%>%` transmet le résultat d'une fonction comme premier argument de la suivante. On enchaîne ainsi les étapes de gauche à droite au lieu d'imbriquer les fonctions : `x %>% f() %>% g()` équivaut à `g(f(x))`.

Question 7.3

Réponse : a) Vrai ; b) Faux ; c) Faux ; d) Vrai ; e) Vrai

a), d) et e) `group_by()` crée un groupe par pureté ; `summarize()` calcule ensuite, pour chaque groupe, le nombre de diamants (`n()`) et le prix moyen.

b) 1790 est le **nombre** de diamants de pureté maximale (IF) ; leur prix moyen est d'environ 2865 USD.

c) Les effectifs varient de 741 (I1) à 13 065 (SI1).

Question 7.4

Réponse : a) Vrai ; b) Faux ; c) Vrai ; d) Faux ; e) Faux

a) et b) `arrange()` trie par ordre croissant ; pour l'ordre décroissant, on écrit `arrange(desc(mpg))`.

d) Sans affectation (`<-`), le résultat est seulement affiché : `mtcars` ne change pas.

e) La Cadillac Fleetwood apparaît en **premier** : c'est l'une des voitures les moins efficaces (10,4 mpg).

Question 7.5

Réponse : a) Vrai ; b) Vrai ; c) Vrai ; d) Vrai ; e) Faux

a) et b) Le critère `taille_f <= 160` vaut TRUE ou FALSE ; `count()` compte les lignes de chaque groupe.

c) Chaque appel à `rnorm()` tire de nouvelles valeurs (sauf si l'on fixe `set.seed()`).

d) `as.data.frame()` transforme le vecteur en data frame.

e) `rnorm(500, ...)` crée 500 valeurs, donc 500 lignes, pas 100.

Question 7.6

Réponse : a) Vrai ; b) Vrai ; c) Faux ; d) Vrai ; e) Faux

On peut aussi chercher le post dans les données : `igposts %>% filter(grepl("CpHHbGWunhI", URL)) %>% select(User.Name, Type, Reposts)`. Le post vient de **zendaya**, il est de type **Album** et compte 284 Reposts.

e) Zendaya est américaine.

Question 7.7

Réponse : Vrai

CSV signifie *comma-separated values* : chaque ligne du fichier est une observation, et les valeurs sont séparées par des virgules.

Question 7.8

Réponse : Vrai

`as.factor()` convertit un vecteur textuel ou numérique en facteur, p. ex. `mtcars$cyl <- as.factor(mtcars$cyl)`.

Question 7.9

Réponse : Faux

Dans un fichier CSV (*comma-separated values*), les valeurs sont séparées par des **virgules** (parfois des points-virgules), pas par des espaces.

Question 7.10

Réponse : Vrai

Si le séparateur décimal est mal indiqué, les nombres sont lus comme du texte. L'argument `dec` de `read.csv()` le précise ; `read.csv2()` est prévu pour les fichiers avec virgule décimale et point-virgule comme séparateur.

Question 7.11

Réponse : Faux

Les métriques ont été relevées le 5 octobre 2026 sur les pages Instagram publiques : elles reflètent l'état des posts à cette date, pas en 2023.

Question 7.12

Réponse : Faux

L'unité n'est pas le compte Instagram : chaque ligne de `igposts` est une observation associant un **post** à un compte qui l'a publié ou co-publié. Le fichier contient 84 lignes correspondant à 82 posts distincts provenant de 33 comptes. Un même compte apparaît donc sur plusieurs lignes.

Question 7.13

Réponse : Vrai

Toutes les cellules contenant `#N/A` sont lues comme `NA`, la valeur manquante de R. Sans cet argument, une colonne numérique contenant `#N/A` serait lue comme du texte. (Le fichier `igposts.csv` du cours n'a pas de valeurs manquantes : la valeur par défaut, `na.strings = "NA"`, suffit.)

Chapitre 8 : Manipuler des données : exercices

Question 8.1

Réponse : a) Faux ; b) Vrai ; c) Faux ; d) Vrai ; e) Vrai

a) La page d'aide `?women` indique des femmes âgées de **30 à 39 ans**.

b) Le résumé donne 165,1 cm et 62,0 kg.

c) Le jeu de données ne contient aucune femme européenne : aucune comparaison n'est possible.

d) `dim(women)` renvoie 15 2.

e) Les multiplications par 2,54 et 0,4536 font la conversion.

Question 8.2

Réponse : 5

```
igposts %>% filter(User.Name == "badgalriri") %>% count()
```

Question 8.3

Réponse : 6

```
igposts %>% filter(User.Name == "selenagomez") %>% count()
```

Question 8.4

Réponse : 60

```
igposts %>% count(Type)
```

`count()` affiche le nombre de lignes de chaque type : 60 albums. (Un album co-publié par deux comptes apparaît sur deux lignes : ces 60 lignes correspondent à 59 posts distincts.)

Question 8.5

Réponse : 14

```
mtcars %>% filter(mpg >= 20) %>% count()
```

« Au moins 20 » inclut 20 : on utilise `>=`, pas `>`.

Question 8.6

Réponse : cristiano

```
igposts %>% arrange(desc(Followers.at.0bservation)) %>% select(User.Name) %>% head(1)
```

Question 8.7

Réponse : agustd

```
igposts %>% arrange(desc(Reposts)) %>% select(User.Name, Reposts) %>% head(1)
```

Question 8.8

Réponse : Vrai

```
igposts %>%  
  mutate(ratio = Comments / Followers.at.Observation) %>%  
  arrange(desc(ratio)) %>%  
  select(Account, ratio) %>%  
  head()
```

KIARA arrive en tête, que l'on compare les posts un par un ou la moyenne par compte.

Question 8.9

Réponse : Faux

La première ligne du résultat est la **Lotus Europa**, suivie de la Honda Civic. La Toyota Corolla a le mpg le plus élevé, mais pas le meilleur rapport mpg / poids.

Chapitre 9 : Révision 3 : manipulation de données

Question 9.1

Réponse : leomessi

Triez `igposts` par `Total.Interactions` décroissant : le premier post est celui de leomessi (22 626 965 interactions). Ouvrez son URL pour voir de quoi il s'agit : <https://www.instagram.com/p/CpLxnFINJZn/>

Question 9.2

Réponse : c) Créer des séquences de fonctions pour simplifier le code.

Le *pipe* `%>%` transmet le résultat d'une fonction comme premier argument de la suivante. On enchaîne ainsi les étapes de gauche à droite au lieu d'imbriquer les fonctions : `x %>% f() %>% g()` équivaut à `g(f(x))`.

Question 9.3

Réponse : a) Vrai ; b) Faux ; c) Faux ; d) Vrai ; e) Vrai

a), d) et e) `group_by()` crée un groupe par pureté ; `summarize()` calcule ensuite, pour chaque groupe, le nombre de diamants (`n()`) et le prix moyen.

b) 1790 est le **nombre** de diamants de pureté maximale (IF) ; leur prix moyen est d'environ 2865 USD.

c) Les effectifs varient de 741 (I1) à 13 065 (SI1).

Question 9.4

Réponse : a) Vrai ; b) Faux ; c) Vrai ; d) Faux ; e) Faux

a) et b) `arrange()` trie par ordre croissant ; pour l'ordre décroissant, on écrit `arrange(desc(mpg))`.

d) Sans affectation (`<-`), le résultat est seulement affiché : `mtcars` ne change pas.

e) La Cadillac Fleetwood apparaît en **premier** : c'est l'une des voitures les moins efficaces (10,4 mpg).

Question 9.5

Réponse : a) Vrai ; b) Vrai ; c) Vrai ; d) Vrai ; e) Faux

a) et b) Le critère `taille_f <= 160` vaut TRUE ou FALSE ; `count()` compte les lignes de chaque groupe.

c) Chaque appel à `rnorm()` tire de nouvelles valeurs (sauf si l'on fixe `set.seed()`).

d) `as.data.frame()` transforme le vecteur en data frame.

e) `rnorm(500, ...)` crée 500 valeurs, donc 500 lignes, pas 100.

Question 9.6

Réponse : a) Vrai ; b) Vrai ; c) Faux ; d) Vrai ; e) Faux

On peut aussi chercher le post dans les données : `igposts %>% filter(grepl("CpHHbGWunhI", URL)) %>% select(User.Name, Type, Reposts)`. Le post vient de **zendaya**, il est de type **Album** et compte 284 Reposts.

e) Zendaya est américaine.

Question 9.7

Réponse : Faux

Dans un fichier CSV (*comma-separated values*), les valeurs sont séparées par des **virgules** (parfois des points-virgules), pas par des espaces.

Question 9.8

Réponse : Vrai

Si le séparateur décimal est mal indiqué, les nombres sont lus comme du texte. L'argument `dec` de `read.csv()` le précise ; `read.csv2()` est prévu pour les fichiers avec virgule décimale et point-virgule comme séparateur.

Question 9.9

Réponse : Faux

Les métriques ont été relevées le 5 octobre 2026 sur les pages Instagram publiques : elles reflètent l'état des posts à cette date, pas en 2023.

Question 9.10

Réponse : Faux

L'unité n'est pas le compte Instagram : chaque ligne de `igposts` est une observation associant un **post** à un compte qui l'a publié ou co-publié. Le fichier contient 84 lignes correspondant à 82 posts distincts provenant de 33 comptes. Un même compte apparaît donc sur plusieurs lignes.

Question 9.11

Réponse : Vrai

Toutes les cellules contenant `#N/A` sont lues comme `NA`, la valeur manquante de R. Sans cet argument, une colonne numérique contenant `#N/A` serait lue comme du texte. (Le fichier `igposts.csv` du cours n'a pas de valeurs manquantes : la valeur par défaut, `na.strings = "NA"`, suffit.)

Question 9.12

Réponse : a) Faux ; b) Vrai ; c) Faux ; d) Vrai ; e) Vrai

- a) La page d'aide ?women indique des femmes âgées de **30 à 39 ans**.
- b) Le résumé donne 165,1 cm et 62,0 kg.
- c) Le jeu de données ne contient aucune femme européenne : aucune comparaison n'est possible.
- d) `dim(women)` renvoie 15 2.
- e) Les multiplications par 2,54 et 0,4536 font la conversion.

Question 9.13

Réponse : 6

```
igposts %>% filter(User.Name == "selenagomez") %>% count()
```

Question 9.14

Réponse : 60

```
igposts %>% count(Type)
```

`count()` affiche le nombre de lignes de chaque type : 60 albums. (Un album co-publié par deux comptes apparaît sur deux lignes : ces 60 lignes correspondent à 59 posts distincts.)

Question 9.15

Réponse : 14

```
mtcars %>% filter(mpg >= 20) %>% count()
```

« Au moins 20 » inclut 20 : on utilise `>=`, pas `>`.

Question 9.16

Réponse : cristiano

```
igposts %>% arrange(desc(Followers.at.Observation)) %>% select(User.Name) %>% head(1)
```

Question 9.17

Réponse : Vrai

```
igposts %>%  
  mutate(ratio = Comments / Followers.at.Observation) %>%  
  arrange(desc(ratio)) %>%  
  select(Account, ratio) %>%  
  head()
```

KIARA arrive en tête, que l'on compare les posts un par un ou la moyenne par compte.

Question 9.18

Réponse : Faux

La première ligne du résultat est la **Lotus Europa**, suivie de la Honda Civic. La Toyota Corolla a le mpg le plus élevé, mais pas le meilleur rapport mpg / poids.

Question 9.19

Réponse : 10

ncol(reponses_socio) ou dim(reponses_socio) : le nombre de colonnes correspond au nombre de variables (naissance, genre, permis, suisse, origine, fraterie, domicile, trajet, note_ecole, note_uni).

Question 9.20

Réponse : 17

table(reponses_socio\$genre) ou reponses_socio %>% count(genre).

Chapitre 10 : Statistiques descriptives

Question 10.1

Réponse : a) Faux ; b) Vrai ; c) Faux ; d) Vrai ; e) Vrai

Les statistiques descriptives **résumant** les données de l'échantillon (effectifs, fréquences, moyennes, dispersion, graphiques). Les tests d'hypothèse et l'analyse de relations causales relèvent des statistiques **inférentielles**, qui viennent généralement ensuite.

Question 10.2

Réponse : a) Faux ; b) Faux ; c) Vrai ; d) Vrai ; e) Faux

- a) Le nombre de frères et sœurs est une variable de comptage asymétrique, pas une loi normale.
- b) Un histogramme du nombre de frères et sœurs ne dit rien sur la forme des familles.
- c) 7 étudiant·es ont 3 frères et sœurs, contre 5 enfants uniques.
- d) La barre la plus haute est celle d'un frère ou d'une sœur (33 étudiant·es).
- e) Personne n'indique 7 frères et sœurs : le maximum est de 5. Vérifiez avec `table(reponses_socio$fraterie)`.

Question 10.3

Réponse : 74

`nrow(reponses_socio)` ou `dim(reponses_socio)` : le nombre de lignes correspond au nombre d'observations.

Question 10.4

Réponse : 10

`ncol(reponses_socio)` ou `dim(reponses_socio)` : le nombre de colonnes correspond au nombre de variables (naissance, genre, permis, suisse, origine, fraterie, domicile, trajet, note_ecole, note_uni).

Question 10.5

Réponse : 51

`table(reponses_socio$genre)` ou `reponses_socio %>% count(genre)`.

Question 10.6

Réponse : 17

`table(reponses_socio$genre)` ou `reponses_socio %>% count(genre)`.

Question 10.7

Réponse : 34

```
reponses_socio %>%  
  group_by(suisse) %>%  
  summarize(trajet_moyen = mean(trajet))
```

Pour suisse == "Oui", la moyenne vaut 33,52 minutes, soit 34 une fois arrondie.

Question 10.8

Réponse : Homme

```
reponses_socio %>%  
  group_by(genre) %>%  
  summarize(trajet_moyen = mean(trajet)) %>%  
  arrange(trajet_moyen)
```

arrange() trie par ordre croissant : la première ligne, Homme (20,6 minutes), a le trajet moyen le plus court.

Question 10.9

Réponse : Vrai

La fonction write.csv() écrit un data frame dans un fichier CSV, p. ex. write.csv(ice_age_df, "ice_age_df.csv").

Question 10.10

Réponse : Vrai

summary() affiche pour chaque variable numérique le minimum, les quartiles, la médiane, la moyenne et le maximum.

Question 10.11

Réponse : Vrai

```
reponses_socio %>%  
  group_by(suisse) %>%  
  summarize(trajet_moyen = mean(trajet))
```

Les étudiants sans passeport suisse mettent en moyenne 21,5 minutes, contre 33,5 pour les Suisses·ses. Attention : le groupe sans passeport suisse ne compte que 10 personnes. Pour savoir si une telle différence est due au hasard, il faudrait un test (session 11).

Question 10.12

Réponse : Vrai

Effectifs, moyennes et graphiques montrent souvent déjà les principales tendances ; les statistiques inférentielles testent ensuite si elles se généralisent à la population.

Chapitre 11 : Statistiques inférentielles I : le test t

Question 11.1

Réponse : b) Rejeter l'hypothèse nulle (H_0).

La valeur p (0.00016) est bien inférieure au seuil de 0.05 : on rejette H_0 . La moyenne de l'échantillon diffère significativement de la valeur testée.

Question 11.2

Réponse : a) Vrai ; b) Vrai ; c) Faux ; d) Faux ; e) Faux

Le test t à deux échantillons compare **deux moyennes** : H_0 « les moyennes sont égales » (a) contre H_1 « elles diffèrent » (b). Il ne teste ni la taille des populations ni les variances. Pour comparer trois groupes, on utilise une ANOVA.

Question 11.3

Réponse : a) Vrai ; b) Faux ; c) Vrai ; d) Faux ; e) Vrai

b) Corrélation n'est pas causalité : une troisième variable ou le hasard peuvent expliquer une corrélation.

d) La taille moyenne des femmes en Suisse est d'environ 165 cm : un échantillon dont la moyenne est de 180 cm n'est manifestement pas représentatif.

Question 11.4

Réponse : a) Vrai ; b) Faux ; c) Vrai ; d) Faux ; e) Vrai

a) Sous *sample estimates*, **mean of x** (les joueuses) vaut 183,86 cm ; **mean of y** (les hommes néerlandais) vaut 184,66 cm.

b), d) et e) La valeur p (0,40) est supérieure à 0,05 : on ne rejette pas H_0 . Dans l'échantillon, les hommes néerlandais sont un peu plus grands, mais cet écart de 0,8 cm n'est pas significatif ; l'intervalle de confiance à 95 % (de -2,66 à 1,06 cm) contient 0.

c) Les hypothèses portent sur les populations : H_0 postule que les deux moyennes sont égales ($\mu_1 = \mu_2$), H_1 qu'elles diffèrent.

Question 11.5

Réponse : e) Toutes les autres réponses sont incorrectes.

La valeur p est la probabilité d'observer des résultats au moins aussi extrêmes que ceux de l'échantillon **si l'hypothèse nulle était vraie**. Elle se calcule en supposant H_0 vraie : ce n'est donc ni la probabilité que H_0 soit vraie (a, c), ni la probabilité que les résultats soient « dus au hasard » (b). Cette dernière lecture est très répandue, mais elle inverse la condition : elle décrit $P(H_0 \mid \text{données})$ au lieu de $P(\text{données} \mid H_0)$. La valeur p ne dit rien non plus sur la normalité des données (d).

Question 11.6

Réponse : Vrai

L'erreur standard diminue quand la taille de l'échantillon augmente : avec de très grands échantillons, même une petite différence donne une valeur p très faible. « Significatif » ne veut donc pas dire « important ».

Chapitre 12 : Statistiques inférentielles II : ANOVA et régression

Question 12.1

Réponse : a) Faux ; b) Vrai ; c) Faux ; d) Vrai ; e) Vrai

a) La ligne du milieu est la **médiane**, pas la moyenne.

b) La nageoire la plus longue chez les manchots Adélie (210 mm) dépasse la plus courte chez les manchots papous (203 mm).

c) et d) Un graphique ne teste rien : il faut un test (ici une ANOVA) pour conclure à une différence significative.

e) Les deux points isolés appartiennent aux Adélie (172 mm et 210 mm).

Question 12.2

Réponse : a) Faux ; b) Vrai ; c) Vrai ; d) Vrai

a) Avec trois variables indépendantes, le modèle ne se résume pas à une droite dans un nuage de points à deux dimensions.

b) et c) Dans $y \sim x_1 + x_2 + x_3$, la variable à gauche du $\sim$ est la variable dépendante.

d) Les valeurs p de **wt** ($< 0,001$), **qsec** ($< 0,001$) et **am** (0,047) sont toutes inférieures à 0,05.

Question 12.3

Réponse : 4202

```
library(palmerpenguins)
mean(penguins$body_mass_g, na.rm = TRUE) # 4201.754
```

Deux manchots n'ont pas de poids (**NA**) : sans **na.rm = TRUE**, le résultat est **NA**.

Question 12.4

Réponse : 3862

```
penguins %>%
  filter(sex == "female") %>%
  summarize(poids_moyen = mean(body_mass_g)) # 3862.273
```

Question 12.5

Réponse : Vrai

Une corrélation seule ne prouve pas la causalité, mais ici un mécanisme connu (il faut plus d'énergie pour déplacer une masse plus grande) justifie l'hypothèse d'un effet causal du poids sur la consommation.

Question 12.6

Réponse : Vrai

Un R^2 de 0,75 signifie que le modèle explique 75 % de la variance de y . On le lit dans `summary(modele)` (*Multiple R-squared*).

Chapitre 13 : Révision 4 : statistiques

Question 13.1

Réponse : a) Faux ; b) Faux ; c) Faux ; d) Vrai

Le nuage de points place une variable numérique sur chaque axe. L'histogramme montre la distribution d'une variable numérique ; le diagramme à barres compte des catégories ; la boîte à moustaches compare une variable numérique entre catégories.

Question 13.2

Réponse : a) Faux ; b) Vrai ; c) Faux ; d) Vrai ; e) Vrai

Les statistiques descriptives **résumant** les données de l'échantillon (effectifs, fréquences, moyennes, dispersion, graphiques). Les tests d'hypothèse et l'analyse de relations causales relèvent des statistiques **inférentielles**, qui viennent généralement ensuite.

Question 13.3

Réponse : a) Faux ; b) Faux ; c) Vrai ; d) Vrai ; e) Faux

- a) Le nombre de frères et sœurs est une variable de comptage asymétrique, pas une loi normale.
- b) Un histogramme du nombre de frères et sœurs ne dit rien sur la forme des familles.
- c) 7 étudiant·es ont 3 frères et sœurs, contre 5 enfants uniques.
- d) La barre la plus haute est celle d'un frère ou d'une sœur (33 étudiant·es).
- e) Personne n'indique 7 frères et sœurs : le maximum est de 5. Vérifiez avec `table(reponses_socio$fraterie)`.

Question 13.4

Réponse : 34

```
reponses_socio %>%  
  group_by(suisse) %>%  
  summarize(trajet_moyen = mean(trajet))
```

Pour suisse == "Oui", la moyenne vaut 33,52 minutes, soit 34 une fois arrondie.

Question 13.5

Réponse : Homme

```
reponses_socio %>%  
  group_by(genre) %>%  
  summarize(trajet_moyen = mean(trajet)) %>%  
  arrange(trajet_moyen)
```

`arrange()` trie par ordre croissant : la première ligne, **Homme** (20,6 minutes), a le trajet moyen le plus court.

Question 13.6

Réponse : Vrai

`summary()` affiche pour chaque variable numérique le minimum, les quartiles, la médiane, la moyenne et le maximum.

Question 13.7

Réponse : Vrai

```
reponses_socio %>%  
  group_by(suisse) %>%  
  summarize(trajet_moyen = mean(trajet))
```

Les étudiants sans passeport suisse mettent en moyenne 21,5 minutes, contre 33,5 pour les Suisses·ses. Attention : le groupe sans passeport suisse ne compte que 10 personnes. Pour savoir si une telle différence est due au hasard, il faudrait un test (session 11).

Question 13.8

Réponse : Vrai

Effectifs, moyennes et graphiques montrent souvent déjà les principales tendances ; les statistiques inférentielles testent ensuite si elles se généralisent à la population.

Question 13.9

Réponse : b) Rejeter l'hypothèse nulle (H_0).

La valeur p (0.00016) est bien inférieure au seuil de 0.05 : on rejette H_0 . La moyenne de l'échantillon diffère significativement de la valeur testée.

Question 13.10

Réponse : a) Vrai ; b) Vrai ; c) Faux ; d) Faux ; e) Faux

Le test t à deux échantillons compare **deux moyennes** : H_0 « les moyennes sont égales » (a) contre H_1 « elles diffèrent » (b). Il ne teste ni la taille des populations ni les variances. Pour comparer trois groupes, on utilise une ANOVA.

Question 13.11

Réponse : a) Vrai ; b) Faux ; c) Vrai ; d) Faux ; e) Vrai

b) Corrélation n'est pas causalité : une troisième variable ou le hasard peuvent expliquer une corrélation.

d) La taille moyenne des femmes en Suisse est d'environ 165 cm : un échantillon dont la moyenne est de 180 cm n'est manifestement pas représentatif.

Question 13.12

Réponse : a) Vrai ; b) Faux ; c) Vrai ; d) Faux ; e) Vrai

a) Sous *sample estimates*, **mean of x** (les joueuses) vaut 183,86 cm ; **mean of y** (les hommes néerlandais) vaut 184,66 cm.

b), d) et e) La valeur p (0,40) est supérieure à 0,05 : on ne rejette pas H_0 . Dans l'échantillon, les hommes néerlandais sont un peu plus grands, mais cet écart de 0,8 cm n'est pas significatif ; l'intervalle de confiance à 95 % (de -2,66 à 1,06 cm) contient 0.

c) Les hypothèses portent sur les populations : H_0 postule que les deux moyennes sont égales ($\mu_1 = \mu_2$), H_1 qu'elles diffèrent.

Question 13.13

Réponse : e) Toutes les autres réponses sont incorrectes.

La valeur p est la probabilité d'observer des résultats au moins aussi extrêmes que ceux de l'échantillon **si l'hypothèse nulle était vraie**. Elle se calcule en supposant H_0 vraie : ce n'est donc ni la probabilité que H_0 soit vraie (a, c), ni la probabilité que les résultats soient « dus au hasard » (b). Cette dernière lecture est très répandue, mais elle inverse la condition : elle décrit $P(H_0 \mid \text{données})$ au lieu de $P(\text{données} \mid H_0)$. La valeur p ne dit rien non plus sur la normalité des données (d).

Question 13.14

Réponse : Vrai

L'erreur standard diminue quand la taille de l'échantillon augmente : avec de très grands échantillons, même une petite différence donne une valeur p très faible. « Significatif » ne veut donc pas dire « important ».

Question 13.15

Réponse : a) Faux ; b) Vrai ; c) Faux ; d) Vrai ; e) Vrai

- a) La ligne du milieu est la **médiane**, pas la moyenne.
- b) La nageoire la plus longue chez les manchots Adélie (210 mm) dépasse la plus courte chez les manchots papous (203 mm).
- c) et d) Un graphique ne teste rien : il faut un test (ici une ANOVA) pour conclure à une différence significative.
- e) Les deux points isolés appartiennent aux Adélie (172 mm et 210 mm).

Question 13.16

Réponse : a) Faux ; b) Vrai ; c) Vrai ; d) Vrai

- a) Avec trois variables indépendantes, le modèle ne se résume pas à une droite dans un nuage de points à deux dimensions.
- b) et c) Dans $y \sim x_1 + x_2 + x_3$, la variable à gauche du $\sim$ est la variable dépendante.
- d) Les valeurs p de `wt` ($< 0,001$), `qsec` ($< 0,001$) et `am` (0,047) sont toutes inférieures à 0,05.

Question 13.17

Réponse : 4202

```
library(palmerpenguins)
mean(penguins$body_mass_g, na.rm = TRUE) # 4201.754
```

Deux manchots n'ont pas de poids (**NA**) : sans `na.rm = TRUE`, le résultat est **NA**.

Question 13.18

Réponse : 3862

```
penguins %>%  
  filter(sex == "female") %>%  
  summarize(poids_moyen = mean(body_mass_g))    # 3862.273
```

Question 13.19

Réponse : Vrai

Une corrélation seule ne prouve pas la causalité, mais ici un mécanisme connu (il faut plus d'énergie pour déplacer une masse plus grande) justifie l'hypothèse d'un effet causal du poids sur la consommation.

Question 13.20

Réponse : Vrai

Un R^2 de 0,75 signifie que le modèle explique 75 % de la variance de y. On le lit dans `summary(modele)` (*Multiple R-squared*).

Références

Les références suivent les normes APA (7^e édition, adaptation française). Les vidéos, cours en ligne et outils sont accompagnés d’une courte note.

- Allaire, J. J., Teague, C., Scheidegger, C., Xie, Y., Dervieux, C., & Woodhull, G. (2026). *Quarto* (Version 1.9.37) [Logiciel]. Zenodo. <https://doi.org/10.5281/zenodo.5960048>
- Anderson, E. (1935). The irises of the Gaspé Peninsula. *Bulletin of the American Iris Society*, 59, 2–5.
- Arnold, J. B. (2026). *ggthemes: Extra themes, scales and geoms for 'ggplot2'* (Version 6.0.0) [Paquet R]. CRAN. <https://doi.org/10.32614/CRAN.package.ggthemes>
- Barnier, J. (2024, 19 décembre). *Introduction à R et au tidyverse*. <https://doi.org/10.5281/zenodo.6382598>
- Barr, D., DeBruine, L., & Van Lissa, C. J. (2026). *webexercises: Create interactive web exercises* (Version 1.2.0) [Paquet R]. CRAN. <https://doi.org/10.32614/CRAN.package.webexercises>
- Brown, K. W. (2021, 23 janvier). *Colors in R*. RPubs. <https://rpubs.com/kylewbrown/r-colors>
- Chang, W. (2018). *R graphics cookbook: Practical recipes for visualizing data* (2^e éd.). O’Reilly Media. <https://r-graphics.org/>
- Chat Sceptique. (2020, 16 avril). *Introduction à R et RStudio (ft. mon chat radioactif)* [Vidéo]. YouTube. https://www.youtube.com/watch?v=sav3Mbe0_DM
Tutoriel en français d’environ une heure de Nathan Uyttendaele.
- CrowdTangle Team. (2023). *CrowdTangle* [Logiciel]. Facebook, Menlo Park, California, United States.
Outil fermé en 2024.
- DataCamp. (s.d.). *Introduction to R programming* [Cours en ligne]. Consulté le 6 octobre 2026, à l’adresse <https://www.datacamp.com/courses/free-introduction-to-r>
Cours gratuit : vecteurs, matrices, facteurs, data frames et listes.
- Fisher, R. A. (1936). The use of multiple measurements in taxonomic problems. *Annals of Eugenics*, 7(2), 179–188. <https://doi.org/10.1111/j.1469-1809.1936.tb02137.x>
- freeCodeCamp.org. (2019, 6 juin). *R programming tutorial – Learn the basics of statistical computing* [Vidéo]. YouTube. https://www.youtube.com/watch?v=_V8eKsto3Ug
Cours d’introduction générale à R d’environ deux heures, par Barton Poulson.
- Gorman, K. B., Williams, T. D., & Fraser, W. R. (2014). Ecological sexual dimorphism and environmental variability within a community of Antarctic penguins (genus *Pygoscelis*). *PLoS ONE*, 9(3), e90081. <https://doi.org/10.1371/journal.pone.0090081>
- Goulet, V. (2026). *Programmer avec R*. <https://vigou3.gitlab.io/programmer-avec-r/>
- HarvardX. (s.d.). *Data science: R basics* [Cours en ligne]. edX. Consulté le 6 octobre 2026, à l’adresse <https://www.edx.org/learn/r-programming/harvard-university-data-science-r>

basics

Cours de Rafael Irizarry, gratuit en auditeur · rice libre.

- Henderson, H. V., & Velleman, P. F. (1981). Building multiple regression models interactively. *Biometrics*, 37(2), 391–411. <https://doi.org/10.2307/2530428>
- Horst, A. M., Hill, A. P., & Gorman, K. B. (2020). *palmerpenguins: Palmer Archipelago (Antarctica) penguin data* (Version 0.1.0) [Paquet R]. Zenodo. <https://doi.org/10.5281/zenodo.3960218>
- Insee. (s.d.). *utilitr : une documentation collaborative sur l'usage de R*. Consulté le 6 octobre 2026, à l'adresse <https://book.utilitr.org/>
- Larmarange, J. (2026, 20 août). *guide-R : Guide pour l'analyse de données d'enquêtes avec R*. <https://larmarange.github.io/guide-R/>
- Latzer, M., Festic, N., Kappeler, K., & Odermatt, C. M. (2023). *Internetanwendungen und deren Nutzung in der Schweiz 2023*. Universität Zürich. <https://doi.org/10.5167/uzh-251581>
- Long, J. D., & Teetor, P. (2019). *R cookbook* (2^e éd.). O'Reilly Media. <https://rc2e.com/>
- McNeil, D. R. (1977). *Interactive data analysis: A practical primer*. John Wiley & Sons.
- Müller, S.-M., Floris, J., Rohrmann, S., Staub, K., & Matthes, K. L. (2022). Body height among adult male and female Swiss Health Survey participants in 2017: Trends by birth years and associations with self-reported health status and life satisfaction. *Preventive Medicine Reports*, 29, 101980. <https://doi.org/10.1016/j.pmedr.2022.101980>
- Peeters, S. (2026). *Zeeschuimer* (Version 1.14.3) [Logiciel]. Zenodo. <https://doi.org/10.5281/zenodo.22660960>
- Posit team. (2026). *RStudio: Integrated development environment for R* (Version 2026.09.0+174) [Logiciel]. Posit Software, PBC. <https://posit.co/>
- R Core Team. (2026). *R: A language and environment for statistical computing* (Version 4.6.1) [Logiciel]. R Foundation for Statistical Computing. <https://doi.org/10.32614/R.manuals>
- StanfordOnline. (s.d.). *R programming fundamentals* [Cours en ligne]. edX. Consulté le 6 octobre 2026, à l'adresse <https://www.edx.org/learn/r-programming/stanford-university-r-programming-fundamentals>
- Gratuit en auditeur · rice libre.
- StatQuest with Josh Starmer. (2015, 3 février). *R-squared, clearly explained!!!* [Vidéo]. YouTube. <https://www.youtube.com/watch?v=2AQKmw14mHM>
- StatQuest with Josh Starmer. (2017a, 24 juillet). *Linear regression, clearly explained!!!* [Vidéo]. YouTube. https://www.youtube.com/watch?v=nk2CQITm_eo
- StatQuest with Josh Starmer. (2017b, 25 juillet). *Linear regression in R, step-by-step* [Vidéo]. YouTube. https://www.youtube.com/watch?v=u1cc1r_Y7M0
- StatQuest with Josh Starmer. (2017c, 9 octobre). *The normal distribution, clearly explained!!!* [Vidéo]. YouTube. <https://www.youtube.com/watch?v=rzFX5NWojp0>
- StatQuest with Josh Starmer. (2018, 26 juillet). *Logistic regression in R, clearly explained!!!!* [Vidéo]. YouTube. https://www.youtube.com/watch?v=C4N3_XJJ-jU
- StatQuest with Josh Starmer. (2020a, 22 mars). *p-values: What they are and how to interpret them* [Vidéo]. YouTube. <https://www.youtube.com/watch?v=vemZtEM63GY>
- StatQuest with Josh Starmer. (2020b, 5 juillet). *Hypothesis testing and the null hypothesis, clearly explained!!!* [Vidéo]. YouTube. <https://www.youtube.com/watch?v=0oc49DyA3hU>
- StatQuest with Josh Starmer. (2022, 18 novembre). *Multiple regression in R, step by step!!!*

- [Vidéo]. YouTube. <https://www.youtube.com/watch?v=mno47Jn4gaU>
- Summer Institutes in Computational Social Science. (s.d.). *SICSS boot camp*. Consulté le 6 octobre 2026, à l'adresse https://sicss.io/boot_camp/
- Série de vidéos d'introduction au code pour les sciences sociales, présentée par Chris Bail (Duke University).
- Udacity. (s.d.). *Data analysis with R* [Cours en ligne]. Consulté le 6 octobre 2026, à l'adresse <https://www.udacity.com/course/data-analysis-with-r--ud651>
- Cours gratuit ; page mise à jour le 4 mars 2022.
- U.S. Environmental Protection Agency. (s.d.). *fuelconomy.gov*. Consulté le 6 octobre 2026, à l'adresse <https://fuelconomy.gov/>
- Source des données mpg du paquet ggplot2.
- van Atteveltdt, W. (2020, 1 octobre). *Introduction to R & Rstudio* [Vidéo]. YouTube. https://www.youtube.com/watch?v=PVhZD5MINYM&list=PLjXODJ_lGN_V2ntvV2CN_GvzZ6Qm5km9L
- Première vidéo de la liste de lecture « Data Analysis with R ».
- van Atteveltdt, W., Trilling, D., & Arcila Calderón, C. (2022). *Computational analysis of communication*. Wiley-Blackwell. <https://cssbook.net/>
- Wickham, H. (2016). *ggplot2: Elegant graphics for data analysis*. Springer. <https://doi.org/10.1007/978-3-319-24277-4>
- Wickham, H., Averick, M., Bryan, J., Chang, W., McGowan, L. D., François, R., Golemund, G., Hayes, A., Henry, L., Hester, J., Kuhn, M., Pedersen, T. L., Miller, E., Bache, S. M., Müller, K., Ooms, J., Robinson, D., Seidel, D. P., Spinu, V., ... Yutani, H. (2019). Welcome to the tidyverse. *Journal of Open Source Software*, 4(43), 1686. <https://doi.org/10.21105/joss.01686>
- Wickham, H., Çetinkaya-Rundel, M., & Golemund, G. (2023). *R for data science: Import, tidy, transform, visualize, and model data* (2^e éd.). O'Reilly Media. <https://r4ds.hadley.nz/>
- Wickham, H., François, R., Henry, L., Müller, K., & Vaughan, D. (2026). *dplyr: A grammar of data manipulation* (Version 1.2.1) [Paquet R]. CRAN. <https://doi.org/10.32614/CRA N.package.dplyr>
- Xie, Y. (2026). *knitr: A general-purpose package for dynamic report generation in R* (Version 1.52) [Paquet R]. <https://yihui.org/knitr/>